

THE COMPLETE SYSTEM DESIGN INTERVIEW GUIDE

THE SYSTEM DESIGN INTERVIEW PLAYBOOK

For Java Backend Developers, Freshers & 1-3 Years of Experience

Covering system design fundamentals, Java + Spring Boot mapping, scalability, Redis, Kafka, databases, microservices, and real interview design questions asked at product and service companies.

WHAT THIS PLAYBOOK COVERS

- System Design Fundamentals & the 5-Step Interview Framework
- Core Building Blocks: Load Balancers, Caching, Queues, CDNs
- Java + Spring Boot Mapping for Every Design Component
- Redis Caching, Kafka & Async Communication Patterns
- SQL vs NoSQL, Database Scaling, Sharding & Replication
- Reliability, Rate Limiting, Security & Observability
- 15 Real Interview Design Questions, Fully Solved
- Ready-to-Use Answer Templates for Interview Day

LEGAL

Copyright & Disclaimer

Copyright

© 2026 javainterviewready. All rights reserved. This guide, including all text, diagrams, code samples, and design content, is the intellectual property of javainterviewready and may not be copied, reproduced, redistributed, or resold in whole or in part without written permission.

Personal Use Only

This guide is licensed for personal, individual use. Sharing your copy, uploading it to public drives, or reselling it undermines the work that went into building it and is not permitted.

No Guarantee of Outcome

This guide is built to help you think and communicate like an engineer during system design interviews. It does not guarantee a job offer, a specific interview outcome, or a particular company's hiring decision. Your results depend on practice, communication, and the specific interview panel.

Accuracy

Every effort has been made to keep explanations, diagrams, and Java/Spring Boot mappings accurate and current. Tools, frameworks, and interview patterns evolve, so treat this guide as a strong foundation, not the final word.

GETTING STARTED

How to Use This Guide

This playbook is built to be read module by module, in order, if you are starting from zero. If you already know the fundamentals, jump straight to the module that matches your gap.

If you are a complete fresher

Start at Module 1 and go straight through to Module 13 before touching the interview questions in Module 14. The building blocks in Modules 4 to 13 are what let you actually reason through those questions instead of memorizing them.

If you already know the basics

Skim Modules 1 to 3 for the interview framework, then go directly to Module 14 and treat every question as a mock interview. Come back to Modules 4 to 13 whenever a question exposes a gap.

The night before an interview

Use Module 16 (Cheat Sheet + Interview Tips) and the Bonus Answer Templates. They are built for a fast, final pass, not for learning something new.

While you read

Pay attention to the callout boxes. Interview Focus boxes tell you what the interviewer is actually listening for. Common Mistake boxes point out where freshers lose marks, most often by jumping to a solution before asking clarifying questions.

WHAT'S INSIDE

Table of Contents

MODULE 01	What is System Design and Why Interviewers Ask It	1
MODULE 02	How to Approach Any System Design Question	13
MODULE 03	Requirement Gathering and Scope Definition	27
MODULE 04	Core Building Blocks of System Design	42
MODULE 05	REST API Design for System Design Interviews	63
MODULE 06	SQL vs NoSQL: When to Pick What	77
MODULE 07	Database Scaling Basics	90
MODULE 08	Caching with Redis and Spring Boot	105
MODULE 09	Message Queues & Async Communication	122
MODULE 10	Designing for Scale	143
MODULE 11	Reliability, Fault Tolerance and Failure Handling	158
MODULE 12	Rate Limiting, Security and Authentication	181
MODULE 13	Logging, Monitoring, and Observability	200
MODULE 14	Real Interview Design Questions	218
Q1	URL Shortener	218
Q2	Chat Application	229
Q3	Notification System	236
Q4	File Upload System	246
Q5	Food Delivery App	261
Q6	E-Commerce Order System	276
Q7	Payment System	290
Q8	Ride Booking App	302
Q9	Instagram Feed	315
Q10	YouTube Video Upload	330
Q11	BookMyShow Ticket Booking	346
Q12	Rate Limiter	361
Q13	Logging System	377
Q14	Online Code Judge	391
Q15	WhatsApp Last Seen / Online Status	407

WHAT'S INSIDE

Table of Contents

MODULE 15	Java + Spring Boot System Design Mapping	419
MODULE 16	System Design Cheat Sheet + Interview Tips	441
BONUS	System Design Interview Answer Templates	454

javainterviewready

MODULE 1

What is System Design and Why Interviewers Ask It

Understand what system design actually means, why companies test it even for freshers, and what “good enough” looks like at your level.

Part of The System Design Interview Playbook, for Java Backend Developers, Freshers & 1-3 Years of Experience.

javainterviewready

TOPIC 01

What “System Design” Actually Means

Ask five engineers what system design means and you'll get five different answers. One will talk about scalability. Another will say it's about databases. Someone else will just say “drawing boxes and arrows” and laugh, because that's honestly what it feels like the first time you see one of these interviews.

Here's the version that actually matters for you right now. System design is the skill of deciding how the different parts of a software system talk to each other, store data, and stay up when things go wrong, before you write a single line of code. It's planning, not building. You're answering questions like: where does this data live, what happens when 10,000 people hit this endpoint at once, and what breaks first when something fails.

If you've ever built a Spring Boot project where everything sat in one application, one database, and a handful of REST controllers, you've already done a tiny version of system design. You just didn't call it that. The interview version of this skill is the same idea, scaled up, with someone asking you follow-up questions to see how you think under pressure.

INTERVIEW FOCUS

Interviewers don't expect you to design Netflix in 45 minutes. They want to see whether you can take a vague problem, ask the right questions, and build something reasonable step by step. The destination matters less than how you got there.

A lot of freshers think system design means memorizing diagrams for Uber, WhatsApp, and Instagram and reproducing them on a whiteboard. That gets you maybe 30% of the way. The other 70% is reasoning ability. If the interviewer changes one constraint, like “what if writes are 10x more frequent than reads,” can you adjust your design on the spot, or does your answer fall apart because you memorized a fixed picture?

A simple way to think about it

Every system design problem, no matter how big, breaks down into the same handful of decisions repeated at different scales:

- Where does the request enter the system, and how does it get routed?
- Where does data get stored, and what shape does it take (SQL, NoSQL, cache)?
- What happens synchronously, and what can be pushed to run later (async)?
- What happens when one piece fails. Does the rest of the system survive?
- How does the system grow when traffic grows 10x or 100x?

Once you internalize that these five questions repeat across almost every design problem, the topic stops feeling infinite. It becomes a pattern you apply, not a fact you memorize.

Diagram: A request, simplified to its core decisions

```
Client Request
  |
  v
Entry Point (API Gateway / Load Balancer)
  |
  v
Service (business logic lives here)
  |           |
  v           v
Database    Cache / Queue
```

WHERE FRESHERS GO WRONG

A common mistake freshers make is jumping straight into naming technologies. “I’ll use Kafka here, Redis there, microservices everywhere.” That’s not designing, that’s name-dropping. Interviewers notice immediately when someone throws out buzzwords without explaining why that piece is needed for this specific problem.

TOPIC 02

What System Design Is NOT

Before going any further, it helps to clear up what this round is not testing. Most of the fear around system design interviews comes from a wrong picture of what’s expected. Here’s the corrected version.

System design is not memorizing the famous diagrams. You can find “Uber architecture” or “WhatsApp system design” posts everywhere online, and yes, looking at a few of them helps you recognize patterns. But if your entire prep is screenshotting diagrams and trying to redraw them from memory, you’ll fall apart the moment the interviewer changes one detail of the question. They’re not asking you to reproduce a diagram. They’re asking you to think.

It’s also not about throwing in every trendy tool you’ve heard of. Kafka, Redis, Kubernetes, microservices, these are real tools that solve real problems, but they are not decorations you sprinkle on a design to look advanced. A fresher who says “I’ll add Kafka for async processing” without being able to explain what would actually go wrong without it is, frankly, easy to catch out. Interviewers ask one follow-up question and the whole thing falls over.

WHERE FRESHERS GO WRONG

If you can't answer "what breaks if I remove this component," you probably added it for the wrong reason. That one question separates candidates who understand their design from candidates who memorized a shape.

It's also not about drawing the most complex diagram you can fit on the board. Some candidates think more boxes means more impressive. In reality, a cluttered diagram with components you can't justify is worse than a simple diagram you can defend completely. Interviewers have sat through enough of these to tell the difference instantly.

And it's definitely not about giving a perfect, production-grade architecture at fresher level. Nobody expects that from you, and trying to fake that confidence usually backfires, because the follow-up questions will expose the gaps anyway. It's safer, and frankly more impressive, to give a simple design and reason about it well than to give an elaborate one you can't defend.

So what is it actually testing?

Reasoning. Tradeoffs. The ability to explain a decision clearly, out loud, to someone who's listening for how you think, not just what you draw. That's the whole game. Everything else, the boxes, the arrows, the tool names, is just vocabulary you use to express that reasoning.

Mini Example: URL Shortener at Fresher Level

Here's what that looks like in practice. Say the interviewer asks you to design a URL shortener. A fresher answer that's actually solid doesn't need to start big. It can start as simple as this:

Starting point: the simplest version that works

```
Client
  |
  v
URL Shortener Service
  |
  v
Database
```

That's it. One service takes the long URL, generates a short code, saves the mapping in the database, and returns it. When someone visits the short URL, the service looks it up and redirects. No cache, no load balancer, nothing fancy. At fresher level, saying this out loud and explaining why it works for the basic case is already a reasonable start.

Then the interviewer will almost always push further. “What if this gets a million requests a day?” That’s your cue to evolve the design, not throw it out and start over:

After the scale question: same idea, scaled up

```
Client
|
v
Load Balancer
|
v
URL Shortener Service
|       |
v       v
Redis Cache  Database
```

The load balancer shows up because one instance of the service can’t handle that traffic alone. The cache shows up because redirects are read-heavy, the same popular short links get hit over and over, and there’s no reason to make a database round trip every single time. Notice that nothing here is random. Each addition answers a specific question the interviewer asked. That’s the entire skill, in miniature.

WHAT INTERVIEWERS NOTICE QUICKLY

Whether you ask about requirements before jumping into a diagram.

Whether every component on your diagram has a reason you can state in one sentence.

Whether you can talk through tradeoffs instead of just picking an option.

Whether you stay calm when the interviewer changes a constraint midway through.

Whether you connect the design to something you actually know, like Spring Boot, instead of speaking only in abstractions.

TOPIC 03

Why Companies Test This Even for Freshers

If you’re 0-3 years into your career, this question has probably crossed your mind: “I haven’t built anything at scale. Why is this company asking me to design a chat app that handles a billion messages a day?” It’s a fair question, and the honest answer is that the interviewer isn’t testing whether you’ve done it before. They’re testing how you think when you haven’t.

Companies hire freshers expecting to train them on the actual codebase, the actual infrastructure, the actual scale problems specific to that company. What they can't easily train is structured thinking. Can you take an ambiguous problem and not freeze? Can you ask good questions instead of guessing? Can you explain a tradeoff instead of picking an answer and hoping it's right? That's what the round is actually measuring.

What this looks like from the other side of the table

Having sat on the interviewer side of this for a while, here's the pattern that actually separates candidates at the fresher and early-career level. It's rarely about who knows more buzzwords. It's about who stays calm and structured when the problem gets harder mid-interview.

- Candidate A jumps straight to drawing boxes without asking what the system actually needs to do. Five minutes in, the interviewer realizes the design solves the wrong problem.
- Candidate B asks two or three sharp clarifying questions, states an assumption out loud, and then starts simple. Even if the design is basic, the thinking is visible and the interviewer can follow along.

Candidate B usually does better, even with a weaker final design, because the interviewer isn't grading the diagram on a whiteboard like an art piece. They're grading your decision-making out loud.

WHAT INTERVIEWERS ACTUALLY EXPECT AT YOUR LEVEL

Clear communication of your thought process.

Reasonable, not perfect, requirement gathering.

Basic awareness of scale (what changes at 100 users vs 1 million).

Ability to connect the design to something you've actually built, even a small project.

Comfort saying "I'm not sure, but here's how I'd approach it" instead of going silent.

Companies also test this early because system design awareness compounds. An engineer who understands why a cache exists, why async processing matters, why a single database becomes a bottleneck, writes better code from day one. They don't need someone to explain later why their feature broke under load. That's the real reason this shows up even in entry-level interviews, not because they expect you to have already designed Twitter.

In real projects, this matters earlier than you'd expect

Even a small Spring Boot service for an internal tool runs into these questions. Should this endpoint hit the database directly or read from a cache? Should this notification be sent synchronously inside the request, or pushed to a queue so the user doesn't wait? These are system design decisions at a small scale, and the habits you build answering them in interviews carry directly into how you write production code.

TOPIC 04

What “Good Enough” Looks Like at Your Level

This is the part most guides skip, and it's the part that actually reduces anxiety before the interview. Nobody at the 0-3 year mark is expected to produce a production-grade architecture for a system handling a billion users. If you walk in thinking you need to, you'll freeze the moment the problem gets hard.

“Good enough” at your level means something specific. It means you can take a vague prompt like “design a URL shortener” and turn it into a workable system in layers. Start simple. Add pieces as the interviewer pushes on scale, reliability, or edge cases. You're not expected to arrive with the final architecture in your head. You're expected to build toward it, visibly, in front of them.

Layered thinking beats a perfect-looking diagram

Here's a sequence that tends to work well for freshers and early-career engineers. Not because it's the official method, but because it matches how interviewers actually evaluate growth in real time during the conversation.

- Start with the simplest design that solves the core functional requirement. One service, one database, no caching, no async, nothing fancy.
- State it out loud as a baseline: “At my level, I'd start with this simple version and then improve it as we talk about scale.”
- When the interviewer pushes on load, latency, or failure, add exactly the component that solves that specific problem. Don't add five things at once.
- Explain why each addition exists. “I'm adding a cache here because this is a read-heavy endpoint and most reads are for the same handful of popular records.”

TIP

If you don't know where to start, start with the request path. Where does it enter, what does it touch, where does the response come from. That alone gives you a baseline diagram to improve, instead of staring at a blank page.

Many beginners design this in reverse. They try to think of the most impressive, scalable answer first, throw in Kafka, Redis, sharding, and microservices on the first pass, and then can't explain why half of it is there when asked. That usually backfires. An interviewer who hears unexplained buzzwords will dig in immediately, and that's exactly where the interview falls apart.

What “good enough” does not mean

It doesn't mean a shallow answer. Saying “we'll just add a load balancer and a database” without explaining the reasoning isn't simplicity, it's vagueness, and interviewers can tell the difference immediately. Good enough means simple but justified. Every box on your diagram should have an answer ready for “why is this here?”

INTERVIEW MOMENT

A realistic follow-up question you should expect after any design: “What's the single point of failure in what you just drew?” If you can answer that confidently, even if the answer is “the database right now, and here's how I'd fix that next,” you're in good shape. If you've never thought about it, that's the gap to close before your next interview.

TOPIC 05

Where Java and Spring Boot Fit Into All of This

This guide keeps coming back to Java and Spring Boot for a reason. Most system design content online is written in an abstract, language-agnostic way, which is fine for theory but doesn't help you when an interviewer asks “okay, how would you actually build that service?” That question lands differently when you can answer it in terms of code you've actually written.

Every concept in this playbook eventually maps to something concrete in the Spring ecosystem. A load balancer connects to Spring Cloud LoadBalancer and service discovery. A cache connects to @Cacheable and Redis. Async processing connects to Kafka, @Async, or Spring's event system. None of these are separate topics from system design. They're the implementation layer underneath it.

How a typical request flows through a Spring Boot service

```
Client
|
v
Controller  (@RestController, handles HTTP)
|
v
Service      (business logic, transactions)
|
v
Repository   (Spring Data JPA / JdbcTemplate)
|
v
Database
```

If you already understand this flow from building REST APIs, you have more system design intuition than you think. The interview version just zooms out. Instead of one service, you're now asking: do we need more than one service? Instead of one database call, you're asking: should this be cached, and for how long? The underlying Controller, Service, Repository pattern doesn't disappear, it just becomes one box in a bigger picture.

JAVA / SPRING BOOT MAPPING

Throughout this playbook, every major concept gets a direct mapping. Caching maps to @Cacheable and Redis. Rate limiting maps to Bucket4j or Spring Cloud Gateway filters. Circuit breakers map to Resilience4j. Authentication maps to Spring Security with JWT or OAuth2. You'll see this pattern repeat across every module, because it's what makes the difference between knowing system design in theory and being able to explain it as a Java backend developer.

If I were mentoring someone preparing for these interviews, the advice I'd give first is this: don't treat system design as a separate subject from the Spring Boot work you're already doing. Every time you build a feature, ask yourself one extra question. What would happen here if traffic was 100x higher? You don't need to fix it. Just noticing it is the habit that makes you good at this round.

TOPIC 06

How System Design Rounds Actually Run (So Nothing Feels Like a Surprise)

A lot of nervousness around this round comes from not knowing the shape of it beforehand. Coding rounds feel familiar because you've practiced hundreds of problems. System design feels unfamiliar because most people walk in without ever having seen one play out end to end.

The typical structure of a 45 minute round

- 0-5 minutes: The interviewer gives you a vague prompt. "Design a URL shortener" or "Design a notification system." That's usually it. No extra detail unless you ask.
- 5-12 minutes: You ask clarifying questions and state assumptions. This is where requirement gathering happens, and it matters more than people expect.
- 12-30 minutes: You build the design in layers, starting simple, narrating your thinking, and adding components as you go.
- 30-42 minutes: The interviewer pushes on scale, failure scenarios, or a specific tradeoff. This is usually where the real signal comes from.
- 42-45 minutes: Wrap-up, questions for the interviewer, sometimes a quick recap of the design.

Notice how much of that time is just talking and reasoning, not drawing. If you walk in expecting to spend 40 minutes drawing a perfect diagram, you'll run out of time and miss the part that's actually being scored, which is the conversation around the diagram.

TRADEOFF

There's a real tradeoff between speed and depth here. Rushing to a "complete" design in the first ten minutes looks efficient, but it usually means you skipped requirement gathering, and the interviewer has to course-correct you later. Taking time upfront to ask questions feels slower, but it almost always produces a design that needs fewer corrections later. Interviewers consistently prefer the second approach, even if it looks less polished in the first five minutes.

A mistake that shows up constantly at this level

Staying silent while thinking. In a coding interview, a minute of silence is normal because you're working through logic. In a system design interview, silence is much more costly, because the interviewer has nothing to evaluate during that time. Thinking out loud, even messily, gives them something to react to and guide. "I'm trying to decide between a single database and splitting reads and writes, let me think through this" is a far better fifteen seconds than dead silence followed by an answer.

INTERVIEW MOMENT

If you ever feel stuck mid-interview, it's completely fine to say: "Let me back up and reconsider this part of the design." Interviewers see this constantly, and it almost always reads better than pushing forward with something you already know is wrong.

How to Sound Confident Even When You Don't Know Everything

Here's something that took me a while to learn myself: confidence in this round has nothing to do with knowing every answer. It's about how you talk when you don't. Freshers often go quiet the moment they hit something unfamiliar, as if admitting a gap will end the interview. It won't, but going silent might.

A few lines do most of the work here, and you can use them almost word for word:

- "At my level, I would start with a simple design and improve it step by step."
- "I'm not fully sure about the production-level details, but I can explain the direction I'd take."
- "I would first identify the bottleneck before adding a new component."

Notice none of these are excuses. They're honest framing that still moves the conversation forward. An interviewer hearing "I'm not fully sure about the production details, but here's the direction" gets way more signal than silence, and more respect than a confident-sounding guess that falls apart under one follow-up question.

This matters more than it sounds like it should. Plenty of candidates with decent technical instincts undersell themselves because they think hesitation reads as weakness. In a system design round, controlled, honest hesitation reads as maturity. Reckless confidence is what actually costs you points.

TOPIC 07**What to Expect From the Rest of This Playbook**

This module was the foundation. From here, the playbook moves through the actual building blocks one at a time: load balancing, caching, databases, message queues, API gateways, authentication, and the tradeoffs that connect them. Each module follows the same mentoring approach used here, but the structure of each topic will vary depending on what fits the concept best.

By the time you reach the classic design questions later in this playbook, like URL shorteners, chat systems, and payment platforms, you won't be learning new patterns from scratch. You'll be applying the same five core decisions from Topic 1, just with more components and sharper tradeoffs involved.

A short checklist before you move to Module 2

- Can you explain, in your own words, what system design means without using the word “scalability” as a crutch?
- Do you understand why companies test this even without prior production experience?
- Can you describe what a layered, “start simple and improve” answer looks like?
- Do you see how your existing Spring Boot knowledge (Controller, Service, Repository) already connects to system design thinking?

If most of those feel shaky, that's completely normal this early in the playbook. The goal of Module 1 wasn't to make you design-ready. It was to remove the fog around what this round actually tests, so the components you learn next have somewhere to attach.

Next: Module 2 shows you how to approach any system design question using a simple step-by-step interview framework.

How to Approach Any System Design Question

A simple 5-step framework. What to say first, what to draw, and how to structure your answer from start to finish.

Part of **The System Design Interview Playbook**

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

MODULE 2 · OVERVIEW

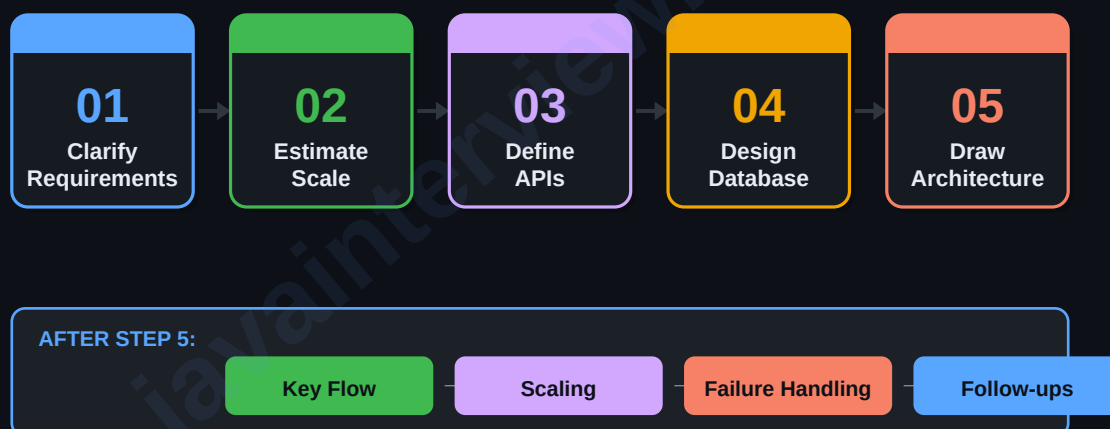
The 5-Step Framework

Most freshers fail system design interviews not because they don't know the tech. They fail because they panic, jump straight into drawing boxes, and lose the interviewer in the first two minutes.

The interviewer isn't expecting a perfect design. They're watching how you think. Whether you ask the right questions. Whether your decisions are driven by the problem or just habit.

This framework gives you a reliable structure for any system design question. Practice it enough and it becomes automatic.

The 5-Step System Design Interview Framework



BEFORE YOU START

This framework works for everything. URL shortener, chat app, payment system, Instagram feed. The same 5 steps apply every time. What changes is what you put inside each step.

STEP 1

STEP 1 OF 5

Clarify Requirements

The most common mistake in system design interviews is jumping straight into drawing. Don't. Take 3-5 minutes at the start to ask questions. This tells the interviewer you think before you act.

Requirements split into two buckets. You need both before you touch the whiteboard.

Functional Requirements

- What the system does
- User actions & features
- Core capabilities

Example:

- Create short URL
- Redirect user to original URL

Non-Functional Requirements

- How well the system works
- Scale, latency, availability
- Consistency, durability, security

Example:

- Redirect must be fast (<50ms)
- System must be highly available

EXAMPLE QUESTIONS: URL SHORTENER

Functional:

- Can users create custom short URLs, or auto-generated only?
- Do short URLs expire? Who controls the TTL?
- Do users need to be logged in to create a short URL?
- Should we track click analytics per URL?

Non-Functional:

- Redirect latency should be under 50ms.
- System should be highly available (99.9%+ uptime).
- Read-heavy. Expected 10:1 read-to-write ratio.

WHAT INTERVIEWERS ACTUALLY LOOK FOR

Most freshers answer the functional part fine. But when non-functional requirements come up, they go blank.

The interviewer wants to hear something like:

"Since this is read-heavy, I will optimize the read path and add caching early in the design."

That one line shows you understand system tradeoffs.

COMMON MISTAKE

Some candidates ask 10+ questions before starting. That signals you can't move without hand-holding. Ask 4-6 targeted questions, confirm assumptions, and proceed.

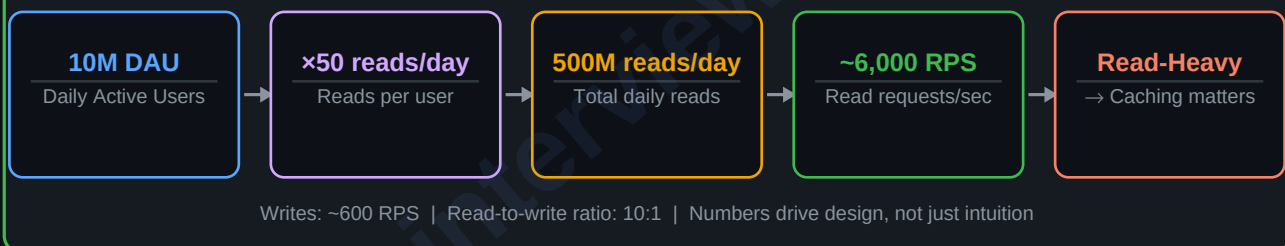
STEP 2**STEP 2 OF 5**

Estimate the Scale

You don't need to be precise here. The goal is to show that your design decisions are driven by numbers, not just intuition.

If you say 'we need caching' without knowing the read volume, an interviewer will push back. But if you say '10M DAU, 10:1 read-to-write ratio, so caching makes sense': that is a much stronger answer.

QUICK BACK-OF-THE-ENVELOPE ESTIMATION



Round numbers are fine in interviews. 580 becomes 600. 5,800 becomes 6,000. No one is checking your arithmetic. They're checking your approach.

HOW TO SAY IT

"Let me do a quick back-of-the-envelope. 10M DAU, ~50 reads per user per day. That is 500M reads/day, roughly 6,000 reads/sec. This is clearly read-heavy, so caching will be important in the design."

STEP 3**STEP 3 OF 5**

Define the APIs

Before drawing a single box, write out the key API endpoints. This is the contract between the client and your system. It forces you to think clearly about what data goes in and what comes out.

Method	Endpoint	Purpose	Spring Boot
POST	/api/v1/urls	Create short URL	@PostMapping
GET	/api/v1/{shortCode}	Redirect to original	@GetMapping
GET	/api/v1/urls/{code}/analytics	Click analytics	@GetMapping
DELETE	/api/v1/urls/{shortCode}	Delete short URL	@DeleteMapping

IN SPRING BOOT

@RestController + @RequestMapping("/api/v1")

POST /urls → @PostMapping → createShortUrl(@RequestBody CreateUrlRequest req)

GET /{code} → @GetMapping → redirect() returns ResponseEntity with 301/302

Controller calls @Service, which checks Redis first, then falls back to @Repository (JPA).

STEP 4

STEP 4 OF 5

Design the Database

Two things matter here: what tables or collections you need, and which database type you pick. Both need justification. Don't just say 'I'll use Postgres' without a reason.

SQL

PostgreSQL · MySQL

- Strong consistency
- ACID transactions
- Complex queries
- Schema migrations
- Best for relational data

Good for:

- Orders, Users, Products
- Financial transactions

NoSQL

MongoDB · DynamoDB · Cassandra

- Flexible schema
- Horizontal scaling
- High write throughput
- Eventually consistent
- Best for key-value / document

Good for:

- URL mappings
- Session data, Logs

HOW TO DECIDE QUICKLY IN AN INTERVIEW

Is the data relational? Need ACID? → SQL

Flexible schema or very high write throughput? → NoSQL

For URL shortener: shortCode → originalUrl is simple key-value. Either works.

SQL is fine here since the data is small and reads dominate. Redis handles hot lookups.

URL SHORTENER DATABASE SCHEMA

Table: urls

id BIGINT PRIMARY KEY (auto-increment)

short_code VARCHAR(10) UNIQUE, NOT NULL ← add INDEX here

original_url TEXT NOT NULL

user_id BIGINT FK → users.id (nullable)

expires_at TIMESTAMP nullable

created_at TIMESTAMP DEFAULT NOW()

Table: url_clicks (analytics, write-heavy, separate table)

id BIGINT PRIMARY KEY

url_id BIGINT FK → urls.id

clicked_at TIMESTAMP

country VARCHAR(5)

SPRING BOOT + JPA MAPPING

@Entity class Url { ... } → maps to urls table

@Repository UrlRepository extends JpaRepository

findByShortCode(String code) → fast lookup by short_code

Add @Index on short_code. Every redirect hits this column millions of times.

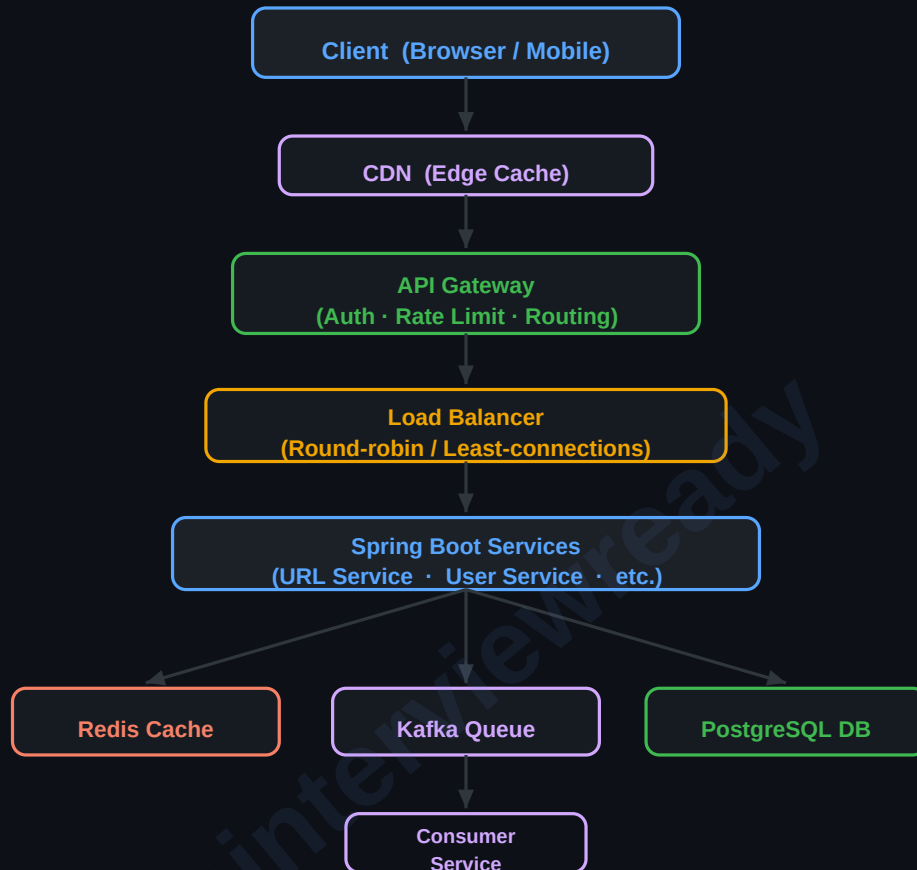
STEP 5

STEP 5 OF 5

Draw the High-Level Architecture

Now you draw. But not random boxes. You draw a system that directly handles the requirements and scale you've already discussed. Trace the request from client to database and justify every layer.

Standard Backend Architecture Flow



NARRATE YOUR DIAGRAM

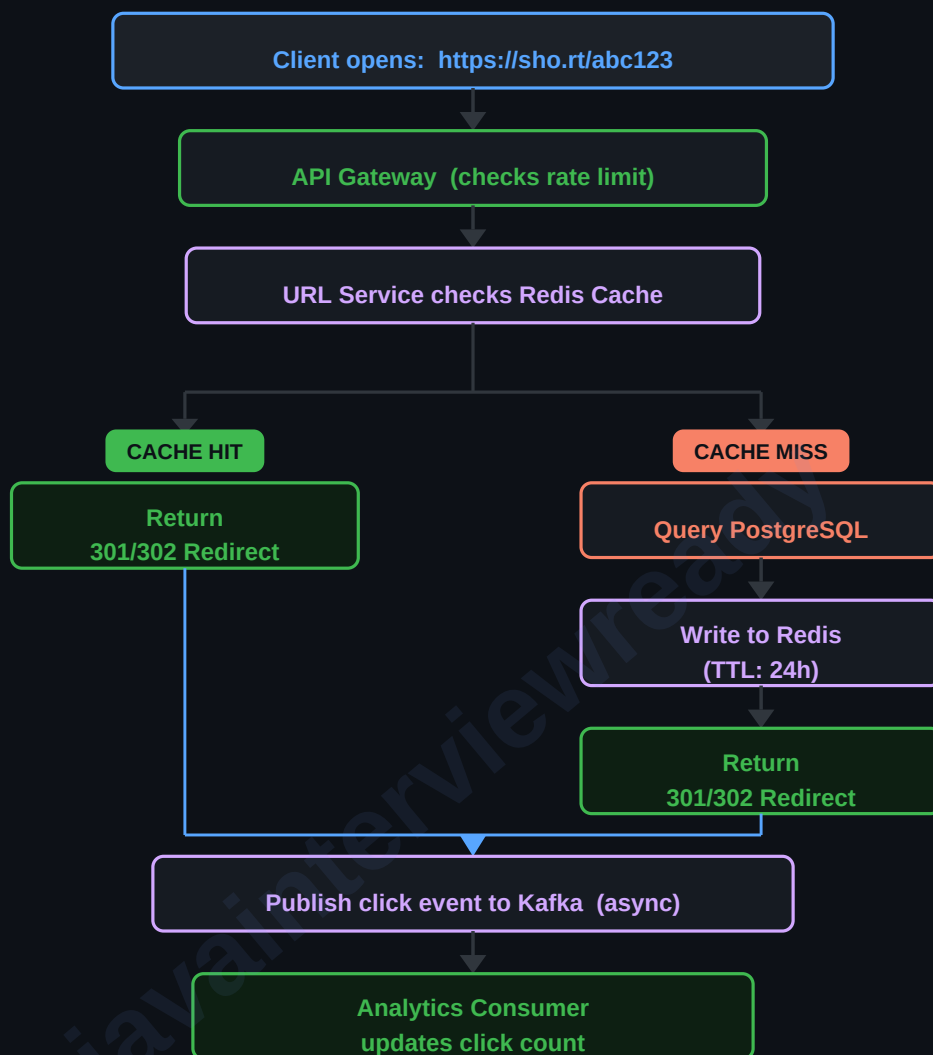
Don't just draw the boxes in silence. Say why each one is there.

"I'm adding Redis in front of the database here because short URL lookups are read-heavy. The same URL gets hit thousands of times. Caching the mapping avoids repeated DB queries."

That one sentence is worth more than a perfectly drawn diagram.

After the Architecture: Key Flow Walkthrough

Many candidates stop after drawing the diagram. That's a mistake. Walk the interviewer through the most important flow. For a URL shortener, that's the redirect.



Scaling Approach

- **Horizontal scaling:** Run multiple service instances behind the load balancer.
- **Database read replicas:** Route all reads to replicas. Writes go to primary only.
- **Redis Cluster:** If single-node Redis isn't enough, partition across nodes.
- **Kafka partitions:** Add more partitions and consumers to parallelize async work.
- **Auto-scaling:** Spring Boot apps on Kubernetes or AWS ECS scale under load automatically.

Common Failure Scenarios

Every system fails eventually. Proactively bring up 2-3 failure scenarios after your architecture. It shows maturity.

Failure	What Happens	How to Handle
Redis is down	All reads hit the database directly	Fall back to DB. System slows but stays alive
DB is slow	API response times spike, timeouts for users	Read replicas, connection pool limits, circuit breaker
Service crash	Load balancer shows unhealthy, stops sending traffic	Health checks + auto-restart via Kubernetes/ECS
Kafka consumer lag	Analytics/notifications are delayed	Monitor lag, scale consumer instances, add partitions
Duplicate request	Same URL created twice, or payment charged twice	Idempotency key in request. Check before insert.

Interview Time Allocation

You don't need to be perfectly on schedule. But if you're still doing requirements at the 20-minute mark, something went wrong.

INTERVIEW TIME ALLOCATION		
Typical 45-minute system design round		
STEP 1	Clarify Requirements	3-5 min
STEP 2	Estimate Scale	3-5 min
STEP 3	Define APIs	5-7 min
STEP 4	Design Database	5-7 min
STEP 5	Draw Architecture	10-12 min
AFTER	Key Flow Walkthrough	5 min
AFTER	Scaling + Failures	5 min
AFTER	Handle Follow-ups	remaining

Mistakes Freshers Make

1 Jumping straight into drawing

What freshers do:

Starts drawing boxes within 30 seconds. No questions asked. The design ends up missing key requirements.

Better approach:

Always start with requirements. Say it out loud: 'Before I start, let me clarify a few things.'

2 Only designing the happy path

What freshers do:

Designs perfect flows where everything works. Never mentions what happens when the DB is down or Kafka is lagging.

Better approach:

After finishing the architecture, proactively bring up 2-3 failure scenarios. It shows production experience.

3 Numbers without reasoning

What freshers do:

Says '10 million users' but can't explain what that means for the design choices made.

Better approach:

Tie your numbers to decisions. '10M DAU means ~6,000 reads/sec, so caching becomes necessary here.'

4 Adding components without justification

What freshers do:

Puts Kafka, Redis, Elasticsearch, and Cassandra in the diagram without explaining why any of them are there.

Better approach:

Every component you draw should be justified. If the interviewer asks 'why Kafka here?', have an answer ready.

5 Freezing on follow-up questions

What freshers do:

Does well until the interviewer asks 'how would you handle 10x traffic?' and goes completely silent.

Better approach:

Prepare 3-4 follow-up scenarios: scaling, failure handling, alternative design choices, and cost tradeoffs.

How This Maps to Spring Boot

The 5-step framework is language-agnostic, but since you're working in Java, here's how each step maps to real Spring Boot components. You don't need to write code in the interview. But knowing the mapping makes your explanations sound real.

System Design Step	Spring Boot Mapping	Example
APIs (Step 3)	<code>@RestController</code> <code>@RequestMapping</code>	<code>createShortUrl()</code> <code>redirect()</code>
Database (Step 4)	<code>@Entity</code> <code>@Repository (JPA)</code>	<code>Url.java</code> <code>UrlRepository</code>
Redis Cache (Step 5)	<code>@Cacheable</code> <code>RedisTemplate</code>	Cache <code>shortCode</code> lookup results
Kafka (Step 5)	<code>KafkaTemplate</code> <code>@KafkaListener</code>	Publish click event Analytics consumer
Auth (Step 5)	Spring Security JWT filter	Protect <code>POST /urls</code> <code>@PreAuthorize</code>
API Gateway (Step 5)	Spring Cloud Gateway Bucket4j filter	Route requests Rate limiting

How to Say It in the Interview

These are ready-to-use lines. Not scripts. Natural phrases that make your reasoning clear to the interviewer.

STARTING THE ANSWER

"I will clarify requirements first so I do not design the wrong system."

AFTER ESTIMATION

"Since this system is read-heavy, caching becomes important. I will design the read path to hit Redis first."

BEFORE DRAWING

"I will define the APIs first so the input and output are clear before I draw anything."

AFTER THE DIAGRAM

"I will start simple and then improve the design step by step based on scale and failure scenarios."

FAILURE HANDLING

"If Redis fails, the system falls back to the database. It stays functional, just slightly slower."

MODULE 2 QUICK REFERENCE: 5-STEP FRAMEWORK

Part 1 of 2 · Steps 1 – 3

STEP 1 · CLARIFY REQUIREMENTS

- Ask 4-6 focused questions (not 10, not 2)
- Functional: what does the system do? What features?
- Non-Functional: scale, latency, availability, consistency
- Identify read-heavy vs write-heavy before drawing

STEP 2 · ESTIMATE SCALE

- DAU → reads per user → daily total → RPS
- Use round numbers. Precision is not the point.
- Let numbers justify decisions: caching, DB type
- Read:Write ratio shows where the bottleneck forms

STEP 3 · DEFINE APIS

- Write 3-5 endpoints for the core flows only
- Method + path + request body + response body
- Maps to @RestController in Spring Boot
- Defines client-system contract before any diagram

MODULE 2 QUICK REFERENCE: 5-STEP FRAMEWORK

Part 2 of 2 · Steps 4 – 5 + After Step 5

STEP 4 · DESIGN DATABASE

- SQL for relational data + ACID transactions
- NoSQL for flexible schema + high horizontal scale
- Sketch main tables and show key indexes
- short_code column needs an index because it hits millions of times

STEP 5 · DRAW ARCHITECTURE

- Client → Gateway → LB → Services → Cache / DB / Queue
- Justify every component you add
- Walk a key flow end-to-end after drawing
- Discuss scaling and failure handling proactively

AFTER STEP 5: complete these before wrapping up:

Key Flow

Scaling

Failure Handling

Follow-up Qs

FINAL THOUGHT

The interviewer knows you can't design a production-grade system in 45 minutes. They're not grading the diagram. They're grading how you think, how you communicate, and whether your decisions make sense.

This framework gives you the structure to show exactly that.

Next: Module 3 covers requirement gathering and scope definition, including exactly what questions to ask before designing anything.

MODULE 3

Requirement Gathering and Scope Definition

The questions you must ask before designing anything. Functional vs non-functional requirements. How to define scope so you don't go off-track in interviews.

Part of The System Design Interview Playbook, for Java Backend Developers, Freshers and 1-3 Years of Experience.

javainterviewready

TOPIC 01

The Step Almost Everyone Skips

Here's a pattern I've seen in maybe two hundred system design interviews by now. The interviewer says "design a parking lot system" or "design an inventory service," and the candidate picks up the marker and starts drawing before the interviewer has even finished the sentence. Box for the API. Box for the database. Arrow connecting them. Confident start.

Five minutes later, I ask one question: "How many users are we talking about?" and the candidate freezes, because their entire design was built on an assumption that was never checked. Maybe they designed for ten users when the real number is ten million. Maybe they designed for heavy writes when the system is actually read-heavy. Either way, we now spend the rest of the interview unwinding a design instead of building one.

This is the single most avoidable mistake in this entire interview format, and it has nothing to do with technical knowledge. It's a habit problem. Jumping straight into drawing feels productive. It looks like progress. But it's the equivalent of writing code before reading the ticket properly. You'll get something on the screen, it just might not be the thing anyone asked for.

INTERVIEW FOCUS

Interviewers are not being difficult when they wait for you to ask questions. Most of them are quietly hoping you will, because a candidate who clarifies scope before designing is already showing the instinct that separates a junior engineer from someone who can be trusted with ambiguous real-world tickets.

Requirement gathering in a system design interview isn't a formality you rush through to get to the "real" part. It is the real part. Everything you draw after this point is downstream of the assumptions you lock in during these first few minutes. Get the scope wrong, and a technically correct design can still be the wrong answer.

What this looks like in an actual project

You'll recognize this pattern from real work too, even if you haven't called it "requirement gathering" before. Someone on your team says "we need an API to fetch user orders." If you start building immediately, you might build it assuming there are a few dozen orders per user. Then it turns out some enterprise customers have fifty thousand orders, and your endpoint that returns everything in one response times out in production. A two-minute conversation about expected data volume would have caught that before you wrote a single line.

System design interviews compress that same real-world habit into a five to ten minute window at the start of the conversation. The skill you're practicing here is not theoretical. It's the exact same instinct that keeps you from shipping a broken API in your actual job.



This order matters more than it looks. Each step depends on the one before it, scope before functional details, functional before non-functional, and assumptions locked in before a single box gets drawn. Skip ahead, and you're designing against guesses instead of answers.

TOPIC 02

Functional vs Non-Functional Requirements

These two terms get thrown around a lot, and most freshers can recite the textbook definition without actually knowing how to use the distinction in an interview. Here's the concrete version instead of the abstract one.

Functional requirements are about what the system does. The features. If you removed one, the system would no longer do something it's supposed to do.

Non-functional requirements are about how well the system does it. Latency, availability, consistency, scale. Remove one of these and the system still runs, just badly. A URL shortener that takes eight seconds to redirect still "works," it's just unusable.

Functional vs Non-Functional Requirements

FUNCTIONAL REQUIREMENTS

- What the system does
- User actions
- Core features
- Example: create short URL, redirect user

NON-FUNCTIONAL REQUIREMENTS

- How well the system works
- Scale, latency, availability
- Quality expectations
- Example: redirect should happen fast, service should not go down

TIP

A quick way to tell them apart on the fly: functional requirements usually show up as verbs a user performs (create, fetch, update, delete, search). Non-functional requirements usually show up as adjectives describing quality (fast, reliable, consistent, available).

Why interviewers care about this split specifically

Because most of the interesting engineering decisions in a system design interview come from non-functional requirements, not functional ones. The functional requirements of a chat application (send a message, receive a message, see online status) are honestly not that complicated. What makes the design interesting is the non-functional layer underneath: can messages arrive out of order, what happens if a user is offline, how much delay is acceptable before a message shows up on the other side.

If you only gather functional requirements and skip the non-functional ones, you'll produce a design that technically does what was asked but completely misses what the interviewer actually wanted to evaluate. I've seen candidates design a flawless CRUD API for a notification system and never once ask about delivery guarantees, which is the entire point of that question.

A short list to keep in your head

You don't need to recite all of these every time, but having this list available in your head means you won't blank out when asked "what else would you want to know?"

- **Functional:** what actions can a user take, what data do they create or read, what are the core use cases.
- **Scale:** how many users, how many requests per second, read-heavy or write-heavy.
- **Latency:** how fast does a response need to come back, is this user-facing or backend.
- **Availability:** can this system go down for a few minutes, or does it need to survive failures without interruption.
- **Consistency:** does every read need the absolute latest data, or is slightly stale data acceptable.
- **Storage:** how much data are we storing, and for how long.
- **Security:** does this involve authentication, authorization, or sensitive data.

Notice that functional requirements come first on that list and everything after it is non-functional. In real interviews, you'll usually nail down the functional side fast (it's often given to you directly in the prompt) and then spend more time negotiating the non-functional side, because that's where the actual design decisions get made.

TOPIC 03

The Questions You Should Actually Be Asking

Knowing the categories above is one thing. Knowing what to actually say out loud in the first two minutes of an interview is a different skill, and it's the one that tends to fall apart under pressure. Here's a sequence that holds up well in practice.

Start with what the system does, not how big it is

If the prompt is vague, like "design a notification system," your first question shouldn't be about scale. It should be about scope. "Are we sending notifications through push, email, SMS, or all three?" "Are these triggered by user actions, or scheduled, or both?" Nailing down what the system actually does prevents you from designing something impressive that solves the wrong problem.

WHERE FRESHERS GO WRONG

A common mistake here is asking scale questions before scope questions. Knowing you have ten million users doesn't help if you haven't figured out whether the system needs to support email notifications, push notifications, or both. Scope first, scale second.

Then move to scale, and be specific

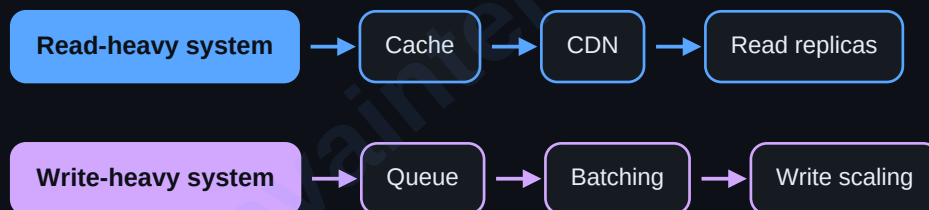
Once scope is clear, ask about scale in a way that produces a usable number, not a vague impression. “Is this for a small internal tool or a consumer app with millions of users?” works fine as an opener, but push for something more concrete if the interviewer gives you room. “Roughly how many requests per second should this handle at peak?” gives you something you can actually design around.

If the interviewer says “you tell me,” that’s not them avoiding the question, that’s them handing you the decision. State an assumption out loud and move on. “I’ll assume around one million daily active users with a 10:1 read to write ratio, and I’ll adjust if that’s off.” That one sentence does more for your evaluation than five minutes of silent thinking.

Ask about read vs write pattern directly

This single question shapes more of your design than almost anything else, so don’t skip it. A read-heavy system (like a news feed or a product catalog) wants caching, read replicas, and CDN usage high on your list. A write-heavy system (like a logging pipeline or an order processing system) wants you thinking about queues, batching, and write throughput instead.

How Read/Write Ratio Changes Design



Latency and availability, in plain language

Don’t ask “what’s the latency requirement” like you’re reading from a form. Ask in a way that shows you understand why it matters. “Is this on the critical path of a user action, like a checkout button, or is it something that can run in the background, like sending a confirmation email?” That question tells you more than a number would, because it tells you whether the user is staring at a loading spinner waiting for this to finish.

Same with availability. Instead of asking for an uptime percentage you’ll never use directly, ask what happens if the system goes down. “If this service is unavailable for two minutes, does the business lose money, or does it just mean a delayed notification?” The answer tells you how much redundancy and failover thinking the design actually needs.

TOPIC 04

Defining Scope So the Interview Doesn't Run Away From You

Here's a failure mode that has nothing to do with requirements gathering itself, but happens right after it. The candidate asks great questions, gets great answers, and then tries to design every single thing the system could possibly need. Forty-five minutes later, they've covered ten components at a shallow depth instead of four components with real depth.

Scope definition means deciding, out loud, what you will and will not cover, and saying so before you start drawing. This single habit prevents the most common way system design interviews go badly, which isn't a wrong answer, it's an unfinished one.

INTERVIEW MOMENT

A strong way to open this part of the conversation: "Given the time we have, I'll focus on the core flow of creating and retrieving an order, and I'll touch on notifications and payment only at a high level unless you'd like me to go deeper there." This tells the interviewer exactly where your attention is going and gives them the chance to redirect you if they disagree.

Most design prompts have a "main flow" and several "supporting flows." For an e-commerce order system, the main flow is placing an order and having it processed correctly. Supporting flows are things like order history, refunds, and inventory sync. If you spend equal time on all of them, you'll run out of time before going deep on the part the interviewer actually cares about, which is almost always the main flow.

How to scope without sounding like you're avoiding work

There's a wrong way to do this. Saying "I'll skip authentication and error handling to save time" sounds like you're cutting corners. Saying "I'll assume authentication is handled by an existing identity service, and I'll mention failure handling for the core flow specifically, since that's usually where the interesting tradeoffs are" sounds like a decision, because it is one. Same outcome, completely different signal.

In real projects, this same instinct keeps a sprint from ballooning. A ticket that says "add search to the product page" can quietly grow into full text search, filters, sorting, autocomplete, and typo correction if nobody draws a line around what "version one" actually means. Scoping in an interview is the same skill, just compressed into a few sentences instead of a sprint planning meeting.

TOPIC 05

Turning Requirements Into Rough Numbers

Once you have functional scope and the big non-functional questions answered, there's a step a lot of guides skip entirely: turning those answers into numbers you can actually design against. You don't need to be a math person for this. You need comfort with rough estimates, because that's genuinely what the job is.

Say the interviewer agrees on one million daily active users for a URL shortener, with each user creating two short links a day on average and clicking links ten times as often as they create them. You don't need a calculator for this, just enough comfort with the numbers to reason out loud.

Rough Capacity Thinking, Talked Through Out Loud



That's the whole exercise. You're not trying to land on an exact number, you're trying to get a sense of order of magnitude, because that's what actually changes your design. Twenty-three writes a second is something a single well-tuned database instance can handle without sweating. Twenty-three thousand writes a second is a completely different conversation involving sharding and write-optimized storage.

TRADEOFF

Don't fall into the trap of doing precise math the interviewer didn't ask for. Spending three minutes calculating exact storage in terabytes when a rough estimate would do just burns time you needed for the actual design. The estimation is a tool to inform decisions, not a deliverable on its own.

Storage estimates work the same way. If each short URL record is roughly 500 bytes including metadata, and you're creating 2 million a day, that's about 1 gigabyte a day, or roughly 365 gigabytes a year. That number alone tells you something useful: this is not a small dataset, but it's also not a big-data problem requiring exotic storage. A regular relational database with decent indexing handles this fine for years before you'd need to think about partitioning.

TOPIC 06

Where Freshers Usually Go Wrong Here

Most of this module has been about what to do. It's worth spending a little time on what actually goes wrong, because the mistakes here are surprisingly consistent across candidates, and recognizing the pattern in yourself is half the fix.

Treating every requirement as equally important

A fresher will sometimes ask five or six good questions and then try to design around all of them with equal weight. In practice, one or two non-functional requirements usually dominate a given system. For a chat app, it's message ordering and delivery guarantees. For a ride booking app, it's latency on matching. Spend your depth on the one or two that matter most for this specific system, instead of giving shallow attention to all of them.

WHERE FRESHERS GO WRONG

If you find yourself spending equal time on every requirement, ask yourself which one the interviewer actually emphasized when answering your questions. That's almost always your signal for where to go deep.

Asking questions but not using the answers

This one is sneaky because it looks fine in the moment. A candidate asks great requirement questions, gets clear answers, and then designs something that doesn't actually reflect those answers. If the interviewer told you this is read-heavy with relaxed consistency, and your design has no caching layer anywhere, that disconnect is exactly what gets flagged afterward. The questions only count if they visibly shape the design that follows.

Confusing “I don't know” with “I won't decide”

When an interviewer says “you decide” on scale or consistency, some freshers freeze, as if the lack of a given number means they can't proceed. In real projects, you almost never get handed exact numbers either. A senior engineer doesn't wait for permission to estimate, they make a reasonable assumption, say it out loud, and keep moving. “I'll assume moderate consistency is fine here since this isn't financial data” is a perfectly good answer when nobody gave you a firm rule.

Spending too long here

The opposite mistake also shows up. A candidate who read about requirement gathering and decided it's the most important part of the interview can sometimes spend fifteen or twenty minutes on it, asking question after question without ever drawing anything. At that point you've correctly identified the problem and failed to solve it. This phase exists to set up the design, not replace it.

TOPIC 07

Where This Shows Up in Actual Spring Boot Work

Requirement gathering can feel like a soft skill that's disconnected from actual code, so it's worth being direct about where it shows up in a real Spring Boot codebase, because it shows up constantly, just under different names.

Non-functional requirements around latency and read patterns directly decide whether you reach for `@Cacheable` and Redis on a service method, or whether you leave it hitting the database every time. A read-heavy endpoint with relaxed consistency requirements is a textbook candidate for caching. A write path where every request needs to see the absolute latest state usually isn't, or needs careful cache invalidation if it is.

Availability requirements decide whether a Spring Boot service needs to be stateless and horizontally scalable behind a load balancer from day one, or whether a single instance with a scheduled restart window is genuinely fine for an internal tool. Both are valid engineering decisions. The requirement, not your personal preference for “doing it properly,” should be what decides this.

JAVA / SPRING BOOT MAPPING

Scale and read/write ratio decide your persistence choice too. High write throughput with simple lookups often points toward something like Kafka feeding a write-optimized store, consumed asynchronously. Complex relational queries with moderate volume usually keep you in Spring Data JPA on Postgres or MySQL without needing anything exotic. The requirement comes first, the technology choice follows it, never the other way around.

Even something as basic as DTO design connects back to this. If a non-functional requirement says responses need to stay under 200 milliseconds for a mobile client on a weak connection, that's a real argument for trimming your response DTOs instead of returning your full JPA entity graph with five levels of nested relationships. The requirement justifies the code decision, rather than you doing it because it's "best practice" without being able to say why.

If I were mentoring someone on this specifically, I'd say: the next time you pick up a ticket at work, write down the non-functional requirements for it before writing code, even if nobody asked you to. You'll start noticing how often the actual implementation choice was hiding inside a requirement nobody had stated out loud.

TOPIC 08

A Framework You Can Reuse in Every Interview

By now you've seen a lot of individual pieces. Here's how to fold them into something you can actually run through under pressure, without sounding like you're reading from a checklist.

- Clarify scope first. What does this system do, and what's explicitly out of scope for this conversation.
- Confirm the core user flow. Out of everything this system could do, which one flow is the main thing you'll design deeply.
- Ask about scale in a way that gives you a usable number, and state an assumption out loud if the interviewer hands the decision back to you.
- Identify whether this is read-heavy or write-heavy, since that one answer reshapes most of your later decisions.
- Get a sense of latency and availability expectations by asking what the user experiences if this is slow or down, not by asking for a raw percentage.
- Do a rough capacity estimate out loud if it's relevant. Order of magnitude only.

None of this needs to take more than five to seven minutes. If it's taking longer, you're probably overthinking the requirements phase instead of moving into the design. Requirement gathering exists to set you up to design well, not to replace the design itself.

How Requirements Change Design

Read-heavy requirement



Redis cache / CDN / read replicas

Write-heavy requirement



Queue / batching / write scaling

Low-latency requirement



Avoid slow sync calls / optimize critical path

High-availability requirement



Multiple instances / load balancer / failover

WHAT INTERVIEWERS NOTICE QUICKLY

Whether you ask about scope before scale, instead of jumping straight to numbers.

Whether you turn vague answers into concrete assumptions instead of staying stuck.

Whether you ask about read versus write pattern without being prompted.

Whether you can explain why a requirement changes your design, not just that it does.

Whether you keep this phase tight instead of spending half the interview here.

TOPIC 09

A Worked Example: Ride Booking App

Reading a list of questions is one thing. Seeing how an actual exchange plays out is more useful, so here's a realistic version of how the first few minutes might go if an interviewer asked you to design a ride booking app, the kind of prompt you'll see in plenty of interviews.

Interviewer: "Design a ride booking app, something like Uber."

A weak opening here is trying to design the whole of Uber. A strong opening narrows scope immediately, out loud, before asking anything else.

You: "Before I start, I want to scope this down a bit. Should I focus on the core flow of a rider requesting a ride and getting matched with a driver, or do you also want me to cover things like payments and ratings?"

Interviewer: "Let's focus on requesting a ride and matching with a driver. You can mention payment at a high level."

That single exchange just saved you from spending fifteen minutes designing a payment system nobody asked you to go deep on. Scope is locked. Now move to scale, and don't ask in a way that invites a vague answer.

You: "Roughly what scale should I design for? Something like a city-level service with a few hundred thousand riders, or larger?"

Interviewer: "Let's say a large city, around 500,000 riders and 50,000 drivers active at a given time."

Good, now you have a usable number. Next, the question most candidates forget entirely for this specific prompt: is this read-heavy or write-heavy, and what's the latency expectation. For ride matching, this one matters more than usual, because the answer is unusually strict.

You: "Matching a rider with a nearby driver sounds latency-sensitive, since the user is waiting on the screen. Should I assume we need a match within a few seconds?"

Interviewer: "Yes, ideally under five seconds."

INTERVIEW MOMENT

Notice what just happened. You didn't ask "what's the latency requirement" like a form field. You proposed a real constraint based on how the product actually behaves, and asked the interviewer to confirm it. That's a stronger signal than a generic question, because it shows you understand the product, not just the interview format.

One more question worth asking for this specific prompt, since it shapes the data model directly: does driver location update continuously, or only when requested. This decides whether you're dealing with a steady stream of location writes or occasional lookups.

You: "Should I assume drivers send their location every few seconds even when idle, so we always have a fairly fresh location for matching?"

Interviewer: "Yes, assume every four seconds."

At this point you have everything you need to start designing. Notice the whole exchange took six questions, not twenty, and each one produced a concrete answer you can design against. Compare that to

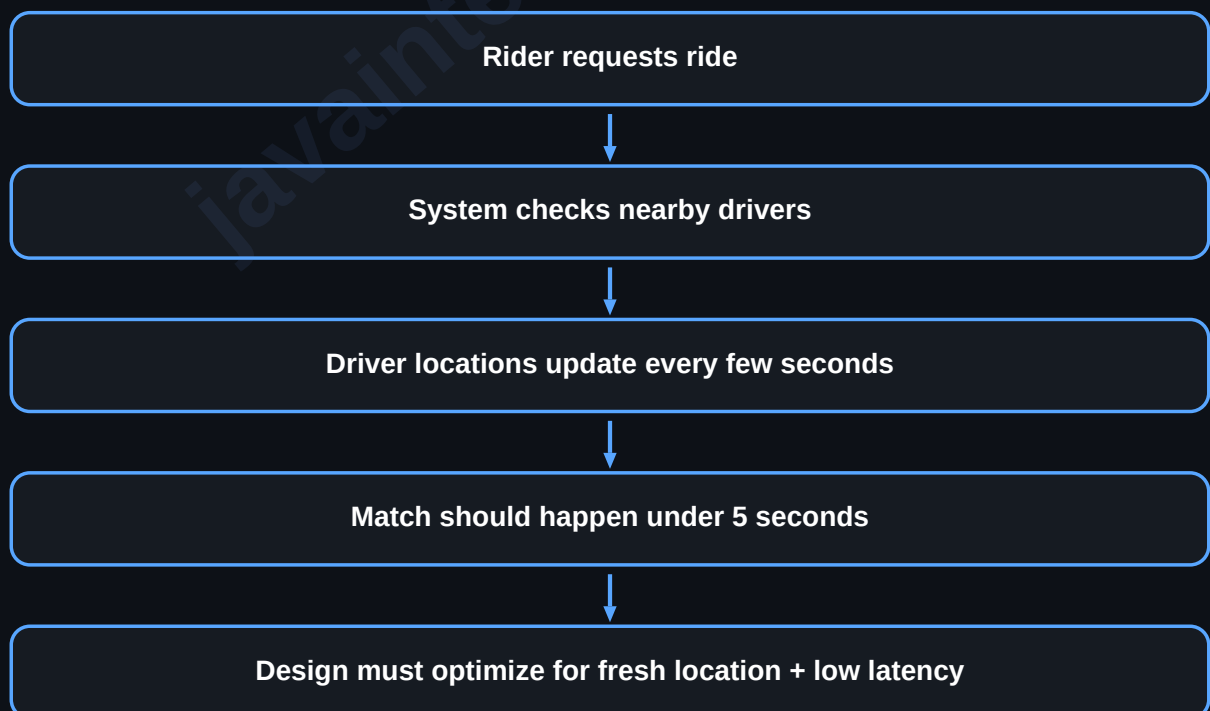
a candidate who spent the same six exchanges asking generic questions like “what database should I use” (a design question disguised as a requirements question, asked far too early).

What this conversation actually locked in

- Scope: ride request and driver matching only, payment mentioned at a high level.
- Scale: 500,000 riders, 50,000 active drivers.
- Latency: match within five seconds, this is on the critical path of the user experience.
- Write pattern: drivers send location updates every four seconds, meaning this is a high-frequency write workload even before a single ride is requested.

Every one of those four bullets becomes a direct input into the design. Five-second matching rules out anything that depends on a slow batch job. A high-frequency location write stream from fifty thousand drivers points toward something that can absorb frequent small writes without hammering a relational database for every single update, which is exactly the kind of decision a non-functional requirement should be driving.

Ride Booking Requirement Flow



TOPIC 10

Wrapping Up This Module

A short checklist before you move to Module 4

- Can you explain the difference between functional and non-functional requirements using your own example, not the textbook one?
- Do you have a default sequence of questions you'd ask for almost any design prompt?
- Can you turn a rough user count into requests per second without a calculator?
- Do you see how a non-functional requirement like latency or consistency directly decides a Spring Boot implementation choice, like caching or DTO shape?
- Could you run a short scoping conversation like the ride booking example above without it turning into ten generic questions?

If you're still unsure how this plays out across a full design question end to end, that's expected at this point. This module gave you the questions to ask before designing. In the next module, we will connect those answers to the actual building blocks you use while designing real backend systems.

Next: Module 4 introduces the core building blocks of system design, including load balancers, API gateways, databases, caches, message queues, CDNs, and object storage.

MODULE 4

Core Building Blocks of System Design

Load Balancer · API Gateway · Database · Cache · Message Queue · CDN · Object Storage

Every large system is built from a small set of reliable components. This module walks through each one, what it does, when you need it, and how to talk about it in an interview.

Why These Seven Components Matter

Before you can design any system in an interview, you need to know your building blocks. Not definitions. Not textbook explanations. You need to know what problem each one solves and when to reach for it.

Most freshers walk into a system design interview and either ignore these components or use them randomly without reason. An interviewer can tell immediately. The good answers always start with understanding what each component does at its core.

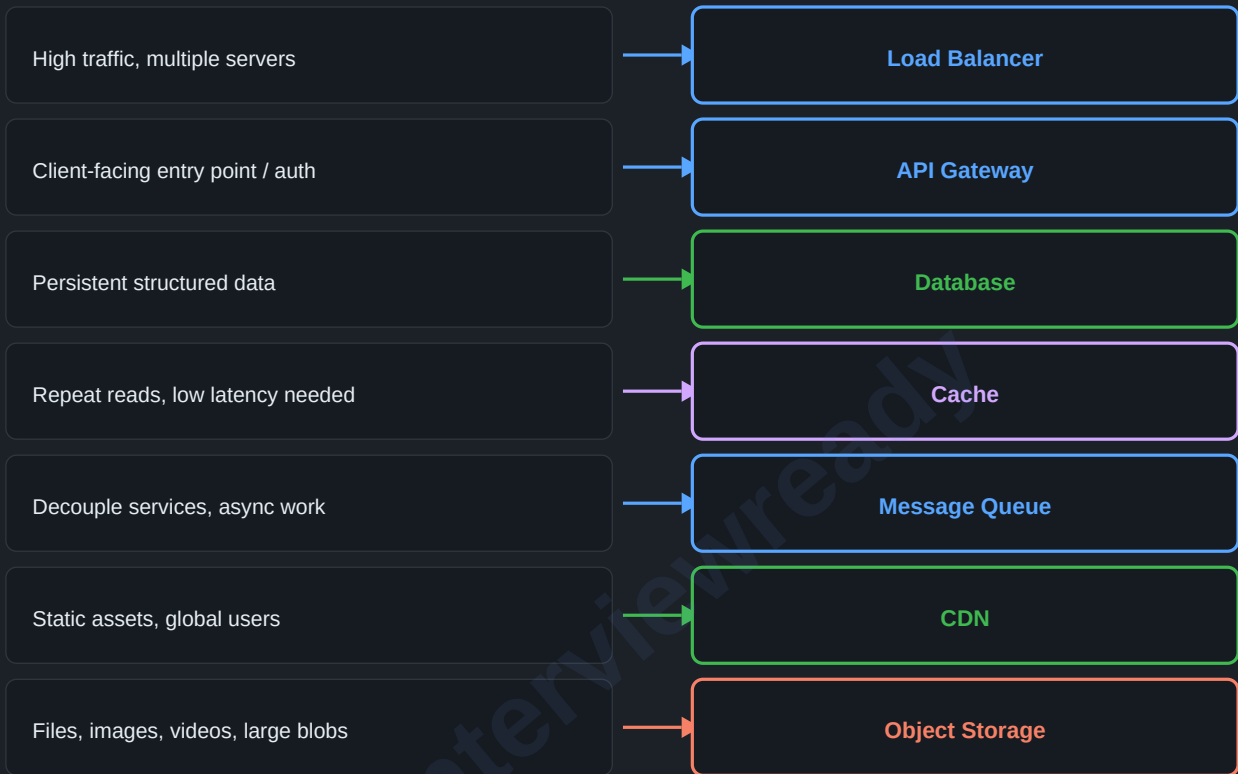
Think of these seven as your toolbox. Every backend system you will ever design uses some combination of them. Know when to pick each tool and, just as importantly, when not to.

SYSTEM DESIGN: 7 CORE BUILDING BLOCKS



These seven components show up in almost every system design question. You do not use all of them every time, but you need to know all of them.

QUICK DECISION MAP: When to Reach for Each Component



TOPIC 01

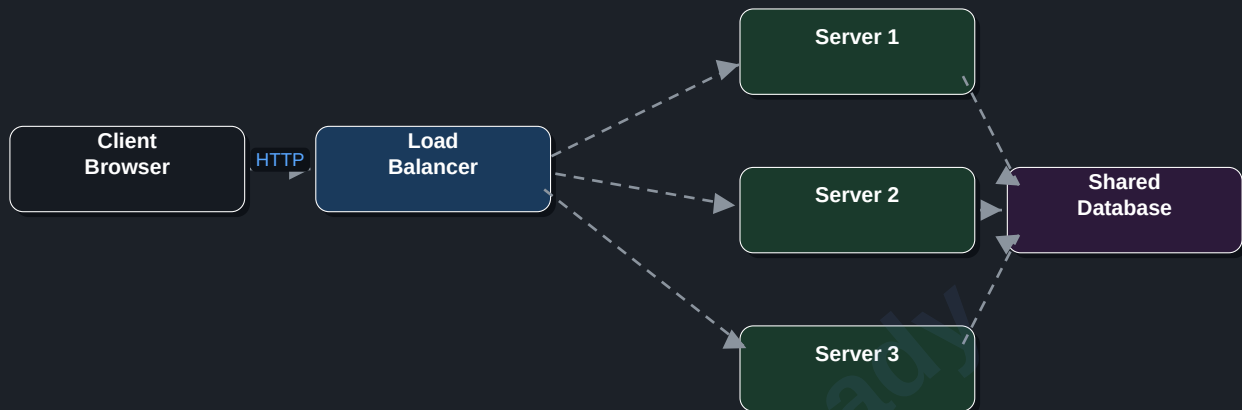
Load Balancer

Imagine your app suddenly gets 10,000 requests per second. One server cannot handle that. You spin up five servers. Now what? How does the traffic get split across them? That is exactly what a load balancer does.

A load balancer sits in front of your servers and distributes incoming requests across them. If one server goes down, the load balancer stops sending traffic there. The user never notices. That is availability done right.

How It Works

Load Balancer Architecture



Common Algorithms

Round Robin is the default. Request 1 goes to Server 1, request 2 to Server 2, and so on. Simple, works well when servers have equal capacity.

Least Connections sends the next request to whichever server has the fewest active connections. Better when some requests take much longer than others.

IP Hash maps each client IP to a specific server, so the same user always hits the same backend. Useful when you need sticky sessions.

Layer 4 vs Layer 7

Layer 4 vs Layer 7 Load Balancing

Layer 4 (Transport)

- Routes based on IP and TCP port
- Does not read HTTP content
- Faster, lower overhead
- Cannot route based on URL path
- Good for raw TCP/UDP traffic

Layer 7 (Application)

- Reads HTTP headers, URL, cookies
- Can route /api to one backend
- Can route /images to another
- SSL termination happens here
- Used in modern web systems

In most web system designs you will talk about Layer 7. It is more flexible. Layer 4 comes up when you are dealing with very high throughput or non-HTTP protocols.

What Happens If It Goes Down?

The load balancer becomes a single point of failure if you only run one. Production setups run two: one active, one standby. If the primary fails, the secondary takes over. This is called active-passive failover.

JAVA + SPRING BOOT MAPPING

Spring Cloud LoadBalancer is the standard client-side load balancer in Spring Boot. When Service A calls Service B, Spring Cloud LoadBalancer picks which instance of B to call. Service discovery (via Eureka or Consul) tells it which instances are available. On the infrastructure side, Nginx and AWS ALB handle server-side load balancing before traffic reaches your Spring Boot app.

INTERVIEW FOCUS

If they ask 'how would you handle 10x traffic?', your first answer should include horizontal scaling plus a load balancer. Do not jump to database sharding first. Scale the app tier before the data tier.

WHERE FRESHERS GO WRONG

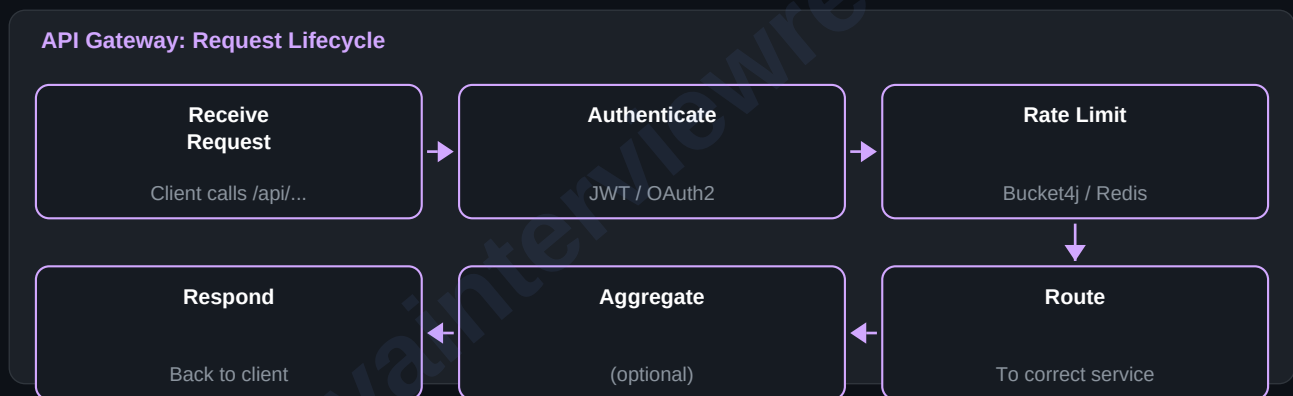
A lot of freshers say 'I will add a load balancer' and stop there. The interviewer then asks: 'What algorithm? What if the load balancer goes down?' Have answers for both. Also, if your backend is stateful (storing session data in-memory), load balancing breaks session consistency. Always use a shared session store like Redis when you are behind a load balancer.

API Gateway

Once you move to microservices, you have a problem. The client does not want to call 10 different services on 10 different ports. It wants one clean entry point. That entry point is the API gateway.

But it is more than just routing. The API gateway is the right place to handle authentication, rate limiting, request logging, and protocol translation. You put cross-cutting concerns here so every microservice does not have to repeat them.

What It Handles



Key Responsibilities

Feature	What It Does
Routing	Forward /orders to Order Service, /users to User Service
Authentication	Validate JWT before the request reaches any microservice
Rate Limiting	Block users who exceed request limits
Load Balancing	Can distribute across service instances
SSL Termination	Handles HTTPS, backend services can use plain HTTP
Request/Response Transform	Convert from REST to gRPC, add headers, modify payload
Logging & Tracing	Central place to log all inbound traffic

API Gateway vs Load Balancer

This trips up a lot of people. They are not the same thing, even though both sit in front of your services.

API Gateway vs Load Balancer: Key Differences	
Load Balancer	API Gateway
• Distributes traffic across instances • Works at Layer 4 or 7 • Does not understand business logic • No auth or rate limiting • Focused on availability	• Routes to different services by path • Always Layer 7 • Understands auth, headers, tokens • Can do rate limiting, transformation • Focused on API management

In production, both exist together. Load balancer handles traffic to gateway instances. Gateway handles routing to microservices.

JAVA + SPRING BOOT MAPPING

Spring Cloud Gateway is the standard choice. You configure routes in `application.yml` or programmatically using Java DSL.

Key features: predicates (match by path, header, method), filters (add JWT validation, rate limiting via Redis, response transforms), and built-in Reactor Netty for non-blocking I/O.

For rate limiting: Spring Cloud Gateway + `spring-boot-starter-data-redis` + `RequestRateLimiter` filter using Redis token bucket.

For auth: a `GlobalFilter` that reads the `Authorization` header, validates JWT using `spring-security-oauth2-jose`, then passes `userId` downstream as a header.

TRADEOFF

The API gateway can become a bottleneck if it does too much. Keep heavy business logic in the services, not in the gateway. If the gateway goes down, everything goes down. Run it in a clustered setup and keep it stateless.

INTERVIEWER MOMENT

Interviewer: 'Where do you handle JWT validation in a microservices architecture?' Good answer: 'At the API gateway. That way each microservice trusts that the request is already authenticated and can focus on its own logic. The gateway extracts the user ID from the token and passes it as a header.'

TOPIC 03

Database

Every system stores data somewhere. The database is that place. What trips freshers up is not knowing which type to pick and why. In interviews, the choice between SQL and NoSQL matters less than your reasoning for it.

SQL vs NoSQL: The Real Difference

SQL vs NoSQL: When to Use Which

SQL (Relational)

- Structured data with fixed schema
- Complex joins and relationships
- ACID transactions needed
- Financial, inventory, user data
- Examples: MySQL, PostgreSQL

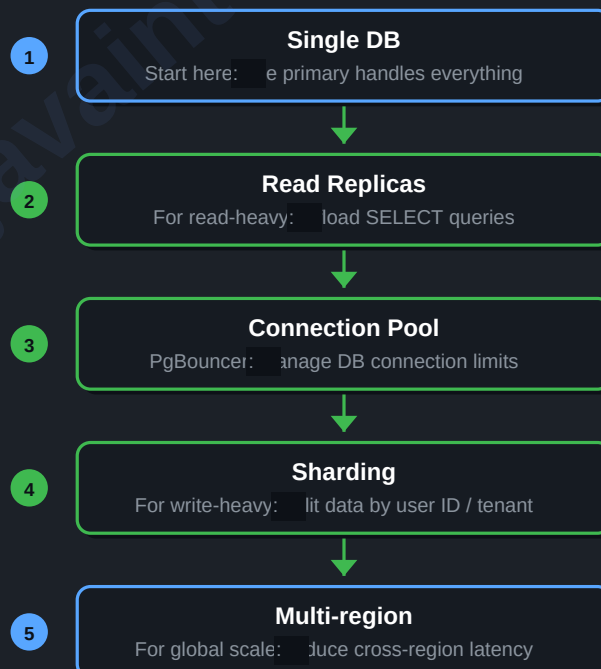
NoSQL (Non-Relational)

- Flexible or evolving schema
- Massive write throughput needed
- Horizontal scaling is a priority
- Unstructured/semi-structured data
- Examples: MongoDB, Cassandra, DynamoDB

There is no universally correct answer. Most large systems use both. User profiles in PostgreSQL. Activity feed in Cassandra. Search in Elasticsearch.

Scaling a Database

Database Scaling Path



Read Replicas

If your system is 80% reads, do not put everything on the primary database. Add read replicas. Writes go to primary, reads go to replicas. This is the first scaling step most production teams take.

Sharding

When the data volume grows beyond one server, you shard. You split the data across multiple databases by some key, usually user ID or tenant ID. Shard 1 holds users 1-1M, Shard 2 holds users 1M-2M, and so on.

Sharding adds complexity. Cross-shard queries are painful. Re-sharding when you grow is painful. Avoid it until you genuinely need it.

Indexes

Missing indexes on large tables will kill your query performance. If you are filtering or joining on a column frequently, that column needs an index. The tradeoff is that indexes slow down writes and take extra storage.

TRADEOFF

SQL: Strong consistency, ACID, complex queries supported, harder to scale horizontally. NoSQL: Easy horizontal scale, flexible schema, eventual consistency in most setups. Pick based on your data shape and access patterns, not what is trending.

JAVA + SPRING BOOT MAPPING

Spring Data JPA with Hibernate is the standard ORM stack. Use `@Entity` for tables, `@Repository` for data access, and keep business logic in `@Service`.

`@Transactional` handles transaction boundaries. For read-only operations, use `@Transactional(readOnly = true)` to route to read replicas in some configurations.

Connection pooling: HikariCP is bundled with Spring Boot. Configure pool size based on your thread count and DB connection limits.

For NoSQL: Spring Data MongoDB or Spring Data Cassandra follow the same repository pattern. Spring Data abstracts the storage layer so your Service layer stays clean.

WHERE FRESHERS GO WRONG

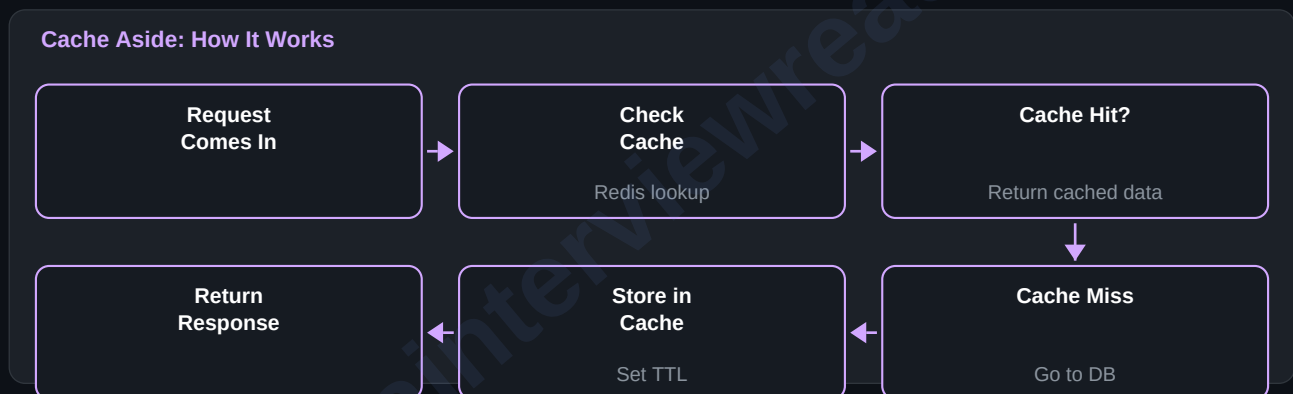
In interviews, freshers say 'I will use MongoDB because it scales better.' That is a lazy answer. MongoDB does not automatically scale. You still need replica sets, sharding, proper indexes. State your actual reason: 'I will use MongoDB because the document schema is flexible and changes frequently' or 'because it is denormalized for fast reads.'

Cache

Some data gets read thousands of times and almost never changes. Hitting the database every time for that data is wasteful. A cache stores a copy of that data in memory so you can get it back in under a millisecond.

The most common use case: a user profile page. You fetch the user from the database once, store it in Redis, and for the next 5 minutes every request reads from Redis. Your database gets a fraction of the load.

Cache Aside Pattern



Cache aside (also called lazy loading) is the most common pattern. The application checks the cache first. On a miss, it goes to the database, then writes the result to the cache before returning. The cache is populated only when data is actually requested.

Cache Eviction Strategies

Strategy	What It Does	Use When
LRU (Least Recently Used)	Removes the least recently accessed item	General purpose, most common
LFU (Least Frequently Used)	Removes the least frequently accessed item	When popularity matters more than recency
TTL (Time to Live)	Item expires after a set time	Data that becomes stale after some period
FIFO (First In, First Out)	Removes the oldest item regardless of access	Simple queues or ordered data

In most production setups you use TTL combined with LRU. TTL ensures staleness does not build up. LRU ensures the hot data stays in memory.

Cache Invalidation: The Hard Part

Putting data in the cache is easy. Knowing when to remove it is hard. If the user updates their profile but the cache still has the old version, they see stale data.

Three approaches: TTL expiry (let the data expire naturally after X seconds), write-through (update the cache every time you write to the database), or explicit invalidation (delete the cache key when the data changes).

JAVA + SPRING BOOT MAPPING

@Cacheable from Spring Cache is the simplest way to add caching.

@Cacheable(value = "users", key = "#userId") on a method will cache the return value in Redis automatically on first call and return the cached version on subsequent calls.

@CacheEvict removes a cache entry when data is updated. Put it on your update/delete methods.

Spring Boot + spring-boot-starter-data-redis wires Redis as the cache provider. Configure RedisTemplate or use the auto-configured CacheManager.

For distributed rate limiting or session caching, use StringRedisTemplate or ReactiveRedisTemplate.

WHERE FRESHERS GO WRONG

Cache stampede is a real production problem. When a cached item expires, 100 requests all miss the cache at the same moment and all slam the database at once. Fix it with a mutex lock on the cache key, or use a slightly random TTL jitter so keys do not all expire at the same time.

INTERVIEW FOCUS

When asked 'how do you reduce database load?', always mention caching. Then mention the eviction strategy and how you handle staleness. Most freshers mention Redis but do not explain TTL or invalidation.

TOPIC 05

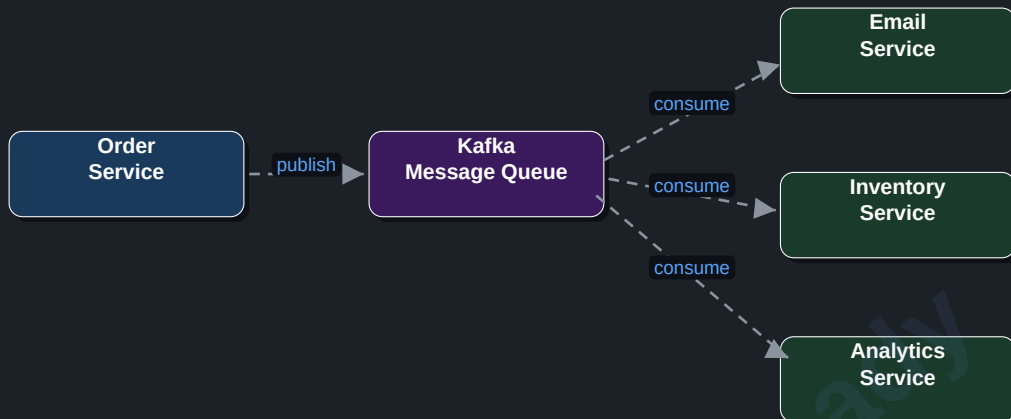
Message Queue

A user places an order. Your system needs to: update inventory, send an email, notify the delivery partner, update analytics. If you do all of this in a single synchronous request, you are making the user wait for all of it. And if the email service is down, the whole order fails.

A message queue breaks this. The order service publishes an event: 'Order created.' The inventory service, email service, and analytics service each consume that event independently. The order API returns immediately. Everything else happens in the background.

How It Decouples Services

Message Queue: Async Processing Flow



Key Concepts

Concept	What It Means
Producer	The service that sends messages to the queue
Consumer	The service that reads and processes messages
Topic	A named channel; producers write to it, consumers subscribe to it
Consumer Group	Multiple consumers sharing the load from one topic
Offset	Kafka's pointer to where a consumer has read up to
Dead Letter Queue	Where messages go when they fail after retry attempts

Kafka vs RabbitMQ

Kafka vs RabbitMQ: High-Level Comparison

Apache Kafka

- Built for high throughput at scale
- Messages are stored on disk (durable)
- Consumers can replay past messages
- Pull-based: consumers control their pace
- Better for event streaming and logging

RabbitMQ

- Better for task queue patterns
- Messages removed after consumption
- Push-based to consumers
- Simpler setup for small teams
- Better for short-lived, fire-and-forget tasks

For most system design interviews at the fresher level, Kafka is the expected answer for async event-driven systems. Know when to use it and how to describe it.

What Can Go Wrong

Consumer lag happens when the consumers cannot keep up with the producers. In Kafka, you fix this by adding more consumer instances, up to the number of partitions.

Duplicate messages happen when a consumer processes a message but crashes before committing the offset. The same message gets redelivered. Design your consumers to be idempotent: processing the same message twice should produce the same result.

JAVA + SPRING BOOT MAPPING

Spring Kafka is the standard integration. `@KafkaListener` on a method makes it a consumer.

`KafkaTemplate` is used to produce messages.

For idempotent consumers: use a deduplication key stored in Redis or a DB table. Before processing, check if you have already handled this message ID.

`@EnableKafka` in your config class activates listener containers.

`ConsumerConfig.AUTO_OFFSET_RESET_CONFIG` controls whether to start from earliest or latest.

INTERVIEWER MOMENT

Interviewer: 'What happens if your Kafka consumer crashes mid-processing?' Good answer: 'The offset was not committed, so Kafka redelivers the message. To handle this safely, my consumer should be idempotent. I check a processed_events table before doing any work, and only process if the message ID has not been seen before.'

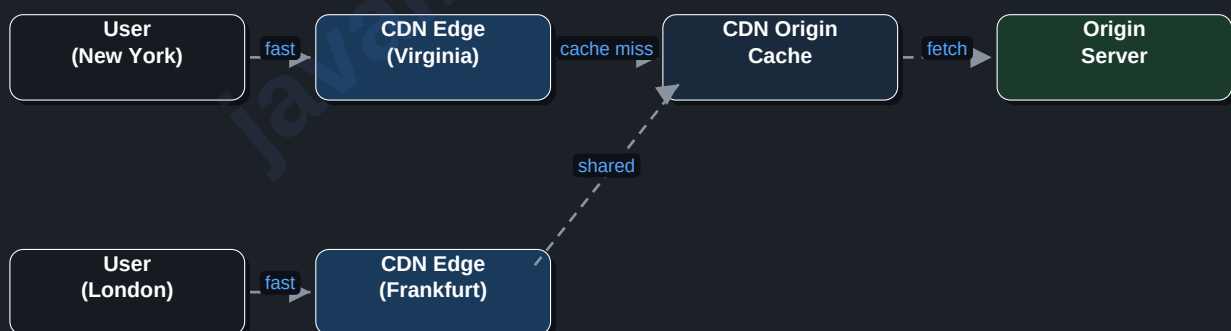
TOPIC 06

CDN (Content Delivery Network)

Your server is in Mumbai. A user in New York requests your app. Every asset, every image, every CSS file has to travel from Mumbai to New York. That is slow. A CDN fixes this by caching your static content on servers distributed around the world. The New York user gets assets from a CDN node in Virginia.

What a CDN Caches

CDN: How Content Gets Served



What Goes Through a CDN

Static assets are the obvious ones: images, JavaScript bundles, CSS files, fonts, videos. These change rarely, so caching them for hours or days makes sense.

Some CDNs also cache dynamic API responses if you configure cache-control headers correctly. But for most interview answers, focus on static asset delivery.

Cache Invalidation at CDN Level

You deploy a new version of your frontend. The old CSS is still cached on CDN nodes. Users see broken styles.

Solution: cache busting. Include a hash in the file name: `styles.a3f9b1.css`. When the file changes, the hash changes. The CDN sees it as a new URL and fetches fresh content.

TRADEOFF

CDNs reduce latency and origin server load significantly. The tradeoff is that serving stale content is a real risk if cache headers are not set correctly. For systems with user-generated content (photos, avatars), your CDN needs to handle cache invalidation via API or URL-based versioning.

INTERVIEW FOCUS

CDN comes up in any design involving media: Instagram, YouTube, food delivery app with images. Mention it when you have static content or when the system has global users. Saying 'I will serve profile images and media through a CDN to reduce latency' is exactly the kind of statement interviewers want to hear.

JAVA + SPRING BOOT MAPPING

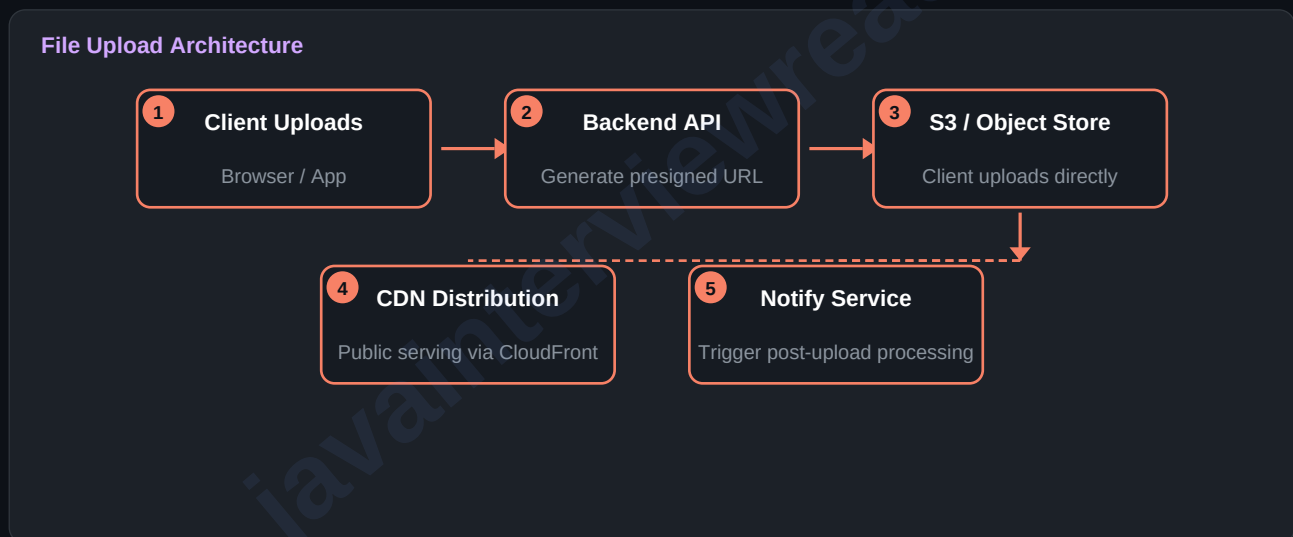
A Spring Boot application usually does not manage CDN directly. You configure your CDN (CloudFront, Cloudflare, Akamai) at the infrastructure level. In code, you return CDN URLs for media instead of your own server URLs. When an object is stored in S3, your service generates the CloudFront URL and returns it in the API response instead of the raw S3 URL.

Object Storage

User uploads a profile photo. Where does it go? Not in the database. Storing binary files in a relational DB is almost always a bad idea. It bloats the database, makes backups slow, and kills performance.

Object storage is built for exactly this. Large files, any format, cheap storage, built-in redundancy. S3 (Amazon), GCS (Google), Azure Blob are the major options. You store a file, you get back a URL. That is it.

How File Upload Works in System Design



The cleanest approach is the presigned URL pattern. The client asks your backend for a temporary upload URL. The backend generates a presigned URL from S3 and returns it. The client uploads the file directly to S3 using that URL. Your backend never touches the file bytes. This keeps your server fast and scales well.

Object Storage vs File System vs Block Storage

Type	What It Is	Best For
Object Storage	Key-value store for files. Each file is an 'object' with metadata.	Images, videos, backups, documents
File System	Hierarchical directory structure. Familiar to developers.	Code, config, small shared files
Block Storage	Raw storage blocks, like a hard drive. OS mounts it.	Databases, VMs, low-level I/O

Design Decisions

Access control matters. Files can be public (a profile photo anyone can see) or private (a document only the uploader can see). Public files get served via CDN. Private files require presigned download URLs with expiry.

For video platforms, large file uploads are split into chunks (multipart upload). Each chunk uploads separately. If one fails, only that chunk needs to retry.

JAVA + SPRING BOOT MAPPING

AWS SDK for Java (aws-sdk-java-v2) with the S3 client handles uploads and downloads.

To generate a presigned URL in Spring Boot:

`S3Presigner.create()` creates the signer.

`PutObjectRequest` defines the bucket and key.

`PresignedPutObjectRequest` gives you the URL with expiry.

Store the S3 key (not the full URL) in your database. Generate the CDN or presigned URL dynamically at read time. This way, if your CDN domain changes, you do not need to update millions of rows.

WHERE FRESHERS GO WRONG

A common mistake: storing the full S3 URL in the database. If you switch CDN provider or S3 region, all those URLs break. Store only the object key (path) and construct the URL dynamically in your service layer.

PUTTING IT ALL TOGETHER

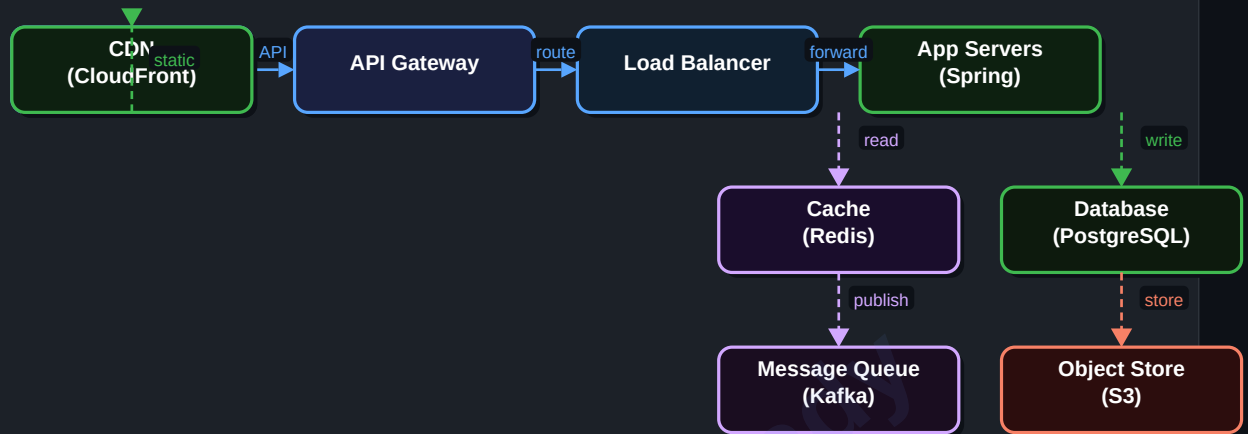
All 7 Components: Quick Reference

Real systems combine these components. No single component is a silver bullet. The skill is knowing which combination solves which problem.

Component	Core Job	When to Add It	Java / Spring Boot
Load Balancer	Distribute traffic across servers	Multiple server instances needed	Spring Cloud LoadBalancer, Nginx, AWS ALB
API Gateway	Single entry point, auth, routing	Microservices architecture	Spring Cloud Gateway
Database	Persist structured data	Any system storing user or business data	Spring Data JPA, Hibernate, MySQL/PostgreSQL
Cache	Speed up repeated reads	Read-heavy, low-latency requirements	@Cacheable, Redis, Spring Cache
Message Queue	Async communication, decouple services	Long-running tasks, downstream systems	Spring Kafka, KafkaTemplate, @KafkaListener
CDN	Serve static content from edge	Global users, media-heavy systems	CloudFront + S3, served via CDN URL
Object Storage	Store large files and blobs	Images, videos, file uploads	AWS S3 SDK, presigned URLs

How a Realistic System Uses All of Them

Production System: All 7 Components Working Together



This is the kind of architecture you sketch in system design interviews. You do not need every component for every question. But you need to know when each one earns its place.

INTERVIEW FOCUS

At the end of any design, the interviewer may ask: 'Walk me through a request from start to finish.' Practice this for each system. It shows you understand the flow, not just the components. A good answer touches each component and explains what it does for that specific request.

WHERE FRESHERS GO WRONG

Do not add all seven components to every design. A simple admin tool does not need Kafka or a CDN. Overengineering is a red flag. Add each component only when you can explain exactly what problem it solves in this specific system.

Next: Module 5 explains REST API design for system design interviews, including clean endpoints, status codes, pagination, versioning, and Spring Boot mapping.

MODULE 5

REST API Design for System Design Interviews

Status codes, versioning, pagination, error handling, and Spring Boot mapping for real system design interviews.

Part of The System Design Interview Playbook, for Java Backend Developers, Freshers & 1-3 Years of Experience.

javainterviewready

TOPIC 1

Why API Design Shows Up in Every Round

Here is something that surprises a lot of freshers. In a system design interview, you are rarely asked to "design REST APIs" as a standalone topic. It almost always shows up inside a bigger question. You are designing a URL shortener, or a food delivery backend, and somewhere in the middle the interviewer says, "okay, walk me through the API for this."

At that point, a lot of candidates just list endpoints. `POST /shorten`, `GET /url/id`, done. That is not wrong, but it tells the interviewer almost nothing about how you think. What they are actually checking is whether you treat the API as a contract that other teams, other services, and future versions of your own code depend on.

In real projects, this usually becomes a problem when a mobile app team built against your API six months ago, and now you want to change a field name. If your API was designed carelessly, that change breaks their app in production. Interviewers who have shipped real systems know this pain, and they are listening for whether you know it too.

INTERVIEW FOCUS

When an interviewer asks you to design the API for a feature, they are quietly testing five things at once: resource naming, correct HTTP method usage, status code discipline, how you handle scale through pagination, and how you handle change through versioning. You do not need to announce all five. Just make sure your answer naturally touches each one as you go.

This module walks through each of those areas the way you would actually use them while designing a system on a whiteboard or a shared doc, not as five disconnected definitions.

TOPIC 2

Think in Resources, Not Actions

A wrong design choice that almost every fresher makes at least once: naming endpoints after actions instead of resources. You see a requirement like "cancel an order" and your first instinct is to write `POST /cancelOrder`. It reads fine in English. It is still a mistake.

REST treats everything as a resource, a noun, something that has state. The order is the resource. Cancelling it is just a change in that resource's state, which means it belongs in the HTTP method and the request body, not in the URL itself.

RESOURCE NAMING, SIDE BY SIDE

AVOID

- `POST /cancelOrder/102`
- `GET /getUserOrders/55`
- `POST /createPaymentForOrder`
- Verb baked into the path
- A new endpoint for every action

PREFER

- `PATCH /orders/102` → status: CANCELLED
- `GET /users/55/orders`
- `POST /orders/102/payments`
- Noun based, hierarchical paths
- The method carries the action

Notice the second example, `GET /users/55/orders`. Nesting resources like this tells the reader exactly what relationship you are modeling, an order belongs to a user, without needing a sentence of documentation. Keep nesting to one or two levels though. Once you see paths like `/users/55/orders/102/items/9/reviews`, it is usually a sign that "reviews" deserves its own top level resource with a filter, something like `GET /reviews?itemId=9`.

JAVA + SPRING BOOT MAPPING

In Spring Boot, this maps directly onto how you organize `@RestController` classes. One controller per resource, not one controller per action. `OrderController` owns `@PatchMapping("/orders/{id}")` for status changes, instead of a separate `CancelOrderController` sitting next to it. If you find yourself naming a controller after a verb, that is usually the same mistake showing up in your code structure.

TOPIC 3

Choosing HTTP Methods, and Idempotency

Most freshers can recite GET, POST, PUT, DELETE without thinking. Fewer can explain idempotency clearly, and that is usually the actual follow up question. An operation is idempotent if calling it once or calling it five times in a row leaves the system in the exact same state. This matters far more than it sounds, because networks fail, clients retry, and load balancers sometimes deliver the same request twice.

Method	Typical Use	Idempotent	Where Freshers Slip
GET	Read a resource or a collection	Yes	Putting side effects (like logging a view count update) directly inside a GET handler.
POST	Create a new resource, or trigger an action	No	Assuming POST is always safe to retry without an idempotency key.
PUT	Replace a resource completely	Yes	Using PUT for a partial update, which silently wipes fields the client did not send.
PATCH	Update part of a resource	Usually	Forgetting that a PATCH that increments a counter is not idempotent, even though PATCH itself usually is.
DELETE	Remove a resource	Yes	Returning 404 on the second delete call instead of treating it as already satisfied.

That DELETE row trips people up in interviews more than you would expect. If a resource is already deleted, calling DELETE on it again did not fail, the end state the client wanted is already true. Many teams return 204 No Content either way, instead of 404, specifically to keep the operation idempotent from the client's point of view.

INTERVIEW MOMENT

A realistic follow up: "Your mobile client has a flaky network and retries a POST /orders request. How do you avoid creating two orders?" A weak answer says "the client should not retry." A stronger answer says "POST is not idempotent by default, so I would have the client send an idempotency key, and the server checks that key before creating anything new." We will build on this exact scenario again in Topic 9.

TOPIC 4

Status Codes That Actually Mean Something

Here is a pattern you will see in a lot of junior level code: every single response comes back as HTTP 200, and the real outcome, success or failure, is buried inside a JSON field like "status": "error". It works, technically. It also throws away one of the most useful parts of HTTP, and interviewers notice immediately.

Status codes exist so that load balancers, monitoring tools, API gateways and client libraries can all react correctly without parsing your response body. A 500 can trigger an alert. A 429 can trigger a backoff and retry. A 200 with an error hidden inside triggers nothing, until a customer complains.

Code	Meaning	When You Actually Use It
200	OK	Successful GET, PUT, or PATCH with a body to return.
201	Created	Successful POST that created a new resource. Include the new resource's location.
204	No Content	Successful DELETE, or a PUT/PATCH with nothing useful to return.
400	Bad Request	The request itself is malformed, missing fields, wrong types, failed validation.
401	Unauthorized	No valid identity at all. The caller has not authenticated.
403	Forbidden	Identity is known, but it is not allowed to do this action.
404	Not Found	The resource path does not point to anything that exists.
409	Conflict	The request conflicts with current state, for example a duplicate order, or an outdated version.
429	Too Many Requests	Rate limit hit. Should usually include a Retry-After header.
500	Internal Server Error	Something failed on your side that the client could not have prevented.

401 versus 403 is the pair that gets mixed up the most in interviews. A simple way to keep them straight, 401 means "I don't know who you are," 403 means "I know who you are, and you still can't do that."

JAVA + SPRING BOOT MAPPING

In Spring Boot, you rarely set status codes by hand on every method. `ResponseEntity<T>` lets you return the right code per case, `ResponseEntity.status(HttpStatus.CREATED).body(order)` after a successful POST. For errors, a centralized `@ControllerAdvice` class with `@ExceptionHandler` methods keeps your controllers clean, one place maps `ResourceNotFoundException` to 404, `ValidationException` to 400, and so on, instead of scattering try/catch blocks everywhere.

TOPIC 5

Designing Error Responses Clients Can Use

Getting the status code right is half the job. The other half is what you put in the body. A vague error like `{"error": "something went wrong"}` forces the client team to message you on Slack to figure out what actually happened. A well designed error response answers that question by itself.

A CONSISTENT ERROR SHAPE, REUSED ACROSS EVERY ENDPOINT

```
{
  "timestamp": "2026-06-25T10:42:00Z",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "Quantity must be greater than zero",
  "path": "/orders/102/items",
  "traceId": "8f2a91c4"
}
```

Two fields are doing the most work here. `error` is a stable machine readable code, the kind a client can branch logic on. `message` is for humans, it can change wording without breaking anything. And `traceId` is what your support team and your logging system both search for when a customer reports an issue. Many beginners design this directly with just an `error` string, but that creates a problem later when you need to localize messages or correlate logs across services.

WATCH OUT

Never let a raw stack trace or a database error message reach the client. Besides looking unprofessional, it leaks internal details, table names, query structure, sometimes even library versions, that an attacker can use. Catch it, log the full detail server side with the `traceId`, and return the clean shape above to the client.

If I were designing this in Spring Boot, I would usually start with one shared `ApiError` record, and build every exception handler around returning that same shape. Consistency here matters more than getting the exact field names perfect on day one.

JAVA + SPRING BOOT MAPPING

A typical setup, a `GlobalExceptionHandler` annotated with `@ControllerAdvice`, catching specific exceptions first (`MethodArgumentNotValidException` for Bean Validation failures from `@Valid`), and a generic `Exception` handler last as a safety net that still returns the same JSON shape with a generic 500.

TOPIC 6

Pagination, Before Your List Endpoint Falls Over

Here is a common production problem that almost never shows up in tutorials. A `GET /orders` endpoint works fine in testing with twenty rows. Six months later there are four million rows, the endpoint has no pagination, and a single request to it locks up a database connection for ten seconds. Pagination is not a nice to have, it is what keeps a read endpoint usable as the table grows.

There are two common approaches, and interviewers like asking which one you would pick and why.

OFFSET VS CURSOR BASED PAGINATION

OFFSET PAGINATION

- `?page=3&size=20`, simple to implement
- Database still scans and skips earlier rows
- Gets slower as the offset grows
- Items can shift between pages if rows are inserted or deleted mid scroll
- Fine for admin panels, small or stable tables

CURSOR PAGINATION

- `?after=lastSeenId&size=20`
- Query jumps straight to the cursor using an index
- Performance stays flat regardless of table size
- No shifting, new rows do not disturb your position
- Standard choice for feeds, chat history, large lists

If this were an Instagram style feed or a chat history screen, cursor pagination is almost always the right call, because users scroll forward through time and the data keeps growing underneath them. For something like an internal reports dashboard where someone wants to jump to "page 7" directly, offset pagination is simpler and the tradeoff is acceptable.

TRADEOFF

The tradeoff in one line: offset pagination gives you random access to any page number but degrades at scale, cursor pagination gives you consistent performance but only lets you move forward and backward from where you are. Most interviewers are happy if you can state this tradeoff clearly, even if you lean towards offset for the simple version of your design first.

JAVA + SPRING BOOT MAPPING

Spring Data makes offset pagination almost free, a `Pageable` parameter on your repository method gives you page, size and sort handling out of the box. Cursor pagination needs a bit more hand rolling, typically a custom query with a `WHERE id > :cursor` clause backed by an index on that column, returning a `Slice<T>` instead of a full `Page<T>` so you skip the expensive total count query.

TOPIC 7

Versioning, So You Can Change Your Mind Later

At some point, you will need to change an API in a way that breaks old clients, renaming a field, changing a response structure, removing something nobody warned you was load bearing. Versioning is how you make that change without taking down every app that has not updated yet.

Strategy	Looks Like	Tradeoff
URI versioning	<code>/v1/orders</code> , <code>/v2/orders</code>	Easy to read, easy to route at the gateway level, but the version leaks into every path and every client bookmark.
Header versioning	Accept: <code>application/vnd.app.v2+json</code>	Keeps URLs clean, plays nicer with REST purity, but it is easy to forget the header and a little harder to test quickly in a browser.
No versioning, additive only	Same <code>/orders</code> path forever	Works as long as every change is backward compatible, adding optional fields only, never renaming or removing anything clients depend on.

In real projects, URI versioning wins more often than the purist answer would suggest, mainly because it is so much easier to operate. You can route `/v1/*` and `/v2/*` to different services entirely at the API Gateway, run them side by side, and retire v1 once usage drops to zero. That said, if an interviewer is looking for the textbook REST answer, header based versioning is the one that keeps your URI pointing at the same resource regardless of version.

INTERVIEW FOCUS

At my level, I would not try to defend one versioning strategy as objectively correct. I would say something like, "I will start with no explicit versioning and keep every change backward compatible for as long as possible, and move to URI based versioning the moment a change has to break existing clients." That answer shows judgment, not just memorized options.

JAVA + SPRING BOOT MAPPING

URI versioning maps cleanly onto Spring, separate `@RequestMapping("/v1/orders")` and `@RequestMapping("/v2/orders")` controllers, sometimes in entirely separate packages so v1 logic is not tangled with v2 logic. Header versioning usually goes through a custom `RequestCondition` or content negotiation setup, more flexible, more setup, which is part of why URI versioning is the more common choice on real teams.

TOPIC 8

Request and Response Shapes, Not Raw Entities

A mistake that is easy to make when you are moving fast, returning your JPA entity directly from the controller. It feels efficient, one class, no mapping code. It also quietly creates three problems at once.

- Every column on that table is now part of your public API, including ones you never meant to expose, like an internal audit flag or a password hash on a User entity.
- Lazy loaded relationships can trigger extra queries during serialization, or throw an exception entirely once the transaction has already closed.
- Bidirectional relationships, an Order pointing to a User who points back to a list of Orders, can serialize into an infinite loop.

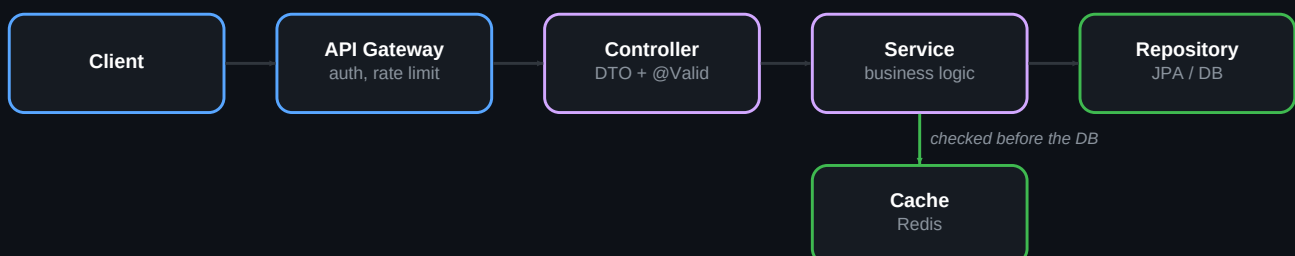
The fix is a thin layer of DTOs, plain classes that represent exactly what the API promises to return, no more and no less. It is a little more code up front. It pays for itself the first time you need to change your database schema without breaking every client that depends on the response shape.

JAVA + SPRING BOOT MAPPING

Controller takes the request, maps it to a request DTO validated with `@Valid` and Bean Validation annotations like `@NotNull` and `@Positive`. Service layer works with the entity. Before returning, you map the entity back to a response DTO, by hand, with a mapper class, or with a library like MapStruct once the number of fields gets large enough that manual mapping turns repetitive.

Here is the request flow this creates end to end, including where a cache typically sits for a read heavy endpoint.

REQUEST FLOW FOR A TYPICAL READ ENDPOINT



TOPIC 9

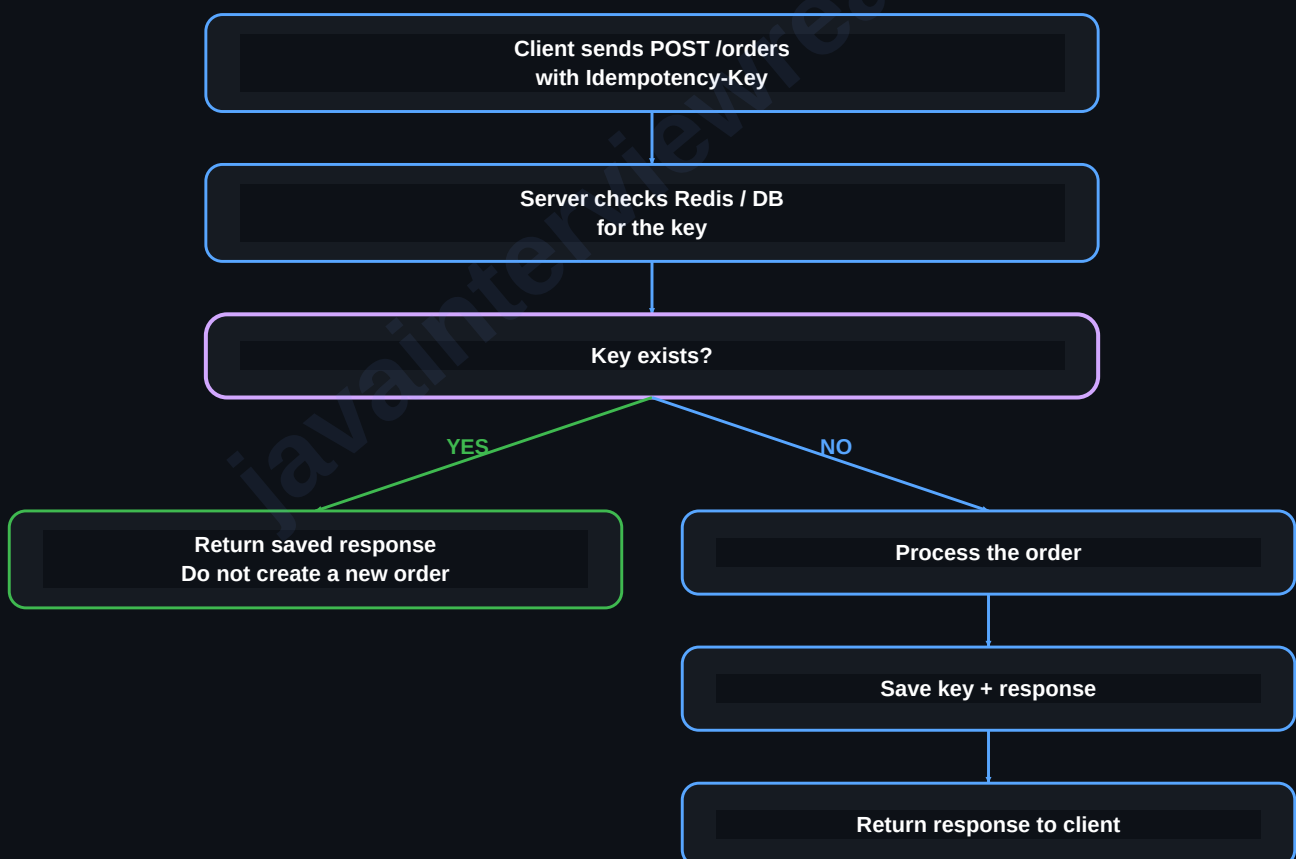
Idempotency Keys, for the Requests You Cannot Retry Twice

We touched on this in Topic 3. Now let's design it properly, because it shows up constantly in payment and order systems, and it is a favorite follow up question for exactly that reason.

Picture a checkout flow. The client sends `POST /orders`, the server creates the order and charges the card, but the response times out before it reaches the client. The client, reasonably, retries. Without any protection, you now have two orders and two charges for one purchase.

The fix is an idempotency key, a unique value the client generates once per logical operation, usually a UUID, sent as a header on every attempt of that same request.

IDEMPOTENCY KEY FLOW



Notice that this check has to happen before the order is created, and the key plus the resulting response have to be saved together, ideally in the same transaction or close to it. If you save the key after creating the order, a second request arriving in that small gap still slips through and creates a duplicate.

WATCH OUT

A duplicate order is not just an annoying bug, in a payment system it means a customer gets charged twice. This is exactly the kind of failure scenario interviewers expect you to bring up yourself when designing anything involving money, without being asked directly.

JAVA + SPRING BOOT MAPPING

A common implementation, a `HandlerInterceptor` or a thin `@Aspect` around the order creation endpoint reads the `Idempotency-Key` header, checks Redis with a short TTL using `SETNX` style semantics, and either short circuits with the stored response or lets the request continue. Some teams instead enforce a unique constraint on the key column in the orders table itself as a second line of defense.

IDEMPOTENCY IMPLEMENTATION CHECKLIST

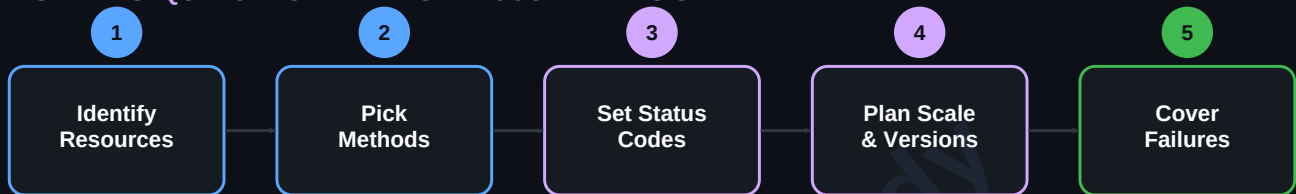
- ✓ Client sends a unique key with every attempt
- ✓ Server checks the key before processing anything
- ✓ Response is saved together with the key
- ✓ Same response is returned on every retry
- ✓ Redis TTL or a DB unique constraint backs the check

TOPIC 10

How to Walk Through Your API Design Out Loud

In interviews, you do not need a perfect API on the first pass. You need to explain your thinking clearly, in an order that shows you considered the right things before the interviewer had to ask. Here is the rough sequence that tends to land well.

A SIMPLE SEQUENCE FOR TALKING THROUGH API DESIGN



You do not have to narrate each step as a separate labeled section out loud, that would sound rehearsed. Just make sure your answer covers all five naturally as you sketch the endpoints. Many candidates only do step one and two, list the resources and methods, then stop. The candidates who stand out are the ones who keep going into status codes, scale, and failure on their own.

INTERVIEW FOCUS

Useful phrases for this part of the interview: "Since this list can grow large, I would add cursor based pagination here." "For the create endpoint, I would expect clients to retry on timeout, so I will add idempotency key support." "I will keep the response shape consistent across endpoints so the error format is predictable for whoever is consuming this API."

TOPIC 11

Interview Tradeoffs Worth Having Ready

These are the decisions interviewers come back to most often. You do not need to memorize a script, just know which way you would lean and why.

Decision	Pick This When	Pick The Other When
Offset vs cursor pagination	Offset: small or stable tables, users need to jump to a specific page number.	Cursor: large or fast growing tables, infinite scroll feeds, chat history.
URI vs header versioning	URI: you want simple routing at the gateway and easy manual testing.	Header: you want the cleanest possible REST URLs and your clients can manage custom headers easily.
Synchronous response vs async with a callback	Sync: the client needs the result immediately to continue, like a payment confirmation.	Async: the operation is slow, like video processing, so you return 202 Accepted and notify later.
Strict request validation vs lenient parsing	Strict: financial or safety critical fields, where a wrong value is worse than a rejected request.	Lenient: optional, low risk fields, where ignoring an unknown field is friendlier to evolving clients.
Single combined endpoint vs multiple focused endpoints	Combined: the client always needs all the data together, fewer round trips matter more, think a mobile home screen.	Focused: different clients need different slices of the data, and you want independent caching per piece.

TOPIC 12

Quick Revision Cheat Sheet

One pass before your interview. If you can explain the right column for each row below in your own words, you are in good shape for the API design part of any system design question.

Area	Remember This
Naming	Resources are nouns. The action lives in the HTTP method, not the path.
Methods	GET, PUT, DELETE are idempotent. POST is not, PATCH usually is, unless it increments something.
Status codes	201 for created, 204 for no content, 409 for conflicts, 401 means unknown identity, 403 means known but not allowed.
Errors	One consistent JSON shape everywhere, with a machine readable code, a human message, and a trace id.
Pagination	Offset is simple but slows down at scale. Cursor stays fast but only moves forward and backward.
Versioning	Stay backward compatible as long as you can. Move to URI versioning the moment a change has to break clients.
DTOs	Never return JPA entities directly. Map to a response shape you control.
Idempotency keys	Required for any unsafe operation a client might retry, especially anything involving money.

Next: Module 6 explains SQL vs NoSQL, when to choose each database type, and how database decisions connect to Java, JPA, Hibernate, and real backend systems.

MODULE 6

SQL vs NoSQL

When to Pick What

How to choose the right database for any system, and how to defend that choice when the interviewer pushes back.

Covers JDBC, JPA and Hibernate from the Java side, plus the database decisions that actually come up in backend interviews.

javainterviewready

TOPIC 01

The Database Question That Trips Up Most Freshers

Here's a pattern I've seen in interview after interview. The interviewer asks something like, "design an e-commerce order system," and within thirty seconds the candidate says "I'll use MongoDB because it's flexible and scalable."

That sentence alone tells the interviewer the candidate has memorized a buzzword pair instead of understanding the tradeoff. Flexible compared to what? Scalable for which kind of load? Nobody followed up with a reason, so there isn't one.

This module is not really about SQL versus NoSQL as two competing technologies. It's about teaching you to ask a different question first: what does my data actually look like, and what do I need to do with it fast? Once you can answer that, the database choice mostly falls out on its own.

INTERVIEW MOMENT

If an interviewer asks "SQL or NoSQL for this system," they are not testing whether you know both exist. They're testing whether you can reason about access patterns, consistency needs, and relationships in the data before reaching for a tool name.

In real projects, this decision usually gets made once, early, and then everyone lives with it for years. Migrating a production database after the fact is one of the more painful things a backend team can be asked to do. That's part of why interviewers care so much about how you think through it, not just which answer you land on.

TOPIC 02

What You're Really Buying When You Pick SQL

Most people describe relational databases as "tables with rows and columns," which is true but misses the point. The actual thing you're paying for is a set of guarantees the database enforces for you, so you don't have to write that logic yourself in application code.

Three of those guarantees matter most in interviews:

WHAT SQL GIVES YOU

ACID Transactions ■ All steps succeed or roll back ■ Useful for orders, payments, inventory	Foreign Keys ■ Database protects relationships ■ Avoids invalid references	Joins ■ Query related data together ■ Useful for user, orders, items
--	---	---

Notice none of those three things are about speed. People assume relational databases are slow and NoSQL is fast, but that's not really the comparison. A well-indexed Postgres table handles thousands of reads per second without trouble. The real tradeoff is about structure and correctness guarantees, not raw throughput.

TIP

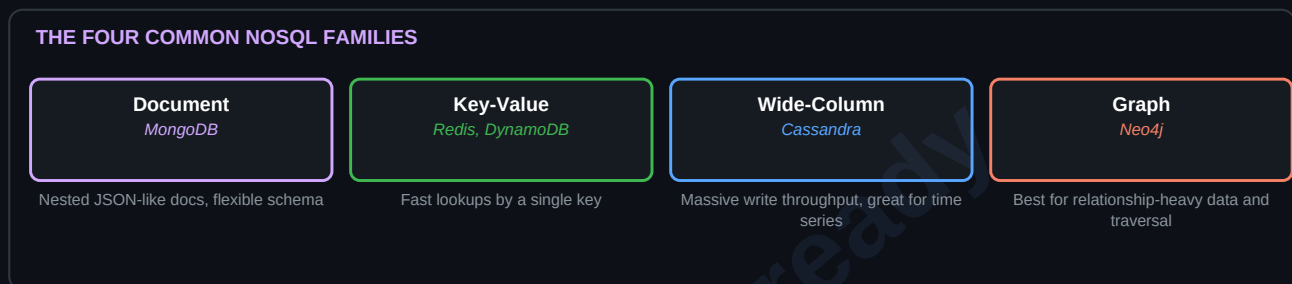
When you're not sure which database to pick, ask yourself: "if two of these writes happen at the same time, does it matter if one is incomplete?" If yes, that's usually a strong pull toward SQL.

Many beginners design the schema directly without asking how the data will actually be queried. You end up with a clean-looking ER diagram that falls apart the moment someone needs to fetch "all orders for a user in the last 30 days, sorted by status" and there's no index supporting that access pattern. Design the queries first, then the tables.

TOPIC 03

NoSQL Isn't One Thing. Stop Treating It Like One.

This is where a lot of answers fall apart in interviews. "NoSQL" is an umbrella term for at least four genuinely different storage models, and they solve different problems. Saying "I'll use NoSQL" is a bit like saying "I'll use a vehicle" without saying if you mean a bicycle or a cargo truck.



You don't need expert depth in all four for a 0-3 year interview, but you should know which one solves which problem, because the question "why not just use MongoDB for everything" comes up a lot more than people expect.

Document stores like MongoDB work well when your data naturally nests. A product catalog where each product has a variable set of attributes is a good fit. You're not fighting a rigid schema for every new product type.

Key-value stores like Redis or DynamoDB exist for one job: get a value back instantly when you already know the key. Session storage, leaderboard scores, rate-limit counters. If your access pattern is "give me record X by its ID," a key-value store will outperform almost anything else.

Wide-column stores like Cassandra are built for write-heavy, append-style data at huge scale, things like IoT sensor readings or activity logs where you're writing constantly and reading by time range. Graph databases like Neo4j matter when the relationships between records are the actual point of the query, like "friends of friends" in a social app.

WARNING

Picking MongoDB because "it's web-scale and flexible" without a specific reason is one of the fastest ways to lose credibility in an interview. Interviewers have heard that line a hundred times and it usually means there's no real reasoning behind it.

TOPIC 04

The Framework I Actually Use to Decide

Forget memorizing pros and cons lists for a second. When I'm actually designing a system, or when I'm listening to a candidate design one, there are really three questions doing most of the work.

Question 1: Does this data have strong relationships that need to stay consistent? Orders linked to users linked to payments linked to inventory. If a bug could let an order exist without a valid user, that's a sign you want the database enforcing the relationship, not your application code.

Question 2: How will I query this data, mostly? If you'll be running complex queries that join multiple entities and filter on several conditions, SQL's query engine is going to save you a lot of pain. If you're always fetching by a single known key, that complexity is wasted overhead.

Question 3: What happens if I need to scale writes massively? This is where NoSQL options, especially wide-column and key-value stores, start to pull ahead. They were built from day one to horizontally scale writes across many machines, which relational databases can do too, but with a lot more operational effort.

Lean SQL when

- Relationships matter and must stay correct
- You need transactions across multiple tables
- Query patterns are varied and evolve over time
- Data structure is mostly fixed and well understood

Lean NoSQL when

- Access pattern is simple: fetch by key, mostly
- Schema needs to change often without migrations
- Write volume is extremely high and growing fast
- Data is naturally nested or relationship-light

Putting It on a Decision Map

QUICK DECISION MAP

Read-heavy, complex joins



PostgreSQL / MySQL + indexes

Write-heavy, simple key lookup



DynamoDB / Cassandra

Needs millisecond cache reads



Redis in front of main DB

Flexible, evolving document shape



MongoDB

In real projects, this usually becomes a problem when a system starts with one clear access pattern and slowly grows a second one nobody planned for. A typical example: you build a product catalog in MongoDB because the schema is flexible, and eighteen months later product management wants complex reporting across categories, prices, and inventory. Now you're writing aggregation pipelines that look suspiciously like SQL joins, just harder to read.

INTERVIEW FOCUS

A strong answer sounds like: "Orders and payments need strict consistency, so I'd put those in PostgreSQL. Product search and catalog browsing is read-heavy and doesn't need strong consistency, so I might cache that layer in Redis." That single sentence shows you can split a system by access pattern instead of picking one database for everything.

TOPIC 05

Your SQL Database Is Fine. Until It Isn't.

Here's something that surprises freshers: a single PostgreSQL or MySQL instance can comfortably handle a system with hundreds of thousands of users. People jump to NoSQL way earlier than they need to because they assume relational databases hit a wall at some small number. They don't. The wall shows up later, and there are steps before you reach it.

The Usual Order of Moves

When a relational database starts struggling under load, this is roughly the order engineers reach for solutions, cheapest first.

SCALING A RELATIONAL DATABASE, IN ORDER



Most performance problems in interviews and in real production systems are actually missing-index problems, not "wrong database" problems. Before reaching for a bigger architectural change, ask whether the slow query even has an index supporting it. I've seen production incidents get fixed with a single index that should have existed from day one.

Read replicas solve a specific problem: too many reads hitting one machine. You copy the data to one or more secondary instances and send read traffic there, while writes still go to the primary. This works great for read-heavy systems like a news feed or product catalog. It does nothing for write-heavy load, because all writes still funnel through one primary.

Sharding is the real architectural shift. You split your data across multiple database instances based on some key, often user ID or region, so writes spread out too. This is also where things get genuinely harder. Joins across shards don't work the way they used to, and you need to think carefully about which shard key keeps related data together.

TRADEOFF

Sharding a relational database gets you closer to NoSQL-style scale, but you give up some of the convenience that made SQL attractive in the first place, like simple cross-entity joins. At some point the question stops being "SQL or NoSQL" and becomes "how much operational complexity am I willing to take on to keep relational guarantees at this scale."

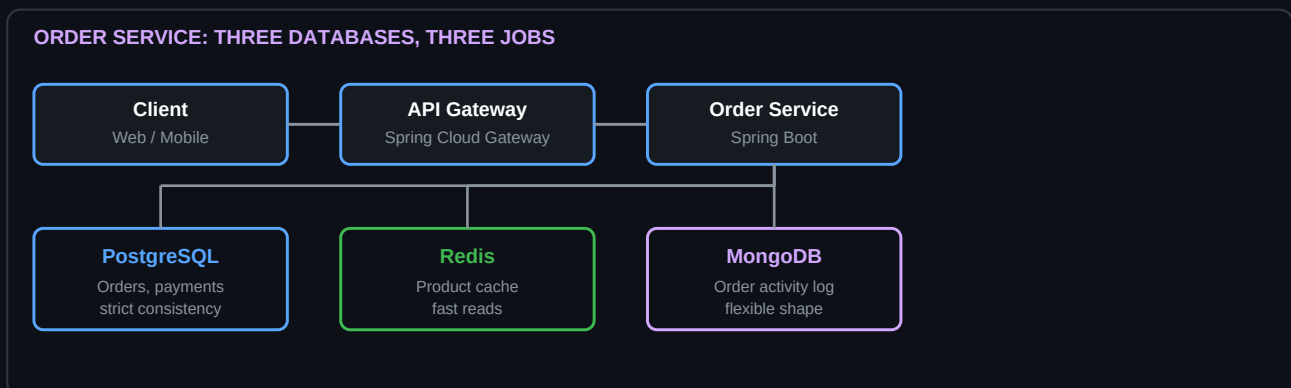
This is exactly where interviewers expect you to show judgment instead of reciting a tier list. If I were designing this in Spring Boot, I would usually start with a single PostgreSQL instance, add proper indexes early, put Redis in front of the hottest read paths, and only talk about sharding once the interviewer pushes the scale number high enough that a single primary genuinely can't keep up.

TOPIC 06

Most Real Backends Don't Pick One. They Mix.

Here's something the SQL versus NoSQL framing hides: almost every production system I've worked on uses more than one type of database, each one chosen for a specific part of the system. This is usually called polyglot persistence, though you don't need to use that term in an interview. Just show that you understand why the split happens.

Take a typical e-commerce order system. Walk through what each piece of data actually needs, and the database choice for each piece stops being a guess.



PostgreSQL holds orders and payments because a half-completed payment is a real business problem, and the relationships between order, user, and payment need to stay correct. Redis caches product details because product pages get read constantly and don't need to be perfectly fresh to the millisecond. MongoDB stores order activity events (status changes, delivery updates) because that data is naturally a flexible, growing list per order, and you're not running complex joins against it.

JAVA MAPPING

In Spring Boot, this usually means three different repository layers in the same service: a JPA repository for the PostgreSQL order and payment tables, a RedisTemplate or Spring Cache @Cacheable layer for product data, and a Spring Data MongoDB repository for the activity log. The service layer coordinates all three, but each repository only talks to its own datastore.

This is where freshers often go wrong in interviews: they assume the interviewer wants one database for the whole answer, so picking three feels like cheating or overcomplicating things. It's the opposite. Naming a specific database for a specific piece of data, with a reason, is a stronger answer than a single confident-sounding pick for everything.

TOPIC 07

JDBC, JPA, Hibernate: Stop Mixing These Up

I've interviewed candidates with two years of Spring Boot experience who couldn't explain the difference between JPA and Hibernate. That's a problem, because it usually means they've been copy-pasting repository code without understanding what's actually running underneath it.

These three things sit at different layers, and once you see the layering, the confusion mostly goes away.



JDBC is the foundation. It's the raw Java API for connecting to a database, sending SQL, and reading back results. You write SQL strings yourself, manually map `ResultSet` rows into objects, and manage connections by hand. Nobody writes plain JDBC in a modern Spring Boot app, but it's worth knowing it's there because everything above it eventually calls down into it.

JPA (Java Persistence API) is not a library you can run on its own. It's a specification, a set of rules: how `@Entity`, `@Id`, and `@OneToMany` should behave, what an `EntityManager` should do. JPA defines the contract, nothing more.

Hibernate is the most common implementation of that contract. When you annotate a class with `@Entity` in a Spring Boot app, Hibernate is the engine underneath that actually generates the SQL, manages the session, and talks to JDBC. You're coding against the JPA interface, but Hibernate is doing the work.

Spring Data JPA sits on top of all of it. It's the layer that lets you write an interface with a method like `findByIdAndStatus(...)` and have Spring generate the query for you, no implementation needed. This is what most Spring Boot developers interact with daily, which is exactly why people blur the lines between the four.

INTERVIEW FOCUS

A common follow-up: "if you're using Spring Data JPA, why do you need to know Hibernate exists?" Good answer: because Hibernate's behavior leaks through in real situations like lazy loading, the N+1 query problem, and how flush timing affects what's actually sent to the database. Spring Data JPA hides the wiring, not the consequences.

Where People Get Burned: The N+1 Problem

This is the most common real-world Hibernate mistake, and it comes up in interviews too. You fetch a list of orders, then loop through them and access `order.getItems()` for each one. If that relationship is lazily loaded, Hibernate fires one query to get the orders, then a separate query per order to get its items. A hundred orders means a hundred and one queries instead of two.

Approach	What happens
Lazy fetch, naive loop	1 query for orders + 1 query per order for items. Slow at scale.
JOIN FETCH in JPQL	Pulls orders and items in a single query using a join.
@EntityGraph	Declarative way to tell Spring Data JPA which relations to fetch eagerly for a query.

WARNING

If you say "I'd use JPA" in an interview without being able to explain lazy loading or the N+1 problem, an interviewer with real production experience will probe exactly there. It's one of the fastest ways to tell who has actually run a Spring Boot app under load versus who has only followed a tutorial.

On the NoSQL side, the Java mapping looks different but the same instinct applies. Spring Data MongoDB gives you a repository abstraction similar to JPA, but there's no join concept underneath it, you're working with documents. Spring Data Redis (through `RedisTemplate` or the `@Cacheable` annotation) is usually about caching values, not modeling complex relationships at all.

TOPIC 08

When the Database Falls Over, Not If

Every database fails eventually, whether that's a slow query under load, a node going down, or a replica falling behind. What changes between SQL and NoSQL is how that failure shows up and what you have to do about it. This is a section freshers almost never prepare for, and it's exactly the kind of follow-up question that separates a confident answer from a memorized one.

Relational Database Failures

Failure	What Happens	How to Handle
Primary down	Writes stop completely until a replica is promoted or the primary recovers.	Set up automated failover. Don't rely on someone noticing at 2am.
Connection pool exhaustion	App opens more connections than HikariCP allows. New requests time out waiting.	Size the pool correctly and monitor active connections, even though the DB itself is healthy.
Lock contention	Two transactions update the same row at once. One waits, and may time out.	Keep transactions short. This is the cost of the consistency you asked for.

NoSQL / Cache Failures

Failure	What Happens	How to Handle
Eventual consistency surprise	A write isn't visible on a read right after. Code assuming read-after-write gets hard-to-reproduce bugs.	Don't assume read-after-write consistency. Check what your store actually guarantees.
Hot partition	One partition key (a viral post, a celebrity user) gets far more traffic than the rest.	Pick a partition key that spreads load evenly. Watch for skew early.
Cache stampede	A hot key expires, a flood of requests all miss the cache at once, and they all hit the DB together.	Stagger TTLs, or lock so only one request rebuilds the cache while others wait briefly.

TIP

A simple fix for cache stampede: stagger cache expiry times slightly instead of setting every key to expire at the exact same TTL, or use a lock so only one request rebuilds the cache while others wait briefly.

Five Tradeoffs Interviewers Actually Want to Hear

01 Consistency vs Availability

SQL with strong consistency means a write blocks until it's confirmed durable. Many NoSQL systems favor availability, accepting a small window where data isn't fully synced yet, in exchange for staying responsive during failures.

02 Flexibility vs Enforced Structure

NoSQL lets you change the shape of your data without a migration. SQL forces you to define structure upfront, which costs you flexibility but buys you data integrity you don't have to write yourself.

03 Simple Scaling vs Rich Querying

Key-value and wide-column stores scale writes horizontally with little effort. Relational databases give you joins and ad-hoc queries, but scaling writes takes real engineering work like sharding.

04 Operational Simplicity vs Specialization

Running one PostgreSQL instance well is simpler than running three different database technologies. Polyglot persistence solves real problems but adds real operational cost: more systems to monitor, back up, and patch.

05 Familiar Tooling vs Purpose-Built Performance

JPA, Hibernate, and Spring Data give a mature, well-understood toolchain for SQL. A purpose-built store like Redis or Cassandra often performs better for its specific job, but your team has to learn and maintain another tool.

INTERVIEW MOMENT

At your level, you're not expected to have a perfect answer for every tradeoff. What interviewers are actually listening for is whether you can say "here's the tradeoff, and here's why I'd accept it for this specific system," instead of pretending one option has no downside.

If you remember nothing else from this module, remember this: the question is never really "SQL or NoSQL." It's "what does this specific piece of data need, and which tool gives me that with the least extra work." Answer that, and the database name you say out loud stops being a guess.

MODULE 6 · QUICK REVISION

Module 6 Quick Revision Cheat Sheet

One pass before the interview. If a row doesn't ring a bell, go back to that topic.

Area	Remember This
SQL	Tables, fixed schema, ACID, foreign keys, joins. Correctness over flexibility.
NoSQL	Umbrella term for document, key-value, wide-column, graph. Not one thing.
When to choose SQL	Relationships matter, you need transactions, query patterns vary.
When to choose NoSQL	Simple key lookup, schema changes often, very high write volume.
Read replicas	Copies for read traffic. Writes still go through one primary.
Sharding	Splits data across instances by a key. Spreads writes, complicates joins.
Polyglot persistence	Using multiple databases in one system, each for a specific job.
JDBC	Raw Java API that sends SQL to the driver. The foundation underneath everything.
JPA	Specification only. Defines how @Entity, @Id, relationships should behave.
Hibernate	The implementation that fulfills the JPA spec and talks to JDBC.
Spring Data JPA	Repository interfaces on top of JPA. Less boilerplate, derived queries.
N+1 problem	Lazy loading in a loop fires one query per row. Fix with JOIN FETCH or @EntityGraph.
Database failure handling	Know what fails (primary down, pool exhaustion, hot partition) and how to respond.

Next: Module 7 explains database scaling basics, including indexing, replication, partitioning, sharding, read replicas, and common database bottlenecks.

Database Scaling Basics

Indexing, replication, partitioning, sharding, read replicas, caching, and common database bottlenecks.

Part of **The System Design Interview Playbook**

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

MODULE 7 · OVERVIEW

Why Database Scaling Matters

In almost every system design interview, the interviewer will eventually ask: 'What happens when your database can't handle the load?' Most freshers say 'add more servers' without being able to explain how or why.

Database scaling is one of those topics where the gap between knowing the words and knowing what to actually do becomes obvious very quickly. This module closes that gap.

Indexing

TOPIC 1

**Replication &
Read Replicas**

TOPIC 2

Partitioning

TOPIC 3

Sharding

TOPIC 4

**Caching for
DB Load**

TOPIC 5

**Common
Bottlenecks**

TOPIC 6

HOW TO USE THIS MODULE

Each section covers one scaling technique. For each one, understand: what problem it solves, when to reach for it, and how to explain it in an interview.

You do not need to memorize every detail. You need to explain the thinking behind each choice.

TOPIC 1

TOPIC 1 OF 6

Indexing

If someone asks why queries are slow and your first answer isn't 'missing index', you're already behind. Indexing is the first thing you reach for before considering any architectural change.

An index is a separate data structure that the database maintains to speed up lookups. Without it, every query scans the entire table row by row. With it, the database can jump directly to the right row in $O(\log n)$ time using a B-Tree.

HOW A DATABASE INDEX WORKS

WITHOUT INDEX

user_id=1001

user_id=2234

user_id=3892

user_id=8761

user_id=4412 ✓

Scanned 5 rows (full scan)

INDEX (sorted)

4412 → row 5

8761 → row 4



user_id=4412 ✓

1 lookup $O(\log n)$

Types of Indexes

Index Type	Best For	Example
B-Tree	Range queries, ORDER BY	WHERE age > 25
Hash	Exact equality lookups	WHERE email = 'x@y.com'
Composite	Multi-column filters	WHERE city='NY' AND age>18
Partial	Subset of rows	WHERE deleted = false
Covering	Index-only scans	SELECT col in index cols

The Cost Side of Indexes

Every index you add speeds up reads but slows down writes. When you insert or update a row, the database has to update every index on that table too. On a write-heavy table with 10 indexes, every insert touches 11 data structures.

In real projects this becomes a problem when someone indexes every column 'just to be safe' and then wonders why inserts are taking 200ms. Pick indexes based on your actual query patterns, not speculation.

COMMON MISTAKE

Adding a composite index in the wrong column order. If your query is WHERE city = 'NY' AND age > 18, the index must be (city, age) not (age, city). The leftmost column in a composite index is the leading key. The DB can only use the index starting from the left.

IN SPRING BOOT + JPA**@Entity annotation level:**

```
@Table(indexes = { @Index(name = "idx_user_email", columnList = "email") })
```

Or use Flyway / Liquibase migration:

```
CREATE INDEX idx_order_user_id ON orders(user_id);
```

In Spring Data, methods like `findByEmail()` work fine but if email is not indexed, you get a full table scan at 1M rows. Use `EXPLAIN ANALYZE` in PostgreSQL to check if your index is being hit.

WHAT TO SAY IN AN INTERVIEW

"Before I consider any architectural change, the first thing I check is whether the slow queries are missing indexes. I would run `EXPLAIN ANALYZE` to see the query plan and look for sequential scans on large tables."

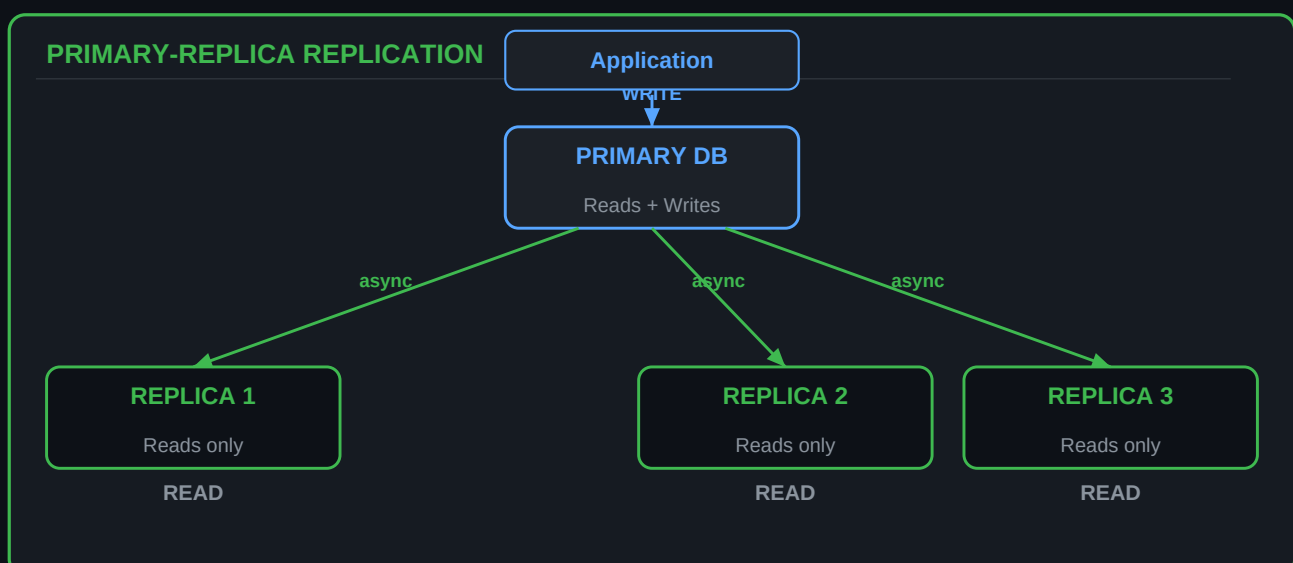
"If the column is part of a `WHERE` clause or `JOIN` condition, it should probably have an index. The tradeoff is that indexes slow down writes, so I only add them where they measurably help."

TOPIC 2**TOPIC 2 OF 6**

Replication and Read Replicas

Your app starts getting 10,000 reads per second. One database can't keep up. What do you do? You don't immediately shard. You add read replicas.

Replication means you run multiple copies of your database. One node is the primary and handles all writes. The others are replicas that receive a copy of every change and serve read traffic.



Sync vs Async Replication

Replication can be synchronous or asynchronous. The difference matters a lot in interviews.

SYNC VS ASYNC REPLICATION

Synchronous:

The primary waits for at least one replica to confirm before acknowledging the write. You get strong consistency but higher write latency. A network hiccup between primary and replica can slow down every write.

Asynchronous (default in most systems):

The primary writes and immediately returns success. Replicas catch up in the background. Much faster writes, but replicas can be slightly behind. If primary fails, some recent data might not be on any replica yet.

For most web apps: async replication is fine. For financial transactions: sync or semi-sync.

Replication Lag

This trips up a lot of junior developers. After a write, if you immediately read from a replica, you might get stale data. The replica hasn't caught up yet.

Classic example: user updates their profile picture, gets redirected to their profile page, but still sees the old photo. The write hit the primary but the read hit a replica that's 50ms behind.

Solutions: read your own writes from the primary, or use sticky sessions to route a user's reads to the primary for a short window after they write.

IN SPRING BOOT

Route reads and writes to different datasources using `AbstractRoutingDataSource`:

`@Primary DataSource` → primary DB (writes)

`@Qualifier("replica") DataSource` → replica (reads)

Or use Spring Data JPA `@Transactional(readOnly = true)` with a routing datasource.

The `readOnly` hint tells Hibernate to skip dirty checking and can be used to route to read replicas.

FOLLOW-UP QUESTION

"What happens if the primary fails?"

Answer: You promote one replica to primary. In managed services like AWS RDS, this is automatic (Multi-AZ failover). In self-managed setups you need tooling like Patroni or ProxySQL. Failover usually takes 30-60 seconds, during which writes are unavailable.

TOPIC 3

TOPIC 3 OF 6

Partitioning

Partitioning is when you split one large table into smaller physical pieces, but they still live inside the same database. It is not the same as sharding. This distinction matters in interviews.

The most common type is range partitioning by date. Your orders table gets split into orders_2023, orders_2024, orders_2025. Queries for 'all orders this year' only scan the 2024 partition instead of 100 million rows.

Types of Partitioning

- **Range partitioning:** Split by a range of values. Most common with dates. WHERE created_at > '2024-01-01' only scans the 2024 partition.
- **Hash partitioning:** Apply a hash function to a column. Distributes rows evenly. Good for balancing load when there's no natural range.
- **List partitioning:** Split by discrete values. WHERE region = 'US' goes to the US partition, 'EU' goes to the EU partition.

PARTITIONING

- Same physical machine
- Single database, multiple tables
- Managed by the DB engine itself
- No app-layer complexity
- Easier to query across parts

Example:

```
orders_2024_q1, orders_2024_q2
```

SHARDING

- Different physical machines
- Multiple database nodes
- App or proxy layer routes queries
- More complex to maintain
- Scales beyond a single server

Example:

```
shard0.db, shard1.db, shard2.db
```

WHEN TO REACH FOR PARTITIONING

You have a single table growing past 50-100 million rows, and most queries filter by the partition key (like date). Before sharding, try partitioning. It's built into PostgreSQL natively and adds almost no operational complexity.

In PostgreSQL: CREATE TABLE orders PARTITION BY RANGE (created_at); Then add child tables for each time window.

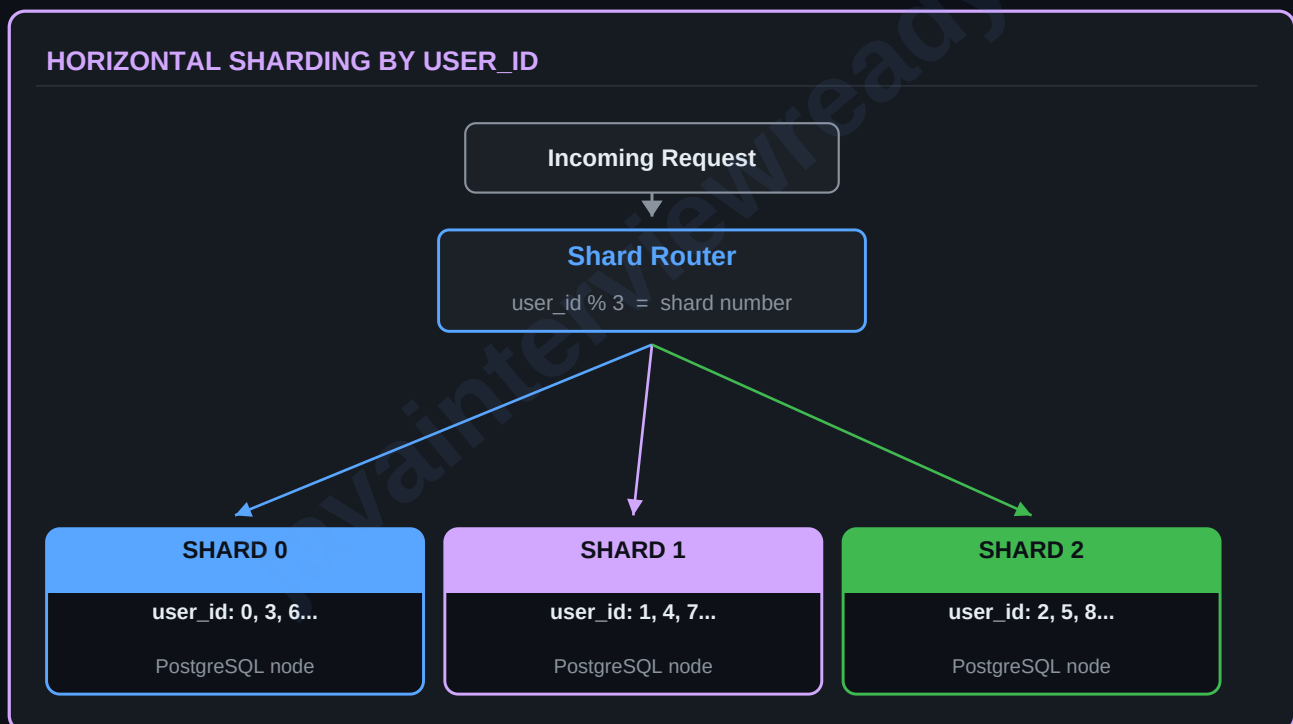
TOPIC 4

TOPIC 4 OF 6

Sharding

Sharding is what you reach for when a single database server, even a very powerful one, cannot hold your data or handle your write volume. You split your data across multiple completely separate database nodes.

The core idea: pick a shard key, apply a function to determine which node a row belongs to, and route every query through a shard router. Each node only stores and serves a fraction of the total data.



Choosing a Shard Key

This is where most interview answers fall apart. Picking the wrong shard key creates a 'hot shard', where one node gets most of the traffic while others sit idle.

- **Good shard keys:** High cardinality, evenly distributed, aligns with your query patterns. `user_id` for a social app. `order_id` for an e-commerce system.
- **Bad shard keys:** Country code (most users might be US). Status column (most rows are 'active'). Timestamp without hashing (all new writes go to the latest shard).

THE CROSS-SHARD JOIN PROBLEM

Once you shard, JOIN queries across shards become very expensive or impossible. If user data is on shard 0 and their orders are on shard 1, you cannot JOIN them in SQL.

Solutions:

1. Co-locate related data on the same shard (put user and their orders on the same shard by hashing user_id).
2. Denormalize: embed the data you need instead of joining.
3. Handle joins at the application layer, not the database layer.

WHEN AN INTERVIEWER ASKS ABOUT SHARDING

Don't jump to sharding as your first answer. Say: 'I would first check indexes, then add caching, then try read replicas. Sharding adds significant operational complexity and I would only reach for it when single-node options are exhausted.'

This shows maturity. Junior engineers shard everything. Senior engineers only shard when they have to.

IN SPRING BOOT

Sharding is typically handled by a proxy layer, not Spring Boot itself:

- Option 1: Use Apache ShardingSphere with Spring Boot. Shard key routing configured in YAML.
- Option 2: Vitess (YouTube's sharding layer) for MySQL.
- Option 3: Application-level routing with AbstractRoutingDataSource and custom shard key resolver.

For most interview contexts, saying 'ShardingSphere or Vitess handles routing, and we co-locate by user_id' is a complete and correct answer.

TOPIC 5**TOPIC 5 OF 6**

Reducing DB Load with Caching

Before adding read replicas or sharding, caching can absorb 80-90% of your read traffic without touching the database at all. This is why it belongs in any database scaling conversation.

The typical pattern: check Redis first. If the data is there, return it. If not, query the database, store the result in Redis with a TTL, then return it. The next 1000 requests for the same data hit Redis, not PostgreSQL.

Cache-Aside vs Write-Through

CACHING STRATEGIES

Cache-Aside (most common):

App checks cache first. On miss, reads from DB and writes to cache. Simple. Works for read-heavy data.
Risk: stale data if TTL is too long.

Write-Through:

Every write goes to DB and cache simultaneously. Cache is always fresh. Cost: every write is slightly slower.
Good when reads and writes are both frequent.

Write-Behind (Write-Back):

Write to cache immediately, sync to DB asynchronously. Fastest writes. Risk: data loss if cache goes down before flush.

SPRING BOOT + REDIS

```
@Cacheable(value = "users", key = "#userId")
```

```
public User getUser(Long userId) { ... } // reads from Redis first
```

```
@CacheEvict(value = "users", key = "#user.id")
```

```
public void updateUser(User user) { ... } // clears cache on update
```

Config in application.properties:

```
spring.cache.type=redis
```

```
spring.data.redis.host=localhost
```

```
spring.data.redis.port=6379
```

HOW TO TALK ABOUT CACHING IN DB SCALING CONTEXT

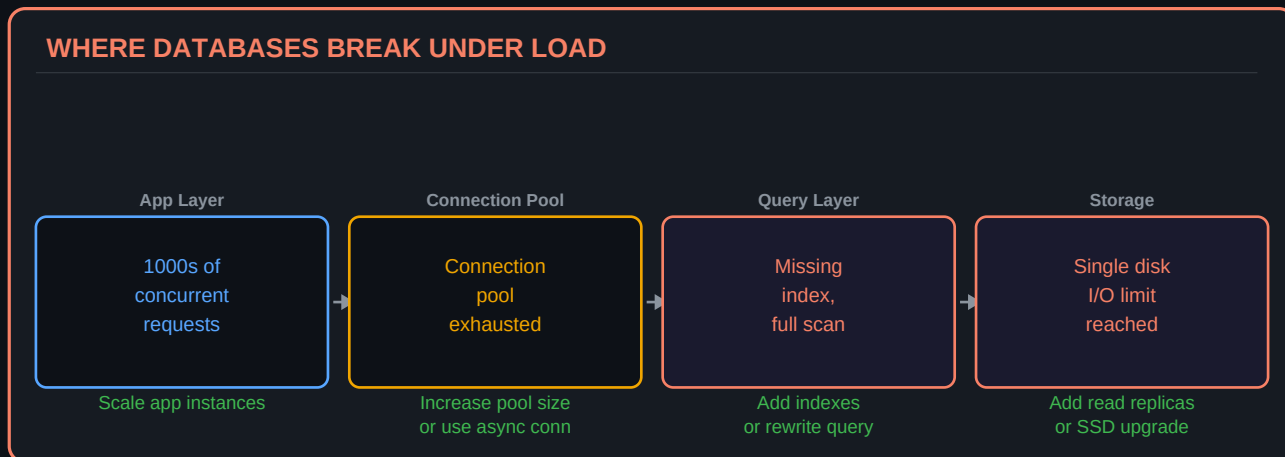
"My first step for read load is caching. If 80% of reads are for the same product catalog data, I can put that in Redis with a 5-minute TTL. That eliminates most of the DB read load before I even consider read replicas or sharding."

TOPIC 6

TOPIC 6 OF 6

Common Database Bottlenecks

In production, databases don't fail spectacularly. They degrade slowly. Response times creep up. Connection pool errors start appearing. Query times spike during peak hours. Knowing what to look for separates good engineers from great ones.



The N+1 Query Problem

This is one of the most common performance killers in Spring Boot apps and interviewers love to ask about it.

You fetch 100 orders. Then for each order, your code calls `getUser(order.getUserId())`. That's 1 query to get orders + 100 queries to get users = 101 total queries. At scale, this destroys your database.

FIXING N+1 IN SPRING BOOT

Problem (N+1 queries):

```
List orders = orderRepo.findAll();
for (Order o : orders) { User u = userRepo.findById(o.getUserId()); }
```

Fix 1: Use JOIN FETCH in JPQL:

```
@Query("SELECT o FROM Order o JOIN FETCH o.user WHERE o.status = :status")
```

Fix 2: Use @EntityGraph to eagerly load associations:

```
@EntityGraph(attributePaths = {"user"})
```

```
List findByStatus(String status);
```

Connection Pool Exhaustion

Your app creates a pool of database connections at startup. Under heavy traffic, if every thread needs a connection and the pool is empty, threads start waiting. If they wait too long, you get timeout errors even if the DB itself is fine.

TUNING HIKARI CONNECTION POOL

```
spring.datasource.hikari.maximum-pool-size=20 # default is 10
```

```
spring.datasource.hikari.minimum-idle=5
```

```
spring.datasource.hikari.connection-timeout=30000 # ms
```

```
spring.datasource.hikari.idle-timeout=600000
```

Rule of thumb: pool size = (core count * 2) + effective spindle count.

For async I/O or SSD, keep it lower. Bigger pools can actually hurt performance due to contention.

Slow Queries and Lock Contention

- Slow queries: Missing indexes cause full table scans. Always check EXPLAIN ANALYZE before adding indexes randomly.
- Lock contention: Long-running transactions hold row locks and block other queries. Keep transactions short. Avoid doing API calls or heavy computation inside a transaction.
- Dead locks: Two transactions each waiting for the other's lock. PostgreSQL detects and kills one. Log these and redesign the query order.

WHAT NOT TO DO INSIDE A TRANSACTION

Common mistake in Spring Boot: putting external API calls, file I/O, or Kafka publishes inside a @Transactional method.

@Transactional

```
public void processOrder(Order order) {  
    orderRepo.save(order); // holds DB lock  
    paymentApi.charge(order); // 2-second external call  
    notificationService.send(); // another network call  
} // lock held for entire 2+ seconds
```

Keep transactions focused on DB work. Do async work after the transaction commits.

Putting It Together: The Scaling Decision Map

When someone asks 'how would you scale your database?', walk through this order. Don't jump straight to sharding.

DATABASE PROBLEM -> SOLUTION MAP**PROBLEM**

Slow read queries

→

Add indexes on filtered columns

Too many reads for 1 DB

→

Add read replicas, route reads there

DB too big for 1 server

→

Shard horizontally by a partition key

Hot rows / hot partition

→

Hash the shard key, distribute writes

Joins across shards

→

Denormalize, or use a service layer

High write throughput

→

Write-behind cache, async batch inserts

THE SCALING ORDER IN INTERVIEWS**A common interviewer question: 'Your app has 10 million users and the DB is slow. What do you do?'**

Walk through this order:

1. Check for missing indexes first. Free fix, zero ops cost.
2. Add caching (Redis). Absorb 80% of read traffic without changing the DB.
3. Add read replicas. Route SELECT queries away from the primary.
4. Partition large tables by date or range. Reduce scan size within the same DB.
5. Shard only when single-node approaches are exhausted. Only when you truly need it.

Interview Tradeoffs You Should Know**MORE INDEXES VS FEWER INDEXES**

More indexes speed up reads but slow down writes and increase storage. Add only what your queries actually need.

SYNC VS ASYNC REPLICATION

Sync gives you zero data loss on failover but adds write latency. Async is faster but replicas can lag by seconds.

PARTITIONING VS SHARDING

Partitioning is simpler: one machine, same DB. Sharding gives more scale but adds routing complexity and loses JOIN capability.

CACHE-ASIDE VS WRITE-THROUGH

Cache-aside is simpler and works for read-heavy patterns. Write-through keeps cache always fresh but costs more on writes.

VERTICAL SCALING VS HORIZONTAL SCALING

Vertical scaling (bigger server) is simple and fast to apply, but has a ceiling and single point of failure. Horizontal (more nodes) scales indefinitely but needs distributed coordination.

What Can Go Wrong

In interviews, showing you've thought about failure modes is as important as showing you know the happy path.

PRIMARY DB GOES DOWN

Replica promotion kicks in. Managed services (RDS Multi-AZ) do this automatically in ~30-60 seconds. You need health checks and a load balancer or proxy that detects the failover.

REPLICA FALLS BEHIND

You start seeing stale reads. Detect this by monitoring replication lag in your DB metrics. Route time-sensitive reads to the primary, or alert and investigate the bottleneck.

HOT SHARD AFTER SHARDING

One shard is getting 90% of traffic. Usually caused by a bad shard key. Fix: re-shard with a better key or add a hash prefix to the shard key to distribute load.

CACHE INVALIDATION BUG

Old data in Redis after a write. Users see stale product prices or profile data. Fix: always evict or update the cache key on write. Use short TTLs on volatile data.

N+1 AT SCALE

Your app worked fine at 1000 users but at 1 million, it fires 50 DB queries per request. The DB gets overwhelmed. Fix with JOIN FETCH, @EntityGraph, or DataLoader patterns.

Module 7 Quick Reference

DATABASE SCALING CHEAT SHEET

INDEXING

Use when: slow reads on large tables

Types: B-Tree (default), Hash, Composite, Partial, Covering

Cost: slows down writes. Add only where queries need them.

Spring Boot: `@Index` on `@Table`, or Flyway migration

REPLICATION

Use when: too many reads for one DB node

Primary handles writes. Replicas handle reads.

Async replication: faster writes, small risk of lag

Spring Boot: `AbstractRoutingDataSource` or `@Transactional(readOnly=true)`

PARTITIONING

Use when: one table has 50M+ rows, queries filter by partition key

Same machine, different table chunks. No cross-node routing needed.

Types: Range (date), Hash, List (region)

SHARDING

Use when: single server can't hold the data or write volume

Different physical machines. Pick shard key carefully.

Problem: cross-shard joins. Co-locate related data by shard key.

Spring Boot: `ShardingSphere`, `Vitess`, or `AbstractRoutingDataSource`

CACHING (before DB scaling)

Redis with `@Cacheable` absorbs 80% of read traffic first.

Cache-aside: check Redis first, fallback to DB.

Always consider caching before adding read replicas.

BOTTLENECKS TO KNOW

N+1 queries: fix with `JOIN FETCH` or `@EntityGraph`

Connection pool exhaustion: tune `HikariCP` pool size

Long transactions with external calls: move async work outside `@Transactional`

DATABASE SCALING LADDER

Try each rung before moving to the next

1

Fix slow queries with indexes

Free fix. Check EXPLAIN ANALYZE first. Add B-Tree indexes on WHERE / JOIN columns.

2

Add Redis caching for repeated reads

Absorbs 80% of read traffic before it hits the DB. Use @Cacheable in Spring Boot.

3

Add read replicas for read-heavy traffic

Route SELECT queries to replicas. Primary handles writes only.

4

Partition large tables

Same machine, split into time or range buckets. Works within a single DB node.

5

Shard only when single-node options are exhausted

Different machines, different nodes. Adds routing complexity and loses JOIN support.

FINAL THOUGHT

Every rung on the ladder adds operational complexity. Indexes are free. Caching is easy. Read replicas are manageable. Partitioning adds some config. Sharding is a major commitment.

In interviews, showing you know this order and can justify each decision is what separates a good answer from a great one.

Next: Module 8 covers caching strategies in depth, including cache eviction, TTL design, and common caching patterns in microservices.

MODULE 8

Caching with Redis and Spring Boot

Redis fundamentals, Spring Cache, @Cacheable, eviction strategies, cache invalidation, and when NOT to cache.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

MODULE OVERVIEW

Module 8 Overview

Ten topics, building from the basic question of why caching exists all the way to how to defend your caching decisions in an interview.

Topic 1 Why Caching Exists	Topic 2 Redis Fundamentals
Topic 3 Cache-Aside Pattern	Topic 4 Spring Cache and @Cacheable
Topic 5 Cache Eviction Strategies	Topic 6 Cache Invalidation and Stampede
Topic 7 When NOT to Cache	Topic 8 What Happens When Redis Goes Down
Topic 9 How to Talk About Caching in Interviews	Topic 10 Quick Revision

WHAT THIS MODULE COVERS

- Why caching exists and what problem it actually solves in a backend system
- What Redis is under the hood, and why backend teams reach for it specifically
- The cache-aside pattern, and how it maps to a real Spring Boot service
- Spring's caching abstraction: @Cacheable, @CachePut, @CacheEvict
- Eviction strategies: TTL, LRU, LFU, and how Redis decides what to drop
- Cache invalidation, stale data, and the cache stampede problem
- When NOT to cache, and why this question matters more than people think
- What happens to your service when Redis goes down
- How to talk about caching decisions in a system design interview

TOPIC 8.1

Why Caching Exists

Picture a fairly normal Spring Boot service. It has an endpoint that returns a product's details. Nothing fancy, a Controller calls a Service, the Service calls a Repository, the Repository runs a query against Postgres or MySQL. It works fine in your local setup and it works fine in the demo.

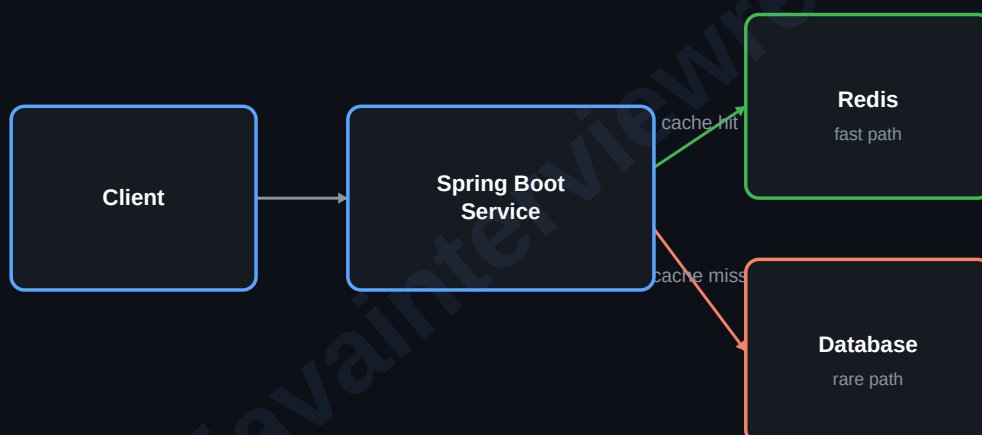
Now this product becomes popular. The same product page is opened by thousands of users every minute. Each one of those requests goes through the same path: Controller, Service, Repository, database. The database is doing the same read, over and over, for data that has not changed in the last ten minutes.

This is the exact moment where most freshers say "we will just scale the database" and stop there. That is not wrong, but it is not the first move either. Scaling a database is expensive and slow to react. Adding a cache in front of it is cheap, fast, and solves the actual problem, which is that you are repeating work that does not need to be repeated.

The core idea behind caching

Caching means keeping a copy of frequently accessed data somewhere closer and faster than its original source, usually in memory, so that repeated requests for the same data do not have to pay the full cost again. The database remains the source of truth. The cache is just a fast, temporary shortcut to data you have already fetched once.

In real projects, this usually becomes a problem the moment a single piece of data is read far more often than it is written. A product catalog, a user's profile, a leaderboard, a configuration value, all of these get read thousands of times for every one time they actually change. That gap between reads and writes is exactly where caching earns its place.

WITHOUT A CACHE**WITH A CACHE IN FRONT OF THE DATABASE****Small scale vs large scale**

At small scale, with a handful of users, you will not notice this problem at all. The database can comfortably answer a few requests per second without anyone complaining. This is exactly why freshers sometimes struggle to explain why caching matters, they have never personally felt the database groan under load.

At larger scale, the same read can be requested hundreds or thousands of times per second. Without a cache, every single one of those requests becomes a database query, a network round trip, disk access, and query execution. The database becomes the slowest and most fragile part of your system, not because the design is bad, but because it is doing work it does not need to repeat.

INTERVIEW FOCUS

Most freshers answer “why do we cache” with “to make it faster.” That answer is not wrong, but interviewers want to hear the reasoning behind it: you are reducing repeated load on a slower resource (the database) for data that is read far more often than it changes. Mentioning the read-to-write ratio is what separates a memorized answer from an understood one.

TOPIC 8.2

Redis Fundamentals

You could technically build a cache with a plain Java HashMap sitting inside your service. People do this for small, single-instance applications. The problem shows up the moment you run more than one instance of your service, which in production you almost always will. Each instance would have its own HashMap, with its own copy of the data, and no way to know what the others are holding.

Redis solves this by living outside your application, as a separate in-memory data store that every instance of your service can talk to. All instances see the same cached data, because the data lives in one shared place instead of inside each JVM.

What Redis actually is

Redis is an in-memory key-value store. Data is held in RAM, which is what makes reads and writes extremely fast compared to a disk-backed database. It also supports persistence to disk, through snapshotting (RDB) or an append-only log (AOF), so a restart does not always mean losing everything, though most teams treat cached data as something that can be safely rebuilt if it is lost.

Redis is also single-threaded for command execution, which sounds like a limitation but is actually a big part of why it stays predictable. There is no internal locking between threads fighting over the same key, so individual commands are fast and consistent. Heavy use of Redis usually scales by running multiple Redis nodes, not by adding threads inside one.

Beyond simple key-value pairs

A common misconception among freshers is that Redis only stores plain strings. In reality it supports several data structures, and picking the right one for the job matters in interviews:

- **String:** the simplest form, used for cached JSON blobs, counters, session tokens
- **Hash:** a map of fields inside one key, useful for caching an object like a user profile
- **List:** ordered collection, useful for recent activity feeds or simple queues
- **Set:** unordered unique values, useful for tags or membership checks

- **Sorted Set:** values with a score, this is what powers leaderboards and ranking features

In-Memory HashMap	Redis
Lives inside one JVM instance	Lives outside the application, shared across instances
Lost the moment you scale horizontally	Stays consistent across every running instance
No expiry or eviction support by default	Built-in TTL and eviction policies
Zero network cost, but zero sharing	Small network hop, but data is shared and durable

JAVA + SPRING BOOT MAPPING

Spring Boot talks to Redis through **Spring Data Redis**, usually with the **Lettuce** client (the default since Spring Boot 2). You add the `spring-boot-starter-data-redis` dependency, configure `spring.data.redis.host` and `spring.data.redis.port`, and Spring auto-configures a `RedisConnectionFactory` and a `RedisTemplate` for you. From there, either use `RedisTemplate` directly for fine control, or use Spring's caching annotations (covered next) for a cleaner, declarative approach.

In interviews, when someone asks “why Redis and not just Memcached,” the honest answer is that Memcached is simpler and purely a cache, while Redis gives you richer data structures, persistence options, pub/sub, and built-in support for patterns like leaderboards and rate limiters. If your need is genuinely simple key-value caching at very large scale, Memcached is a fair answer too. The tradeoff is simplicity versus flexibility.

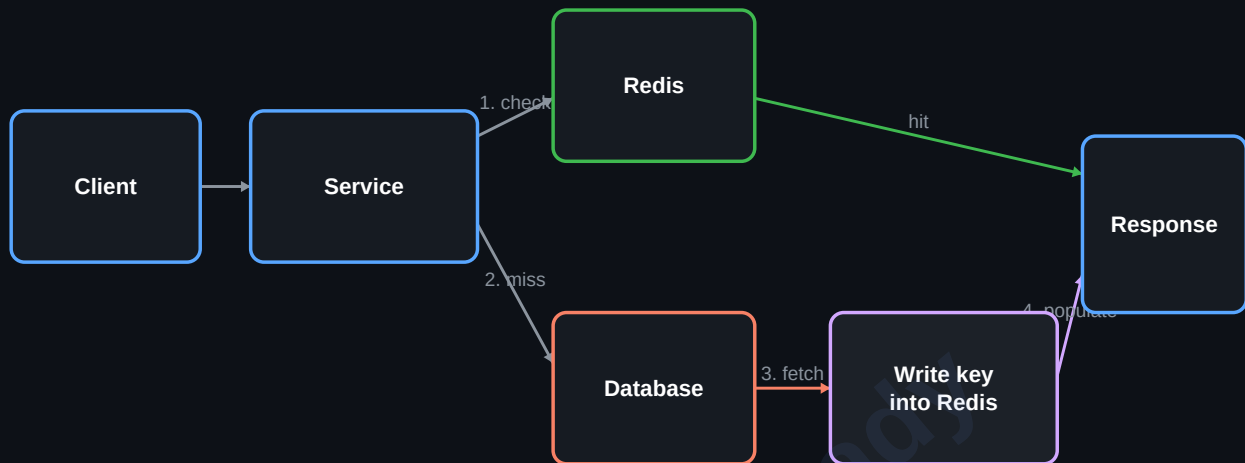
TOPIC 8.3

Where Caching Fits: The Cache-Aside Pattern

If I were designing this in Spring Boot, I would almost always start with the cache-aside pattern, also called lazy loading. It is the pattern you will use the most in real projects, and it is also the one interviewers expect you to be able to draw and explain without hesitation.

How cache-aside actually works

The application, not Redis, is responsible for managing the cache. When a request comes in, the service first checks Redis. If the data is there (a cache hit), it is returned immediately and the database is never touched. If the data is missing (a cache miss), the service falls back to the database, fetches the data, stores a copy of it in Redis for next time, and then returns it to the caller.

CACHE-ASIDE (LAZY LOADING) FLOW

Notice what this means for the first request after the cache is empty, or after a key has expired. That first request is always slower, because it has to pay the full database cost plus the cost of writing into Redis. Every request after that, until the key expires again, is fast. This is exactly the tradeoff you are accepting with cache-aside, and it is worth saying out loud in an interview rather than pretending caching makes every single request fast.

The other two patterns, briefly

You do not need to implement these for most fresher-level interviews, but you should be able to name them and explain the difference if asked:

- **Write-through:** every write goes to the cache and the database together, so the cache is always fresh, at the cost of slower writes
- **Write-behind (write-back):** writes go to the cache first and are flushed to the database later in the background, fast writes but a real risk of losing data if the cache crashes before the flush happens

TRADEOFF: CACHE-ASIDE VS WRITE-THROUGH

Cache-aside keeps your database as the single source of truth and only fails soft, a cache miss just means a slower request, not a wrong one. The cost is that the first reader after an eviction always pays full price, and a busy key that keeps expiring can repeatedly hammer the database. Write-through avoids stale reads but slows down every write. There is no universally correct choice here, the right one depends on whether your workload is read-heavy or write-heavy.

TOPIC 8.4

Spring Cache Abstraction and @Cacheable

Writing the cache-aside logic by hand, checking Redis, falling back to the database, writing the result back, is not hard, but it adds boilerplate to every method that needs caching. Spring's caching abstraction removes that boilerplate. You annotate a method, and Spring handles the check-fetch-store logic around it for you.

Enabling caching

APPLICATION CLASS

```
@SpringBootApplication
@EnableCaching
public class PlaybookApplication {
    public static void main(String[] args) {
        SpringApplication.run(PlaybookApplication.class, args);
    }
}
```

With Redis on the classpath and configured, Spring auto-configures a `RedisCacheManager` as the backing store for the cache abstraction. That means `@Cacheable` and friends end up storing their data in Redis without you writing any Redis-specific code inside your service.

@Cacheable: cache a method's return value

PRODUCTSERVICE.JAVA

```
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    public Product getProductById(Long id) {
        // Only runs on a cache miss
        return productRepository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }
}
```

The first time `getProductById(101)` is called, the method body actually runs and the result is stored in Redis under the cache name `products` with key `101`. The next call with the same `id` skips the method entirely and returns straight from Redis. This is the cleanest way to apply cache-aside in Spring Boot, the annotation is doing exactly what you would have written by hand.

@CachePut, @CacheEvict, and @Caching

`@Cacheable` alone is not enough once your data can change. You need a way to update or remove a cached value when the underlying row changes, otherwise you end up serving stale data forever.

PRODUCTSERVICE.JAVA (WRITE PATHS)

```

@CachePut(value = "products", key = "#product.id")
public Product updateProduct(Product product) {
    // Always runs, and the return value replaces the cached entry
    return productRepository.save(product);
}

@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {
    // Always runs, and the matching cache entry is removed
    productRepository.deleteById(id);
}

@Caching(evict = {
    @CacheEvict(value = "products", key = "#id"),
    @CacheEvict(value = "productList", allEntries = true)
})
public void archiveProduct(Long id) {
    productRepository.archive(id);
}

```

Annotation	Runs the method?	What it does
@Cacheable	Only on a miss	Returns cached value if present, otherwise runs and stores result
@CachePut	Always	Always runs the method, then updates the cache with the new result
@CacheEvict	Always	Always runs the method, then removes the matching cache entry
@Caching	Always	Groups multiple cache operations on a single method

A mistake many freshers make is putting `@Cacheable` on the update method as well, thinking it will “keep things cached.” That is backwards. `@Cacheable` on a write path would skip the actual update on a cache hit, which is exactly wrong. Reads use `@Cacheable`, writes use `@CachePut` or `@CacheEvict`.

TIP

Use `unless` or `condition` on `@Cacheable` to avoid caching things you should not, for example `@Cacheable(value = "products", key = "#id", unless = "#result == null")` stops Spring from caching a null result, which would otherwise keep getting returned for an id that genuinely does not exist yet but might be created later.

TOPIC 8.5

Cache Eviction Strategies

RAM is not free and not unlimited. A Redis instance is given a fixed amount of memory, and at some point new data needs to come in while old data needs to leave. Deciding what leaves is the eviction strategy, and interviewers like asking about this because it reveals whether you understand caching as a system, not just an annotation you copy-pasted.

Time-based expiry: TTL

The simplest and most common approach is giving every cached key a Time To Live. After that duration, Redis removes the key automatically, whether or not memory is under pressure. TTL is less about memory management and more about freshness, you are saying “this data is allowed to be wrong for at most N minutes.”

TTL CONFIGURATION

```
@Cacheable(value = "products", key = "#id")
public Product getProductById(Long id) { ... }

// Configured centrally, per cache name, in your RedisCacheManager bean
RedisCacheConfiguration config = RedisCacheConfiguration
    .defaultCacheConfig()
    .entryTtl(Duration.ofMinutes(10));
```

Memory-based eviction: LRU and LFU

When Redis actually runs out of allowed memory, TTL alone is not enough, some keys without an expiry might still need to go. This is where Redis's `maxmemory-policy` setting decides what gets dropped first:

- **LRU (Least Recently Used):** removes the key that has not been accessed for the longest time, the most common default in practice
- **LFU (Least Frequently Used):** removes the key with the fewest total accesses, useful when some keys are accessed rarely but recently
- **Random:** removes a random key with an expiry set, cheap but unpredictable
- **noeviction:** refuses new writes once memory is full instead of dropping anything, used when losing cached data silently is not acceptable

FRESHNESS POLICY

Policy	Removes	Good fit when
TTL expiry	Keys past their set lifetime	Data has a known freshness window

MEMORY-PRESSURE POLICIES (maxmemory-policy)

Policy	Removes	Good fit when
allkeys-lru	Least recently used key, any key	General-purpose caching, most common choice
allkeys-lfu	Least frequently used key, any key	Some keys are hot and reused constantly
volatile-lru	Least recently used key, with a TTL set	Mixed cache + persistent keys in same instance
noeviction	Nothing, rejects new writes instead	You would rather fail loudly than serve nothing

INTERVIEW MOMENT

If an interviewer asks “how does Redis decide what to evict,” naming LRU alone is an okay answer. Naming LRU, mentioning that it is configurable through maxmemory-policy, and explaining that you would still set explicit TTLs for freshness regardless of the eviction policy, is a much stronger answer. It shows you see TTL and eviction as two separate concerns, freshness versus memory pressure, not one setting.

TOPIC 8.6

Cache Invalidation and the Stampede Problem

Caching is easy to explain and surprisingly easy to get wrong in production. Most of the pain does not come from reading data, it comes from keeping cached data in sync with the database, and from what happens at the exact moment a popular key disappears.

Invalidation: telling the cache something changed

When a row in your database changes, the cached copy of it does not know that on its own. Someone has to remove it or update it, either through `@CacheEvict` / `@CachePut` on the write path, or by letting TTL expire it eventually. Skipping this step is how teams end up serving a user their old cart total or an old order status long after it actually changed.

Many beginners design this directly as “cache everything, evict on every write,” but that creates an issue later when a single write needs to invalidate several related cache entries, a product update might need to clear the product detail cache, the category listing cache, and the search index cache all at once. This is exactly why `@Caching` with multiple `@CacheEvict` entries exists.

The cache stampede problem

Now picture a single, very popular key, say the homepage banner data, with a 5 minute TTL. The moment it expires, if your service is handling a thousand requests per second, all thousand of them can miss the cache at almost the same instant. Every single one falls through to the database at once, for the exact same piece of data. This sudden spike is called a cache stampede, and it can take down a database that was otherwise handling load just fine.

CACHE STAMPEDE: ONE EXPIRED KEY, EVERY REQUEST MISSES AT ONCE



How this is actually handled

- **Request coalescing / locking:** the first request that misses takes a short lock and fetches from the database, others wait briefly and reuse that one result instead of all hitting the database themselves
- **Probabilistic early expiration:** refresh the key slightly before it actually expires, with some randomness, so not every instance refreshes at the exact same second
- **Staggered TTL:** add a small random offset to TTLs across similar keys so they do not all expire at the same moment

FAILURE SCENARIO

A cache stampede is one of the few caching failures that can be worse than having no cache at all, because the database gets hit by a sudden, synchronized spike instead of a smooth, distributed load. At a fresher level, simply being aware this can happen, and being able to name request coalescing or staggered TTLs as a fix, already puts you ahead of most candidates.

TOPIC 8.7

When NOT to Cache

This is the topic that actually separates strong candidates from average ones. Almost everyone can explain what caching is. Far fewer people can explain where it stops making sense, and an interviewer who hears you reach for Redis for absolutely everything will start wondering if you understand it at all.

Data that changes faster than it is reused

If a value changes on almost every write and is rarely read more than once before it changes again, caching adds overhead without adding benefit. A live stock ticker updating every second, or a counter that increments on every single request, gains very little from a cache layer that needs to be invalidated just as often as it is read.

Data that must always be correct, right now

A bank account balance right before a withdrawal is a good example. Serving a cached, slightly stale balance could let someone withdraw money they no longer have. Anywhere strong consistency matters more than speed, payments, inventory counts during checkout, seat availability at the exact moment of booking, you either skip the cache for that specific read or accept a very short TTL and back it with strict validation at write time.

Data with low reuse

Caching only pays off when the same data is requested again before it changes or expires. A report generated for one user, one time, with no one else likely to request the exact same report, is not a good caching candidate. You would be spending memory to store something that will probably never be read a second time.

Very large objects

Caching a 50 MB file or a huge serialized object eats memory fast and defeats the purpose of an in-memory store, which is meant to hold many small, hot pieces of data, not a handful of large ones. Large binary content usually belongs in object storage with a CDN in front of it, not in Redis.

Good fit for caching	Bad fit for caching
Read far more often than written	Written almost as often as read
Stale-for-a-bit is acceptable	Must be correct at the instant of read
Same data requested repeatedly	Requested once, by one user, rarely again
Small, frequently accessed objects	Very large files or blobs

WATCH OUT

If you find yourself adding a cache and then immediately writing complicated logic to keep it perfectly in sync with every possible write, that is usually a sign the data was not a good caching candidate in the first place. The fix is often to cache something else, like a slower-changing summary, rather than caching the volatile value directly.

TOPIC 8.8

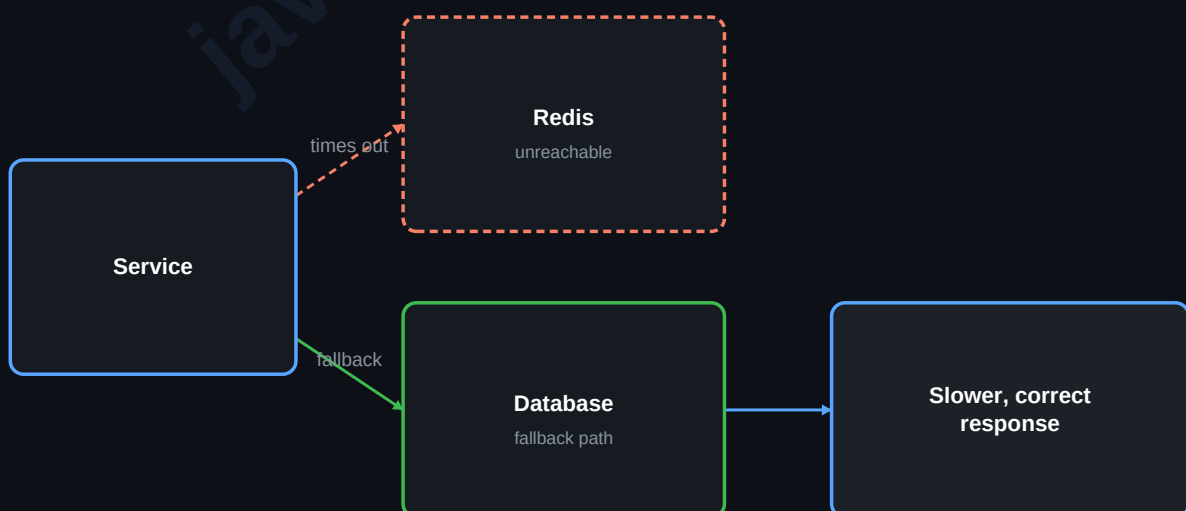
What Happens When Redis Goes Down

A question that comes up a lot in interviews, and that freshers rarely prepare for, is simply: what happens to your service if Redis becomes unreachable. The honest, good answer is that your service should keep working, just slower, not crash.

Treat the cache as optional, never as a dependency

A cache should never be something your application cannot function without. If a Redis call fails, times out, or refuses a connection, the correct behavior is to fall back to the database directly, log the failure, and continue serving the request. Users feel a slower response. They should never see an error page because the cache, a performance optimization, happened to be unavailable.

GRACEFUL DEGRADATION WHEN REDIS IS UNAVAILABLE



Circuit breakers stop a slow cache from making things worse

A Redis instance that is slow to respond, not fully down, is actually more dangerous than one that is completely unreachable. Threads can pile up waiting on a slow cache call, which can exhaust your connection pool and slow down the whole service. A circuit breaker watches for repeated failures or timeouts and, once a threshold is crossed, stops calling Redis for a short period, sending every request straight to the database instead.

RESILIENCE4J CIRCUIT BREAKER AROUND THE CACHE CALL

```
@CircuitBreaker(name = "redisCache", fallbackMethod = "getProductFromDb")
public Product getProductById(Long id) {
    return redisCacheService.get(id);
}

public Product getProductFromDb(Long id, Throwable t) {
    return productRepository.findById(id)
        .orElseThrow(() -> new ProductNotFoundException(id));
}
```

JAVA + SPRING BOOT MAPPING

In Spring Boot, this protective layer is usually built with **Resilience4j**. It can wrap the Redis call with a circuit breaker, a timeout, and a fallback method, all configured through properties or annotations, without you writing manual retry loops or try-catch blocks scattered across your service.

There is also a real risk worth naming: if Redis goes down during a traffic spike, every request that would have been a cache hit suddenly becomes a database read. This can look very similar to a cache stampede, except triggered by an outage instead of an expiry. Teams that run Redis in production usually plan for this with database connection pool limits and rate limiting, so a Redis outage degrades performance instead of taking the whole system down.

TOPIC 8.9

How to Talk About Caching in an Interview

By this point you understand the mechanics. This section is about presentation, because two candidates who know exactly the same things about Redis can come across very differently based on how they explain their thinking.

What most freshers say

“We will add Redis caching to make it fast.” That sentence is technically true and says almost nothing. It does not say what is being cached, why, for how long, or what happens when it goes wrong.

What interviewers actually want to hear

Something closer to: “Since product details are read far more often than they change, I would cache the product object in Redis using cache-aside, with a TTL of around 10 minutes. On a write, I would evict that key explicitly through CacheEvict rather than waiting for the TTL, since stale product data after an update would be a noticeable bug.” That single sentence covers what, why, the pattern, the freshness policy, and the write-path handling, in plain language.



A realistic follow-up, and how to handle it

A common follow-up is: “what if two requests update the same product at almost the same time, and both try to refresh the cache?” You do not need a perfect answer. A solid one acknowledges the race, mentions that the last write to Redis wins, and that this is usually acceptable for a cache since the database write order is what actually determines correctness, the cache is just trailing behind it briefly.

INTERVIEW FOCUS

At your level, you are not expected to design a flawless distributed caching system from memory. You are expected to reason out loud: name the pattern, name the tradeoff, and show you know caching can fail. “At my level, I would first explain the simple design and then improve it step by step” is a completely fair thing to say, and it usually lands better than pretending you have a perfect answer for every edge case.

TRADEOFF: CACHE READ VS DATABASE READ

If asked to compare a cache read against a database read directly, the honest framing is: a cache trades a small chance of staleness for a large drop in latency and database load. A database read trades that speed for always being current. Most systems do not need to choose one everywhere, they cache the read-heavy paths and go straight to the database for the few paths that truly need real-time correctness.

TOPIC 8.10

Quick Revision

Use this page the night before an interview. If you can explain every row below out loud, in your own words, without checking the rest of the module, you are ready for this topic.

Concept	One-line takeaway
Why cache	Avoid repeating reads against a slower resource for data that rarely changes
Redis	Shared, in-memory key-value store, multiple data structures, sits outside the JVM
Cache-aside	Check cache, on a miss fetch from DB and populate cache, then return
@Cacheable	Caches a read method's result, skips the method body on a hit
@CachePut / @CacheEvict	Used on write paths, always run the method, then update or remove the entry
TTL	Time-based freshness control, independent of memory pressure
LRU / LFU	Memory-based eviction policy, decides what to drop when Redis is full
Cache stampede	Many requests miss an expired popular key at once, overload the DB
When not to cache	Write-heavy, strongly consistent, low-reuse, or very large data
Redis down	Fall back to the DB directly, degrade gracefully, never hard-fail

TIP

If you remember nothing else from this module, remember this sentence and be ready to defend it: "I cache data that is read far more often than it changes, I keep the database as the source of truth, and I make sure my service still works correctly, just a little slower, if the cache is not there." That sentence alone answers most of what this module covered.

WHAT'S NEXT

Next: Module 9 explains message queues and async communication, including Kafka basics, producer-consumer model, Spring Boot + Kafka integration, and real-world use cases.

Message Queues & Async Communication

Why async beats direct API calls, Kafka basics, producer-consumer model, Spring Boot + Kafka integration, and real use cases.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

MODULE OVERVIEW

What This Module Covers

This module covers one of the most important concepts in distributed systems: async communication. You'll learn why synchronous API chains fail at scale, how message queues solve those problems, and how Kafka fits into real Java backend systems.

#	Topic
01	The Problem With Direct API Calls
02	What Is a Message Queue?
03	Sync vs Async Communication
04	Kafka Deep Dive
05	Producer-Consumer Model
06	Topics, Partitions and Consumer Groups
07	Spring Boot + Kafka Integration
08	Real-World Use Cases
09	Failure Handling in Async Systems
10	When NOT to Use Kafka
11	Interview Tradeoffs

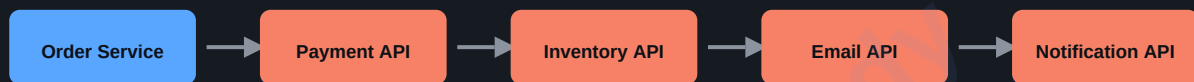
TOPIC 01

The Problem With Direct API Calls

Picture this: a user places an order. Your Order Service needs to charge payment, reduce inventory, send a confirmation email, update analytics, and push a notification to the delivery partner. All from one request.

The easiest code is direct HTTP calls. Order Service calls Payment Service, waits, then Inventory, waits, then Notification, waits. Five services, five network hops, one user request blocking on all of them.

DIRECT CALL CHAIN (THE PROBLEM)



Every service must be healthy for a single order to succeed. One slow service blocks the entire chain.

WARNING

This is the trap most freshers fall into. They chain service calls and never think about what happens when one is slow or down.

If Payment takes 3s and Notification takes 2s, the user waits 5+ seconds. If Notification is down, the whole order fails even though payment already succeeded.

Why This Breaks at Scale

- **Tight coupling:** Notification Service down = order fails. That should never happen.
- **Cascading failures:** One slow downstream service makes everything upstream slow.
- **Wasted user latency:** User waits for analytics updates they'll never see in real time.
- **Retry chaos:** Each service needs to know the retry policies of every dependency it calls.

INTERVIEW MOMENT

Q: 'How would you improve reliability in a microservices system?'

Wrong: 'Add more retries and timeouts.'

Better: 'Separate the critical path from non-critical operations. The order confirmation must be fast and synchronous. Background tasks like notifications and analytics go into a message queue and get handled asynchronously.'

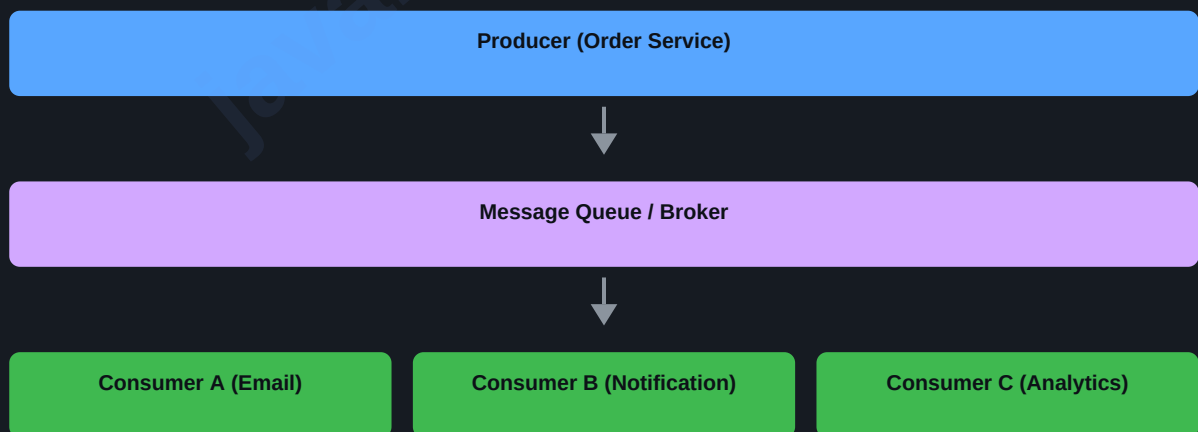
TOPIC 02

What Is a Message Queue?

A message queue is a buffer between services. One service writes a message to the queue and moves on. Another picks it up and processes it at its own pace. The sender and receiver never talk directly.

That's the key idea: **decoupling the producer from the consumer**. The queue holds the message until someone is ready to handle it.

MESSAGE QUEUE: CORE IDEA



Producer doesn't wait. If Consumer B crashes, A and C still work fine.

Key Properties

- **Decoupling:** Producer and consumer don't need to know about each other.

- **Durability:** Messages stored on disk. Consumer crash doesn't lose messages.
- **Buffering:** Slow consumers don't cause producer timeouts. Messages wait.
- **Fan-out:** One message consumed by multiple independent consumers.

TIP

Popular options: Kafka, RabbitMQ, Amazon SQS, Google Pub/Sub.

In interviews, Kafka is the safest choice for high-throughput distributed systems. RabbitMQ works well for simpler task queue scenarios. Mention both and explain the tradeoff - that's what interviewers want to hear.

TOPIC 03

Sync vs Async Communication

Not every operation should be async. Knowing when to use which one is what separates a thoughtful answer from a generic one.

SYNC VS ASYNC COMMUNICATION

Synchronous

- Caller waits for response
- Tight coupling between services
- Simple to debug
- Fails if downstream is down
- Good for auth, payment, DB reads

Asynchronous

- Caller does not wait
- Queue-based communication
- Harder to trace
- Retries handled by consumer
- Good for emails, notifications, analytics

The Critical Path vs Background Tasks

Any user action has two parts: the critical path (must be fast and synchronous) and background work (can be async). The rule is simple - if the user needs to see the result immediately, keep it sync. Everything else can be async.

SYNC VS ASYNC DECISION

User login	→	Sync: real-time auth check needed
Send email OTP	→	Async: queue + background sender
Place order + pay	→	Sync: needs confirmation
Order confirm email	→	Async: side effect after payment OK
Update analytics	→	Async: never user-facing
Check inventory	→	Sync: needed before order proceeds
Push notification	→	Async: non-critical, can retry

INTERVIEW FOCUS

Follow-up: 'How do you decide what to make async?'

If the user needs the result to continue their action, it must be sync. If it is a side effect that happens after the main action, make it async.

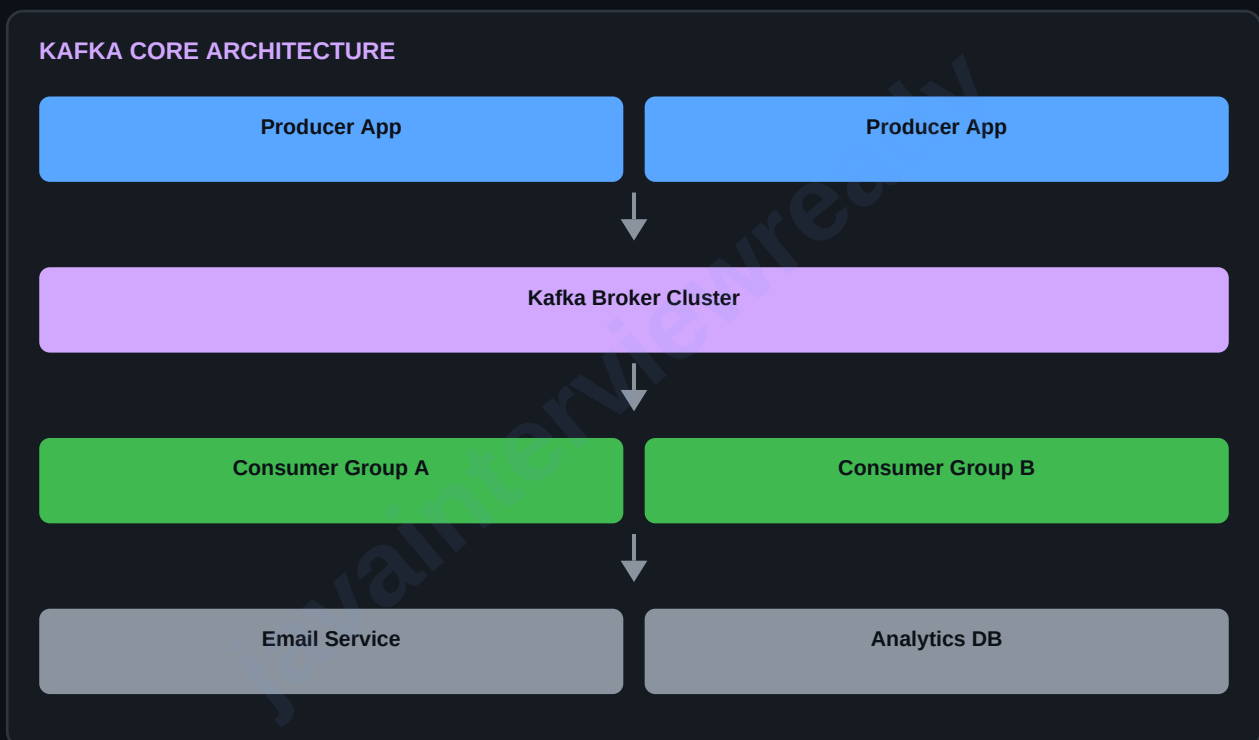
Ask: 'Does the user need to wait for this?' If not, put it in a queue.

TOPIC 04

Kafka Deep Dive

Apache Kafka is not a traditional message queue. It's an event streaming platform. The difference matters. In a traditional queue, once a message is consumed it's gone. In Kafka, messages stay on disk for a configurable retention period. Multiple consumers can read the same message independently.

Kafka was built at LinkedIn to handle billions of events per day. If scale is involved - millions of events - Kafka is almost always the right architecture choice.



Multiple producers write to Kafka. Multiple independent consumer groups each read the full stream independently.

Core Kafka Concepts

Broker

A Kafka broker is a server in the cluster. Kafka runs as 3+ brokers in production for fault tolerance. Each broker stores some partitions.

Topic

A named stream of messages - like a log file that services write to and read from. Examples: **order-events**, **payment-events**, **notification-events**.

Partition

Each topic splits into partitions. A partition is an ordered, immutable sequence. Messages in a partition are always read in order. Partitions enable horizontal scaling - each one can live on a different broker.

Offset

Each message in a partition has a unique sequential number - its offset. Consumers track their offset. If a consumer crashes, it restarts from where it left off.

TIP

In interviews, skip Zookeeper and KRaft internals. Focus on: topics, partitions, producers, consumers, consumer groups, offsets, and retention.

Those seven concepts cover 90% of Kafka interview questions at the 1-3 year level.

TOPIC 05

Producer-Consumer Model

The producer publishes messages to a topic. The consumer subscribes and processes them. Simple in theory, but the details matter in interviews.

PRODUCER TO CONSUMER FLOW



How the Producer Works

The producer picks the topic and partition. By default, Kafka distributes messages in round-robin. If you provide a **message key**, all messages with that key go to the same partition - that's how you get ordering for a specific entity like a user or order.

WARNING

Common fresher mistake: assuming Kafka guarantees global ordering across all partitions.

It doesn't. Kafka only guarantees ordering within a single partition.

If you need all events for user ID 42 in order, use the user ID as the partition key. All those messages route to the same partition.

How the Consumer Works

Consumers pull messages from Kafka - it's pull-based, not push. They read at their own pace and commit offsets to tell Kafka where they got to.

Committing offsets can be automatic or manual. In production, use manual commit. Process the message first, then commit the offset. If you auto-commit and then crash mid-processing, that message is lost.

Behavior	Auto Commit	Manual Commit
When offset is saved	Periodically, time-based	After message is processed
Risk of message loss	Yes - crash before processing finishes	No
Risk of duplicate processing	Low	Yes - crash after process, before commit
Best for	Non-critical high-throughput logs	Critical ops like payments and orders

TOPIC 06

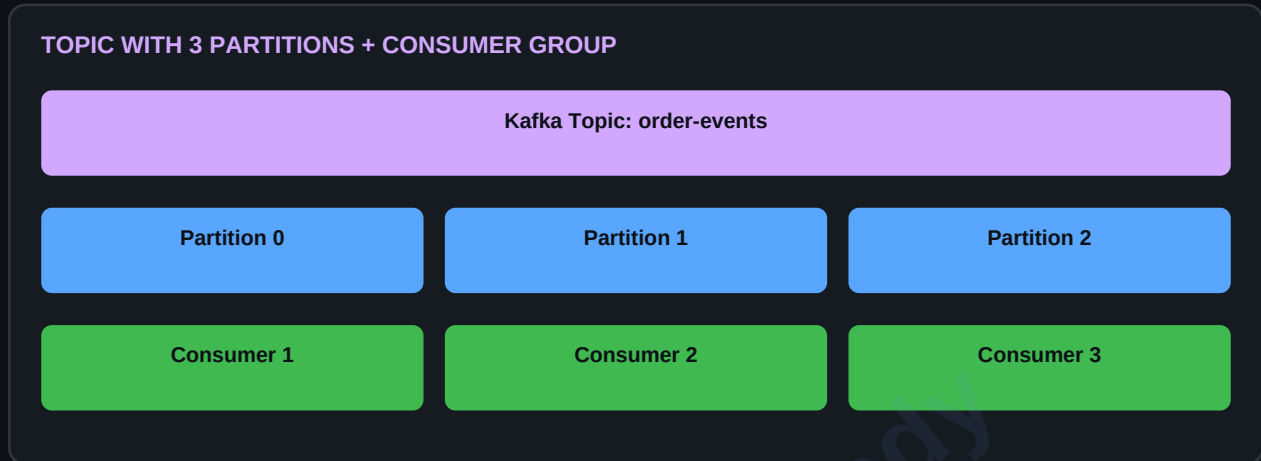
Topics, Partitions & Consumer Groups

This is where Kafka gets powerful for scale. Understanding partitions and consumer groups is what separates a surface answer from a solid one in interviews.

Partitions and Parallelism

More partitions = more parallelism. A topic with 3 partitions lets 3 consumers read in parallel - each handles one partition. You don't make one consumer faster. You add more consumers and more

partitions.

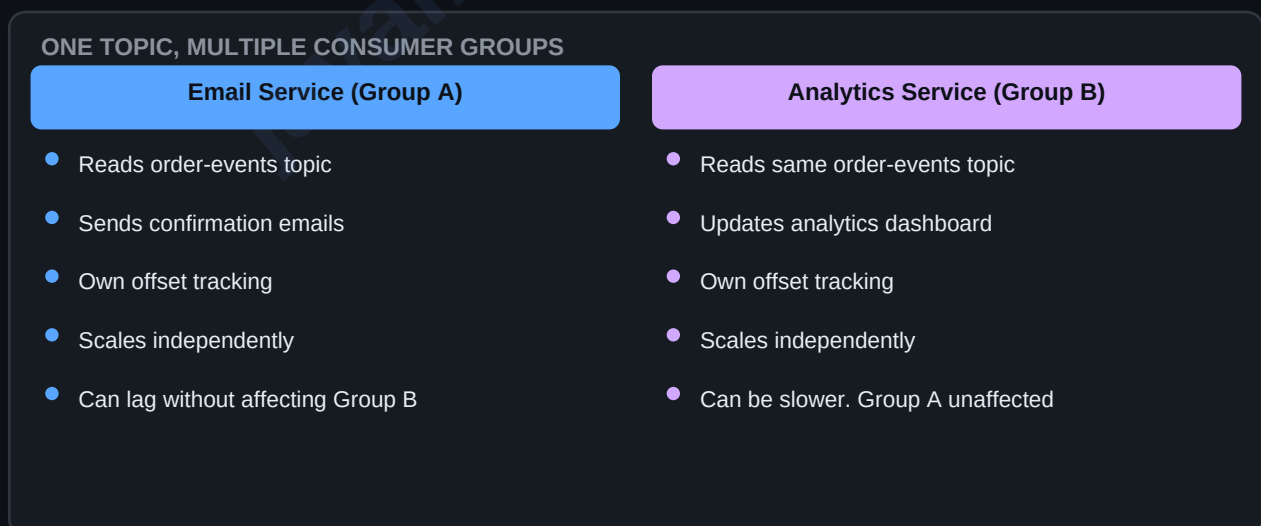


3 partitions = max 3 active consumers in a group. Extra consumers beyond that number sit idle.

Consumer Groups

A consumer group is a set of consumers working together to consume a topic. Each partition is assigned to exactly one consumer in the group. No duplicate processing within the group.

But different groups read the same topic independently. Email Service as Group A and Analytics as Group B both read every message from the same topic. Neither affects the other's offset.



INTERVIEW FOCUS

Q: 'What happens if I add more consumers than partitions?'

The extra consumers sit idle. Kafka assigns each partition to exactly one consumer in a group. 3 partitions + 5 consumers = 2 consumers doing nothing.

This is why partition count is a capacity planning decision you make upfront, not something you change easily later.

TOPIC 07

Spring Boot + Kafka Integration

Most candidates explain Kafka concepts fine but stumble when asked how it looks in actual Spring Boot code. This section covers everything you need to know.

Maven Dependency

```
<dependency>
<groupId>org.springframework.kafka</groupId>
<artifactId>spring-kafka</artifactId>
</dependency>
```

application.yml

```
spring:
kafka:
bootstrap-servers: localhost:9092
producer:
key-serializer: org.apache.kafka.common.serialization.StringSerializer
value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
consumer:
group-id: order-service-group
auto-offset-reset: earliest
key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
```

Producer: Publishing an Event

```
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderPlaced(OrderEvent event) {
        // key = orderId routes same order to same partition
        kafkaTemplate.send("order-events", event.getOrderId(), event);
    }
}
```

Consumer: Listening to Events

```
@Service
public class NotificationConsumer {
    @KafkaListener(topics = "order-events", groupId = "notification-group")
    public void handleOrderPlaced(OrderEvent event) {
        notificationService.sendConfirmation(event.getUserId());
    }
}
```

SPRING BOOT KAFKA INTEGRATION FLOW

REST Controller



Order Service



KafkaTemplate.send()



Kafka Broker



@KafkaListener Consumer

JAVA MAPPING

KafkaTemplate -> publish events from the Service layer
@KafkaListener -> consume events in any @Service or @Component
KafkaAdmin + @Bean NewTopic -> create topics on app startup
DefaultErrorHandler + DeadLetterPublishingRecoverer -> handle failures
ProducerFactory / ConsumerFactory -> configure serialization and connection

TOPIC 08

Real-World Use Cases

When you explain async communication in an interview, don't just say 'for decoupling.' Give concrete examples. Interviewers want to see that you know where this applies in real systems.

ASYNC USE CASE DECISION RULE

User needs immediate result

Keep it synchronous

Background side effect

Send to Kafka

Multiple consumers, one event

Use Kafka topics + groups

Simple one-worker task

RabbitMQ or SQS is enough

Use Case	Topic / Event	Why Async?
Order placed	order-events	Email, push notification, inventory update happen in background
User registered	user-registered	Welcome email, analytics event, referral check are not real-time
Payment completed	payment-events	Receipt email, loyalty points, seller notification are side effects
Video uploaded	video-upload-events	Transcoding, thumbnail, CDN push are async processing pipelines
Log aggregation	app-logs	100 services ship logs to Kafka, consumed by ELK or Splunk
Activity tracking	user-activity	Clicks and searches logged to Kafka for real-time analytics
Inventory update	inventory-events	Warehouse publishes changes, storefronts consume to refresh stock

Notification System - End to End

Here's how this pattern makes a notification system reliable. Order Service publishes one event. Three notification consumers (email, SMS, push) each belong to different consumer groups and process the event independently.

NOTIFICATION FLOW VIA KAFKA



If push notifications are down, email is completely unaffected. When push recovers, it reads from its last offset and catches up. The order itself was never at risk.

INTERVIEW MOMENT

Q: 'How would you design a notification system that doesn't block the main order flow?'

Answer: After the order is saved and payment confirmed, Order Service publishes an order-placed event to Kafka. A Notification Service is a Kafka consumer that reads the event and sends email, SMS, and push notifications. The user already got their confirmation on screen. Notifications are side effects in the background. If they fail, the consumer retries from the offset. The order is not affected.

TOPIC 09

Failure Handling in Async Systems

Async systems fail differently from sync ones. When an HTTP call fails, you get an exception immediately. When a Kafka consumer fails, messages can pile up silently, or the same message gets processed twice.

Design for these failure modes upfront. In interviews, the follow-up is almost always 'what happens when X fails?'

Consumer Crashes Mid-Processing

With manual offset commit, if the consumer crashes after processing but before committing, it re-processes the same message on restart. This is **at least once delivery**. Your consumer logic must be idempotent - processing the same message twice should produce the same result.

TIP

How to make consumers idempotent: check if the action was already done before doing it.

Before sending an email for order ID 12345, check if a record already exists in the DB for that order. If yes, skip it. If no, send and record it. Clean and simple duplicate handling.

Consumer Lag

Consumer lag means messages pile up in Kafka faster than the consumer can process them. Left unchecked: delayed notifications, stale data, and memory pressure on the broker.

Fix: add more consumers up to partition count, optimize processing logic, or switch to batch consumption. Monitor lag with Prometheus and Grafana in production.

Dead Letter Topic (DLT)

If a message keeps failing after retries, don't let it block the queue. Move it to a dead-letter topic. Engineers inspect it later, fix the issue, and replay.

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    DeadLetterPublishingRecoverer recoverer =
    new DeadLetterPublishingRecoverer(template);
    return new DefaultErrorHandler(
    recoverer, new FixedBackOff(1000L, 3)
    );
}
```

This retries 3 times with a 1-second backoff. After that, the message goes to the dead-letter topic automatically.

Failure	Detection	Recovery
Consumer crashes	Consumer group offset stalls	Restart - reads from last committed offset
Processing fails	Exception in @KafkaListener	Retry with backoff, then send to DLT
Kafka broker down	Producer gets timeout or error	Retry with acks=all, min.insync.replicas >= 2
Consumer lag spikes	Prometheus metric alert fires	Scale up consumers to match partition count
Duplicate messages	Same message processed twice	Idempotency check before acting on it

TOPIC 10

When NOT to Use Kafka

This separates a thoughtful engineer from someone who adds Kafka to everything. Kafka has real operational overhead - a distributed cluster that needs monitoring, replication, partition tuning, and consumer group management.

USE KAFKA VS USE SOMETHING SIMPLER

Use Kafka When

- High throughput events
- Multiple consumers need same event
- Message replay is needed
- Real-time streaming pipeline
- Event sourcing or audit log

Avoid Kafka When

- Low volume task queue
- Only one consumer per message
- Small team, no Kafka ops capacity
- Simple background jobs
- SQS or RabbitMQ is already available

Simpler Alternatives Worth Knowing

- **@Async + Spring Events:** In-process async. No infrastructure needed. Good for monolith or low-volume background tasks.
- **Spring @Scheduled:** Periodic background jobs like batch emails at midnight.
- **RabbitMQ:** Better for simple task distribution and point-to-point messaging. Much less setup than Kafka.
- **Amazon SQS:** Managed queue on AWS. Great when you don't want to run your own broker.

INTERVIEW FOCUS

When asked 'why Kafka over RabbitMQ?', cover three points:

1. Kafka retains messages on disk for a configurable time. RabbitMQ deletes once consumed.
2. Kafka supports multiple independent consumer groups reading the same topic. RabbitMQ delivers each message to one consumer.
3. Kafka scales via partitions and handles millions of events per second. RabbitMQ is simpler but doesn't match that throughput.

Then add: 'For a simple task queue with retries, RabbitMQ is valid. Kafka makes more sense when I need event replay or fan-out at scale.'

TOPIC 11

Interview Tradeoffs

These are the tradeoffs that come up when async communication is part of your design. Practice articulating both sides clearly. Interviewers don't want 'X is better' - they want to hear when each option makes sense.

TRADEOFF 1: SYNC VS ASYNC

Sync is simpler, easier to trace, and gives instant feedback. Async is more resilient and scalable but adds complexity and makes debugging harder. Use sync for critical path operations like auth and payment confirmation. Use async for side effects like emails, notifications, and analytics.

TRADEOFF 2: AT LEAST ONCE VS EXACTLY ONCE DELIVERY

Kafka default is at least once. You may process the same message twice on failures. Exactly once is supported via Kafka transactions but adds overhead. In most systems, make consumers idempotent and accept at least once. It is simpler and fast enough for the majority of use cases.

TRADEOFF 3: MORE PARTITIONS VS FEWER PARTITIONS

More partitions means more parallelism and throughput. But it also means more file handles on the broker, longer rebalance times when consumers join or leave, and more operational complexity. Start with a reasonable number and scale based on actual load measurements, not guesses.

TRADEOFF 4: AUTO OFFSET COMMIT VS MANUAL COMMIT

Auto-commit is simpler but you risk losing messages if the consumer crashes before processing finishes. Manual commit gives you control at the cost of handling commit failures explicitly. For anything business-critical, use manual commit.

TRADEOFF 5: SEPARATE TOPICS VS SHARED TOPIC

Separate topics per domain keeps concerns clean and lets each topic have different retention and partition settings. Shared topics are simpler but mix unrelated messages and complicate consumer filtering logic. Prefer separate topics in production.

MODULE SUMMARY

Async communication shows up in almost every system design question because almost every real system has background tasks, notifications, or high-throughput event streams.

The pattern is always the same: identify what the user is waiting for, keep that synchronous. Push everything else to a queue. Let consumers handle it at their own pace with their own retry logic.

In Spring Boot, this maps to `KafkaTemplate` for producing, `@KafkaListener` for consuming, and a dead-letter topic for failure handling. That's the core you need to explain confidently in any interview.

Concept	What to Remember
Message Queue	Buffer between services. Sender does not wait. Consumer processes when ready.
Kafka Topic	Named stream stored on disk. Multiple consumer groups read it independently.
Partition	Unit of parallelism. Max active consumers in a group equals partition count.
Consumer Group	Consumers sharing the load. Different groups read the same topic independently.
Offset	Position in partition log. Manual commit after processing is safer for critical work.

Concept	What to Remember
Dead-Letter Topic	Failed messages after max retries. Inspect, fix, and replay later.
Idempotency	Processing the same message twice produces the same result. Required for at least once systems.
KafkaTemplate	Spring class used in the Service layer to publish events to Kafka topics.
@KafkaListener	Annotate a Service method to consume messages from a Kafka topic.

Next: Module 10 explains designing for scale, including horizontal vs vertical scaling, stateless services, session management, and load distribution strategies.

MODULE 10

Designing for Scale

Horizontal vs vertical scaling, stateless services, session management, and load distribution strategies for Java backend systems.

PART OF THE SYSTEM DESIGN INTERVIEW PLAYBOOK

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

MODULE 10 OVERVIEW

What This Module Covers

Every system design interview eventually lands on the same question: "how would you scale this?" Most freshers answer with one line, "add a load balancer and more servers," and stop there. That answer tells the interviewer nothing about how you think. This module is about building the actual thinking behind scale: when vertical scaling is fine, why statelessness is the real unlock for horizontal scaling, what happens to sessions once you have ten instances instead of one, and how traffic actually gets spread across those instances in a real Spring Boot setup.

WHAT YOU WILL WALK AWAY KNOWING

- The real difference between scaling up and scaling out, and where each one breaks
- Why state is the enemy of horizontal scaling, and how to remove it from your services
- How session management works once you stop relying on a single server's memory
- How load gets distributed across instances, and what Spring Boot gives you for free

Topics in This Module

01 Vertical vs Horizontal Scaling

When a bigger machine is the right call, and when it just delays the real problem.

02 Stateless Services

Why state is what actually breaks horizontal scaling, and how to design it away.

03 Session Management at Scale

Sticky sessions, Redis-backed sessions, and JWT, with the tradeoffs for each.

04 Load Distribution Strategies

Round robin, least connections, and how Spring Cloud routes traffic in practice.

05 Putting It Together in an Interview Answer

The order interviewers expect, and the mistakes that quietly cost points.

TOPIC 01

Vertical vs Horizontal Scaling

Here is a scenario I have seen play out in real projects more than once. A Spring Boot service starts slowing down under load. Someone on the team says "just bump up the EC2 instance size," and it works. CPU usage drops, response times improve, everyone moves on. Three months later, the same service is struggling again, except now there is no bigger instance left to move to. That is vertical scaling hitting its ceiling, and it always hits one.

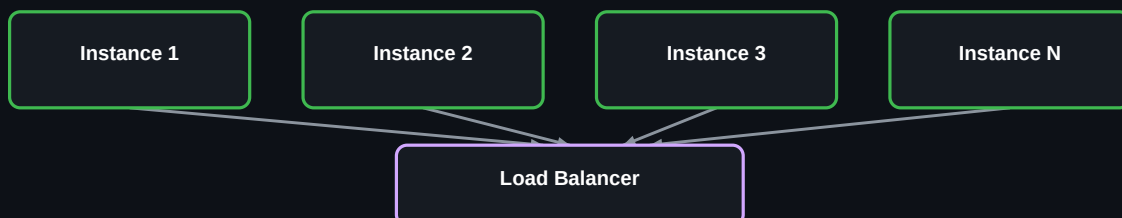
Vertical scaling means making one machine more powerful. More CPU, more RAM, faster disks, same single server. It is the easiest fix because it needs zero code changes. You are not touching your service logic, your database calls, or your deployment pipeline. You are just renting a bigger box.

Horizontal scaling means adding more machines instead of making one machine bigger. Instead of one Spring Boot instance handling everything, you run five or ten instances, and a load balancer spreads traffic across them. This is where things get interesting for an interview, because horizontal scaling is not free. It demands that your service behaves a certain way, and that is exactly what the next topic in this module covers.

VERTICAL SCALING: ONE BOX GETS BIGGER



HORIZONTAL SCALING: MORE BOXES, SAME SIZE



VERTICAL SCALING

- Upgrade CPU, RAM, disk on one machine
- No code or architecture change needed
- Has a hard physical ceiling
- One machine down means the whole service is down
- Good fit for early-stage apps, batch jobs, single-node databases

HORIZONTAL SCALING

- Add more machines behind a load balancer
- Requires stateless, deployable-many-times services
- Scales close to linearly as you add nodes
- One instance dying does not take the system down
- Good fit for growth-stage APIs, read-heavy systems, microservices

INTERVIEW FOCUS

A common fresher answer is "we scale horizontally by adding more servers." That is true but shallow. Interviewers want to hear why that is possible, meaning you understand the service has to be stateless first.

A stronger answer sounds like: "I would start by making sure the service does not store session state locally, then add instances behind a load balancer, and use Redis for anything that needs to be shared across instances."

In real production systems, you usually do both at different points. You scale vertically first because it buys you time without any engineering effort. Once you hit diminishing returns, or once availability starts mattering more than convenience, you move to horizontal scaling. Database servers are a good example: most teams scale the primary database vertically for a long time before they introduce read replicas or sharding, because horizontal scaling at the data layer is genuinely hard.

JAVA + SPRING BOOT MAPPING

In a Spring Boot world, vertical scaling is invisible to your code. You deploy the same JAR to a bigger machine and nothing in your `@RestController` or `@Service` classes changes.

Horizontal scaling is where Spring Boot's design choices start to matter. If your service keeps state in instance variables, local caches, or in-memory maps, running five copies of it behind a load balancer will give you five different, inconsistent versions of that state. That is the bug pattern this module is building toward.

Where Freshers Lose Points Here

The mistake is not picking the wrong scaling strategy. It is not being able to explain the tradeoff. If you say "I would scale horizontally" and the interviewer asks "why not just use a bigger machine," you need an answer ready. Cost is one angle (a single huge machine is usually more expensive per unit of capacity than several smaller ones), but availability is the stronger angle. A single large server is a single point of failure no matter how powerful it is.

VERTICAL VS HORIZONTAL

Vertical scaling trades availability for simplicity. Horizontal scaling trades simplicity for availability and near-linear growth. Neither one is "correct." The right answer in an interview depends on the system's actual requirements: an internal admin tool with ten users does not need horizontal scaling, a public-facing API expecting traffic spikes usually does.

TOPIC 02

Stateless Services

I want to start with a real bug pattern instead of a definition. Imagine a Spring Boot service that handles file uploads. Someone writes a quick feature: track upload progress in a `Map<String, Integer>` stored as an instance variable in the controller, keyed by upload ID. It works perfectly in testing because there is one instance running.

Then the team scales out to three instances behind a load balancer. A user starts an upload, the request lands on instance 1, and that map entry gets created there. Their next request, checking progress, lands on instance 2 because the load balancer round-robins requests. Instance 2 has never heard of that upload ID. The user sees 0 percent progress forever, even though the upload is actually working fine on instance 1. This is what people mean when they say a service "holds state."

A stateless service is one where any instance can handle any request, because the instance itself does not remember anything about previous requests from that user. All the state that matters lives somewhere external and shared: a database, a distributed cache like Redis, or gets passed along in the request itself (like a JWT token carrying the user's identity).

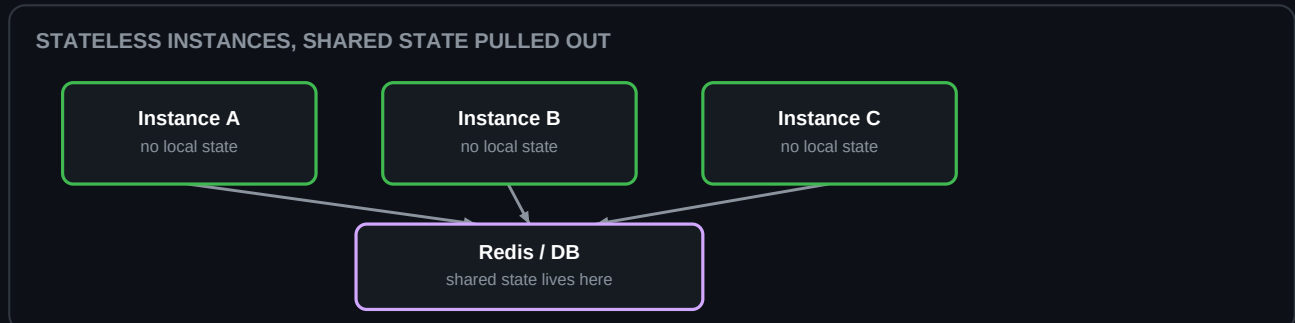
WATCH OUT

Local in-memory maps, static variables, and instance fields that store per-user or per-request data are the most common way a "stateless" Spring Boot service quietly becomes stateful. Singleton beans in Spring are shared across all requests on that instance, so if you put mutable per-user data in one, you have built a trap that only shows up once you scale past one node.

What "State" Actually Means Here

State is anything that needs to persist between requests and that the next request depends on. A logged-in user's session is state. Upload progress is state. A shopping cart that has not been checked out yet is state. The pagination cursor mid-scroll in a feed is state. None of these are wrong to have. The

question is where they live.



How to Actually Make a Service Stateless

In real projects, this usually comes down to a short checklist. Move session data out of the JVM's memory and into Redis, or eliminate server-side sessions entirely using token-based auth. Stop using instance variables for anything that varies by request. Make any "in progress" tracking (uploads, batch jobs, long-running tasks) live in a database row or a Redis key, not a local map. And do not write to the local filesystem expecting it to be there on the next request, because the next request might hit a different instance entirely.

JAVA + SPRING BOOT MAPPING

Spring beans default to singleton scope, which means one instance of that bean per Spring container, shared across every request that hits that node. That is fine for stateless services and repositories. It becomes a problem the moment someone adds a mutable field to a `@Service` or `@Component` bean to "remember" something between calls.

For per-request data that genuinely needs to be request-scoped, Spring gives you `@RequestScope`, but that still only solves the problem within a single request and a single instance. It does not make state available across instances. For that, you need Redis, a database, or the token itself carrying the data (which is exactly what JWT does for identity).

Stateless Does Not Mean No Storage

A mistake I see often: people think stateless means "do not store anything anywhere." That is not it. Stateless means the service instance itself is disposable. You should be able to kill instance A, start a fresh instance D, and have it pick up serving traffic with zero loss of correctness, because nothing important was sitting only in instance A's memory. The database still has all the data. Redis still has the session. The instance was just a temporary worker, not a place where anything lives permanently.

INTERVIEW MOMENT

A realistic follow-up an interviewer might ask: "if your service is stateless, where does a user's shopping cart live between page loads?" The expected answer is that it lives in the database (or Redis if you want it fast and are fine losing it on a cache eviction), keyed by user ID, and the service just reads and writes to that store on every request. The cart is not something the service "holds" in memory across requests.

Why This Matters More Than People Think

Statelessness is not just a scaling requirement, it is what makes rolling deployments, auto-scaling, and failover all work without drama. If your services are stateless, Kubernetes can kill and restart pods whenever it wants, your cloud provider can add instances during a traffic spike and remove them after, and a deployment can roll out new versions one instance at a time without losing any in-flight user data. Take statelessness away and every one of those becomes risky.

TIP

A quick gut check for any service you are designing: if I deleted this instance right now without warning, would any user lose data or get a broken experience? If yes, something is stored in the wrong place.

TOPIC 03

Session Management at Scale

Picture a login flow on a single Spring Boot instance. A user logs in, Spring Security creates an HTTP session, and that session sits in the JVM's memory tied to a session ID stored in a cookie. Every later request carries that cookie, the server looks up the session in memory, and the user stays logged in. Simple, and it works fine until you add a second instance.

The moment you have two instances, that in-memory session only exists on whichever instance created it. If the load balancer sends the next request to the other instance, that instance has no idea who this user is, and they get logged out. This is the exact same root problem as the upload progress example from the last topic, just showing up in a different feature.

Three Real Ways to Solve This

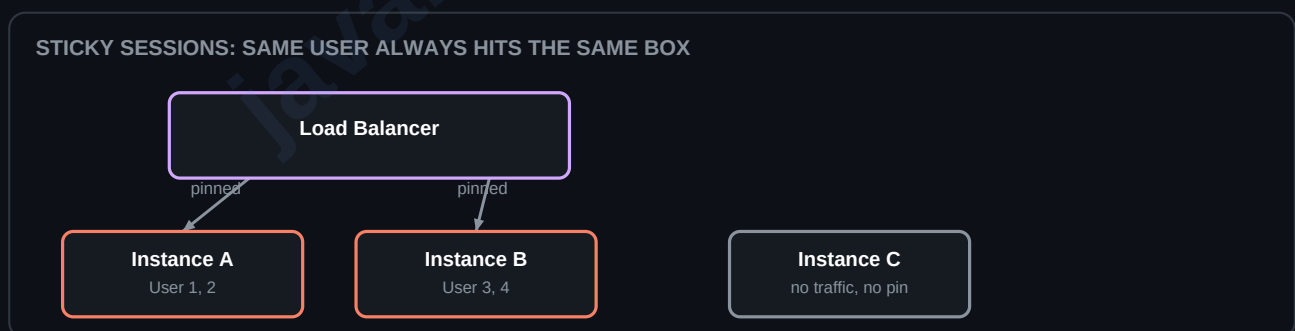
There is not one "correct" answer here, and interviewers usually want you to know the options and when each one fits, not just name one and stop.

Approach	How it works	Tradeoff
Sticky sessions	Load balancer always routes a user to the same instance	Breaks if that instance dies. Uneven load if some users are heavier than others.
Centralized session store	Sessions stored in Redis, any instance can read them	Adds a Redis dependency, but instances stay interchangeable. Most common real-world choice.
Token-based (JWT)	Server signs a token holding identity, client sends it every request	No session storage at all on the server. Harder to revoke a token early.

Sticky Sessions: the Tempting Shortcut

Sticky sessions sound like an easy fix. You configure the load balancer to always send a given user to the same backend instance, usually based on a cookie. No code changes, sessions keep working exactly like the single-instance version. This is why a lot of teams reach for it first.

The problem shows up the moment that instance restarts, crashes, or gets removed during a scale-down. Every user pinned to it loses their session and gets logged out at once. It also defeats some of the point of horizontal scaling, because you are no longer free to route traffic wherever capacity is available. In an interview, sticky sessions are worth mentioning as an option, but you should be ready to explain why most teams move away from them as the system grows.



Notice Instance C has no arrow pointing to it. That is not a gap in the diagram, it is the point: with sticky sessions, a new instance can sit idle while the pinned ones stay loaded, because the load balancer is not free to send anyone there.

Centralized Session Store: the Practical Default

Most real systems land here. Instead of storing the session in the JVM's memory, you store it in Redis. Every instance, on every request, looks up the session in Redis using the session ID from the cookie. Now

it genuinely does not matter which instance handles the request, because none of them are holding the session, they are all just reading from the same shared store.

JAVA + SPRING BOOT MAPPING

Spring Session with a Redis backend is the standard way to do this in a Spring Boot application. You add the `spring-session-data-redis` dependency, configure your Redis connection, and Spring Security's existing session handling automatically starts reading and writing sessions to Redis instead of local memory. Your controllers and security config barely change.

This is also where the line between "caching" and "session storage" gets blurry for freshers. `@Cacheable` is for caching expensive read results. A session store is closer to a small, fast, shared database for short-lived per-user data. Same Redis instance can power both, but they are different responsibilities.

Token-Based Auth: Skipping Server-Side Sessions Entirely

The third option removes the session store question entirely. Instead of the server remembering who you are, the server issues a signed JWT at login containing the user's identity and permissions. The client sends that token on every request, any instance can verify the signature and trust the contents, and nothing needs to be looked up anywhere. This is why JWT shows up so often in microservice architectures: it removes a shared dependency from the authentication path.

The tradeoff that catches people off guard: once a JWT is issued, you cannot easily force it to expire early. If a user changes their password or you need to ban an account immediately, a server-side session can be deleted on the spot. A JWT is valid until it naturally expires, unless you build extra infrastructure like a token blocklist, which brings back some of the shared-state problem you were trying to avoid.

JWT VS SERVER-SIDE SESSIONS

JWT trades easy revocation for zero shared session storage. Redis-backed sessions trade a Redis dependency for instant revocation and simpler logout behavior. Neither is universally right. Short-lived access tokens paired with a refresh token is the common middle ground used in production systems, because it limits how long a compromised or stale token stays valid.

INTERVIEW FOCUS

If you say "I would use JWT" and stop there, expect a follow-up: "how do you log a user out immediately if their token is still technically valid?" Have an answer ready, short expiry times plus a refresh token, or a small denylist in Redis for the rare forced-logout case, instead of going silent.

Where freshers usually go wrong on this topic is treating it as a yes or no question about JWT. The interviewer is not testing whether you know JWT exists. They are checking whether you understand that session data is just another form of state, and that every one of the three approaches above is really just a different answer to "where does that state live once you have more than one server."

TOPIC 04

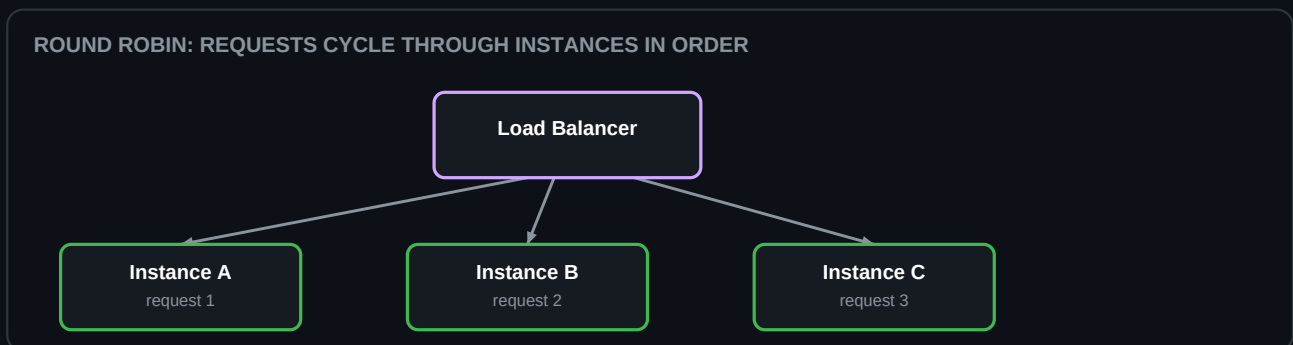
Load Distribution Strategies

Once your services are stateless and sessions are sorted out, the next question is mechanical but still trips people up in interviews: how does a request actually decide which instance to land on? Saying "a load balancer handles it" is true but it is a one-line answer to a question that usually deserves three or four sentences.

The Common Algorithms

Strategy	How it picks an instance	Where it fits
Round robin	Cycles through instances in order, one after another	Default choice when all instances are roughly equal in capacity
Least connections	Sends the request to whichever instance has the fewest active connections	Good when request processing times vary a lot between requests
Weighted routing	Sends more traffic to instances with more assigned weight	Useful during gradual rollouts, or when instances have different hardware specs
IP hash / consistent hash	Derives the target instance from a hash of the client IP or a key	Used when you want the same client to consistently land on the same instance

Round robin is the one most people have heard of, and it is the right default for most stateless APIs. It is predictable and simple. Where it falls short is when requests are not equal in cost. If instance A is still finishing a heavy report-generation request while instance B is idle, blindly sending the next request to whichever is "next in line" can pile work onto an already busy node. That is when least connections becomes the better fit.



Layer 4 vs Layer 7 Load Balancing

This distinction comes up more in senior interviews, but it is worth knowing early because it explains why API Gateways and load balancers are not quite the same thing. A layer 4 load balancer makes routing decisions based on network information, IP address and port, without looking at the actual HTTP request. It is fast but limited. A layer 7 load balancer can look inside the HTTP request itself, the path, headers, even the body, and route based on that.

In a microservices setup, this is why an API Gateway often sits in front of everything. It is doing layer 7 style routing: send `/orders/**` to the order service, `/payments/**` to the payment service, and so on, in addition to handling cross-cutting concerns like authentication and rate limiting before the request even reaches a backend service.

JAVA + SPRING BOOT MAPPING

In a Spring Cloud setup, `Spring Cloud LoadBalancer` handles client-side load balancing between instances of a downstream service, working together with a service registry like Eureka or Consul so it knows which instances currently exist.

`Spring Cloud Gateway` is the layer 7 piece, routing incoming requests to the right microservice based on path, and is also where you would plug in rate limiting filters and authentication checks before traffic reaches your actual services.

Health Checks: the Part People Forget to Mention

A load balancer is only useful if it knows which instances are actually healthy. If instance B has crashed but the load balancer keeps sending it traffic, every user hitting B gets errors even though A and C are perfectly fine. This is why load balancers run periodic health checks, usually a lightweight HTTP call to something like `/actuator/health`, and automatically stop routing to any instance that fails enough checks in a row.

TIP

Spring Boot Actuator gives you a working `/actuator/health` endpoint almost for free. In a real deployment, this endpoint usually checks more than "is the JVM alive," it can also confirm the database connection and Redis connection are working, so a service that is technically running but cannot reach its dependencies gets pulled out of rotation instead of silently failing requests.

What Happens at Small Scale vs Large Scale

At small scale, with two or three instances and predictable traffic, round robin plus a basic health check covers almost everything. You will not notice the difference between load balancing strategies because the traffic volume is forgiving.

At larger scale, the choice of strategy starts to matter. Uneven request costs become more visible, traffic spikes expose any instance that is slightly underpowered, and you start caring about things like zone-aware routing, sending traffic to instances in the same data center region as the client to cut latency. Most teams do not build this themselves. They lean on what their cloud provider's load balancer or service mesh already offers, and focus their own engineering effort on keeping the services themselves stateless and fast.

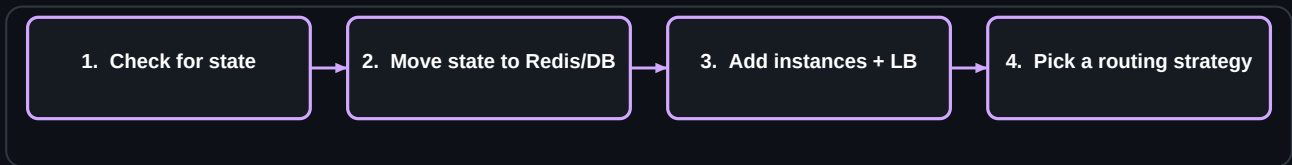
INTERVIEW MOMENT

A fair follow-up question: "what happens if your load balancer itself goes down?" The honest answer is that you usually do not run just one. Cloud load balancers are managed as highly available services themselves, often with multiple nodes behind a DNS entry or a virtual IP, specifically so the load balancer is not the new single point of failure you just spent this whole module trying to remove.

TOPIC 05

Putting It Together in an Interview Answer

Interviewers rarely ask "explain horizontal scaling" in isolation. It usually shows up halfway through a design question, after you have already sketched out a basic architecture. Something like: "this is good for now, but how would this handle ten times the traffic?" That question is testing everything in this module at once, not just one concept.



If I were answering that question out loud, I would walk through it in that order. First, point out anything in the current design that is not stateless yet, sessions stored locally, in-memory caches, anything tied to one specific instance. Then explain how that state moves to a shared store. Only after that does it make sense to talk about adding more instances and a load balancer in front of them, because adding instances to a stateful service does not actually fix anything, it just adds more ways for the bug to show up.

INTERVIEW FOCUS

At my level, I would not try to design the perfect scaled system from the first sentence. I would explain the simple version first, then improve it step by step out loud, scaling instances, then thinking about the database, then caching, then async work. Interviewers consistently say they prefer this over a candidate jumping straight to a final complex diagram with no reasoning shown.

Common Mistakes That Cost Points

- **Saying "add more servers" without checking statelessness first**

Adding instances to a stateful service multiplies the bug, it does not fix it. Always mention removing state before mentioning adding instances.

- **Treating sticky sessions as a long-term solution**

It is a fine stopgap to mention, but if you stop there, the interviewer will ask what happens when that pinned instance dies.

- **Not knowing what a load balancer actually checks before routing traffic**

Health checks come up often as a follow-up. Have a one-sentence answer ready: periodic checks against an endpoint like `/actuator/health`, unhealthy instances get removed from rotation.

- **Assuming JWT has no downsides**

Be ready to explain the revocation tradeoff. It shows you understand JWT, not just that you have heard of it.

Quick Recap

Vertical scaling buys you time without engineering effort, but it has a ceiling and stays a single point of failure. Horizontal scaling removes that ceiling, but only works once your services are genuinely stateless. Session data, upload progress, and anything else that used to live comfortably in one JVM's memory

needs to move to Redis, a database, or a token, before adding more instances actually helps. And once you have multiple instances, a load balancer with a sensible routing strategy and real health checks is what keeps traffic flowing to the instances that can actually handle it.

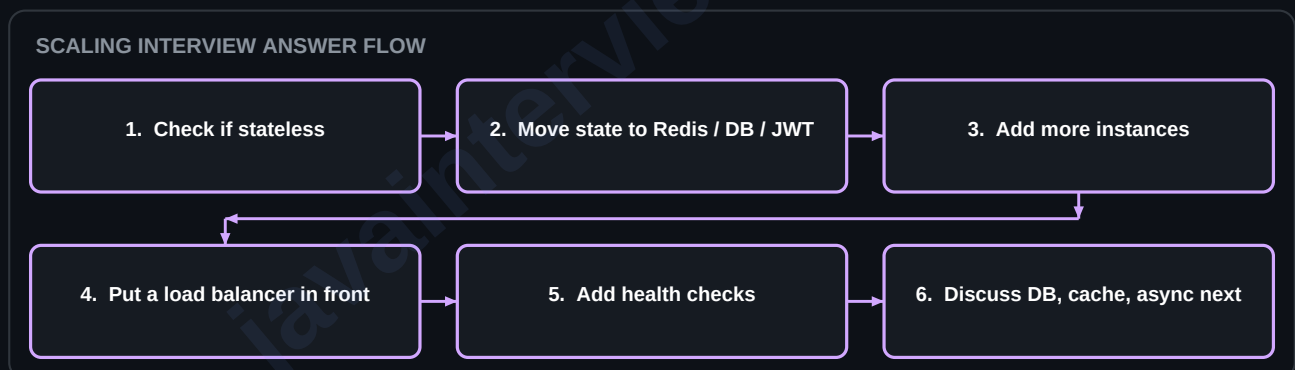
QUICK REVISION

Module 10 Quick Revision Cheat Sheet

Use this page the night before an interview. If you can explain every row in one sentence without looking, you know this module well enough.

Scaling Interview Answer Flow

This is the order to talk through out loud when an interviewer asks how you would scale a design you have already sketched.



Cheat Sheet Table

Area	Remember This
Vertical scaling	Bigger machine, zero code change, hard ceiling, single point of failure
Horizontal scaling	More machines behind a load balancer, needs stateless services first
Stateless service	Any instance can handle any request, nothing important lives only in local memory
Shared state	Sessions, progress, carts: move them to Redis, a database, or a token
Sticky sessions	Pins a user to one instance, breaks if that instance dies, a stopgap not a fix
Redis-backed sessions	Spring Session + Redis, any instance can read the session, the common default

Area	Remember This
JWT	No server-side session storage, but harder to revoke before natural expiry
Load balancer	Decides which instance gets the request, also runs health checks
Round robin	Cycles through instances in order, the right default when instances are equal
Least connections	Sends traffic to the least busy instance, better when request cost varies
Health checks	Periodic calls to an endpoint like /actuator/health, unhealthy instances get pulled
Spring Boot Actuator	Gives you /actuator/health almost for free, can check DB and Redis too
Interview answer order	State first, then instances, then load balancer, then health checks

Next: Module 11 explains reliability, fault tolerance, and failure handling, including retry, timeout, circuit breaker, fallback, idempotency, and graceful degradation.

Reliability, Fault Tolerance and Failure Handling

Retry, timeout, circuit breaker, fallback, idempotency, and graceful degradation for real backend systems.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers and 1-3 Years of Experience

javainterviewready

Module 11 Overview

TOPIC 01 Why Services Fail

TOPIC 02 Timeout

TOPIC 03 Retry and Exponential Backoff

TOPIC 04 Circuit Breaker

TOPIC 05 Fallback Strategies

TOPIC 06 Idempotency

TOPIC 07 Graceful Degradation

TOPIC 08 Java and Spring Boot Mapping

TOPIC 09 Failure Handling in Production

TOPIC 10 Interview Tradeoffs

WHAT YOU WILL LEARN

- ✓ How to stop slow services from blocking threads
- ✓ How to retry safely without causing retry storms
- ✓ How circuit breakers protect the whole system
- ✓ How fallbacks keep features usable during failures
- ✓ Why idempotency matters for payments and orders
- ✓ How these patterns map to Spring Boot and Resilience4j

TOPIC 01

Why Services Fail

In production, things break all the time. Not because engineers are bad, but because distributed systems are inherently unreliable. Networks drop packets. Databases get slow under load. Third-party APIs go down. A memory leak in one service can make it unresponsive for minutes. Your job is not to prevent every failure. It is to build systems that handle failures gracefully and keep working even when parts of the system are broken.

Most freshers design for the happy path. They draw neat boxes and arrows in interviews and assume everything will work. That is exactly what interviewers notice. The moment you start asking what happens when this fails, or how we retry this safely, you sound like someone who has actually shipped code to production.

INTERVIEW SIGNAL

Freshers describe what happens when everything works. Senior engineers talk about what happens when things fail. Just by asking 'what if this service is slow or down?' in your answer, you immediately stand out from most candidates at your level.

Common Failure Scenarios You Will Face

Cascading Failures

Service A is slow. B calls A and waits. B's thread pool exhausts. B becomes slow. C calls B and crashes. One slow service brings down the whole system.

Thundering Herd

Cache expires. 10,000 requests hit the database simultaneously. Database buckles. Everything slows. A single cache miss causes a system-wide problem.

Partial Failure

Your recommendation service is down but product listing still works. If not handled, the whole page fails instead of just the recommendations.

Duplicate Requests

User clicks Pay Now twice. Network retry sends the request twice. Without idempotency, you charge the user twice.

External Dependency Failure

Payment gateway returns 500. Email service is rate-limiting you. You cannot control these dependencies but you need to handle them.

The Reliability Patterns This Module Covers

1

Timeout

Stop waiting after N ms. Do not block forever.

2

Retry

Try again, but with delay and a limit.

3

Circuit Breaker

Stop calling a broken service. Give it time to recover.

4

Fallback

When primary fails, serve something useful.

5

Idempotency

Safe to call multiple times. Same result always.

6

Graceful Degradation

Reduce features, not uptime.

TOPIC 02

Timeout: The Most Underrated Safety Net

If there is one pattern most freshers skip, it is timeout. It seems too simple to mention. But I have seen production incidents where a single downstream service got slow, and because there was no timeout, threads kept waiting. Thread pool filled up. The upstream service stopped accepting new requests. Entire feature went down.

Timeout means: if you do not hear back from a dependency within N milliseconds, stop waiting and handle it. That handling could be returning an error, using a cached value, or triggering a fallback.

Types of Timeout

Timeout Type	What It Controls	Where to Apply
Connection Timeout	Max wait to establish a connection	DB, HTTP, any TCP connection
Read Timeout	Max wait for response after connecting	REST API calls, DB queries
Write Timeout	Max wait to finish sending data	File uploads, large payloads
Idle Timeout	Close connection if inactive for N ms	Connection pools, WebSocket

In Java and Spring Boot

Spring's WebClient (the modern choice) lets you configure connection and response timeouts on the underlying HttpClient. For database connections, configure them in HikariCP settings in your application.yml.

JAVA: WebClient Timeout Configuration

```
HttpClient httpClient = HttpClient.create()
    .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3000) // 3s connect
    .responseTimeout(Duration.ofMillis(5000)); // 5s read

WebClient client = WebClient.builder()
    .clientConnector(new ReactorClientHttpConnector(httpClient))
    .baseUrl("https://payment-service")
    .build();
```

JAVA: HikariCP Connection Timeout (application.yml)

```
spring:
  datasource:
    hikari:
      connection-timeout: 3000 # 3s to get a connection from pool
      socket-timeout: 10000 # 10s for a query to return
      maximum-pool-size: 20
```

WHERE FRESHERS GO WRONG

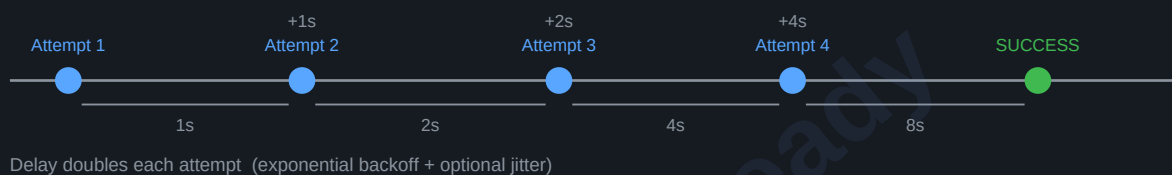
Most freshers either do not set timeouts at all, or set extremely high values like 60 seconds. In interviews, if you mention timeouts with realistic values (3-5 seconds for API calls, 30 seconds for long operations), it shows you understand what a bad user experience looks like.

TOPIC 03

Retry and Exponential Backoff

Retry is simple: if a request fails, try again. But naive retries can make things worse. If 1,000 clients all retry at the same time after a service blip, you get a retry storm that hammers the recovering service. That is why you add backoff.

RETRY WITH EXPONENTIAL BACKOFF



Exponential Backoff with Jitter

With exponential backoff, the delay doubles after each failure: 1s, 2s, 4s, 8s. Jitter adds a small random offset so all retrying clients do not fire at exactly the same moment. This spreads the load on the recovering service.

JAVA: Retry with Exponential Backoff via Resilience4j

```

@Bean
public RetryConfig retryConfig() {
    return RetryConfig.custom()
        .maxAttempts(3)
        .waitDuration(Duration.ofMillis(500))
        .intervalFunction(IntervalFunction
            .ofExponentialRandomBackoff(500, 2.0, 0.5, 5000))
        // 500ms base, doubles each attempt, 50% jitter, 5s max
        .retryOnException(e -> e instanceof IOException)
        .build();
}

@Retry(name = "paymentService", fallbackMethod = "paymentFallback")
public PaymentResponse charge(PaymentRequest req) {
    return paymentClient.post().bodyValue(req)
        .retrieve().bodyToMono(PaymentResponse.class).block();
}

```

What Should and Should Not Be Retried

Retry Safely	Do Not Retry
503 Service Unavailable	400 Bad Request (your bug)
504 Gateway Timeout	401 Unauthorized
Network connection errors	404 Not Found
Transient DB connection failure	Business logic errors
429 Rate limit with backoff	Unique constraint violations

PRODUCTION INSIGHT

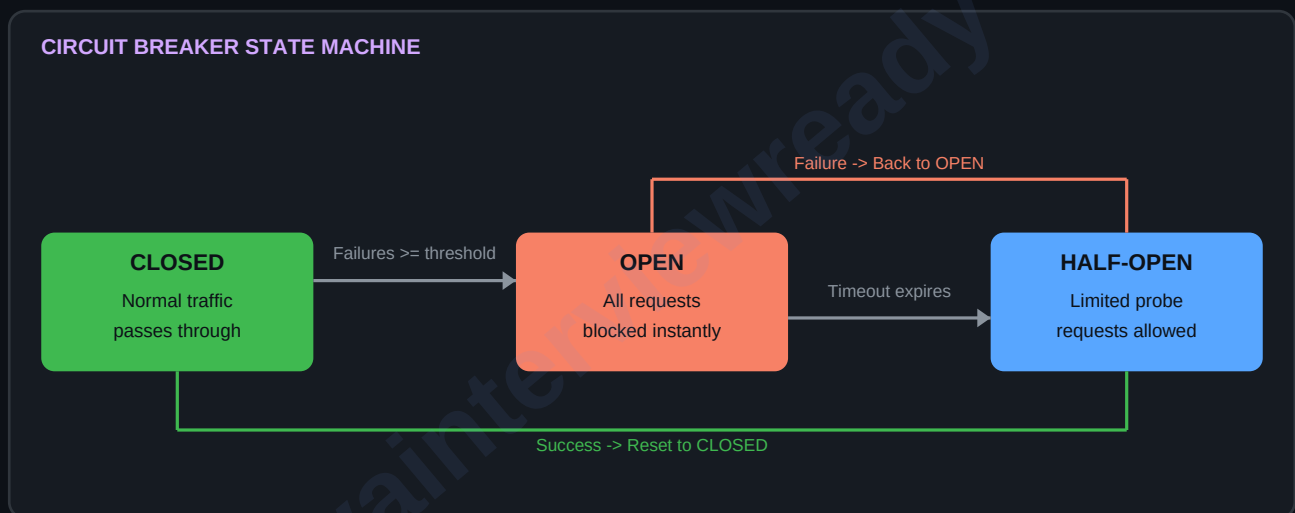
In real projects, the hardest part of retries is identifying what is safe to retry. GET requests are almost always safe. POST requests that create resources are not safe unless you add idempotency keys. Payment retries without idempotency cause double charges. Always think about retries together with idempotency.

TOPIC 04

Circuit Breaker: Fail Fast, Recover Safely

Imagine your payment service is down. Without a circuit breaker, every checkout request tries to reach payment, waits for the timeout, then returns an error. You spend your entire thread pool waiting for a service you already know is down. That is wasteful and it slows everything else down.

A circuit breaker tracks call outcomes. After a threshold of failures, say 5 in 10 seconds, it opens. In the open state, it does not even try to call the failing service. It immediately returns an error or triggers a fallback. After a configured timeout, it enters half-open state to test if the service has recovered.



Three States Explained

State	What Happens	Transition
CLOSED	All requests go through normally.	Opens when failure rate exceeds threshold.
OPEN	Requests fail immediately, no attempt made.	Moves to half-open after wait duration.
HALF-OPEN	Limited requests let through to probe service.	Success: CLOSED. Failure: back to OPEN.

JAVA: Circuit Breaker with Resilience4j

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10000
        permittedCallsInHalfOpenState: 3
        slowCallDurationThreshold: 2000

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackCharge")
    public PaymentResponse charge(PaymentRequest req) {
      return paymentClient.post(req);
    }

    public PaymentResponse fallbackCharge(PaymentRequest req, Throwable t) {
      log.warn("Circuit open, using fallback", t);
      return PaymentResponse.pending(req.getOrderId());
    }
```

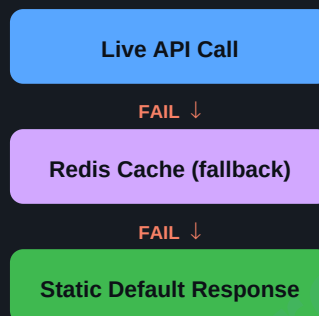
INTERVIEW MOMENT: Retry vs Circuit Breaker

Retry keeps trying when a call fails. Circuit breaker stops trying after too many failures, so you are not wasting threads on a service you already know is down. They work together. Retry handles occasional transient errors. Circuit breaker handles sustained failures. In Spring Boot, Resilience4j gives you both through separate annotations you can stack.

TOPIC 05

Fallback Strategies

A fallback is what your system does when the primary path fails. The goal is not to pretend everything is fine. It is to give the user something useful instead of a blank page or a 500 error.

FALLBACK CHAIN: LAYERED DEFENSE

Fallback Options at a Glance

Fallback Type	Use When	Tradeoff
Cached response	Recent data acceptable, Redis is warm	Data may be stale; set TTL carefully
Default response	Personalisation not critical	Loses personalisation; always safe
Degraded feature	Non-critical feature can be hidden	Users see less; core flow stays intact
Async queue	User does not need immediate result	Eventual processing; added complexity
Failover to backup	Critical service with replica provisioned	Higher infra cost; adds operational overhead

INTERVIEW TIP

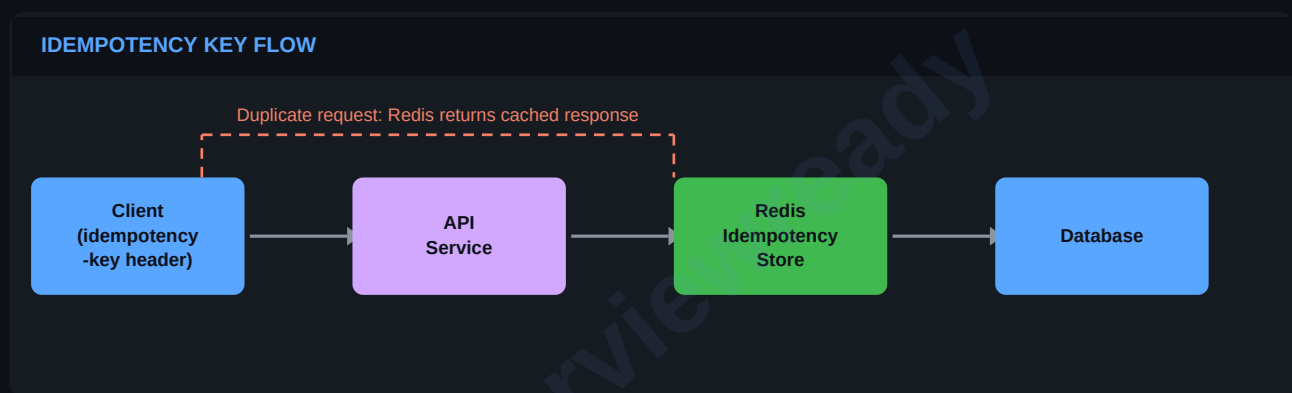
When asked about fallback, mention the tradeoff explicitly: a cached response is fast but might be stale. A default response is always safe but loses personalisation. You choose based on how critical freshness is for that feature. Show that you think in tradeoffs, not just solutions.

TOPIC 06

Idempotency: Safe to Retry

This one trips up a lot of freshers. The question usually comes up in payment or order design interviews: 'What if the user clicks Pay twice? What if the network retries the request?'

Idempotency means: if you call an operation multiple times with the same input, you get the same result. The operation executes once but is safe to call many times. For payments, this means the user never gets charged twice even if the request is sent twice.



How It Works in Practice

The client generates a unique key per operation (a UUID) and sends it in the request header. The server checks if this key has been processed before. If yes, it returns the stored result. If no, it processes the request and stores the result mapped to that key. Redis works well as the idempotency store because lookups are fast and you can set TTL on keys.

JAVA: Idempotency in Spring Boot

```
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> processPayment(
    @RequestHeader("Idempotency-Key") String key,
    @RequestBody PaymentRequest req) {

    String cached = redis.get("idem:" + key);
    if (cached != null) {
        return ResponseEntity.ok(
            objectMapper.readValue(cached, PaymentResponse.class));
    }

    PaymentResponse resp = paymentService.charge(req);
    redis.setEx("idem:" + key, Duration.ofHours(24),
        objectMapper.writeValueAsString(resp));
    return ResponseEntity.ok(resp);
}
```

HTTP Methods and Idempotency

Method	Idempotent?	Notes
GET	Yes	Read-only. Always safe to call multiple times.
PUT	Yes	Replace resource. Same result if called twice.
DELETE	Yes	Second delete returns 404, not an error.
POST	No	Creates resource. Needs idempotency key explicitly.
PATCH	Depends	Set operations: yes. Increment operations: no.

TOPIC 07

Graceful Degradation

Graceful degradation is the art of staying useful even when parts of your system fail. Give users less functionality rather than no functionality.

Think about Netflix. When their recommendation engine has a problem, they do not take down the entire app. They show you a generic list of popular titles. You still watch something. That is graceful degradation in action.

GRACEFUL DEGRADATION: PROGRESSIVE FEATURE LOSS

LEVEL 1: Full Features

All services up. Personalisation, live data, notifications.

LEVEL 2: Partial Degradation

Recommendation service down. Show popular items instead.

LEVEL 3: Core Only

Cache unavailable. Serve DB data with higher latency.

LEVEL 4: Minimal Mode

Most services down. Static page. Maintenance banner shown.

Critical vs Non-Critical Features

CRITICAL FEATURES

- Payment processing
- Order creation
- User login
- Inventory check

NON-CRITICAL FEATURES

- Recommendations
- Product reviews
- Push notifications
- Analytics events

How to Design for Graceful Degradation

Identify critical vs non-critical features

In an e-commerce checkout, adding to cart and payment are critical. Product reviews and recommendations are not. Design failure handling accordingly.

Use feature flags

Toggle features off dynamically when a service is unhealthy. Spring Boot Actuator with feature flags or external config lets you disable features without deploying code.

Define fallback content for every external call

If your call has no fallback plan, it is a single point of failure. Every circuit breaker needs a fallback method defined.

Test failure modes

Use Chaos Monkey (available as a Spring Boot library) to randomly kill services in staging. If it hurts staging, it will hurt production.

PRODUCTION INSIGHT

In real projects, graceful degradation often starts with a late-night incident. A service goes down, the whole feature breaks, you get paged. After fixing it, you add a fallback. Then another. Over time, the system learns to survive individual failures. Good interviews show you already know this before the incident happens.

TOPIC 08

Java and Spring Boot Mapping

Every pattern we covered maps directly to a Resilience4j annotation or Spring Boot component. Here is how the full picture looks in a single service.

JAVA: Full Resilience4j Setup in a Spring Service

```

@Service
public class ProductService {

    @Retry(name = "catalogService", fallbackMethod = "cachedCatalog")
    @CircuitBreaker(name = "catalogService", fallbackMethod = "cachedCatalog")
    @TimeLimiter(name = "catalogService")
    public CompletableFuture<List<Product>> getCatalog() {
        return CompletableFuture.supplyAsync(() ->
            catalogClient.get().retrieve()
                .bodyToFlux(Product.class).collectList().block()
        );
    }

    public CompletableFuture<List<Product>> cachedCatalog(Throwable t) {
        log.warn("Catalog unavailable, returning cached data", t);
        return CompletableFuture.supplyAsync(() ->
            redisTemplate.opsForList()
                .range("catalog:cache", 0, -1)
                .stream().map(this::deserialize).collect(toList())
        );
    }
}

```

Pattern to Spring Boot Mapping

Pattern	Spring / Java Component	Config Location
Timeout	@TimeLimiter, WebClient, HikariCP	application.yml, HttpClient config
Retry	@Retry (Resilience4j), Spring Retry	application.yml resilience4j.retry
Circuit Breaker	@CircuitBreaker (Resilience4j)	application.yml resilience4j.circuitbreaker
Fallback	fallbackMethod param on annotations	Same service class, same signature

Idempotency	Redis (StringRedisTemplate), @RequestHeader	Custom filter or controller logic
Rate Limiting	@RateLimiter, Bucket4j, Gateway filter	Spring Cloud Gateway filters
Graceful Degradation	Feature flags, @ConditionalOn, Actuator	Custom health indicators

JAVA: Resilience4j application.yml All Patterns

```

resilience4j:
  retry:
    instances:
      catalogService:
        maxAttempts: 3
        waitDuration: 500ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
  circuitbreaker:
    instances:
      catalogService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
  timelimiter:
    instances:
      catalogService:
        timeoutDuration: 3s

```

TOPIC 09

Failure Handling in Production

Knowing patterns is one thing. Knowing how failures actually show up in production is what separates someone with real experience from someone who just read about it.

Failure	Symptoms	How to Handle
Database slow or down	High latency, connection pool full, request timeouts	Connection timeout + read timeout. Fallback to Redis cache. Read replica for read traffic. Alert on pool utilisation.
Redis down	Cache misses, sudden DB load spike	Never treat Redis as a hard dependency. Wrap Redis calls in try-catch. Fall through to database silently.
Kafka consumer lagging	Delayed notifications, order processing stuck	Monitor consumer lag. Scale consumers horizontally. Add dead-letter topics for poison pill messages.
External API down	Requests hanging, errors to user	Timeout on the external call. Circuit breaker to stop retrying. Async queue for non-critical operations.
Duplicate request or order	Same order created twice, user charged twice	Idempotency keys on POST endpoints. Unique DB constraint on order_reference_id as last line of defense.
Service returns 500 intermittently	Random failures, users retrying manually	Check logs for root cause. Add retry with backoff on caller. Circuit breaker should open if failure rate is high.

SPRING BOOT ACTUATOR: HEALTH ENDPOINTS

Spring Boot Actuator gives you health endpoints for DB, Redis, and Kafka out of the box. Expose /actuator/health on internal networks only. Circuit breaker state shows up here. Kubernetes readiness probes can use this endpoint to stop routing traffic to an unhealthy pod automatically.

TOPIC 10

Interview Tradeoffs

Knowing the pattern is the minimum. Knowing the tradeoffs is what gets you hired. When you mention a pattern in an interview, always say what it costs you too.

Retry vs Circuit Breaker

Retry is useful for occasional transient failures. But if the downstream service is truly down for minutes, retrying keeps wasting threads. Circuit breaker stops retrying after a threshold and gives the service time to recover. Use both together. Resilience4j lets you stack both annotations on the same method.

Watch out: Aggressive retry without a circuit breaker creates retry storms that make the problem significantly worse.

Aggressive Timeout vs No Timeout

A timeout that is too short causes false failures for legitimately slow operations. No timeout causes cascading failures. Set timeouts based on your SLA. A p99 response time of 2 seconds means your timeout should be around 3-5 seconds, not 60 seconds.

Watch out: Different operations need different timeouts. DB query timeout is not the same as external API timeout.

Synchronous Retry vs Async Queue

Retrying synchronously keeps the user waiting. For non-critical operations like email, SMS, or push notifications, send them to Kafka and retry asynchronously. The user gets an immediate response. The operation retries in the background.

Watch out: Async is not suitable when the user needs immediate confirmation, such as payment status.

Stale Cache as Fallback vs Empty Response

Serving slightly stale data from cache is almost always better than showing an error. The exception is financial data, real-time stock prices, or any data where stale values could cause real harm. Know your data before deciding.

Watch out: Cache TTL must be set thoughtfully. Too short means no protection. Too long means misleading data.

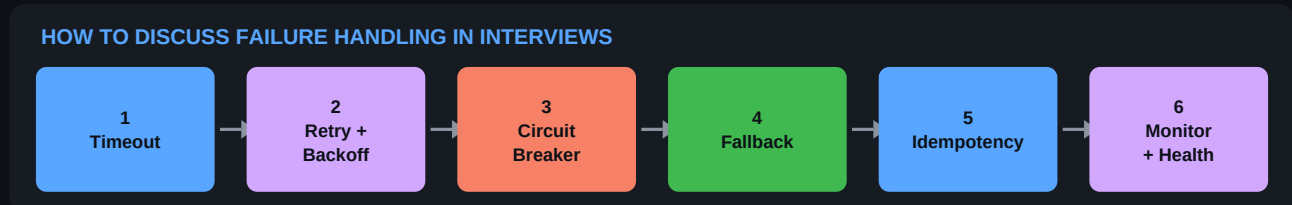
Idempotency Storage in Redis vs Database

Redis is fast but volatile. If Redis restarts, idempotency keys are lost and duplicate requests could go through. For payments, store idempotency keys in the database. For lower-stakes operations, Redis with a long TTL is fine.

Watch out: Redis TTL must outlive your retry window. If retries happen within 24 hours, TTL should be at least 24 hours.

javainterviewready

How to Discuss Failure Handling in Interviews



Structure Your Answer Like This

SAMPLE INTERVIEW LANGUAGE

"Since Service A calls Service B, I want to make sure we handle failures. I would add a timeout on the call, maybe 3 seconds, so we do not block indefinitely if B is slow."

"For transient errors like 503, I would configure Resilience4j retry with exponential backoff: try 3 times before giving up."

"If B is consistently failing, a circuit breaker would stop the retry storm and give B time to recover. The circuit opens after say 5 failures in 10 seconds."

"For the fallback, I would serve cached data from Redis. If Redis is also down, I would return a default response rather than a 500."

"For the payment operation specifically, I would add idempotency keys so even if the client retries, we do not charge the user twice."

The interviewer will not expect you to implement all of this. They want to hear that you know these problems exist and you have a plan for each one.

Common Mistakes in Interviews

X Only designing for the happy path

Drawing a clean diagram with no failure handling tells the interviewer you have not shipped real systems. Always mention at least timeout and fallback.

X Saying 'just retry' without explaining backoff

Retry without backoff or circuit breaker makes failures worse at scale. Always mention backoff and a retry limit.

X Not knowing the difference between retry and circuit breaker

Very common gap. Retry means try again. Circuit breaker means stop trying when things are consistently broken. Know when each one helps.

X Forgetting idempotency for write operations

Especially for payment and order systems. This can cost the company real money. Always mention it when retries are involved in write operations.

X Overcomplicating the answer

You do not need to implement all patterns everywhere. Say what problem each pattern solves and when you would add it. Start simple, then improve.

MODULE 11 QUICK REVISION CHEAT SHEET

Pattern	What It Solves	Spring Boot Mapping
Timeout	Stops waiting forever on slow dependencies	@TimeLimiter, HikariCP, WebClient timeout
Retry	Handles transient failures automatically	@Retry (Resilience4j), Spring Retry
Exponential Backoff	Prevents retry storms on recovering services	IntervalFunction in Resilience4j RetryConfig
Circuit Breaker	Stops calling a broken service; saves threads	@CircuitBreaker (Resilience4j)
Fallback	Keeps features usable when primary fails	fallbackMethod on @Retry or @CircuitBreaker
Idempotency	Prevents duplicate charges and duplicate orders	Idempotency-Key header + Redis or DB store
Graceful Degradation	Reduces features instead of causing full outage	Feature flags, @ConditionalOn, custom health
Dead-Letter Topic	Captures unprocessable Kafka messages safely	DLT config in Spring Kafka listener
Actuator Health Checks	Exposes service health for probes and alerts	/actuator/health, Kubernetes readiness probe
Resilience4j	All patterns in one library for Spring Boot	resilience4j-spring-boot3 dependency

Module 11 complete. Next: Module 12 explains rate limiting, security, and authentication, including JWT, OAuth basics, API keys, rate limiting strategies, and common security mistakes.

Rate Limiting, Security and Authentication

JWT, OAuth basics, API keys, rate limiting strategies, and common security mistakes for Java backend interviews.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

MODULE OVERVIEW

Why this module exists

Most freshers can describe a load balancer. Almost none of them can tell you what happens to your API the moment it goes public and a script starts hitting your login endpoint 400 times a second. That gap is exactly what this module closes.

I have interviewed candidates who designed a clean URL shortener, got the database sharding right, even mentioned CDN caching, and then froze completely when I asked: "how do you stop someone from generating a million short URLs a minute through your public API?" That single question separates people who memorized system design from people who have actually shipped something to production.

Security and rate limiting rarely get their own slide in a system design interview. They show up as a follow-up question after you think you are done. Interviewers ask them precisely because most candidates do not prepare for them. If you can answer this section confidently, you separate yourself from almost everyone else in the interview pipeline that day.

INTERVIEW FOCUS

When an interviewer brings up security mid-design, they are not testing whether you know OWASP by heart. They are testing whether you think about abuse and failure as part of design, not as an afterthought you bolt on later.

What we cover in this module

- Authentication vs authorization, and why interviewers keep mixing candidates up on this
- JWT: structure, signing, validation, and where it actually breaks in production
- Session-based auth vs token-based auth, with a real tradeoff discussion
- OAuth 2.0 basics, just enough to explain "Login with Google" correctly
- API keys, and where they fit when JWT does not make sense
- Rate limiting strategies: token bucket, sliding window, and how to implement them in Spring Boot
- The security mistakes that show up again and again in fresher interviews and fresher code

MODULE OVERVIEW

Topics in this module

01 Authentication vs Authorization**02** JWT Structure and Validation**03** Session-Based vs Token-Based Auth**04** OAuth 2.0 Authorization Code Flow**05** API Keys vs JWT**06** Why Rate Limiting Exists**07** Rate Limiting Strategies**08** What to Rate Limit By**09** Common Security Mistakes**10** Putting It Together in Interviews

TOPIC 01

Authentication vs Authorization

Ask ten freshers to define these two terms and eight of them will give you the textbook line: "authentication is who you are, authorization is what you can do." That answer is correct and also completely useless in an interview, because the interviewer already knows the definition. What they want to see is whether you know where each one lives in a real request.

Here is the version that actually shows up in production systems. A request hits your API Gateway or your Spring Boot app. Before any business logic runs, two questions get answered in order:

INTERVIEW MOMENT

Question 1 (Authentication): Is this a real, logged-in user, or someone pretending to be one? This happens once, usually at login, and the result is a token or session that proves identity for later requests.

Question 2 (Authorization): Given that this is user X, is user X allowed to do this specific thing? This happens on every single request, every single time, because identity does not imply permission.

Where freshers go wrong

The common mistake is treating these as one check. A junior developer writes a filter that checks "is the JWT valid" and stops there, assuming a valid token means the user can do anything. That is how you end up with bugs where a regular user calls `DELETE /admin/users/42` and it works, because the code only checked if the token was real, never whether that user was an admin.

In Spring Boot terms, this split is exactly why Spring Security has two separate concerns. Authentication is handled by an `AuthenticationManager` and a filter chain that runs before the request reaches your controller. Authorization is handled separately, through method security annotations or access rules tied to roles and permissions, not through the token itself.

JAVA + SPRING BOOT MAPPING

Authentication layer: `JwtAuthenticationFilter` extends `OncePerRequestFilter`, validates the token, sets the `SecurityContext`.

Authorization layer: `@PreAuthorize("hasRole('ADMIN')")` on the controller method, or `.requestMatchers("/admin/**").hasRole("ADMIN")` in the security filter chain config.



401 Unauthorized -> invalid or missing token (fails at AuthN)
403 Forbidden -> valid token, but not allowed to do this (fails at AuthZ)

INTERVIEW FOCUS · Common follow-up

"What status code do you return if the token is missing vs if the user is logged in but not allowed?" This trips up a lot of people. Missing or invalid token is 401. Valid token but insufficient permission is 403. Mixing these up in an interview is a small thing that quietly signals you have not actually built this before.

TOPIC 02

JWT: Structure and How It Actually Works

Almost every fresher says "I will use JWT for authentication" in an interview. Almost none of them can explain what stops someone from editing the payload and changing their own user id to someone else's. If you can answer that one question well, you are already ahead.

The three parts, and why each one exists

A JWT is just three base64-encoded pieces joined by dots: `header.payload.signature`. People memorize this but skip the part that matters, which is why the signature is the only piece that actually protects you.

Part	Contains	Can the client read it?	Can the client edit it?
Header	Algorithm used (e.g. HS256), token type	Yes, plain base64	Pointless, server re-verifies
Payload	User id, roles, expiry, custom claims	Yes, plain base64	Pointless, breaks signature
Signature	HMAC or RSA signature of header + payload	Yes, but it's just bytes	No, this is the whole point

WATCH OUT

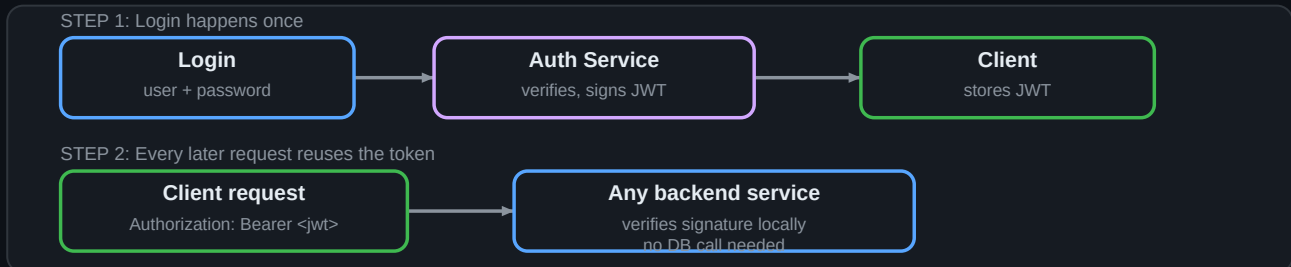
The payload is NOT encrypted. It is just encoded. Anyone can paste your token into jwt.io and read the user id, the roles, everything in plain text. Never put a password, a secret, or sensitive PII directly inside a JWT payload. Treat it like a postcard, not a sealed envelope.

So what actually stops tampering

If someone changes the payload, even one character, the signature no longer matches. When your server recomputes the signature on the incoming token using its secret key (or public key, for RSA) and compares it to the signature attached to the token, the mismatch is instant. That comparison is the entire security model of JWT. There is no database lookup, no session store, nothing else involved in that step.

This is also exactly why JWT is fast. The server does not need to check a database or a cache to know the token is legitimate. It just verifies the signature mathematically. That is the main reason people reach for JWT in microservices: any service with the shared secret (or the public key) can verify a token without calling back

to an auth service.



Where JWT breaks down in real systems

This is the part interview prep content skips, and it is exactly what separates a senior answer from a fresher answer.

Problem 1: You cannot kill a JWT early. A normal session can be deleted from the server's session store the moment a user logs out or gets banned. A JWT is self-contained. Once issued, it stays valid until it expires, even if you ban the user five minutes later. In production, this gets handled with a short expiry (often 15 minutes) plus a refresh token, or with a blocklist in Redis that every request checks against, which quietly brings back the same lookup cost JWT was supposed to avoid.

Problem 2: Token size grows with claims. Every claim you add to the payload travels on every single request, in every header. Freshers sometimes try to stuff a user's entire profile into the JWT to "save a database call." That makes every request heavier and leaks more data if the token is intercepted.

Problem 3: Algorithm confusion attacks. A known JWT vulnerability is when a server accepts the algorithm specified in the token header without restricting it, letting an attacker switch from RS256 to HS256 or to "none" and forge a token. This is why in Spring Boot you always explicitly configure the expected algorithm on the decoder side, you never trust the algorithm field the client sends you.

JAVA + SPRING BOOT MAPPING

In Spring Boot, this usually looks like building a `JwtDecoder` bean with an explicit algorithm and signing key, not letting the library infer it from the token. Libraries: `io.jsonwebtoken (JJWT)` for hand-rolled JWT, or `spring-boot-starter-oauth2-resource-server` if you are validating tokens issued by an external identity provider.

INTERVIEW FOCUS · Common follow-up

"How do you log a user out if you're using JWT?" A lot of candidates freeze here because the honest answer is "you can't, not immediately, unless you add something extra." Say exactly that. Then mention the refresh-token-with-short-access-token pattern, or a Redis-backed blocklist for emergency revocation. Interviewers respect the honest tradeoff far more than a confident wrong answer.

TOPIC 03

Session-Based Auth vs Token-Based Auth

Before JWT became the default answer everyone gives, every login system worked through sessions. Understanding why the industry moved toward tokens, and when sessions are still the better choice, is a question interviewers genuinely like to ask because most candidates have only ever used one of the two.

In a session-based system, the server creates a session after login and stores it (in memory, or more realistically in Redis for anything beyond a single instance). The client just holds a session id in a cookie. Every request, the server looks up that id to find out who the user is.

In a token-based system, the server creates a JWT after login and hands the whole thing to the client. The client sends the token back, and the server verifies it without needing to store anything. The state lives on the client, not the server.

Session-Based

- Server stores session state (Redis or in-memory)
- Easy to revoke instantly, just delete the session
- Needs a shared session store across instances
- Slightly heavier on the server per request
- Natural fit for traditional web apps with cookies

Token-Based (JWT)

- Server stores nothing, token is self-contained
- Hard to revoke early without extra infrastructure
- Scales horizontally without a shared store
- Faster verification, no lookup required
- Natural fit for APIs, mobile apps, microservices

TRADEOFF

This is a textbook "it depends" answer, and that is fine to say out loud in an interview. If you are building a single monolith web app with server-rendered pages, sessions are simpler and give you instant revocation. If you are building an API consumed by a mobile app and three microservices, JWT avoids a shared session store becoming a bottleneck. Most modern backend systems lean token-based specifically because of horizontal scaling, not because tokens are inherently "better."

In real projects, this usually becomes a decision made early and rarely revisited, because switching from sessions to tokens later touches every service that checks identity. If you are designing this from scratch in an interview, just be upfront that you are picking JWT because the system is API-driven and likely to scale across multiple service instances.

TOPIC 04

OAuth 2.0: Just Enough to Explain It Right

"Login with Google" feels like magic until you have actually built the flow once. Most freshers know OAuth exists. Very few can explain what is actually being exchanged behind that button, and that gap shows immediately when an interviewer asks one follow-up question.

The single most important thing to get right: OAuth is not authentication. It is an authorization protocol that lets your app get limited access to a user's data on another platform, without ever seeing their Google

password. "Login with Google" works because Google authenticates the user, then hands your app a token that proves the user said yes to sharing their basic profile info. Your app never touches the password.

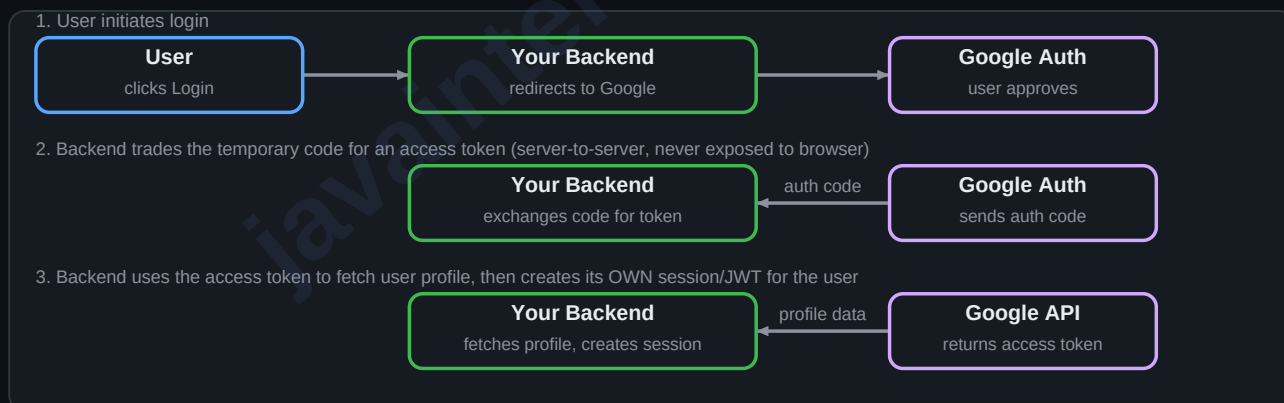
WATCH OUT

Saying "OAuth is how I authenticate users" in an interview is a subtle red flag. The accurate framing is: an identity layer (often OpenID Connect, built on top of OAuth) handles authentication, while OAuth itself is about authorization, granting access to a resource without sharing credentials.

The four roles, in plain language

Role	Who it actually is	Example
Resource Owner	The user	You, logging in
Client	Your application	Your app's backend
Authorization Server	Issues tokens after login	Google's auth server
Resource Server	Holds the protected data	Google's profile API

Authorization Code flow, the one used in real backend apps



Notice the part that freshers usually miss: your backend creates its own token at the end. Google's access token is for talking to Google's APIs. It is not the token your own app uses to identify the user on your own endpoints. Many beginners design this as one single token doing both jobs, but that creates an issue later when you want to revoke your own session without touching the Google grant, or when Google's token expires on its own schedule.

JAVA + SPRING BOOT MAPPING

Spring Boot has first-class support for this through `spring-boot-starter-oauth2-client`. You configure the provider (Google, GitHub, etc.) with client id and secret, Spring handles the redirect and code exchange, and you typically convert the result into your own JWT in an `OAuth2UserService` implementation so the rest of your app does not care how the user originally logged in.

INTERVIEW FOCUS · Common follow-up

"Why not just send the user's Google password to your backend and verify it there?" If you cannot answer this clearly, it usually means you have not internalized the point of OAuth. The answer is trust: your app should never see or store a third-party password. If your backend is compromised, leaked Google credentials cause damage far beyond your app.

TOPIC 05

API Keys: When JWT Is the Wrong Tool

If I'm designing a public API that other companies will integrate with, like a payments API or a weather data API, I would usually start with API keys instead of JWT, and that surprises some candidates who think JWT is always the modern answer.

JWT identifies a logged-in human, with an expiry, refresh flow, and claims about that specific person. An API key identifies a calling application or service, often with no concept of "login" at all. A backend server calling your payment API does not log in every 15 minutes. It just attaches a long-lived key to every request.

	JWT	API Key
Identifies	A logged-in user	A calling application/service
Lifespan	Short (minutes to hours)	Long-lived, manually rotated
Carries claims	Yes (roles, user id, expiry)	No, just an identifier
Typical use	Mobile app, web app sessions	Server-to-server, public APIs
Revocation	Hard before expiry	Instant, just disable the key

SENIOR TIP

A pattern that works well in practice: API key identifies WHICH client is calling (used for billing, rate limit tiers, and basic trust), while a JWT, if present, identifies WHICH user that client is acting on behalf of. Stripe's API does exactly this, your secret key identifies your account, separate from anything about your end customers.

In Spring Boot, API key validation is usually a simple filter that checks a header like `X-API-Key` against a stored value (hashed, never plain text, in your database or a secrets manager), before the request even reaches Spring Security's normal authentication chain. This is also frequently where rate limiting tiers get

applied, since the API key tells you which pricing plan the caller is on.

WATCH OUT

A mistake I still see in real codebases: API keys committed straight into `application.properties` or hardcoded in a constants file, then pushed to a public GitHub repo. Treat API keys exactly like passwords. Store them hashed if you can verify by comparison, load them from environment variables or a secrets manager, and never log them, not even at DEBUG level.

TOPIC 06

Why Rate Limiting Exists

Picture a public sign-up endpoint with no rate limiting. Someone writes a ten-line script that hits it 500 times a second. Your database connection pool, sized for normal traffic, gets exhausted in seconds. Real users start getting timeouts on completely unrelated endpoints, because the same connection pool serves your whole app. Nobody hacked you. Nobody stole data. Your own unprotected endpoint took the system down.

That scenario, not some exotic security exploit, is the real-world reason every public API needs rate limiting. It protects three things at once: your infrastructure from being overwhelmed, your cost (every request to a third-party API or your own DB costs money at scale), and fairness, so one aggressive client cannot starve everyone else.

INTERVIEW FOCUS

When asked "how would you prevent abuse of this API," most freshers jump straight to "add a rate limiter," without saying what kind of abuse it stops, or where in the request path it should sit. Naming the actual failure mode first, like connection pool exhaustion or a brute-force login attempt, then introducing rate limiting as the fix, reads as far more senior.

TOPIC 07

Rate Limiting Strategies

There are a handful of algorithms that come up repeatedly in interviews. You do not need to implement all of them from memory, but you should be able to explain the idea behind each and, more importantly, why the naive one has a real flaw.

Fixed Window Counter (the naive one)

Count requests in a fixed time block, say every 60-second window. Allow up to N requests per window, reset the counter when the window ends. Simple to implement, usually with a single counter in Redis with a TTL.

WATCH OUT · The boundary problem

If your limit is 100 requests per minute, a client can send 100 requests at 0:59 and another 100 at 1:01. That is 200 requests in a 2-second window, technically obeying the rule on paper while completely defeating its purpose. This boundary burst is the exact flaw interviewers expect you to identify.

Sliding Window Log

Instead of resetting a counter, store a timestamp for every request in something like a Redis sorted set, then on each new request, count how many timestamps fall within the last 60 seconds (rolling, not fixed) and drop the old ones. This fixes the boundary burst problem completely, but it is more expensive: you are storing and scanning a timestamp per request instead of incrementing one number.

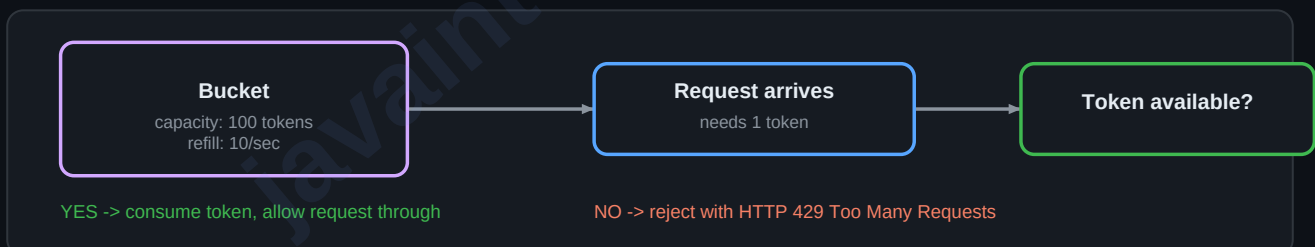
Sliding Window Counter (the practical middle ground)

Most production systems do not use the pure log approach because of its memory cost at scale. Instead they approximate it: keep two fixed window counters (current and previous), and weight the previous window's count based on how much of it overlaps the current rolling window. This gets you sliding-window accuracy without storing every timestamp.

Token Bucket (the one to actually know well)

This is the algorithm that comes up most often in interviews and is also what most real rate limiting libraries implement by default, including Bucket4j in the Java ecosystem.

Picture a bucket that holds a fixed number of tokens, say 100. Tokens refill at a steady rate, say 10 tokens per second, up to the bucket's capacity. Every request consumes one token. If the bucket is empty, the request gets rejected (usually with a 429). If a client has been idle, tokens accumulate up to the cap, letting them burst briefly before settling back into the steady rate.



What makes token bucket the favorite in interviews is that it naturally allows short bursts (a user refreshing a page quickly after being idle) while still enforcing a steady average rate over time. A fixed window cannot express that nuance at all.

Comparing the strategies

Strategy	Handles bursts well	Memory cost	Where it's used
Fixed Window	No, boundary burst flaw	Very low	Quick and dirty limits
Sliding Window Log	Yes, fully accurate	High, per-request storage	Strict billing-tier limits
Sliding Window Counter	Mostly, good approximation	Low	Most production APIs
Token Bucket	Yes, by design	Low	Bucket4j, AWS, Stripe-style APIs
Leaky Bucket	Smooths bursts into steady output	Low	Queue-based traffic shaping

TRADEOFF

There is no universal "best" algorithm. Token bucket is the default reasonable choice for most public APIs because it balances burst tolerance with steady-state enforcement. If you genuinely need zero tolerance for boundary bursts, like enforcing a hard compliance limit, sliding window log is worth its extra memory cost. Saying this out loud, instead of just naming "token bucket" and stopping, is what interviewers are actually listening for.

Where to apply the limit

A question that catches people off guard: should rate limiting live in your Spring Boot service, or in the API Gateway in front of it? In real systems, it usually lives in both, for different reasons.

INTERVIEW MOMENT

At the Gateway: coarse, fast rejection before a bad request even reaches your services. Protects everything downstream at once. This is where Spring Cloud Gateway's `RequestRateLimiter` filter or Redis-backed rate limiting at the edge typically sits.

Inside the service: fine-grained, business-aware limits, like "this specific user can only request a password reset 3 times an hour," which the gateway has no context to enforce.

JAVA + SPRING BOOT MAPPING

Two common Spring Boot approaches: **Bucket4j** for in-process token bucket limiting, often backed by Redis (`bucket4j-redis`) so the limit is shared correctly across multiple instances of your service. Or, at the edge, **Spring Cloud Gateway** filters combined with Redis for a centralized limit before requests even reach your services. A single in-memory counter per instance, without Redis, silently breaks the moment you scale to more than one instance, since each instance ends up enforcing its own separate limit.

TOPIC 08

What You're Actually Limiting By

Saying "I'll rate limit by IP" sounds reasonable until someone points out that an entire office, or an entire country behind a carrier-grade NAT, can share one public IP. Limiting by IP alone punishes innocent users sitting behind the same address as an abuser.

In practice, the key you rate limit on changes based on what you're protecting:

Limit by	Good for	Weakness
IP address	Anonymous/public endpoints, login attempts	Shared IPs punish innocent users, easy to rotate with proxies
User ID / JWT subject	Authenticated endpoints	Useless against pre-login abuse like signup spam
API key	Public API, server-to-server	Doesn't help if the key itself leaks and gets abused
IP + endpoint	Brute-force login protection	Needs care to avoid blocking legitimate retries

For something like a login endpoint specifically, the realistic answer combines two keys: limit by IP to slow down a distributed brute-force attempt, and limit by account/username to stop someone hammering one specific user's password regardless of which IP they rotate through.

What happens when your rate limiter itself fails

This is the question that catches people who only studied the happy path. If your rate limiter depends on Redis and Redis goes down, what happens to every request in your system?

WATCH OUT · Fail open vs fail closed

Fail closed: if the rate limiter can't reach Redis, reject the request. Safe for abuse, but now a Redis blip takes your entire API down, which is often a worse outcome than the abuse you were preventing.

Fail open: if the rate limiter can't reach Redis, let the request through unlimited. Keeps the system available, but leaves you exposed to abuse for as long as Redis is unreachable. Most production systems fail open for rate limiting specifically, and fail closed for authentication, because letting a request through unauthenticated is far more dangerous than letting one through unthrottled for a few seconds.

At my level, I would explain it as: rate limiting is a protective layer, not the core correctness of the system. Losing it briefly is recoverable. Losing your ability to check who someone is, is not. That asymmetry is usually why the two failure modes get treated differently.

REFERENCE

Java + Spring Boot Security Mapping

A quick lookup table for mapping everything covered so far to the actual Spring Boot building blocks. Useful when you want to go from "I understand the concept" to "I know which class I'd reach for."

Concept	Spring Boot Mapping
JWT validation	Spring Security Resource Server / JwtDecoder
Authentication filter	OncePerRequestFilter or SecurityFilterChain
Authorization	@PreAuthorize / requestMatchers
Password hashing	BCryptPasswordEncoder
OAuth login	spring-boot-starter-oauth2-client
API key validation	Custom filter checking X-API-Key
Rate limiting	Bucket4j or Spring Cloud Gateway RequestRateLimiter
Secrets	Environment variables or secrets manager
CORS	CorsConfigurationSource
Error handling	@ControllerAdvice

A minimal Bucket4j example

This is the shape of it, not a full production setup. Capacity of 100 tokens, refilling at 10 tokens a second. If a token is available, the request goes through. If not, it gets a 429.

```
Bandwidth limit = Bandwidth.classic(100,
    Refill.greedy(10, Duration.ofSeconds(1)));

Bucket bucket = Bucket.builder()
    .addLimit(limit)
    .build();

if (bucket.tryConsume(1)) {
    // token available, allow the request
    chain.doFilter(request, response);
} else {
    // no tokens left, reject
    response.setStatus(429);
}
```

TOPIC 09

Security Mistakes That Show Up Again and Again

These aren't theoretical OWASP slide-deck items. These are mistakes I have seen in real fresher codebases and heard in real interview answers, repeatedly enough that they're worth calling out by name.

1. Storing passwords with the wrong hash, or no hash at all

Plaintext passwords in a database is the obvious sin nobody admits to in an interview, but a more common real mistake is using a fast hash like MD5 or SHA-256 directly on the password. Fast hashes exist to be quick, which is exactly the wrong property for password storage, since it makes brute-forcing leaked hashes cheap. The correct tools are slow-by-design: bcrypt, scrypt, or Argon2. In Spring Security, this is just `BCryptPasswordEncoder`, and there's rarely a good reason to swap it for something else.

2. Putting sensitive data inside a JWT payload

Covered earlier, but worth repeating because it keeps happening: the payload is readable by anyone holding the token. Email and user id are usually fine. Password, full card number, or anything you wouldn't want printed on a public note, is not.

3. Trusting client-side validation as the only check

A frontend form that disables the submit button until a field looks valid is a UX nicety, not a security control. Anyone can call your API directly with curl or Postman, skipping the frontend entirely. Every validation that matters for security or data integrity has to be re-checked on the server, in the controller or service layer, every time.

4. No expiry on tokens, or absurdly long expiry

I've seen JWTs issued with a 1-year expiry "so users don't have to log in again." That single token, if leaked once (a logged request, a misconfigured analytics tool, a browser extension), stays exploitable for a year. Short-lived access tokens (15 minutes to an hour) paired with a refresh token is the standard pattern precisely to shrink that exposure window.

5. Exposing internal error details to the client

Returning a raw stack trace, or a message like "SQLException: duplicate key violates constraint users_email_key" in an API response, hands an attacker your table names, column names, and database engine for free. Generic error messages to the client, detailed logs on the server, is the rule. Spring Boot's default error handling exposes too much by default in dev mode; this is exactly what `@ControllerAdvice` and a custom `@ExceptionHandler` are for in production.

6. Missing CORS configuration, or configuring it too loosely

Two opposite mistakes happen here. Either CORS isn't configured at all and the frontend can't call the API from a different origin during development, so someone panics and sets `allowedOrigins("*")` in production. That second one quietly defeats a real protection. Wildcard origins combined with credentials enabled is specifically flagged as unsafe by browsers for good reason: it lets any website make authenticated requests to your API using a logged-in user's browser session.

7. Not rate limiting the login endpoint specifically

A general API rate limit doesn't automatically protect login. Login needs its own tighter, account-aware limit, because it's the single most attractive endpoint for credential-stuffing and brute-force attacks. If your general API allows 1000 requests an hour but login allows the same 1000, you've basically allowed 1000 password guesses an hour per IP.

8. Treating HTTPS as optional internally

"It's just internal service-to-service traffic, HTTPS is overkill" is a sentence I've actually heard. Inside a cloud VPC, the risk is lower but not zero, and compliance audits (PCI-DSS, SOC 2) will fail you on this regardless. The cost of TLS everywhere is small enough now that "it's internal" isn't a strong enough reason to skip it.

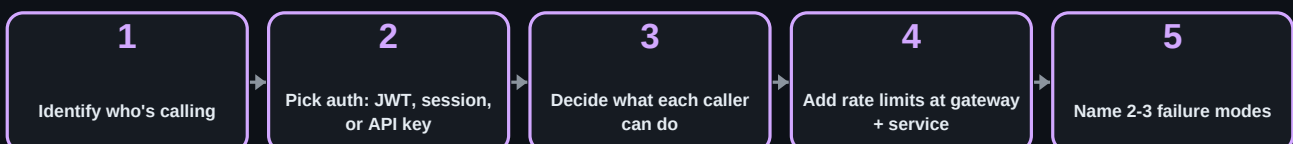
INTERVIEW FOCUS · How to use this section in an interview

You don't need to recite all eight. Pick one or two relevant to the system you're designing and mention them naturally while discussing failure handling or security. For a payment system, password hashing and error message exposure matter more. For a public API, rate limiting and CORS matter more. Showing judgment about which risk applies to which system reads far better than listing all of them mechanically.

TOPIC 10

Putting It Together When Asked Live

When an interviewer drops the question "how would you secure this API and protect it from abuse," mid-design, after you've already drawn your architecture, there's a natural order to walk through it in, rather than dumping everything you know at once.



Notice this doesn't start with "I'll use JWT." It starts with identifying who's actually calling the API, because that decision drives everything after it. A public-facing mobile app, a partner company's backend, and an internal admin dashboard all warrant different answers to the same question.

SENIOR TIP

If you only remember one thing from this module: don't present auth and rate limiting as a checklist you bolted on at the end. Mention them at the point in your design where they're actually relevant, like when you draw the API Gateway, or when you discuss the login endpoint. That placement alone makes the answer feel like real design thinking instead of memorized terms.

Security Answer Flow

- 1 Identify who is calling
- 2 Choose JWT, session, or API key
- 3 Add authorization rules
- 4 Add rate limits
- 5 Protect secrets and errors
- 6 Explain failure behavior

MODULE RECAP

What to carry into your next design

Authentication answers "who is this." Authorization answers "what can they do." Keep them as separate checks in your design, and in your explanation.

JWT trades a database lookup for a signature check. That's the whole value proposition, and also the source of its biggest weakness: you can't kill a token early without adding something extra.

OAuth lets your app borrow trust from another platform without ever holding that platform's password. "Login with Google" is authorization first, with identity riding on top of it.

Token bucket is the rate limiting algorithm worth knowing cold. Know its boundary-burst advantage over fixed windows, and know where it sits, both at the gateway and inside the service.

Most security mistakes aren't exotic. They're decisions made under time pressure that nobody revisited: a hardcoded key, a wildcard CORS rule, a year-long token expiry. Naming these specifically in an interview is worth more than naming OWASP Top 10 from memory.

QUICK REFERENCE

Module 12 Quick Revision Cheat Sheet

One line per concept. Use this the night before an interview, not as your first pass through the module.

Area	Remember This
Authentication	Proves who is calling. Happens once, at login.
Authorization	Decides what that caller can do. Checked on every request.
401 vs 403	401 means no valid identity. 403 means valid identity, no permission.
JWT payload	Readable by anyone. Never put secrets or passwords in it.
JWT signature	The only part that prevents tampering. No DB lookup needed to verify.
JWT revocation	Hard before expiry. Use short expiry plus refresh tokens, or a blocklist.
Sessions	Server-stored state. Easy to revoke, needs a shared store to scale.
OAuth	Authorization, not authentication. Lets you borrow trust without a password.
API keys	Identify an app or service, not a user. Long-lived, instantly revocable.
Token bucket	Allows short bursts, enforces a steady average rate. Default choice.
Sliding window	Fixes the fixed-window boundary burst flaw, at a higher memory cost.
Rate limit key	IP, user ID, or API key, picked based on what you're protecting.
Fail open vs closed	Rate limiters usually fail open. Auth checks usually fail closed.
Password hashing	Use bcrypt, scrypt, or Argon2. Never a fast hash like MD5 or SHA-256.
CORS	Wildcard origin plus credentials is a real vulnerability, not just a dev annoyance.
Security error handling	Generic errors to the client, detailed logs on the server.

Next: Module 13 explains logging, monitoring, and observability, including logs, metrics, alerts, distributed tracing, and health checks.

MODULE 13

Logging, Monitoring, and Observability

Logs, metrics, alerts, distributed tracing, health checks, and production debugging for Java backend systems.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

What This Module Covers

In this module, you will learn the difference between logs, metrics, and traces, why production systems need all three, how to talk about monitoring in an interview without sounding like you memorized a glossary, and how this maps to Spring Boot Actuator, Micrometer, the ELK stack, and distributed tracing tools.

Topic Roadmap

01	Why Observability Is Not Optional
02	Logging
03	Metrics
04	Alerting
05	Distributed Tracing
06	Health Checks
07	When Monitoring Tools Fail
08	Interview Summary and Tradeoffs
09	Quick Revision Cheat Sheet

WHAT YOU WILL LEARN

- Difference between logs, metrics, and traces
- How structured logging helps during incidents
- How Micrometer, Prometheus, and Grafana fit together
- What to alert on and what not to alert on
- How distributed tracing finds slow services
- Liveness vs readiness checks in Spring Boot
- How to explain observability in interviews

TOPIC 13.1

Why Observability Is Not Optional

Picture this. It is 2 AM. Your payment service is returning 500 errors for about 3% of requests. You have no idea why. You SSH into the server, you start grepping through log files by hand, and twenty minutes later you are still not sure if the problem is the database, a downstream API, or your own code. By the time you find it, you have lost orders and a few customers have already tweeted about it.

This is the exact situation observability is built to prevent. Most freshers think monitoring just means "check if the server is up." That is not even close to what production teams actually need. Observability means you can answer three questions quickly, without guessing: what happened, where did it happen, and why did it happen.

If you cannot answer **what, where, and why** within a few minutes during an incident, your system is not observable. It might be running fine, but you are flying blind the moment something breaks.

Three pillars usually come up when people talk about observability. Logs tell you exactly what happened in a specific request. Metrics tell you the overall health and trend of the system over time. Traces tell you how a single request traveled across multiple services. Each one answers a different question, and none of them replace the other two.

THE THREE PILLARS OF OBSERVABILITY



Here is something most candidates miss in interviews. Interviewers do not expect you to design a full observability platform. They want to see that you think about failure as a normal part of the system, not an afterthought you bolt on at the end. If you mention logging, metrics, and tracing naturally while discussing scaling or failure handling, you immediately sound like someone who has actually run a service in production.

TOPIC 13.2

Logging: Your First Line of Defense

Most freshers think logging means `System.out.println` scattered across the codebase. In a real backend, that approach falls apart within weeks. You end up with logs that are unstructured, impossible to search, and missing the context you actually need when something goes wrong at 2 AM.

A production-grade logging setup needs three things: structured output (usually JSON), log levels that mean something consistent across the team, and a way to trace a single request across multiple log lines and services. Without that third part, logs from a microservices system become close to useless during an incident, because you cannot tell which log lines belong to which user request.

Log Levels and What They Actually Mean

LEVEL	WHEN TO USE IT	PRODUCTION VOLUME
ERROR	Something failed and needs attention now	Low, should be rare
WARN	Something unexpected but recoverable happened	Low to moderate
INFO	Key business events: order placed, user signed up	Moderate
DEBUG	Detailed flow for troubleshooting	Disabled in prod usually

In real projects, this usually becomes a problem when teams log everything at INFO level because nobody agreed on a convention early on. Six months later, the logs are so noisy that nobody actually reads them anymore, and the one ERROR line that mattered gets buried under ten thousand routine INFO lines from the same hour.

WATCH OUT

Never log sensitive data. Passwords, card numbers, full auth tokens, and personal identifiers should never appear in plain log output. This is one of the most common mistakes freshers make, and it is also one of the fastest ways to fail a security review.

Structured logging means each log line is a JSON object instead of a free-text sentence. This sounds like a small change, but it changes everything about how useful your logs are. A free-text log line is something a human reads. A structured log line is something a machine can search, filter, and aggregate.


```
// Free-text log (hard to search)
log.info("Order " + orderId + " placed by user " + userId);

// Structured log (queryable in Kibana/Splunk)
log.info("order.placed",
    Map.of("orderId", orderId, "userId", userId, "amount", amount));
```

If I were designing this in Spring Boot, I would usually start with Logback (Spring Boot's default), configure a JSON encoder using `logstash-logback-encoder`, and add a correlation ID filter that injects a unique request ID into the MDC (Mapped Diagnostic Context) for every incoming request. That ID then shows up on every log line generated while handling that request, including log lines from downstream service calls if you propagate it through headers.

JAVA + SPRING BOOT MAPPING

Logging framework: SLF4J as the facade, Logback as the implementation (Spring Boot default).

Structured output: logstash-logback-encoder for JSON formatted logs.

Correlation ID: a Servlet Filter that generates or reads a request ID and puts it into `MDC.put("requestId", id)` at the start of each request, then clears it after.

Propagation across services: pass the ID as an HTTP header (e.g. X-Request-Id) using WebClient or RestTemplate interceptors, and read it back into MDC on the receiving service.

Once logs are structured and tagged with a request ID, they get shipped somewhere centralized. Nobody is grepping log files on individual servers in a system with more than a couple of instances. The standard approach is to ship logs to a centralized store like the ELK stack (Elasticsearch, Logstash, Kibana), Grafana Loki, or a managed service like Datadog or Splunk.

CENTRALIZED LOG PIPELINE



Many beginners design this directly into the application, meaning every service writes logs straight to Elasticsearch over HTTP. That creates an issue later: if Elasticsearch is slow or down, your application threads start blocking on log writes, which is the opposite of what you want during an incident. Log shipping should be decoupled from the request path, usually by writing to local files or stdout and letting an agent like Filebeat or Fluentd tail and forward them asynchronously.

TOPIC 13.3

Metrics: The Pulse of Your System

Here is a question interviewers love to ask after you finish a design: "How would you know if this system is healthy right now, without looking at logs?" The answer is metrics. Logs tell you about one event. Metrics tell you about the system as a whole, over time, as a number you can graph and alert on.

A metric is just a number that changes over time: requests per second, average response time, error rate, CPU usage, queue depth, active database connections. What makes metrics powerful is that they are cheap to collect and cheap to store compared to logs, because you are aggregating numbers instead of storing full text for every single event.

The Four Golden Signals

Google's SRE book popularized four signals that cover almost everything you need to know about a service's health. These come up often enough in interviews that they are worth knowing by name, not just by concept.

SIGNAL	WHAT IT MEASURES	EXAMPLE
Latency	Time taken to serve a request	p99 response time = 420ms
Traffic	Demand on the system	1200 requests/sec
Errors	Rate of failed requests	2.1% of requests return 5xx
Saturation	How full the system is	DB connection pool at 95%

A mistake I see a lot from junior engineers: they track average response time and feel confident the system is fast. Average hides outliers. If 99% of your requests take 50ms but 1% take 8 seconds, your average still looks fine, but one percent of your users are having a terrible experience. That is why production teams track percentiles like p50, p95, and p99 instead of relying on the average alone.

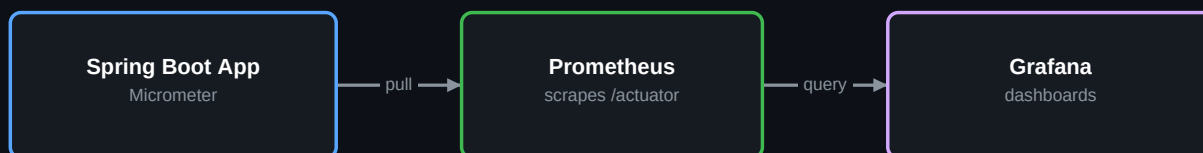
p99 latency of 2 seconds means 99 out of 100 requests finish in under 2 seconds, and 1 out of 100 takes longer. If you have 10,000 requests per minute, that 1% is still 100 unhappy users every minute. Averages would never show you this.

Metrics generally come in a few shapes. Counters only go up, like total requests served. Gauges go up and down, like current memory usage. Histograms bucket values to give you percentile data, which is exactly how response time tracking works under the hood.

In Spring Boot, this is where Micrometer comes in. Micrometer is a metrics facade, similar in spirit to how SLF4J is a logging facade. You write metric code once using Micrometer's API, and it ships that data to

whichever backend you plug in, whether that is Prometheus, Datadog, or CloudWatch.

METRICS PIPELINE WITH MICROMETER



Notice the arrow direction in that diagram. Prometheus pulls metrics from your application on a schedule, usually every 15 seconds, by hitting an endpoint. Your application does not push data to Prometheus. This is a deliberate design choice, and it is a good thing to mention if asked, because it means Prometheus can detect when a service goes silent simply by noticing the scrape fails.

JAVA + SPRING BOOT MAPPING

Metrics facade: Micrometer, bundled with Spring Boot Actuator.

Exposing metrics: add the micrometer-registry-prometheus dependency, which exposes a `/actuator/prometheus` endpoint automatically.

Custom metrics: inject a `MeterRegistry` and call `registry.counter("orders.placed").increment()` or use a `Timer` around a method using the `@Timed` annotation.

Visualization: Grafana reads from Prometheus and renders dashboards for latency, traffic, errors, and saturation.

TOPIC 13.4

Alerting: Knowing Before Your Users Do

Collecting metrics is only half the job. If nobody looks at the dashboard until a customer complains, you have not actually improved anything over the SSH-and-grep approach. Alerting is what turns metrics into something useful at 2 AM, by waking up the right person before things get worse.

A common interview follow-up after you mention monitoring is: "What would you alert on?" The honest answer is not "alert on everything." Too many alerts cause alert fatigue, where engineers start ignoring notifications because most of them turn out to be noise. Good alerting focuses on symptoms that actually affect users, not every internal fluctuation.

Error rate above 5% for 5 minutes	→	Page the on-call engineer
p99 latency doubles vs baseline	→	Alert team channel, investigate
CPU usage above 90% sustained	→	Auto-scale, alert if scaling fails
Disk usage above 85%	→	Warn ahead of time, not an emergency
Single failed health check	→	Log only, do not page anyone

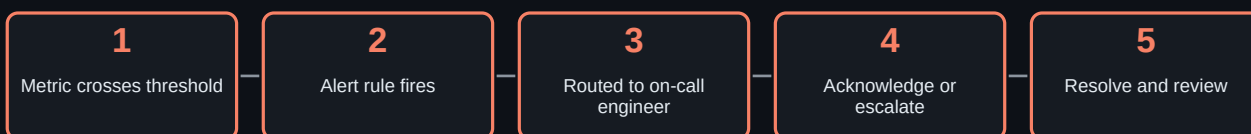
Notice that not every condition pages a human immediately. Pages should be reserved for things that need a person right now. Everything else can be a dashboard you check, a warning in a Slack channel, or a ticket that gets picked up during business hours. This distinction matters a lot in interviews, because it shows you understand the human cost of bad alerting, not just the technical side.

WATCH OUT

A classic mistake: alerting on a single data point instead of a sustained trend. One slow request does not mean the system is down. Alert rules should usually require a condition to hold for a window of time, like "error rate above 5% for 5 minutes," not a single spike that could be a fluke.

In real projects, this usually becomes a problem when a team sets up alerts during a calm period and never revisits them. Six months later traffic has tripled, the thresholds that made sense before are now firing constantly, and the on-call engineer mutes the channel entirely. Alert thresholds need to be revisited as the system grows, the same way capacity planning does.

Tools like Prometheus Alertmanager, Grafana alerting, PagerDuty, and Opsgenie handle the routing piece: deciding who gets notified, through what channel, and with what urgency, plus things like escalation if the first person does not respond in time.



TOPIC 13.5

Distributed Tracing: Following a Request Across Services

Here is a real interview question: "A user reports their order checkout took 9 seconds. Your system has 6 microservices involved in checkout. How do you find out which one is slow?" Logs alone will not answer this cleanly, because you would need to manually stitch together log lines from 6 different services using timestamps and hope they line up. This is exactly what distributed tracing solves.

A trace represents one complete request as it moves through your system. Inside a trace, each individual unit of work, like a database query or a call to another service, is called a span. Spans have a parent-child relationship that mirrors how the request actually flowed, so you end up with a visual waterfall showing exactly where time was spent.

TRACE WATERFALL: CHECKOUT REQUEST (9000ms total)



Looking at this trace, the bottleneck is obvious immediately. Order Service and Inventory Service together account for less than a second. The external bank API call inside Payment Service is eating 7.6 of the 9 seconds. Without tracing, you might have spent an hour suspecting your own database before realizing the slowness was a third-party dependency the whole time.

The way tracing actually works under the hood: when a request enters your system, a trace ID and a span ID get generated. Every time that request triggers a call to another service, the current trace ID is passed along in the request headers, and a new span ID is created for that hop. The tracing backend collects all these spans and reconstructs the full picture using the shared trace ID.

TRADEOFF

Tracing everything vs sampling. Capturing a trace for every single request gives complete visibility but adds overhead and storage cost at high traffic. Most production systems sample, meaning they trace a percentage of requests, often boosted to 100% automatically for requests that error out, since those are the ones you actually need to debug.

In Spring Boot, the standard tool for this used to be Spring Cloud Sleuth, which has since moved into Micrometer Tracing as the underlying instrumentation layer. It auto-instruments your HTTP calls, database queries, and messaging operations so you do not have to manually create spans everywhere. The actual trace data then gets exported to a backend like Zipkin or Jaeger for visualization.

JAVA + SPRING BOOT MAPPING

Instrumentation: Micrometer Tracing (formerly Spring Cloud Sleuth) auto-injects trace and span IDs into outgoing HTTP calls and the MDC.

Trace propagation: headers like traceparent (W3C format) carry the trace context between services automatically when using WebClient or RestTemplate with the tracing bridge.

Backend / visualization: Zipkin or Jaeger collect spans and render the waterfall view shown above.

Sampling config: management.tracing.sampling.probability in application.yml controls what fraction of requests get traced.

What most freshers say in interviews when asked about debugging a slow distributed request is something like "I would check the logs of each service." What interviewers actually want to hear is that you understand tracing exists precisely because checking logs of each service individually does not scale once you have more than two or three services involved in a single request.

TOPIC 13.6

Health Checks: Letting the System Check Itself

A health check is the simplest piece of observability infrastructure you will build, and it is also one of the most misunderstood. A health check endpoint answers one question: is this instance able to serve traffic right now? Load balancers, Kubernetes, and orchestration tools poll this endpoint constantly to decide whether to send traffic to an instance or pull it out of rotation.

A mistake almost every fresher makes the first time they build a health check: they return 200 OK as long as the application process is running, without checking anything else. That tells you the JVM started, not that the service can actually do its job. If the database connection pool is exhausted or a required downstream dependency is unreachable, a "dumb" health check will happily report healthy while every real request fails.

LIVENESS CHECK

- Is the process alive and responsive?
- Should NOT check external dependencies
- Failure means: restart the instance
- Example: can the app respond to HTTP at all

READINESS CHECK

- Can this instance handle real traffic?
- Should check DB, cache, critical dependencies
- Failure means: stop sending traffic, don't restart
- Example: DB connection pool is healthy

This liveness versus readiness distinction matters a lot in container orchestration. If you only have one combined health check and it fails because the database is briefly slow, Kubernetes might restart your application instance, which does nothing to fix a database problem and just adds unnecessary restarts to your incident. Separating the two checks means transient downstream issues pull an instance out of the load balancer rotation without killing it.

If a candidate tells me their health check only verifies the process is running, I usually follow up by asking what happens if the database is unreachable but the JVM is fine. Most freshers pause here. This is your moment to mention readiness checks and show you have thought past the obvious answer.

Spring Boot Actuator gives you this almost for free. The `/actuator/health` endpoint aggregates health indicators automatically: database connectivity, disk space, and any custom indicator you register yourself. Each indicator reports UP, DOWN, or a custom status, and Actuator rolls them up into one overall result.


```
@Component
public class PaymentGatewayHealthIndicator implements HealthIndicator {

    public Health health() {
        if (paymentClient.isReachable()) {
            return Health.up().build();
        }
        return Health.down()
            .withDetail("reason", "payment gateway unreachable")
            .build();
    }
}
```

JAVA + SPRING BOOT MAPPING

Built-in endpoint: /actuator/health, enabled by Spring Boot Actuator.

Liveness group: management.endpoint.health.group.liveness.include configures what counts toward liveness (usually just the app itself).

Readiness group: management.endpoint.health.group.readiness.include adds DB, cache, and critical downstream checks.

Custom checks: implement HealthIndicator for any dependency that matters, like a payment gateway or a message broker connection.

Kubernetes integration: probes point directly at /actuator/health/liveness and /actuator/health/readiness.

TOPIC 13.7

When Your Monitoring Tools Are the Ones Failing

There is an uncomfortable truth that experienced engineers learn the hard way: your logging and monitoring stack can fail too. If you have never thought about this, it is worth sitting with for a second, because it is a great follow-up question interviewers ask once you have described a monitoring setup.

FAILURE	WHAT USERS SEE	WHAT TO DO
Elasticsearch cluster down	Nothing directly, but you lose visibility	Log to local disk as fallback, alert on shipper lag
Prometheus scrape failing	Nothing directly	Alert on "target down" as its own signal
Alertmanager misconfigured	Real incidents go unnoticed	Periodic synthetic alert to test the pipeline
Tracing backend overloaded	Slightly higher latency from overhead	Reduce sampling rate, scale tracing backend

The single most dangerous failure mode in this list is Alertmanager silently misconfigured. Your dashboards still show data, your logs still get written, but nobody gets paged when something actually breaks. From the outside, everything looks fine until a customer escalates, and now you are debugging a production incident and a broken alerting pipeline at the same time.

WATCH OUT

Mature teams run a synthetic check: a deliberate, harmless test alert fired on a schedule that should always reach someone. If that test alert ever stops arriving, you know your alerting pipeline itself is broken, independent of whether anything else in production is having issues.

Another practical decision worth knowing: where do logs go if the centralized logging pipeline is unreachable? The answer in most production setups is that the application keeps writing to local disk or stdout regardless. The shipping agent retries sending logs once the pipeline recovers, and nothing is lost as long as local disk does not fill up first.

At my level, I would explain this to an interviewer by saying that observability tools are themselves a dependency, and dependencies fail. The fix is not to make them unbreakable, because nothing is, it is to make sure their failure mode is graceful and, ideally, detectable on its own.

MODULE SUMMARY

Bringing It All Together for the Interview

If an interviewer asks you to design any backend system and you never once mention how you would know it is healthy in production, that is usually a missed opportunity, not a disqualifier. Weaving observability naturally into your answer, even briefly, signals that you think about systems the way someone who has actually operated one would.

Five Tradeoffs Worth Knowing By Heart

Log everything vs log selectively

Logging everything gives full visibility but costs storage and makes search slower. Most teams log INFO and above in production, with DEBUG available on demand for specific troubleshooting windows.

Trace every request vs sample

Full tracing gives complete data but adds overhead at scale. Sampling, often with 100% capture on errors, balances cost against coverage.

Push metrics vs pull metrics

Pull-based (Prometheus) detects silent failures naturally since a missed scrape is itself a signal. Push-based fits short-lived jobs better, like batch processes that finish before a scrape would happen.

Centralized logging vs local-only

Centralized logging enables cross-service search but adds infrastructure and a new failure point. Local-only is simpler but does not scale past a handful of instances.

Alert on symptoms vs alert on causes

Symptom-based alerts (high error rate) tell you something is wrong for users right now. Cause-based alerts (CPU high) can fire even when users are unaffected, contributing to noise if overused.

INTERVIEW FOCUS

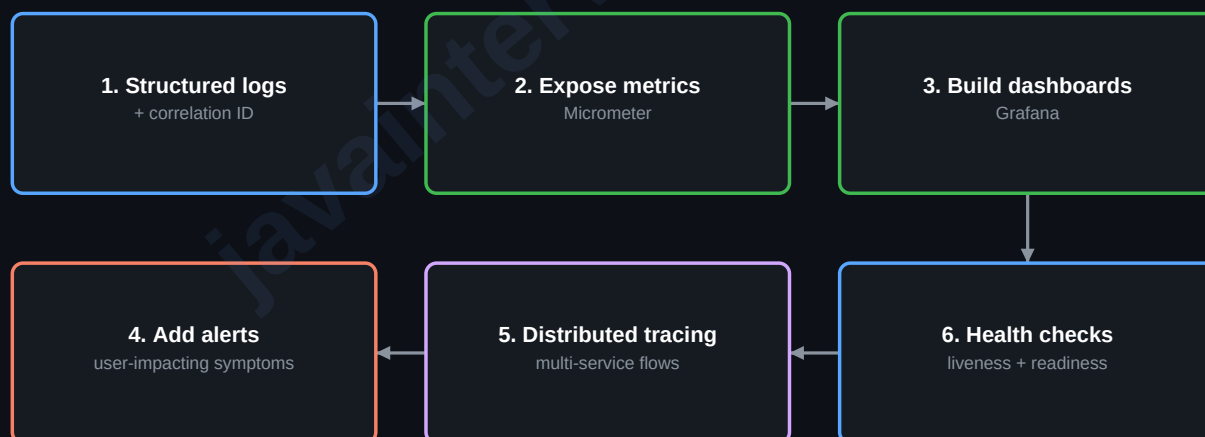
When asked to explain your observability approach, a strong answer covers all three pillars briefly rather than going deep on just one. Something like: "I would add structured logging with a correlation ID, expose key metrics through Micrometer for dashboards and alerts, and add distributed tracing once the system has more than two or three services in the request path." That single sentence covers logs, metrics, and tracing, and ties each one to a concrete reason.

One more thing worth remembering before you walk into an interview: observability is not about using every tool you have heard of. It is about being able to answer what happened, where, and why, quickly, without guessing. Logs, metrics, traces, and health checks are simply the tools that get you there. Everything else, the specific vendor, the specific dashboard, is a detail you can figure out on the job.

INTERVIEW ANSWER STRUCTURE

Observability Interview Answer Flow

When an interviewer asks how you would make a system observable, walking through this order shows a clear, layered way of thinking rather than naming tools at random.

HOW TO BUILD UP AN OBSERVABILITY ANSWER, STEP BY STEP

Notice this flow starts cheap and gets more involved. Structured logs and metrics are low effort and high value, so they come first. Tracing is the most expensive piece to set up correctly, which is why it is usually the last thing added once a system has enough services to justify it.

FINAL REVISION

Module 13 Quick Revision Cheat Sheet

Use this table as a last-minute scan before an interview. Each row is a concept, the question it answers, and where it shows up in a Spring Boot stack.

CONCEPT	WHAT IT ANSWERS	JAVA / SPRING BOOT MAPPING
Logs	What happened in one request?	SLF4J + Logback
Structured logging	Can a machine search this log?	logstash-logback-encoder (JSON)
Correlation ID	Which log lines belong to the same request?	MDC + Servlet Filter + X-Request-Id header
Metrics	Is the system healthy right now, over time?	Micrometer
Golden signals	Latency, traffic, errors, saturation	Micrometer Timer / Counter / Gauge
Prometheus	Who collects the metrics?	/actuator/prometheus (pull-based)
Grafana	How do I visualize metrics?	Dashboards reading from Prometheus
Alerting	Who gets notified, and when?	Alertmanager, PagerDuty, Opsgenie
Distributed tracing	Which service in the chain is slow?	Micrometer Tracing (was Sleuth)
Trace ID and span ID	How do spans link into one trace?	Propagated via traceparent header
Health checks	Can this instance serve traffic?	Spring Boot Actuator
Liveness	Is the process alive? Restart if not.	/actuator/health/liveness
Readiness	Can it handle traffic? Pull from LB if not.	/actuator/health/readiness
Actuator	Where do health and metrics endpoints live?	spring-boot-starter-actuator
Synthetic alert	Is the alerting pipeline itself working?	Scheduled test alert, always expected

MODULE COMPLETE

You now know how to make a Java backend system observable, and how to talk about it in an interview without sounding like you memorized a glossary.

Next: Module 14 covers 15 classic system design questions, including URL Shortener, Chat Application, Notification System, File Upload System, Food Delivery App, E-Commerce Order System, Payment System, Ride Booking App, Instagram Feed, YouTube Video Upload, BookMyShow Ticket Booking, Rate Limiter, Logging System, Online Code Judge, and WhatsApp Last Seen / Online Status.

javainterviewready

QUESTION 01 OF 15 · MODULE 14

URL Shortener

A classic system design question. It looks simple on the surface, which is exactly why interviewers like it. The real signal is not whether you can build a redirect service. It is whether you can reason about read heavy traffic, ID generation, and caching without overthinking the problem.

01 PROBLEM STATEMENT

Most candidates jump straight to ID generation when they hear "URL shortener." Slow down for a second. Before any design, say out loud what you are actually building.

We are building a service that takes a long URL and returns a short alias. When someone opens the short link in a browser, the service looks up the original URL and redirects them there. That is the entire product in one sentence. Everything else, custom aliases, expiry, click analytics, is a feature layered on top of that core flow.

Who uses it? Two kinds of users in practice. Marketing teams and individual users who create short links through a website or API, and a much larger group of anonymous people who simply click those links from social media, emails, or messages. The second group never creates anything. They only trigger redirects, and there are far more of them.

Expected scale for an interview answer: assume something like 100 million new short links created every month, with redirect traffic running at 100x to 1000x that volume. You do not need the exact number from the interviewer. State a reasonable assumption and move on. That is what real engineers do when requirements are not handed to them in full.

INTERVIEW FOCUS

Many freshers describe this system as "a database that stores short codes." That answer stops an interview cold. An interviewer wants to hear that you understand this is a read heavy system where the redirect path has to stay fast and available even when the rest of the system is under pressure.

02 REQUIREMENTS

Do not start drawing boxes yet. Spend two minutes splitting requirements into functional and non-functional. This is the part most freshers skip, and it is the part that makes your answer sound senior.

Functional requirements

Requirement	What it means
Shorten a URL	Accept a long URL and return a short code or alias.

Requirement	What it means
Redirect	Visiting the short URL sends the user to the original long URL.
Custom alias	User can optionally pick their own short code instead of a generated one.
Expiry	A link can optionally stop working after a set date.
Click analytics	Track how many times a short link was opened, at least approximately.

Non-functional requirements

Requirement	What it means
High availability	Redirects should keep working even if parts of the system degrade.
Low latency	A redirect should feel instant, ideally under 100ms.
Uniqueness	Two different long URLs should never end up with the same short code.
Read heavy at scale	Reads will outnumber writes by a large margin, design around that.
Eventual consistency is fine	Click counts do not need to be exact in real time.

Notice what is missing from that list. Strong consistency. This system does not need it everywhere. The short code to long URL mapping needs to be correct and durable, but the click count does not need to be perfectly accurate the second someone clicks. If you call this out yourself, without being asked, that is usually the moment an interviewer relaxes a little and starts treating you as someone who has actually thought about tradeoffs.

02.5 CAPACITY ESTIMATION

You do not need to be fast at mental math here. You need to show the thinking. Pick round numbers and walk through them out loud.

CAPACITY ESTIMATION WALKTHROUGH

Assume 100 million new short links created per month

Write traffic (creation)

100,000,000 / 30 days / 86,400 sec ~ 38 writes/sec average

Peak traffic (5x average) ~ 190 writes/sec

Read traffic (redirects)

Assume a 100:1 read to write ratio

38 x 100 ~ 3,800 reads/sec average, ~19,000/sec at peak

Storage growth

Each row ~ 500 bytes (short code + long URL + metadata)

100,000,000 x 500 bytes ~ 50 GB / month ~ 600 GB / year

Cache sizing

Most traffic hits a small percentage of links (recent + popular)

Caching the hottest few million mappings in Redis covers most of the redirect traffic without holding every row ever created

This estimation is what drives three design decisions later in this answer. Read heavy traffic at 19,000 requests per second tells you caching is not optional. Storage growth of 600GB a year tells you the database choice should scale horizontally without too much trouble. And the read to write ratio tells you the write path can afford a small amount of extra work, like checking for collisions, because writes are rare compared to reads.

03 APIs

Keep the API surface small. Four endpoints cover this entire system. Do not invent extra endpoints just to look thorough, interviewers notice when an API list is padded.

Method & Path	Request / Response	Notes
POST /api/v1/urls	Req: longUrl, customAlias (optional), expiryAt (optional) Res: shortUrl, code, expiryAt	Creates a new short URL. Validates the long URL format first.
GET /{code}	Req: none Res: HTTP 302 redirect to the long URL	The hot path. Called far more often than every other endpoint combined.
GET /api/v1/urls/{code}/stats	Req: none Res: code, longUrl, clickCount, createdAt, expiryAt	Read only. Backed by the eventually consistent click counter.
DELETE /api/v1/urls/{code}	Req: none Res: 204 No Content	Owner can deactivate a link before its natural expiry.

INTERVIEW MOMENT

If an interviewer asks why GET `/` returns a redirect instead of a JSON response with the long URL inside it, the answer is simple. A real redirect lets the browser handle navigation natively, works with every client without custom logic, and lets you control caching behaviour through standard HTTP semantics using 301 versus 302.

04 DATABASE DESIGN

The access pattern here is almost entirely key lookups. Give me a short code, give me back a long URL. That single fact should guide most of this section.

Table: urls

Column	Type	Notes
id	bigint	Internal primary key, used for ID generation if base62 encoding.
short_code	varchar(10)	Unique index. This is the lookup key on every redirect.
long_url	text	The original destination URL.
created_by	bigint, nullable	User ID if the creator was authenticated.
created_at	timestamp	Used for cleanup jobs and basic reporting.
expiry_at	timestamp, nullable	Null means the link never expires.
click_count	bigint	Updated asynchronously, not on the request path.
is_active	boolean	Soft delete flag instead of removing rows outright.

Now the question every interviewer eventually asks. SQL or NoSQL? There is no single correct answer, but you need to argue both sides clearly.

SQL (PostgreSQL)

- Easy to reason about for a small team and a smaller scale.
- Unique constraint on short_code gives you collision protection for free.
- Simple to run admin queries and reports against.
- Vertical scaling and read replicas cover a lot of ground before you hit limits.

NoSQL Key-Value (DynamoDB)

- Built for exactly this access pattern: key in, value out.
- Scales horizontally with far less operational effort at huge volume.
- Lower and more predictable latency on simple key lookups.
- Harder to run ad-hoc analytical queries, you need a separate pipeline for that.

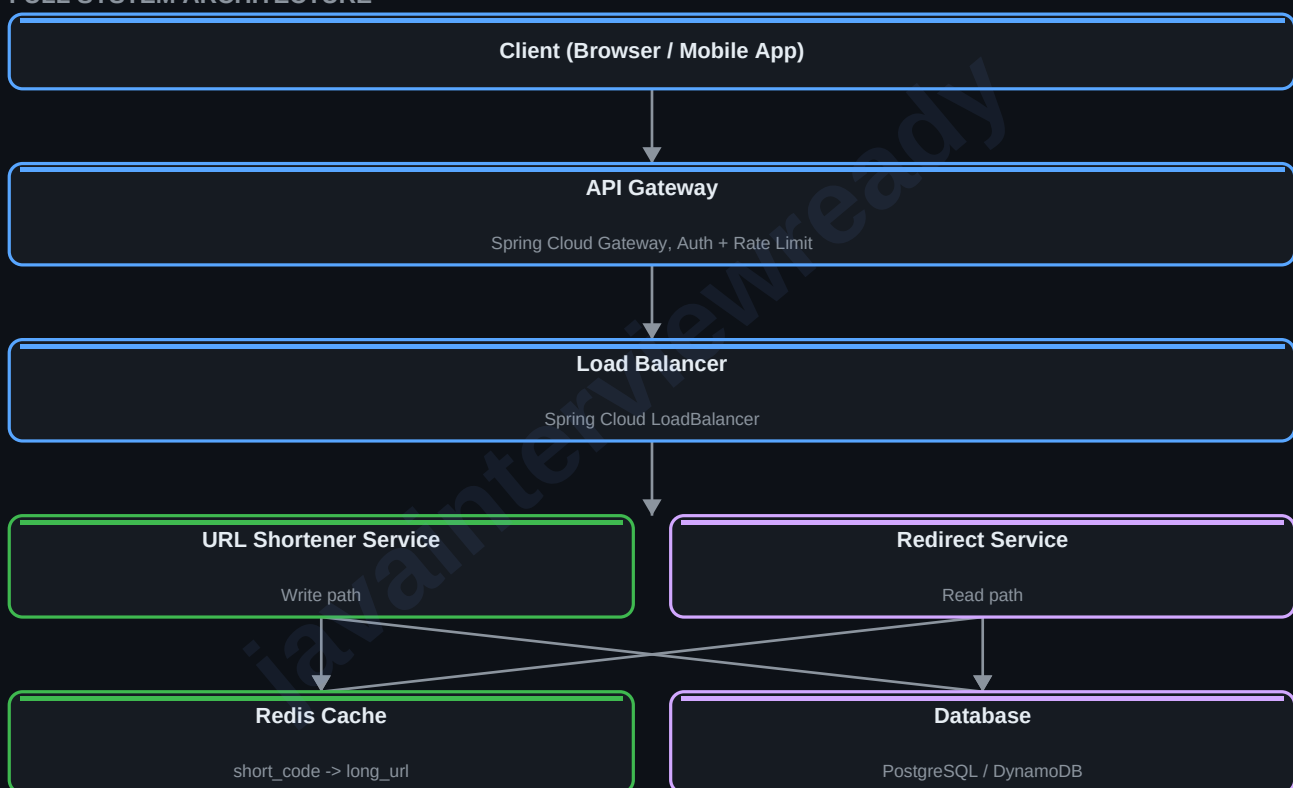
In real projects, this usually becomes a problem when the team picks NoSQL on day one for a product that has no real scale yet. You end up fighting the database for features it was never meant to give you, like

flexible reporting. At this level, the safer interview answer is to start relational, explain that the schema and access pattern will migrate cleanly to a key value store later, and describe what would trigger that migration.

05 HIGH-LEVEL DIAGRAM

This is the diagram you should be able to draw on a whiteboard in under two minutes. Notice the split between the write path and the read path starting at the service layer. That split is the single most important idea in this entire design.

FULL SYSTEM ARCHITECTURE



TIP

Both services touch both the cache and the database, just for different reasons. The write path populates the cache right after writing to the database so the very first redirect for a new link is already fast. The read path checks the cache first and only falls back to the database on a miss.

06 KEY FLOW DIAGRAM

Out of everything in this system, the redirect flow is the one interviewers care about most. It is the path that runs millions of times a day, so walk through both outcomes: a cache hit and a cache miss.

CACHE HIT PATH (MAJORITY OF TRAFFIC)**CACHE MISS PATH (CONTINUES FROM REDIRECT SERVICE)**

If I were explaining this in an interview, I would say it exactly like that: most requests never touch the database at all. The cache miss path exists for correctness, not for the common case. That single sentence usually tells the interviewer you understand why the cache is there in the first place, instead of just knowing that Redis is involved.

07 COMPONENTS EXPLANATION**API Gateway**

Sits in front of both services. Handles authentication for the creation endpoint, applies rate limiting so one user cannot flood the system with short link creation requests, and routes traffic to the right service. The redirect endpoint usually skips heavy auth checks since anyone should be able to click a public link.

URL Shortener Service (write path)

Validates the incoming long URL, generates a short code if the user did not supply a custom alias, checks for collisions, writes the row to the database, and pushes the new mapping into the cache. This service runs far less often than the redirect service, so it can afford slightly heavier work per request.

Redirect Service (read path)

Does one thing. Given a short code, return the long URL as fast as possible. Checks Redis first, falls back to the database on a miss, and asynchronously fires a click event instead of updating a counter inline. This service is the one you scale aggressively, because it absorbs almost all the traffic in this entire system.

Redis Cache

Stores the hot subset of short_code to long_url mappings. Most systems like this follow a Pareto pattern where a small fraction of links account for most of the clicks, so the cache does not need to hold every row ever created, just the ones currently in demand.

Database

The source of truth. Every mapping eventually lives here, whether or not it is currently cached. Many beginners design this directly as the only storage layer and skip the cache entirely, which works fine in a

demo and falls over the moment real traffic shows up.

ID Generation

This deserves its own discussion because it is where most candidates either shine or stumble. There are three common approaches, and you should know the tradeoffs between them.

Approach	How it works	Tradeoff
Auto-increment + base62	Encode the database row's numeric ID into a short base62 string.	Simple and collision free, but IDs are sequential and guessable. A competitor can estimate how many links you create just by watching the pattern.
Random string + retry	Generate a random short code, try to insert it, retry on a unique constraint violation.	No predictable pattern, but retries increase as the keyspace fills up, which hurts write latency at very large scale.
Pre-generated key pool	A separate key generation service pre-computes millions of unique codes ahead of time and hands them out on demand.	No collision checks on the hot write path at all, this is the approach real systems at scale tend to use, at the cost of running one more service.

08 SCALING APPROACH

Scaling this system is mostly about protecting the redirect path. Talk about reads and writes separately, that structure alone makes your answer easier to follow.

Scaling reads

Run multiple stateless instances of the Redirect Service behind the load balancer, since there is no session state tying a request to a specific instance. Push the cache hit ratio as high as you reasonably can, a well tuned cache here should be absorbing well over 90 percent of redirect traffic. For extremely popular links, consider caching at the CDN or edge layer so the request never even reaches your origin servers.

Scaling writes

Writes are rare compared to reads, but they still need a path that does not become a bottleneck. The pre-generated key pool approach removes collision checking from the write path entirely. If the database itself becomes a bottleneck at very large scale, shard by a hash of the `short_code` so writes spread evenly across nodes instead of hitting one table.

Scaling analytics

Do not update `click_count` synchronously inside the redirect request. That turns a fast read into a write on the hottest path in the entire system, and a popular link would create heavy lock contention on a single row. Publish a lightweight click event to Kafka instead, and let a separate consumer aggregate counts in the background.

INTERVIEW FOCUS

A common follow-up question here is: "What if the interviewer asks how you would handle a single link going viral and getting a million redirects in an hour?" The answer is that your cache already protects you, since that one hot key stays in Redis and almost never touches the database. Mention that explicitly, it shows you can reason about hot keys, not just average load.

09 FAILURE HANDLING

Interviewers ask about failure scenarios to see if you have actually operated something in production, not just designed it on paper. Go through these out loud even if not asked.

Failure	What happens	How you handle it
Redis is down	Every redirect would fall through to the database.	Wrap the cache call in a circuit breaker. On repeated failures, stop hitting Redis for a short window and go straight to the database to avoid wasting time on timeouts.
Database is slow	Both the write path and cache miss reads start timing out.	Cached reads keep working fine since they never touch the DB. For writes, return a clear error and let the client retry, do not let one slow dependency take down the whole API.
Duplicate short code	Two requests generate the same code at nearly the same time.	Unique constraint on short_code catches it at the database level. With a pre-generated key pool, this becomes very rare in the first place.
Key generation service is down	New short links cannot be created.	Each service instance keeps a small local buffer of pre-fetched keys so it can survive a brief outage without blocking writes immediately.
Kafka lag for click events	Click counts fall behind the real number of clicks.	This is fine. Click counting is explicitly an eventually consistent feature, delayed counts do not break the redirect itself.

WATCH OUT

Notice that almost none of these failures stop the redirect from working for already cached links. That is not an accident, it is the entire point of putting Redis in front of the database on the read path. If your design does not have that property, that is usually a sign the cache was added late instead of being part of the core design.

10 JAVA + SPRING BOOT MAPPING

This is where you turn the diagram into something you could actually open a pull request for. If I were building this in Spring Boot, here is roughly how the layers would map.

Layer	Spring Boot mapping
Controller	UrlController exposing POST /api/v1/urls, GET /{code}, GET /stats, DELETE /{code}.
Service	UrlShortenerService for the write path, RedirectService for the read path. Keeping them as separate classes mirrors the separation in the diagram.
Repository	Spring Data JPA UrlRepository extends JpaRepository, with a findByShortCode query method.
Entity	UrlEntity with @Id on id, a @Column(unique = true) on shortCode, and standard JPA mappings for the rest of the columns.
DTOs	CreateUrlRequest, CreateUrlResponse, UrlStatsResponse, kept separate from the entity so the API shape does not leak database structure.
Caching	@Cacheable on the long URL lookup method, backed by Spring Cache with a Redis store. @CacheEvict on delete and on expiry cleanup.
Resilience	Resilience4j @CircuitBreaker around the Redis call, with a fallback method that queries the repository directly when the breaker is open.
Async analytics	KafkaTemplate publishes a click event from the Redirect Service. A separate @KafkaListener consumes events and updates click_count in batches.
Rate limiting	Bucket4j filter, or a Spring Cloud Gateway RequestRateLimiter, applied to the creation endpoint only.
Scheduled cleanup	A @Scheduled job that periodically deactivates expired links instead of checking expiry on every single redirect.

REDIRECTSERVICE.JAVA, CACHE-ASIDE WITH CIRCUIT BREAKER FALLBACK

```

@Service
public class RedirectService {

    private final RedisTemplate<String, String> redisTemplate;
    private final UrlRepository urlRepository;

    @CircuitBreaker(name = "redisLookup", fallbackMethod = "fallbackToDb")
    public String resolve(String code) {
        String cached = redisTemplate.opsForValue().get(code);
        if (cached != null) {
            return cached;
        }
        String longUrl = urlRepository.findByShortCode(code)
            .orElseThrow(() -> new UrlNotFoundException(code))
            .getLongUrl();
        redisTemplate.opsForValue().set(code, longUrl);
        return longUrl;
    }

    public String fallbackToDb(String code, Throwable t) {
        return urlRepository.findByShortCode(code)
            .orElseThrow(() -> new UrlNotFoundException(code))
            .getLongUrl();
    }
}

```

JAVA MAPPING

Notice this is cache-aside, written by hand, instead of relying purely on `@Cacheable`. In an interview, you can mention that `@Cacheable` would handle the happy path automatically, but you still want to know what is happening underneath, especially for the fallback behaviour when Redis is unavailable. That distinction is worth saying out loud.

11 INTERVIEW TRADEOFFS

Pick three to five of these depending on how much time is left in the interview. Do not rush through all of them, a well explained tradeoff is worth more than a long list.

Decision	Option A	Option B
ID generation	Sequential (base62 of auto-increment): simple, no collisions, but enumerable and leaks creation rate.	Random / pre-generated pool: not enumerable, but needs collision handling or an extra service.
Storage	SQL: easier admin queries, transactional guarantees, simpler at moderate scale.	NoSQL key-value: near-infinite horizontal scale, lower lookup latency, weaker query flexibility.

Decision	Option A	Option B
Click counting	Synchronous update: always accurate, but adds write load to the hottest path.	Async via Kafka: scales better, counts are eventually consistent.
Caching style	Manual cache-aside: full control over fallback behaviour, more code to maintain.	Spring @Cacheable: less code, but you need to understand what it hides.
Custom alias support	Allowed: better product experience, more validation and namespace management.	Not allowed: simpler system, every code is generated and guaranteed unique.

TRADEOFF

At my level, I would not try to convince an interviewer that one option is universally better. I would explain the situation where I would pick each one. That sentence alone, said with confidence, usually lands better than memorizing a perfect answer for every design question in this book.

QUESTION 02 OF 15 · MODULE 14

Chat Application

WhatsApp and Slack are the usual reference points for this question. The interesting part is not the chat bubble UI. It is keeping millions of long-lived connections open at once and still delivering messages in order, without losing one, when half your users are offline at any given moment.

01 PROBLEM STATEMENT

We are building a messaging system that supports one-to-one chats and group conversations. A sender writes a message, it reaches the receiver in close to real time if they are online, and it waits for them safely if they are not. Every message also needs to survive in history so a user can scroll back and see it later.

This is different from the URL shortener question in one important way. There, the read path and write path were cleanly separated. Here, every message is both a write, since it has to be stored, and a near-instant read for the other participant. That dual nature is what makes connection handling the hard part of this design, not the database.

INTERVIEW FOCUS

A common mistake here is to design this purely as a REST API with polling. That answer technically works but tells the interviewer you have not thought about why real chat apps use persistent connections. Bring up WebSockets yourself in the first two minutes.

02 REQUIREMENTS

Functional	Non-functional
Send and receive 1:1 and group messages	Low latency delivery, ideally under a second when online
Persist full message history	Messages must not be lost, even if a service crashes mid-delivery
Deliver queued messages once a user comes online	Order within a single conversation must be preserved
Basic delivery status: sent, delivered, read	Horizontally scalable to millions of concurrent connections
Support group conversations up to a reasonable member cap	Read receipts and online status can be eventually consistent

Notice the non-functional column leans on two ideas that pull against each other: never lose a message, but also do not wait for perfect consistency everywhere. You will spend most of this answer explaining where you draw that line.

02.5 CAPACITY ESTIMATION

Worth doing here because it directly decides how you handle connections, which is the crux of this question.

CAPACITY ESTIMATION WALKTHROUGH

Assume 50 million daily active users

Concurrent connections

~10% of DAU active at once ~ 5 million open WebSocket connections

One gateway node handling ~50,000 connections ~ 100+ gateway nodes

Message volume

Assume 40 messages/user/day ~ 2 billion messages/day

2,000,000,000 / 86,400 sec ~ 23,000 messages/sec average, ~100,000/sec at peak

Storage growth

Each message row ~ 200 bytes ~ 400 GB/day ~ 140+ TB/year

The number that matters most for the rest of this design is the gateway count. Five million open connections cannot sit on one server, which is exactly why the WebSocket Gateway becomes its own scaled layer instead of living inside the main chat service.

03 APIs

Chat systems are not pure REST. Real-time events travel over a persistent WebSocket connection, while history and setup still go through plain REST calls.

Channel / Path	Payload	Notes
WebSocket: CONNECT	Auth token in handshake	Opens the persistent connection, registers the user in the presence layer.
WebSocket: SEND_MESSAGE	conversationId, clientMsgId, content	clientMsgId is generated by the sender and used for de-duplication on retry.
WebSocket: MESSAGE_ACK	messageId, status	Receiver confirms delivered or read, sender's client updates its UI.
POST /api/v1/conversations	type, memberIds	Creates a 1:1 or group conversation before any messages are sent.
GET /api/v1/conversations/{id}/messages	cursor, limit (query params)	Paginated history fetch, used on chat open and on scrolling up.

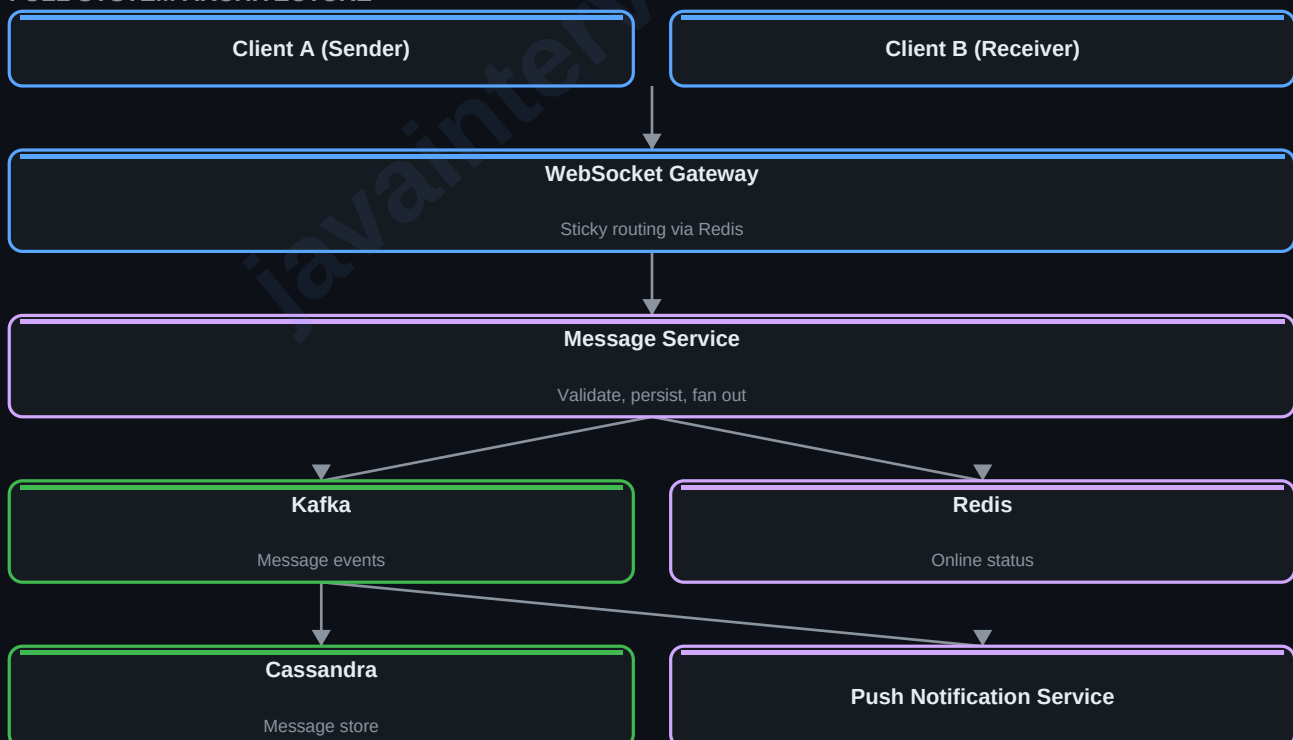
04 DATABASE DESIGN

Three very different access patterns live in this system, so resist the urge to put everything in one database. Match each pattern to the storage it actually fits.

Data	Storage choice	Why
Conversation & member metadata	PostgreSQL	Small, relational, changes rarely. A group's member list is a textbook relational table.
Message history	Cassandra (wide-column)	Append-only, time-ordered, partitioned by conversationId. This is the access pattern wide-column stores were built for.
Presence & connection routing	Redis	Needs to change constantly and expire automatically when a user disconnects, a perfect fit for an in-memory store with TTLs.

TIP

In real projects, this usually becomes a problem when a team tries to force message history into a relational table with a foreign key to every conversation. It works at small scale, then a handful of very active group chats create rows so dense that a single partition becomes a hotspot. Partitioning by conversationId and time bucket from day one avoids that entirely.

05 HIGH-LEVEL DIAGRAM**FULL SYSTEM ARCHITECTURE**

Both clients connect to the same gateway layer, not to each other. The gateway never decides anything on its own, it just holds the connection open and forwards events. All real logic, including who gets the message and when, sits in the Message Service.

06 KEY FLOW DIAGRAM

One flow, two endings. Walk through both, an interviewer will almost always ask what happens when the receiver is not online.

RECEIVER IS ONLINE



Receiver online: push live

RECEIVER IS OFFLINE (CONTINUES FROM THE SAME KAFKA EVENT)



The message only gets written once, by the Kafka consumer that persists to Cassandra. Whether it also gets pushed live or queued for a notification depends entirely on a presence lookup in Redis at that moment. Persistence and delivery are two separate concerns, and keeping them separate is what makes this design resilient.

07 COMPONENTS EXPLANATION

Component	Responsibility
WebSocket Gateway	Holds the persistent connection per device. Stateless about message content, it only knows which user is attached to which node, and writes that mapping into Redis on connect.
Message Service	The only place that touches business logic: validates the sender, resolves the conversation, generates a server-side message ID, and publishes a single event to Kafka.
Kafka	Decouples "a message happened" from "a message was stored" and "a message was delivered." Multiple consumers can react to the same event independently.
Redis (presence)	A simple key-value map of userId to gateway node, with a short TTL refreshed by a heartbeat. If the TTL expires, the user is offline.
Cassandra	Durable, append-only message history, partitioned so that one extremely active conversation cannot slow down lookups for everyone else.
Push Notification Service	Wakes up an offline user's device through APNs or FCM. Completely separate from in-app delivery.

08 SCALING APPROACH

Two scaling questions come up almost every time: how do you scale connections, and how do you scale delivery to a large group without one giant group slowing down everyone.

Connections scale horizontally by simply adding more gateway nodes behind a load balancer that supports sticky sessions, with Redis as the shared map of who is connected where. Ordering within one conversation is preserved by partitioning Kafka on conversationId, so every message for the same conversation lands on the same partition and is processed in the order it arrived.

Group delivery: fan-out on write vs fan-out on read

Fan-out on write

- Copy the message into every member's inbox at send time.
- Reads later are cheap, you just read your own inbox.
- Gets expensive fast for groups with thousands of members.

Fan-out on read

- Store the message once, compute the member's view when they open the chat.
- Write stays cheap no matter how large the group is.
- Reads do more work, which only matters for extremely large groups.

Most consumer chat apps use fan-out on write for normal groups since group sizes are capped, and switch to fan-out on read style for things like broadcast channels with audiences too large to copy a message into every single inbox.

09 FAILURE HANDLING

Failure	How you handle it
Gateway node crashes	Client detects the dropped socket and reconnects through the load balancer to a different node, which re-registers its presence entry in Redis.
Kafka consumer falls behind	Messages are not lost, they are just delayed. The Kafka log holds the event until the consumer catches up.
Duplicate message on retry	The client-generated clientMsgId lets the Message Service detect a retry and return the existing message instead of creating a second one.
Cassandra write fails	Retried with backoff. The event stays in Kafka until the consumer successfully commits the write, so nothing is dropped silently.
Push notification service is down	Acceptable in the short term, the message is still safely stored and will appear the moment the user reconnects and pulls history.

WATCH OUT

If asked how you guarantee no message is ever lost, the honest answer is that you do not guarantee it by being careful, you guarantee it structurally. The message is durable the moment it lands in Kafka, before anyone has tried to deliver it to anyone.

10 JAVA + SPRING BOOT MAPPING

Layer	Spring Boot mapping
WebSocket handling	Spring's WebSocketHandler or STOMP over WebSocket for the gateway layer, with a custom handshake interceptor for auth.
Service	MessageService publishes a MessageEvent to Kafka after validation. PresenceService reads and writes the Redis routing map.
Repository	Spring Data Cassandra MessageRepository for history, Spring Data JPA ConversationRepository for metadata.
Messaging	KafkaTemplate to publish, two separate @KafkaListener consumers: one persists to Cassandra, one checks presence and triggers delivery or push.
Idempotency	clientMsgId checked against a short-lived Redis set before publishing, cheap protection against duplicate sends on flaky connections.
Resilience	Resilience4j @Retry around the Cassandra write, since a transient failure here should not drop a message.
MESSAGEEVENTLISTENER.JAVA: PRESENCE-BASED ROUTING DECISION <pre> @KafkaListener(topics = "message-events", groupId = "presence-router") public void onMessageEvent(MessageEvent event) { String gatewayNode = presenceService.findNode(event.getReceiverId()); if (gatewayNode != null) { gatewayClient.pushTo(gatewayNode, event); } else { pushNotificationService.notify(event.getReceiverId(), event); } } </pre>	

11 INTERVIEW TRADEOFFS

Decision	Option A	Option B
Transport	WebSocket: true real time, but persistent connections are expensive to manage at scale.	Long polling: simpler infra, heavier on servers due to repeated request overhead.
Delivery guarantee	Exactly-once: clean in theory, very hard to guarantee across distributed services.	At-least-once + idempotency key: simpler, practical, and what most real systems do.

Decision	Option A	Option B
Group fan-out	Fan-out on write: cheap reads, expensive for very large groups.	Fan-out on read: cheap writes, more read work for huge audiences.
Message storage	Relational: simpler joins, struggles with very high write volume over time.	Wide-column (Cassandra): built for this access pattern, weaker ad-hoc querying.

HOW TO ANSWER THIS IN 10 MINUTES

1. State the scope out loud: 1:1 and group messaging, real time when online, queued when not, full history. 30 seconds.
2. Call out the dual nature of a message early: it is both a write and a near-instant read. This single sentence sets up the rest of your answer. 30 seconds.
3. Draw the diagram: Client, WebSocket Gateway, Message Service, Kafka, then split into Cassandra and presence-based routing. 2 minutes.
4. Walk through the online and offline delivery paths from the same Kafka event. 2 minutes.
5. Explain fan-out on write versus fan-out on read for groups, and why ordering comes from partitioning Kafka by conversationId. 2 minutes.
6. Cover one failure scenario in depth, duplicate messages and clientMsgId is a strong choice, rather than listing five shallowly. 1.5 minutes.
7. Close with one tradeoff you would revisit if requirements changed, for example exactly-once delivery if the product later needs financial-grade guarantees. 1 minute.

FINAL INTERVIEW CLOSING LINE

For chat systems, the main design idea is simple: WebSocket handles real-time delivery, Kafka makes the message durable and async, Redis tracks presence, and Cassandra stores long-term message history. The tradeoff is between low latency, ordering, and delivery guarantees.

QUESTION 03 OF 15 · MODULE 14

Notification System

Almost every backend you will ever work on eventually needs to email someone, text someone, or push an alert to their phone. The point of this question is not the sending itself, any provider's SDK can do that. It is building one shared system that every other service in the company can call without learning four different provider APIs.

01 PROBLEM STATEMENT

We are building a notification platform that other internal services call when something worth telling a user about happens: an order shipped, a payment failed, a friend request arrived. The caller does not format an email or talk to a push provider directly. It sends an event, and the platform decides how, when, and whether to deliver it.

That last part matters. Unlike a chat message, a notification is not always wanted. A user can mute email receipts but still want push alerts for payment failures. That single fact, that delivery is conditional on user choice, is what separates this question from the previous one, where every message was always delivered somewhere.

INTERVIEW FOCUS

A common mistake is jumping straight into "I'll use Kafka and call SendGrid." That is the easy 20 percent. The interesting 80 percent is templates, preferences, retries, and what happens when a provider starts rejecting your calls at 2am. Spend your time there.

02 REQUIREMENTS

Functional	Non-functional
Send via email, SMS, push, and in-app channels	Calling service must never block waiting on a provider
Render a stored template with caller-supplied data	Survive a single provider outage without affecting other channels
Respect per-user, per-channel preferences	Handle bursty traffic during campaigns, not just steady state
Retry transient failures automatically	At-least-once delivery, with dedup so users do not see duplicates
Avoid sending the same logical event twice	Rate limit outgoing calls so providers do not throttle or ban us
Support both transactional and promotional notification types	Prioritize transactional traffic over promotional traffic under load

Say the word "asynchronous" early and mean it. The caller's job ends the moment it hands the platform an event. Everything after that, rendering, routing, retrying, is this system's problem, not theirs.

Not every notification deserves the same urgency. A payment failure or an OTP is transactional, the user is sitting there waiting for it. A 20 percent off sale is promotional, useful but not urgent. Tag every event with a priority at creation time, and let high priority traffic skip ahead of a marketing backlog rather than queue behind it. This single distinction is what most beginner designs miss entirely.

02.5 CAPACITY ESTIMATION

Brief, but useful, because campaign traffic is spiky in a way most systems in this book are not.

CAPACITY ESTIMATION WALKTHROUGH

Assume 200 million notifications/day across all channels

$200,000,000 / 86,400 \text{ sec} \sim 2,300/\text{sec}$ average

Marketing campaigns can spike this 10x $\sim 23,000/\text{sec}$ at peak

Storage

Each notification record ~ 300 bytes ~ 60 GB/day

Per-channel split (rough)

Email 40%, Push 35%, SMS 15%, In-app 10%

Two things fall out of this. First, the gap between average and peak load means workers have to scale per channel, not as one fixed pool. Second, that 10x spike is exactly why rate limiting toward providers matters more here than in any other question in this book.

03 APIs

This is an internal platform API, called by other backend services, not end users.

Method & Path	Payload	Notes
POST /api/v1/notifications	eventId, userId, templateId, channels[], data	eventId is caller-supplied and doubles as the idempotency key. Returns 202 Accepted.
GET /api/v1/notifications/{eventId}	none	Returns delivery status per channel: queued, sent, failed, or bounced.
PUT /api/v1/users/{id}/preferences	notificationType, channel, optedIn, quietHours, locale	Updates whether a user receives a given notification type on a given channel, and when. A user can block promotional emails but keep payment-failure push alerts on.
POST /api/v1/templates	templateId, version, channel, body, locale	Admin-only. Creates or updates a template, kept separate from caller code.

04 DATABASE DESIGN

Four tables, each answering a different question. Relational fits all of them comfortably here, the volume per table is nowhere near what the chat history table had to deal with.

Table	Key fields	Answers
notifications	event_id (unique), user_id, notification_type, priority, template_id, status (QUEUED, PROCESSING, SENT, PARTIALLY_SENT, FAILED), created_at, updated_at	Did this event get processed, how urgent was it, and what happened to it overall?
notification_attempts	attempt_id, event_id, channel, provider, provider_message_id, status, error_code, error_message, retry_count, attempted_at	What actually happened on each delivery attempt, including the provider's own message id for support lookups.
templates	template_id, version, channel, locale, body	What does this notification actually say, in which language, and which version rendered it.
user_preferences	user_id, notification_type, channel, opted_in, quiet_hours, locale	Is this user allowed to receive this specific notification type on this channel, and at this hour.

Many beginners merge notifications and notification_attempts into one row that gets updated in place. That works until you need to know that the email failed twice before succeeding on the third try. Keep attempts as an append-only log, it costs little and answers questions a single mutable row never can.

Notice user_preferences is keyed by notification_type, not just channel. A user can mute every promotional email they get while still wanting a push alert the moment a payment fails. Modeling preference at the channel level only cannot express that, modeling it at notification_type plus channel can.

TIP

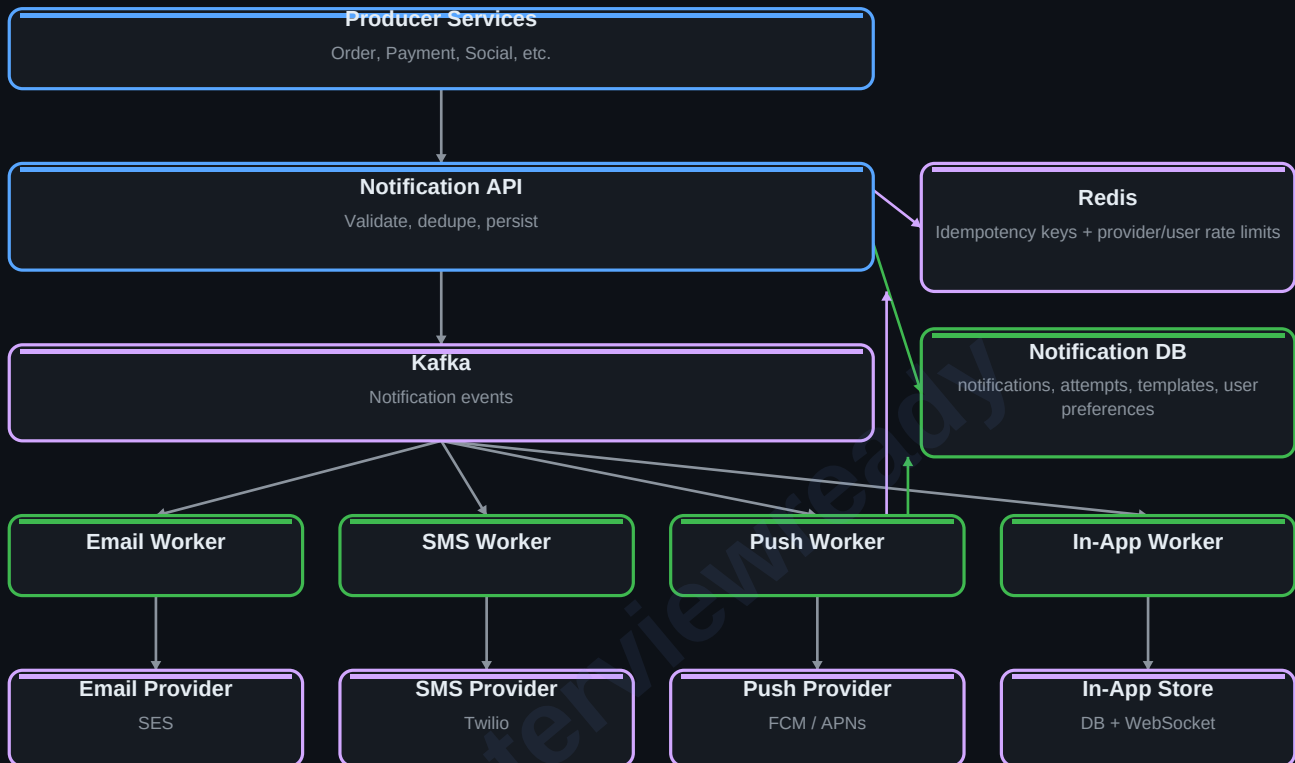
Redis answers "have I seen this eventId" in under a millisecond, but Redis keys expire and Redis can fall over. The real guarantee comes from a unique constraint on event_id in the notifications table. If two requests ever get past the Redis check at the same instant, the database rejects the second insert outright. Redis is the fast path, the unique constraint is the one that can never be skipped.

WATCH OUT

Treat notification payloads as sensitive by default. Do not log the full body of an OTP or a payment alert, and do not carry more personal data into a Kafka event than the channel worker actually needs to render and send it. OTP and payment-related notifications should also expire quickly, a six digit code that is still valid an hour later is a liability, not a feature.

05 HIGH-LEVEL DIAGRAM

FULL SYSTEM ARCHITECTURE



One Kafka topic fans out to four independent workers. Each worker only knows about its own channel and its own provider, so a Twilio outage cannot slow down a single email. Redis and the Notification DB sit beside the pipeline rather than inside it: the API checks Redis, then writes the request and an outbox event to the DB, then each worker writes its own attempt back to the DB and checks Redis again before calling a provider.

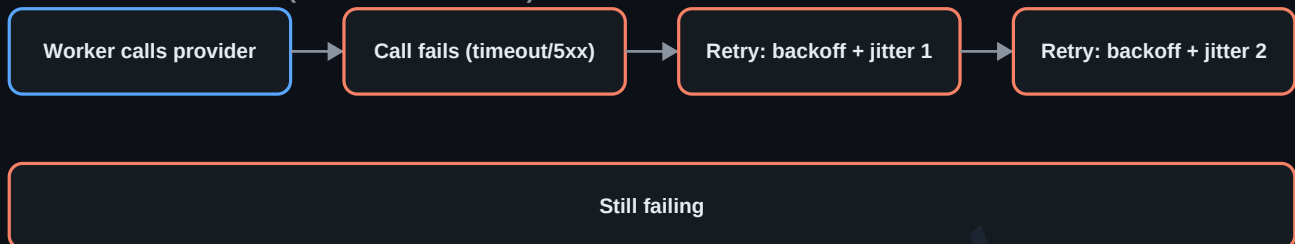
TRADEOFF

This diagram uses one notification-events topic with a separate consumer group per channel, which is the simpler setup and what most teams should start with. The alternative is dedicated topics, email-notifications, sms-notifications, push-notifications, each with its own retention and partition count. Separate topics give stronger isolation, a noisy SMS backlog cannot grow large enough to threaten email throughput, at the cost of more topics and consumer configs to operate.

06 KEY FLOW DIAGRAM

The happy path is not the interesting flow here. What an interviewer actually wants to see is what happens when a provider call fails, since that is the entire reason this system needs a queue in the first place.

PROVIDER CALL FAILS (TRANSIENT ERROR)



AFTER RETRIES ARE EXHAUSTED (CONTINUES FROM ABOVE)



Notice the queue offset is not committed until a retry attempt either succeeds or gets routed to the dead letter topic. That is what guarantees no event quietly disappears during a provider outage, it either gets delivered eventually or it lands somewhere a human can see it.

Not every failure deserves a retry. A timeout or a provider 5xx is transient, the same call might succeed a second later, so retry it with exponential backoff and a little random jitter so a thousand retrying workers do not all hammer the provider at the exact same instant. An invalid phone number or a malformed email address is permanent, no amount of retrying fixes a bad address. Route that straight to the dead letter queue instead of wasting three retry cycles on something that will never succeed.

07 COMPONENTS EXPLANATION

Component	Responsibility
Notification API	Checks the eventId against Redis for idempotency, saves the notification request, and writes an OutboxEvent in the same database transaction. A separate publisher sends the event to Kafka after commit. It never talks to a provider directly.
Kafka	Buffers events so a traffic spike or a slow provider never pushes back onto the services that triggered the notification.
Channel Workers	One consumer group per channel. Before calling a provider, each worker checks user preference, notification priority, the channels the caller actually requested, and any fallback rule, then renders the template and handles retries.
Template Rendering	Loads the right template version and locale, fills in placeholders like {{userName}} or {{orderId}}, and validates every required variable is present before the worker is allowed to send anything.
Redis	Three jobs: idempotency keys with a short TTL, and rate limit buckets scoped per provider, per user, and per notification type.
Dead Letter Queue	Catches anything that exhausted its retries, so one bad event cannot block the partition behind it.

INTERVIEW MOMENT

Routing is a decision, not a lookup. Given a priority, a set of requested channels, and the user's preferences, the worker still needs a rule for when things go wrong. A common product rule: if push fails for a critical payment alert, fall back to email or SMS automatically. A failed push for a promotional notification just gets dropped, it was never worth retrying across channels in the first place.

08 SCALING APPROACH

Scaling this system is less about raw throughput and more about isolating channels from each other, since they fail and spike independently.

Lever	What it does	When to reach for it
Per-channel consumer groups	Scale email workers without touching push workers	Default setup, do this from day one
Rate limiting (provider, user, type)	Token buckets scoped three ways: per provider so a campaign cannot get the account throttled, per user so a bug cannot spam one person, per notification type so promotional volume cannot crowd out transactional traffic	Any time you depend on a third-party API with a quota, which is always

Lever	What it does	When to reach for it
Bulk provider APIs	Batches many sends into one provider call instead of one each	Large marketing campaigns where per-call overhead adds up
Partition by user_id	Keeps all of one user's notifications processed in order	Only if delivery order across channels actually matters to the product

TIP

None of these levers mean anything without visibility. Put dashboards and alerts on delivery success rate, provider error rate, retry count, dead letter queue size, Kafka consumer lag, and end-to-end notification latency. Most outages here are not your bug, they are a provider having a bad day, and a lagging consumer group is usually the first sign of it.

09 FAILURE HANDLING

Failure	How you handle it
Provider rejects calls (rate limited or down)	Circuit breaker opens for that provider so retries back off exponentially instead of hammering a struggling service. If a backup is configured, the worker fails over, SES to SendGrid for email, or a secondary SMS provider when Twilio is rate limited.
Worker crashes mid-send	Kafka offset was never committed, another consumer picks up the same event. The idempotency key prevents a duplicate from actually reaching the user.
Retries exhausted permanently	Event moves to the dead letter queue instead of being dropped or retried forever, with an alert for on-call.
Caller retries the same POST	eventId lookup in Redis returns the existing result instead of creating a second notification.
A buggy producer service spams one user	Per-user rate limit catches it before it ever reaches a provider, independent of the per-provider limit.

WATCH OUT

If an interviewer asks what happens to a notification stuck in the dead letter queue, do not say "we lose it." Say it sits there for a human or an automated replay job to inspect, and once the underlying cause is fixed, it can be republished to the original topic. Lost and quarantined are very different answers.

10 JAVA + SPRING BOOT MAPPING

Layer	Spring Boot mapping
Controller	NotificationController exposing POST /api/v1/notifications and the status lookup endpoint.
Service	NotificationService checks Redis for eventId, saves NotificationEntity and an OutboxEvent inside the same @Transactional method. A separate publisher sends the outbox event to Kafka after the database commit.
Entity	NotificationEntity maps event_id with a JPA unique constraint, the same field Redis already checks, so the database is the backstop if Redis ever expires or fails.
Channel listeners	One @KafkaListener per channel, each in its own consumer group: EmailNotificationListener, SmsNotificationListener, PushNotificationListener, InAppNotificationListener.
Template rendering	A TemplateRenderingService that loads the right version and locale, fills placeholders, and throws before publishing if a required variable is missing.
Provider clients	Thin wrappers around the SES, Twilio, and FCM SDKs, each wrapped in a Resilience4j @CircuitBreaker and @Retry.
Rate limiting	Bucket4j backed by Redis, with separate buckets per provider, per user, and per notification type, all checked before a provider call is attempted.
Repository	Spring Data JPA repositories for notifications, attempts, templates, and preferences.


```

NOTIFICATIONSERVICE.JAVA: IDEMPOTENT SUBMIT WITH OUTBOX EVENT
@Transactional
public NotificationResponse submit(NotificationRequest request) {
    String key = "notif:" + request.getEventId();
    Boolean isNew = redisTemplate.opsForValue()
        .setIfAbsent(key, "1", Duration.ofHours(24));

    if (Boolean.FALSE.equals(isNew)) {
        return NotificationResponse.alreadyQueued(request.getEventId());
    }

    NotificationEntity notification = NotificationEntity.from(request);
    notificationRepository.save(notification);
    outboxRepository.save(
        OutboxEvent.notificationCreated(notification.getEventId())
    );
    return NotificationResponse.queued(notification.getEventId());
}

```


The Redis key handles fast duplicate checks. The database unique constraint on event_id is the final protection. The outbox event is saved in the same transaction as the notification row, so Kafka publishing happens only after the database commit. This avoids the bug where Kafka receives an event for a row that was never committed.

NOTIFICATIONKAFKACONFIG.JAVA: RETRY THEN DEAD LETTER ROUTING

```
@Bean
public DefaultErrorHandler kafkaErrorHandler(KafkaTemplate<?, ?> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);
    var backOff = new ExponentialBackOff(2000L, 2.0);
    backOff.setMaxInterval(30000L);
    backOff.setMaxAttempts(3);
    return new DefaultErrorHandler(recoverer, backOff);
}
```

This single bean is most of the retry and dead letter behaviour from section six. Spring Kafka retries locally with increasing backoff, then automatically republishes the failed record to a .DLT suffixed topic. If you want true random jitter on top of this curve, wrap the BackOff in a small custom implementation, most teams find the exponential curve alone is enough. No manual retry loop required.

11 INTERVIEW TRADEOFFS

Decision	Option A	Option B
Ownership model	Centralized platform: one place for preferences and provider logic, but everyone now depends on it.	Each team sends its own: less shared risk, but duplicated provider code and inconsistent preference handling.
Retry policy	Same policy for every channel: simple to operate.	Per-channel policy: email can wait minutes, push should fail fast since timeliness matters more there.
Dead letter topics	One shared DLT for everything: fewer topics to manage.	Per-channel DLT: faster to tell a Twilio outage apart from an SES outage at a glance.

HOW TO ANSWER THIS IN 10 MINUTES

1. State the scope: one shared platform, four channels, called by other internal services, fully async. 30 seconds.
2. Call out that delivery is conditional on user preference and priority, unlike most systems in this book where every message is always delivered the same way. 30 seconds.
3. Draw the diagram: Producer Services, Notification API, Redis and the DB beside it, Kafka, four channel workers, four providers. 1.5 minutes.
4. Explain the preference and priority check each worker does before sending: channel, notification type, quiet hours, and fallback rules. 1 minute.
5. Explain template rendering: versioned, localized, placeholders filled in, validated before anything is sent. 1 minute.
6. Walk through a provider failure: exponential backoff with jitter, dead letter queue, provider fallback like SES to SendGrid. This is the flow interviewers care about most. 2 minutes.
7. Mention idempotency, eventId in Redis plus a DB unique constraint as the backstop, and rate limiting per provider, per user, and per notification type, in one breath. 1 minute.
8. Cover one scaling lever in depth, per-channel consumer groups is the strongest choice, rather than listing all four shallowly. 1 minute.
9. Close with the real tradeoff: speed versus reliability versus provider safety versus never duplicating a send, and which one you would bend first if forced to choose. 1 minute.

FINAL INTERVIEW CLOSING LINE

For notification systems, the main design idea is simple: Kafka decouples the trigger from the delivery, channel workers isolate one provider's bad day from another's, Redis handles idempotency and rate limiting, and the dead letter queue makes sure nothing fails silently. The tradeoff is between delivery speed, reliability, provider safety, and never duplicating a send, and a senior answer knows which one to bend first when they conflict.

File Upload System

Every backend developer eventually builds an upload feature, and almost every fresher builds it the same wrong way first. This question is not really about files. It is about noticing when your API server has quietly become a bottleneck for something it should never have been doing in the first place.

javainterviewready

01 PROBLEM STATEMENT

We are building a system that lets users upload files: profile pictures, PDFs, videos, invoices, chat attachments, whatever the product needs. The upload itself is the easy part to imagine and the easy part to get wrong. Almost every fresher pictures the file travelling through the Spring Boot API into the database or disk, and that mental model is exactly where this question separates juniors from people who have operated something like this in production.

Who uses it? Regular users uploading a profile photo or a document occasionally, and a second group doing bulk or heavy uploads, video creators, businesses uploading invoices in batches. Your design has to handle both without the second group degrading the experience for the first.

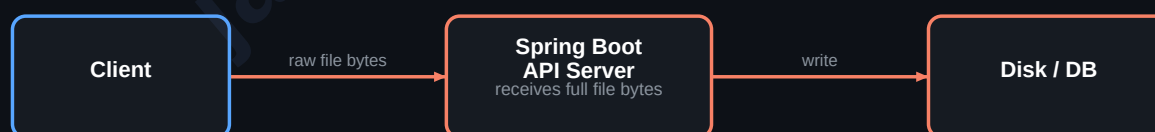
Expected scale for an interview answer: assume something like 5 million daily active users on a mid-size product, with file sizes ranging from a few KB for a profile picture to a few GB for video content. You do not need the interviewer to hand you this number. State it and move forward.

INTERVIEW FOCUS

Most freshers describe this as "an endpoint that accepts a file and saves it." That answer is fine for a college project. It says nothing about what happens when that file is 2GB and ten thousand people are uploading at once. Bring up direct-to-storage upload yourself, in the first two minutes, before anyone has to ask for it.

Here is the design almost everyone starts with, and why it falls apart at real traffic.

THE DESIGN THAT BREAKS AT SCALE



This usually becomes a problem the moment someone uploads a file large enough to make the request slow. The backend holds that connection open the whole time, burning a thread and memory just moving bytes from client to storage. Multiply that by a few thousand concurrent uploads, and your API servers, the same ones handling logins and checkout, are now babysitting file transfers. You end up scaling expensive compute just to handle bandwidth, which is not what compute is good at.

The fix is not a bigger server. The fix is removing the backend from the data path entirely and letting the client talk to object storage directly. That single idea is the spine of this whole answer.

02 REQUIREMENTS

Two minutes here saves you from designing the wrong system. Split functional from non-functional before touching a diagram.

Functional requirements

Requirement	What it means
Upload a file	Accept images, PDFs, videos, and documents up to a defined size limit.
Track upload status	Caller can know whether a file is still uploading, available, or failed.
Retrieve a file	Authorized users can fetch metadata and a way to download the file.
Process after upload	Virus scan, thumbnail generation, compression, or transcoding depending on file type.
Delete a file	Owner can remove a file, which should eventually free the underlying storage too.

Non-functional requirements

Requirement	What it means
Backend stays light	API servers should not carry file bytes through them for large uploads.
Durable storage	Once a file is marked available, it should not silently disappear.
Handle large files	Multi-GB uploads should not time out or block on a single request.
Secure access	Private files should not be reachable through a guessable public URL.
Scalable processing	Thumbnailing or scanning a million files a day should not block uploads.

Notice what is not on that non-functional list: strong read-after-write consistency on processed outputs. If a thumbnail takes three extra seconds to appear after upload, nobody's product breaks. The raw file being safe and durable matters far more than the derived versions being instant, and saying that out loud is usually the moment an interviewer starts paying closer attention to your answer.

02.5 CAPACITY ESTIMATION

Keep this rough. The goal is to show that your storage choice and your backend-load decision are not guesses, they come from a number you worked out loud.

CAPACITY ESTIMATION WALKTHROUGH

Assume 5 million daily active users (matches the problem statement)

Upload volume

Assume 5% of DAU upload at least one file per day ~ 250,000 uploads/day
 $250,000 / 86,400 \text{ sec} \sim 3 \text{ uploads/sec average, } \sim 25 \text{ uploads/sec at peak}$

Storage growth

Mixed average file size across images, PDFs, short videos ~ 4 MB
 $250,000 \times 4 \text{ MB} \sim 1 \text{ TB / day} \sim 350 \text{ TB / year}$

Bandwidth

~1 TB/day inbound, spread across the day, peaks during evening hours
 This entire 1TB never has to pass through your application servers

Why this matters

350 TB a year is routine for object storage like S3, but it would be a real problem for a backend server's local disk or for a relational database's BLOB column

This number is what makes direct-to-storage upload non-negotiable. At roughly 1TB a day, routing every byte through Spring Boot servers means your compute layer scales with bandwidth instead of business logic, which is an expensive way to scale a backend.

03 APIs

Five endpoints, and notice that none of them carry file bytes. That absence is the whole point of this design. The actual upload happens against object storage, not against your API.

Method & Path	Request / Response	Notes
POST /api/v1/files/upload-request	Req: fileName, contentType, sizeBytes Res: fileId, uploadUrl, expiresAt	Validates metadata and returns a short-lived presigned URL. The backend generates the storage key, the client never picks the path.
POST /api/v1/files/upload-complete	Req: fileId, checksum (optional) Res: fileId, status	Client calls this after the object storage upload succeeds. Backend verifies the object before trusting this call, see step 6 in the key flow.
GET /api/v1/files/{fileId}	Req: none Res: fileId, fileName, status, sizeBytes, createdAt	Fetch current metadata. Status tells the caller if processing is still running.
GET /api/v1/files/{fileId}/download-url	Req: none Res: downloadUrl, expiresAt	Generates a short-lived presigned URL instead of exposing a permanent public link.

Method & Path	Request / Response	Notes
DELETE /api/v1/files/{fileId}	Req: none Res: 204 No Content	Soft deletes the metadata row first. Object cleanup in storage happens async afterward.

INTERVIEW MOMENT

If an interviewer asks why the client does not just send the file straight to S3 without asking your backend first, the answer is access control. The backend decides who is allowed to upload, validates file type and size before a single byte moves, and generates the storage key itself so users cannot pick arbitrary paths in your bucket. The presigned URL is also short-lived, ten minutes is typical, so it cannot be leaked and reused later. It is permission to do one specific upload, not an open door.

TIP

For important files, have the client send a checksum or the object storage ETag with upload-complete, and store it. Comparing that against what S3 actually reports lets you catch a corrupted or truncated upload instead of just trusting that the bytes arrived intact. Small detail, but it is the kind of thing that separates a senior answer.

upload-complete is the simpler flow to draw in an interview, the client tells you it is done. In production, many teams also subscribe to object storage event notifications, an S3 ObjectCreated event, and use that as a second, more reliable trigger for the same processing pipeline. That way, even if the client crashes right after uploading and never calls your API, the file is still picked up. Lead with upload-complete for the interview, then mention storage events as the production hardening step.

04 DATABASE DESIGN

Two kinds of data here and they do not belong in the same place. Metadata is small, structured, and queried constantly. File bytes are large, unstructured, and looked up by key. Treat them as two different storage problems from day one.

Table: files

Column	Type	Notes
file_id	UUID	Primary key, also used as part of the storage key.
user_id	bigint	Owner of the file. Indexed for "my files" queries.
original_name	varchar	Display name only, exactly what the client sent. Never used to build a path.

Column	Type	Notes
storage_key	varchar	The real, safe internal path in the bucket. Backend-generated, never user-facing.
file_type	varchar	MIME type, validated against an allowed list on upload-request.
size_bytes	bigint	Confirmed against the actual object size, not trusted from the client alone.
checksum	varchar	Client-sent checksum or the S3 ETag, used to detect a corrupted upload.
status	varchar	UPLOADING, UPLOADED, PROCESSING, AVAILABLE, or FAILED.
visibility	varchar	PRIVATE or PUBLIC, decides whether downloads need a presigned URL.
failure_reason	varchar, nullable	Why processing failed: virus detected, corrupt upload, unsupported codec.
created_at	timestamp	Indexed alongside user_id for listing and cleanup jobs.
processed_at	timestamp, nullable	When async processing finished, null while still PROCESSING.
deleted_at	timestamp, nullable	Soft delete marker. Null means the file is still active.

Table: upload_sessions

Column	Type	Notes
session_id	UUID	Identifies the presigned URL handed to the client.
user_id	bigint	Used to confirm the same user completing the session started it.
file_id	UUID	Links back to the files row created in UPLOADING state.
expected_size	bigint	Compared against the real object size when upload-complete fires.
content_type	varchar	Locked in at request time, checked again before processing.
status	varchar	PENDING, COMPLETED, or EXPIRED.
expires_at	timestamp	A presigned URL that is never used should not stay valid forever.

Many beginners design this with a single files table and call it done. The session table exists for a reason that only shows up in production: you need a place to track an upload that was requested but never

finished, so an expiry job can clean it up instead of leaving an orphaned row sitting in UPLOADING status forever.

One distinction worth saying out loud in the interview: `original_name` is whatever the client sent, purely for display, you never trust it for a file path. `storage_key` is the real, safe path your backend generates, usually something like the `file_id` itself or a hashed prefix plus the `file_id`. Mixing these up is how path traversal bugs happen.

Metadata lives in PostgreSQL or MySQL because it is relational, small, and benefits from indexes and constraints. The actual file bytes go into object storage like S3, never into the database. Storing files as BLOBs works for a college assignment and falls apart the moment your database backups start taking hours because they are hauling around gigabytes of video content that has nothing to do with relational data.

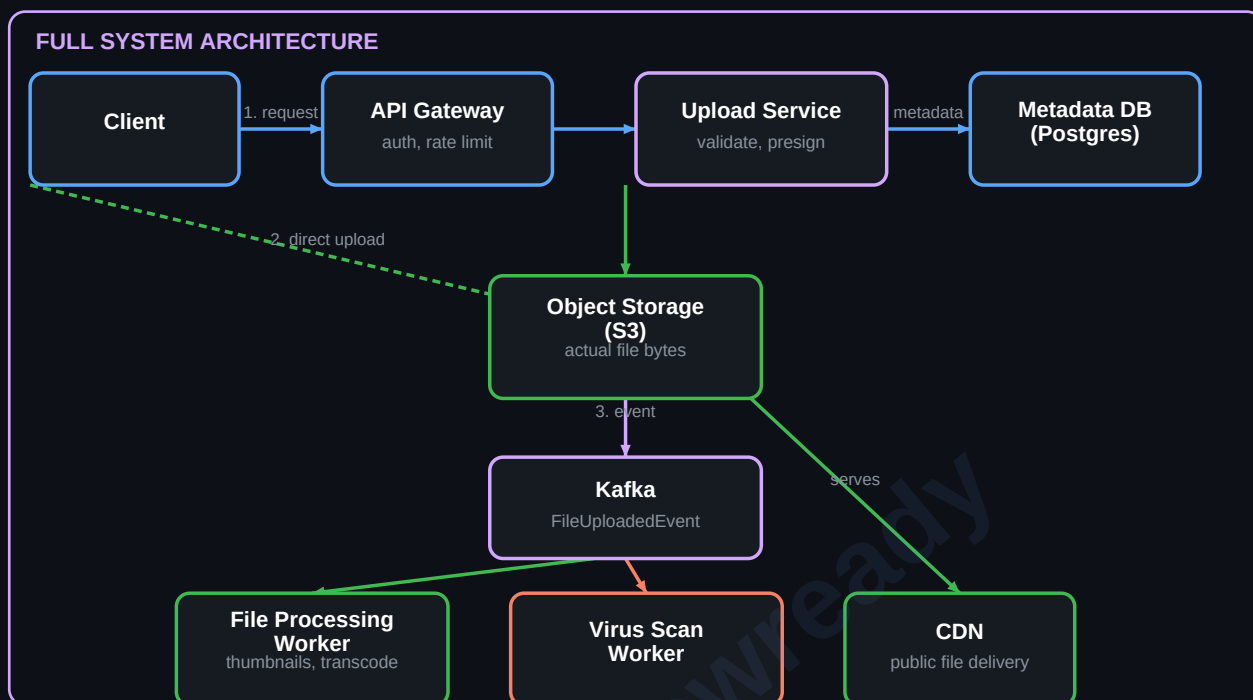
Deletion follows the same soft-first pattern as upload sessions. DELETE sets `deleted_at` on the files row immediately, that is fast and never fails. The actual object cleanup in S3 happens afterward through an async job. If that cleanup call fails or storage is briefly unavailable, the user already sees the file as deleted, and the job just retries later instead of leaving the API hanging on a slow external call.

ACCESS CONTROL

Access control in one breath: users can only access their own private files, decided by `user_id` on the files row. Public files can be served straight through the CDN with no auth check. Private files always go through a short-lived presigned download URL, never a permanent link. Admin or support access to a user's files is a separate, audited path, not a side effect of the normal download endpoint.

05 HIGH-LEVEL DIAGRAM

Look at where the thick line goes in this diagram. The client uploads directly to object storage, never through the Upload Service. The backend's job is permission and metadata, not bytes.

**TIP**

The dashed line is the important one in this diagram. That is the client uploading directly to S3, completely bypassing the Upload Service and the API Gateway. Everything else in this diagram is metadata, permission, and processing, not file bytes.

06 KEY FLOW DIAGRAM

This is the flow an interviewer will ask you to narrate step by step. Walk through it in order, and call out explicitly that step 4 never touches your backend.

- 1 Client calls upload-request with file name, type, and size
- 2 Upload Service validates metadata and creates a files row with status UPLOADING
- 3 Upload Service asks S3Presigner for a presigned PUT URL and returns it to the client
- 4 Client uploads the file bytes directly to object storage using that URL
- 5 Client calls upload-complete once the direct upload finishes successfully
- 6 Upload Service sends a HEAD request to confirm the object exists, checks the real size and content type against what was promised, then marks the file UPLOADED
- 7 Upload Service publishes a FileUploadedEvent to Kafka

8 Workers pick up the event: virus scan, thumbnail generation, or transcoding runs async

9 Once processing clears, file status moves to AVAILABLE and becomes visible to other users

Step 4 never touches your backend. The client uploads file bytes directly to object storage using the presigned URL, so your servers never see that traffic. Step 5 is different: upload-complete is a small metadata call, the client is only telling your backend "I'm done," it carries no file bytes at all.

Notice the gap between step 4 and step 5. The client uploaded successfully, but your backend does not know that yet until the confirm call arrives. That gap is exactly where one of the trickiest failure scenarios in this whole design lives, what happens if the client never calls upload-complete at all, and it shows up again in the failure handling section.

07 COMPONENTS EXPLANATION

Component	Responsibility
API Gateway	Authenticates the user and rate limits how many upload sessions one account can open. Does not touch file bytes at any point.
Upload Service	Validates file type and size, creates the metadata row, talks to S3Presigner to generate the presigned URL, and confirms completion.
Metadata DB	Source of truth for what files exist, who owns them, and what state they are in. Never stores the bytes themselves.
Object Storage (S3)	Where every byte of every file actually lives. Durable, cheap, and built to take traffic directly from clients.
Kafka	Decouples "upload finished" from "processing happened." Lets multiple workers react to the same event independently.
File Processing Worker	Generates thumbnails, compresses images, transcodes video. Runs async so the upload response is never blocked on this work.
Virus Scan Worker	Scans the uploaded object before it is marked AVAILABLE. A file that fails this check never becomes visible to anyone else.
CDN	Sits in front of object storage for public or frequently requested files, so repeated downloads do not all hit S3 directly.

Redis is optional here and only earns its place if you need it. A common use is tracking upload session status briefly so the Upload Service does not have to hit Postgres for every status check, or rate limiting upload-request calls per user using a token bucket. If your scale does not justify it, it is fine to leave it out

and say so.

08 SCALING APPROACH

Scaling uploads

Direct-to-storage upload already does most of the scaling work for you. Object storage like S3 absorbs massive concurrent write traffic, far more than a fleet of application servers could handle doing the same job. Your Upload Service only generates a small JSON response, so it scales like any lightweight stateless service, just add more instances.

For large files, do not treat the upload as one giant request. Use multipart upload: the file is split into chunks, and each chunk uploads directly to object storage in parallel, the same way a single-part upload bypasses your backend. If one chunk fails on a flaky connection, the client only retries that chunk, not the whole multi-GB file. This is the detail that turns "uploads work on my laptop" into "uploads work on a phone with patchy signal."

Scaling reads

Put a CDN in front of files that get downloaded repeatedly: profile pictures, public documents, thumbnails. This keeps S3 from taking repeated read traffic and drops latency for users far from your storage region.

Scaling processing

Workers consuming from Kafka scale independently of the upload path. If thumbnail generation lags, add more consumer instances to that group, the upload itself never slows down since the response already returned before any worker touched the file. One detail people miss: avoid storage keys that create hot partitions. A scheme like `/uploads/2026/06/28/` concentrates today's writes into one prefix. Using the `file_id` itself as the leading path segment spreads writes evenly instead.

TRADEOFF

If asked how you would handle a sudden spike, like a marketing campaign driving ten times the normal upload volume in an hour, the honest answer is that your Upload Service barely notices, since it only issues presigned URLs. The real pressure shows up in the processing workers and the Kafka consumer lag, which is exactly where you would add capacity first.

09 FAILURE HANDLING

This is where interviewers separate people who have only designed systems from people who have operated them. Go through these even if not asked directly.

Failure	What happens	How you handle it
Client upload fails midway	Partial object sits in storage, files row stays UPLOADING.	Session has an expiry. A scheduled job marks expired UPLOADING rows as FAILED and cleans up.
Upload session expires	Presigned URL stops working before the client finished.	Client requests a new upload-request, gets a fresh URL. Old session is discarded.
Confirm called but object missing	Client calls upload-complete but the object never actually landed in S3.	Upload Service checks object existence via a HEAD request before marking UPLOADED. Mismatch returns an error.
Upload succeeds, upload-complete never called	Client uploaded the bytes fine but crashed, lost network, or closed the app before confirming. File row stays stuck in UPLOADING forever.	A storage event notification can trigger processing independent of the client call. As a backstop, a reconciliation job periodically scans UPLOADING sessions and checks via HEAD whether the object actually exists, promoting it if so.
Object storage temporarily down	Presign calls or HEAD checks start failing.	Wrap calls in a circuit breaker. Return a clear retryable error rather than hanging the request.
Virus scan fails	File is flagged unsafe.	File status moves to FAILED or QUARANTINED, never reaches AVAILABLE, and the object is scheduled for deletion.
Processing worker crashes	Kafka event is unacknowledged.	Consumer offset is not committed, so the event is reprocessed by the consumer group automatically.
Duplicate upload-complete call	Client retries the confirm call after a slow network response.	Idempotent by file_id. Second call sees status already UPLOADED and returns the same result.
DB save succeeds, S3 upload fails	Metadata exists for a file that was never actually uploaded.	Row stays UPLOADING and is caught by the same expiry cleanup job as an abandoned upload.
Orphan files in storage	Object exists in S3 with no confirmed metadata row, or metadata was deleted.	Lifecycle policy on the bucket removes incomplete multipart uploads after a set window automatically.

WATCH OUT

Notice how many of these failures resolve through expiry, reconciliation, and cleanup jobs rather than real-time error handling. That is intentional. An upload that fails halfway, or never gets confirmed, is not an emergency, it is a row sitting in an intermediate state that a background job will eventually catch and resolve. Repeatedly failing processing jobs should land in a dead letter queue instead of retrying forever and burning worker capacity.

10 JAVA + SPRING BOOT MAPPING

This is the part that turns the diagram into code you could defend in a pull request review. If I were building this in Spring Boot, here is how the layers would split.

Layer	Spring Boot mapping
Controller & Service	FileController exposes upload-request, upload-complete, GET by id, download-url, and delete. FileUploadService handles validation, presign generation, and status transitions.
Repository & Entity	Spring Data JPA FileMetadataRepository extends JpaRepository. FileEntity carries @Id on fileId, status as an enum, @Column(unique = true) on storageKey.
S3 integration	S3Service wraps AWS SDK's S3Client. Used for HEAD checks on confirm (object exists, real size, content type) and S3Presigner for upload/download URLs.
DTOs	UploadRequestDto, UploadCompleteDto, FileResponseDto, kept separate from the entity.
Messaging & cleanup	KafkaTemplate publishes FileUploadedEvent, consumed by @KafkaListener workers. A @Scheduled job expires stale upload_sessions and marks abandoned files FAILED.
Rate limiting & security	Bucket4j filter on upload-request to stop session abuse. Spring Security confirms the caller owns the file before a download URL or delete.

FILEUPLOADSERVICE.JAVA, PRESIGNED UPLOAD URL + CONFIRM VERIFICATION

CODE

```

@Service
public class FileUploadService {

    private final S3Presigner s3Presigner;
    private final S3Client s3Client;
    private final FileMetadataRepository fileRepository;

    public UploadResponseDto createUploadRequest(UploadRequestDto req, Long userId) {
        validateFileType(req.getContentType());
        String fileId = UUID.randomUUID().toString();
        String storageKey = buildStorageKey(fileId, req.getFileName());

        PutObjectRequest objectRequest = PutObjectRequest.builder()
            .bucket(bucketName).key(storageKey)
            .contentType(req.getContentType()).build();

        PresignedPutObjectRequest presigned = s3Presigner.presignPutObject(
            PutObjectPresignRequest.builder()
                .signatureDuration(Duration.ofMinutes(10))
                .putObjectRequest(objectRequest).build());

        fileRepository.save(new FileEntity(fileId, userId, storageKey,
            req.getSizeBytes(), FileStatus.UPLOADING));

        return new UploadResponseDto(fileId, presigned.url().toString());
    }

    public void confirmUpload(String fileId, String clientChecksum) {
        FileEntity file = fileRepository.findById(fileId).orElseThrow();
        HeadObjectResponse head = s3Client.headObject(HeadObjectRequest.builder()
            .bucket(bucketName).key(file.getStorageKey()).build());

        if (head.contentLength() != file.getSizeBytes()
            || !head.contentType().equals(file.getFileType())) {
            throw new UploadMismatchException(fileId);
        }
        file.setChecksum(clientChecksum != null ? clientChecksum : head.eTag());
        file.setStatus(FileStatus.UPLOADED);
        fileRepository.save(file);
    }
}

```

JAVA MAPPING

Two things worth narrating here. First, the presigned URL has a short expiry, ten minutes, not hours, since that window only needs to cover the upload itself. Second, confirmUpload never trusts the client. It pulls the real size and content type from S3 with a HEAD request and rejects the call if they do not match what was promised at request time.

11 INTERVIEW TRADE-OFFS

Pick three to five depending on time left. A tradeoff explained well beats five tradeoffs rushed through.

Decision	Option A	Option B
Upload path	Through backend: simple first, but carries every byte and doesn't scale with file size.	Direct to object storage: backend stays light, more moving parts to coordinate.
File access	Public URLs: simple, cacheable, but anyone with the link can access it forever.	Presigned URLs: secure, time-limited, adds a small request before download.
Processing timing	Sync before response: caller knows the final state immediately, but waits on scan and processing.	Async via Kafka: response is fast, file briefly sits in a processing state.
File storage	Database BLOB: transactional with your data, but bloats backups past a few MB.	Object storage: built for this, cheap at scale, needs a second system to coordinate.
Upload strategy	Single request: simple for small files, fragile on a retry for large ones.	Multipart/resumable: handles large files and flaky networks, more client complexity.

Common mistakes worth naming: sending large files through the API for every upload, storing files as BLOBs without good reason, skipping file type and size validation, assuming an upload succeeded before object storage confirms it, and forgetting a CDN for repeated downloads. At my level, I would rather name one of these clearly than pretend I have never made a mistake here.

HOW TO ANSWER THIS IN INTERVIEW IN 10 MINUTES

1. State the scope: images, PDFs, and videos get uploaded, processed async, downloaded through secure links.
2. Say the key idea immediately: the backend should never carry file bytes for large uploads. This sets up your whole answer.
3. Draw the diagram: Client, API Gateway, Upload Service, branch into object storage directly and Kafka into workers.
4. Walk the presigned URL flow: request, validate, presign, direct upload, confirm, process async.
5. Cover the database split: metadata in Postgres, bytes in object storage, and why BLOBs break down at scale.
6. Go deep on one failure scenario, the gap between a successful upload and the backend confirming it, rather than listing all of them shallowly.
7. Close with one tradeoff you would revisit if requirements changed.

For file upload systems, the main design idea is simple: the client uploads directly to object storage using a presigned URL, the backend only ever handles metadata and permission, and everything heavy, scanning, thumbnails, transcoding, happens async through Kafka after the file is already safely stored. The tradeoff is between simplicity of routing everything through your own servers and the scalability of getting out of the way and letting object storage do what it is built for.

javainterviewready

Food Delivery App

You might think this is just "take an order, assign a driver, done." It is not. The hard parts are the state machine that controls where an order sits at any moment, the payment and restaurant handoff that can each fail independently, and real-time location tracking that cannot flood your main database. This question tests whether you can keep multiple moving pieces consistent without building a distributed transaction nightmare.

01

Problem Statement

We are building a food delivery platform similar to Swiggy, Zomato, or DoorDash. Four types of users interact with the system: customers who browse and order, restaurants who receive and prepare orders, delivery partners who pick up and drop off, and an admin team for support and operations.

The three core flows are: a customer finding a restaurant and placing an order, the restaurant accepting or rejecting it, and a delivery partner getting assigned and completing the drop-off. Notifications, live tracking, and refunds all sit on top of those flows.

The real challenges are not obvious at first. A payment can succeed while the restaurant is offline. A delivery partner can cancel after picking up. GPS updates arrive every few seconds from thousands of riders. An order can get stuck mid-state if any downstream call times out. Designing around those failure points is what the interviewer actually wants to see.

INTERVIEW FOCUS

Most freshers jump straight to "here is my orders table" without explaining how the order moves between states. An interviewer wants to hear you talk about the state machine before you talk about the database schema.

02

Requirements

Functional Requirements

Requirement	What it means
Browse restaurants by location	Filter open restaurants within a given radius
View menu and pricing	See items, prices, availability for a restaurant
Place order	Customer submits cart, triggers order creation
Make payment	Customer pays; gateway response drives next step
Restaurant accepts or rejects	Restaurant gets notified and responds to the order
Assign delivery partner	System finds and assigns an available rider nearby
Track order status	Customer sees state changes: Preparing, Out for Delivery
Track delivery partner location	Customer sees rider position on map in near real time
Send notifications	Push/SMS alerts at each key state change

Non-Functional Requirements

Requirement	What it means
Low latency for browsing	Menu and restaurant list should load fast; cache is key
Strong consistency for orders	Order state and payment status must be accurate at all times
High availability at peak hours	Lunch and dinner traffic spikes must not degrade service
Scalable location updates	Thousands of GPS pings per second without DB meltdown
Reliable order processing	No lost orders, even if a downstream service is slow
Eventual consistency for tracking	Location and notifications can lag by a few seconds

02.5

Capacity Estimation

Keep this as a thinking exercise. Pick round numbers and show how each estimate drives a design decision.

Metric	Estimate	Impact on Design
Daily active users	5 million	Base for everything else
Orders per day	2 million	0.4 orders/user/day on average
Peak orders per second	~100–200 OPS	5x average; exact number matters less than GPS scale
Restaurant search reads	~10x order volume	Most sessions browse without ordering
GPS location updates	1 ping / 5 sec / active rider	20K active riders = 4,000 pings/sec
Order + event storage	~1 KB per order lifecycle	~2 GB/day for orders and events

TIP

The GPS update rate is what actually defines the scaling problem here. Peak orders at 100–200 OPS is manageable with horizontal scaling. But 4,000 GPS pings per second, every 5 seconds, from every active rider, that is a different problem entirely. It is the one number that forces you away from writing location updates into the main orders table.

03

APIs

Method	Path	Request	Response
GET	/api/v1/restaurants	lat, lng, radius (query params)	200: list of open restaurants with distance
GET	/api/v1/restaurants/{id}/menu	-	200: menu items grouped by category
POST	/api/v1/orders	restaurantId, items[], idempotencyKey	201: orderId, status=CREATED, total
POST	/api/v1/orders/{id}/payment	paymentMethod, amount	200: paymentId, status
POST	/api/v1/orders/{id}/restaurant-response	accepted: true/false, estimatedMinutes	200: order updated
POST	/internal/orders/{id}/assign-delivery	partnerId (service-call only, not client-facing)	200: deliveryAssignmentId

Method	Path	Request	Response
GET	/api/v1/orders/{id}	-	200: full order state, delivery info
PATCH	/api/v1/orders/{id}/status	status (enum)	200: updated order
POST	/api/v1/delivery/location	partnerId, lat, lng, orderId	204: accepted

WARNING

Two important access rules. First: idempotencyKey on POST /orders is not optional. Without it, a network retry creates a duplicate order. Second: the assign-delivery endpoint is internal: only the Delivery Assignment Service calls it, never the customer app. Customers cannot choose their own rider.

04

Database Design

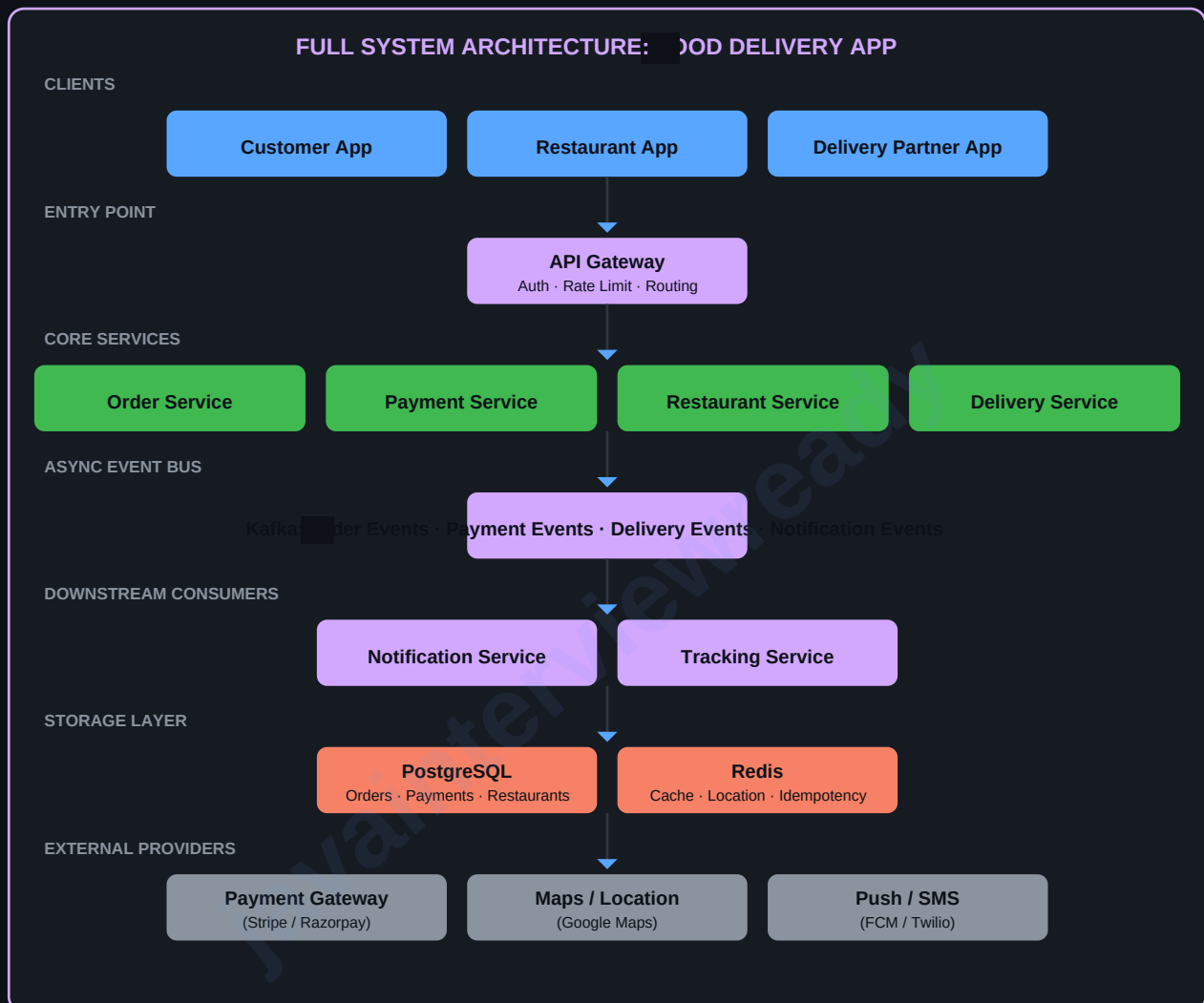
Table	Key Fields
users	id, name, email, phone, role (CUSTOMER / PARTNER / ADMIN)
restaurants	id, name, lat, lng, is_open, rating, cuisine_type
menu_items	id, restaurant_id, name, price, category, is_available
orders	id, customer_id, restaurant_id, status (enum), total, idempotency_key (UNIQUE), created_at
order_items	id, order_id, menu_item_id, quantity, unit_price (price snapshot at order time, never updated)
payments	id, order_id, gateway_ref, status, amount, created_at
delivery_assignments	id, order_id, partner_id, assigned_at, picked_up_at, delivered_at
location_history (optional)	partner_id, lat, lng, recorded_at: 1e-series store or Kafka log only. Not the main DB. Current location lives in Redis.
order_status_events	id, order_id, from_status, to_status, actor, changed_at

Database Choices

Store	Used For	Why
PostgreSQL / MySQL	orders, payments, restaurants, menu_items, delivery_assignments	Relationships and transactions matter here. Payment and order status must stay consistent.
Redis	Menu cache, restaurant availability, current GPS location per rider, idempotency key fast-check	Fast reads, TTL auto-expires stale data. First layer of idempotency guard. DB unique constraint on idempotency_key is the final safety net if Redis expires.
Kafka	Order events, payment events, restaurant responses, delivery assignment, notifications	Async event backbone. Services react independently. Events are replayable on failure.
Elasticsearch (optional)	Restaurant search by location + cuisine + rating	Useful when geo-filtering and full-text grow complex. A DB bounding-box query works at interview scale.

05

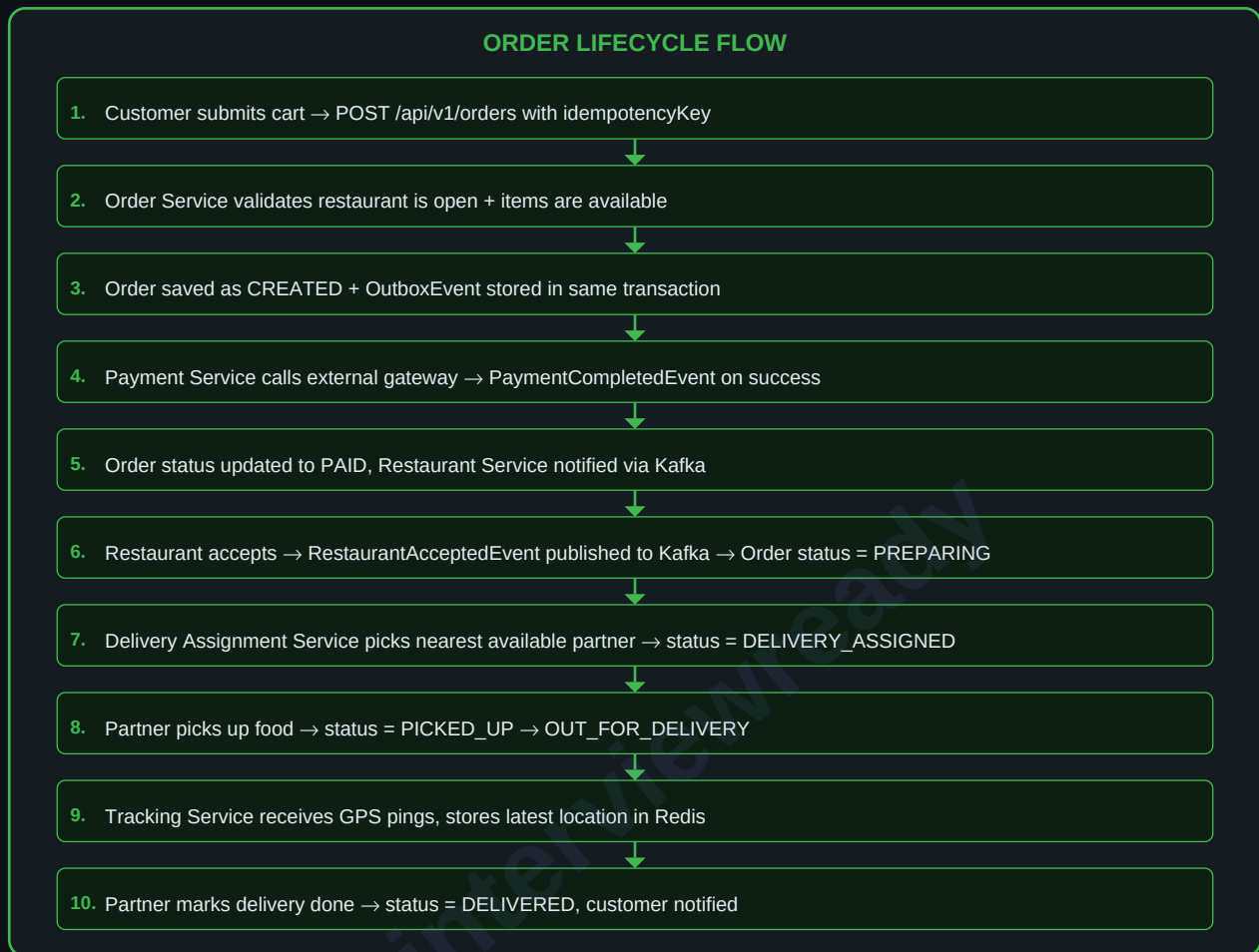
High-Level Architecture Diagram



06

Key Flow: Order Placement to Delivery

This is the flow interviewers care about most. Walk through the full order lifecycle including what happens asynchronously via Kafka.



Note: The outbox publisher sends OrderCreatedEvent to Kafka only after the database commit succeeds.

ORDER STATE MACHINE

CREATED	Order saved, idempotency key stored. Awaiting payment.
PAYMENT_PENDING	Payment gateway call initiated. Waiting for gateway response.
PAID	Payment confirmed. Order sent to restaurant via Kafka event.
PREPARING	Restaurant accepted via RestaurantAcceptedEvent. Kitchen is cooking.
DELIVERY_ASSIGNED	Rider assigned by Delivery Assignment Service. Locked to this order.
PICKED_UP	Rider confirmed food collected from restaurant.
OUT_FOR_DELIVERY	Rider is moving toward the customer. Live GPS tracking active.
DELIVERED	Customer confirmed delivery. Order complete.
CANCELLED	Terminal state. Triggered by customer, restaurant timeout, or system.
REFUNDED	Terminal state after cancellation when payment was already taken.

WARNING

Two things to say unprompted here. First: restaurant acceptance is an event (RestaurantAcceptedEvent), not a separate visible state: → customer sees PREPARING, not RESTAURANT_ACCEPTED. Second: only valid transitions are allowed through the service layer. A client cannot send DELIVERED if the order is still in PAID state. CANCELLED and REFUNDED are separate terminal states: → cancelled does not always mean refunded.

07

Components Explanation

Component	Responsibility
API Gateway	Auth, rate limiting, routing. Enforces role-based access: customers see only their own orders; restaurants see only their orders; delivery partners see only assigned deliveries; admins get support views.
Order Service	Core of the system. Owns the order state machine. Validates carts, creates orders, drives status transitions. Writes OutboxEvents during state transitions. A separate outbox publisher sends those events to Kafka after the database commit.
Payment Service	Calls the external payment gateway. Updates payment status in DB. Publishes PaymentCompletedEvent or PaymentFailedEvent.

Component	Responsibility
Restaurant Service	Manages restaurant metadata, menu, and availability. Notifies restaurant app of incoming orders. Receives accept/reject response.
Delivery Assignment Service	Internal service. Triggered by Kafka event after restaurant accepts. Uses Redis GEO to find nearest available partner. Uses optimistic locking or DB status check to prevent assigning the same partner to two orders simultaneously.
Tracking Service	Receives GPS pings from partners, stores latest position in Redis, exposes live location to customer app.
Notification Service	Subscribes to order events via Kafka. Sends push notifications, SMS, or in-app alerts at each state change.
Redis	Menu cache, restaurant availability cache, current delivery location, and idempotency key store. Fast reads, TTL handles expiry.
Kafka	Async event bus between services. Decouples payment, restaurant, delivery, and notification flows. Events are replayable.
PostgreSQL	Source of truth for orders, payments, restaurants, menu items, and delivery assignments. Handles transactions.
External Payment Gateway	Razorpay, Stripe, or similar. Order Service never trusts payment status until the gateway confirms it.
Maps / Location Provider	Google Maps or similar. Used by Delivery Assignment Service to calculate ETAs and by Tracking Service for routing.

08

Scaling Approach

Concern	Approach
Menu and restaurant browsing	Cache menus in Redis. Use CDN for food images. Scale Restaurant Service horizontally. Add read replicas for browse traffic.
Order placement at peak	Order Service is stateless: scale horizontally. Kafka absorbs bursts without overwhelming downstream services.
GPS location updates	Write partner location to Redis only, not to the main DB on every ping. Store historical path in a Kafka log or time-series store.
Delivery partner assignment	Use Redis GEO commands for fast nearest-partner lookup. At large scale, use a dedicated geo-search index or Elasticsearch.

Concern	Approach
Notification volume	Notification Service reads from Kafka and scales horizontally. Failures do not block order flow since it is fully async.
Database growth	Partition the orders table by month or region at large scale. Archive old orders to cold storage. Use connection pooling.
Read-heavy restaurant search	Move to Elasticsearch for geo + text + filter queries at scale. At interview scale, a DB geo-query with bounding box is fine.

09

Failure Handling

Failure Scenario	How to Handle
Payment succeeds, order update fails	PaymentCompletedEvent in Kafka. Order Service retries. Idempotency key prevents double-processing.
Restaurant rejects after payment	Trigger automatic refund workflow. Publish PaymentRefundEvent. Notify customer. Order moves to CANCELLED.
Payment gateway timeout	Resilience4j @CircuitBreaker around gateway call. Return PAYMENT_PENDING. Poll for confirmation separately.
Duplicate order request	idempotencyKey checked in Order Service against Redis. Return existing order if key already seen.
Restaurant Service is down	Kafka event sits in topic until service recovers. Order stays in PAID state. Alert if stuck too long.
Restaurant does not respond in time	@Scheduled job scans orders in PAID state older than N minutes. Auto-cancels, triggers refund, publishes CancellationEvent, notifies customer.
Delivery partner cancels mid-delivery	Re-trigger Delivery Assignment Service. If no partner found after retries, escalate to ops team.
No available delivery partner	Queue the order. Retry assignment every N seconds. Customer is notified of delay.
Location updates stop	Frontend shows last known position. Alert fires if no ping received for an active order within X seconds.
Notification Service fails	Non-blocking. Order flow is unaffected. Notifications retried from Kafka dead letter topic.

Failure Scenario	How to Handle
Kafka consumer lag	Events are not lost. Consumers catch up. Alert if lag grows beyond threshold. Scale consumer group.
Redis down	Fall back to DB for menu reads and order status. Location tracking degrades gracefully.

javainterviewready

10

Java + Spring Boot Mapping

Component	Spring Boot Mapping
OrderController	POST /orders, PATCH /orders/{id}/status, GET /orders/{id}. Validates input, calls OrderService.
RestaurantController	GET /restaurants, GET /restaurants/{id}/menu, POST /orders/{id}/restaurant-response.
DeliveryController	POST /delivery/location (called by partner app). assign-delivery is internal: triggered by DeliveryAssignmentService via Kafka, not exposed to clients.
OrderService	Core state machine logic. @Transactional for creation + status transitions. Writes OutboxEvents instead of directly publishing Kafka events inside the transaction.
PaymentService	Calls external gateway via WebClient/OpenFeign. Resilience4j @CircuitBreaker wraps gateway call.
DeliveryAssignmentService	Reads nearby partner from Redis GEO. Writes delivery_assignments to DB. Publishes DeliveryAssignedEvent.
TrackingService	Writes lat/lng to Redis. Consumer of partner GPS events. Exposes latest location via GET endpoint.
NotificationService	@KafkaListener on order/payment events. Sends push via FCM or SMS via Twilio.
OrderRepository	Spring Data JPA. findByIdempotencyKey() for dedup. findById() for order history.
RedisTemplate	opsForValue() for menu cache, opsForGeo() for partner location, opsForValue().setIfAbsent() for idempotency.
KafkaTemplate	Used by the outbox publisher to send OrderCreatedEvent, PaymentCompletedEvent, RestaurantAcceptedEvent, and DeliveryAssignedEvent to Kafka after the database commit.
@KafkaListener	Separate consumer groups: order-status-updater, notification-sender, delivery-assigner.
Spring Security + JWT	Customer, restaurant, and partner apps use role-based access. Separate auth flows per actor.
@Transactional	Wraps order creation (order + order_items atomically). Wraps payment status update.
Resilience4j	@CircuitBreaker on payment gateway. @Retry on delivery assignment. @TimeLimiter on slow external calls.

Code Snippet: Order Creation with Idempotency + Outbox Event

```
@Transactional
public OrderResponse createOrder(CreateOrderRequest req) {

    String idemKey = "idem:order:" + req.getIdempotencyKey();
    Boolean isNew = redis.opsForValue()
        .setIfAbsent(idemKey, "1", Duration.ofHours(24));

    if (Boolean.FALSE.equals(isNew)) {
        return orderRepo.findByIdempotencyKey(req.getIdempotencyKey())
            .map(OrderResponse::from)
            .orElseThrow();
    }

    Order order = Order.builder()
        .customerId(req.getCustomerId())
        .restaurantId(req.getRestaurantId())
        .status(OrderStatus.CREATED)
        .idempotencyKey(req.getIdempotencyKey())
        .build();

    orderRepo.save(order);

    // Save event in same transaction:  utbox publisher sends to Kafka after commit
    outboxRepo.save(
        OutboxEvent.orderCreated(order.getId(), order.getRestaurantId())
    );

    return OrderResponse.from(order);
}
```

JAVA MAPPING

Redis setIfAbsent is the fast idempotency check. The database unique constraint on idempotency_key is the final safety net. The Order row and OutboxEvent are saved in the same transaction, so the Kafka event is published only after the database commit by a separate outbox publisher. This avoids the bug where Kafka receives an OrderCreatedEvent for an order that was never committed.

In real code, if the database unique constraint catches a duplicate idempotency_key, return the existing order instead of failing the request.

11

Interview Tradeoffs

Decision	Option A	Option B
Monolith vs Microservices	Faster to start, harder to scale order and tracking independently.	Each domain scales on its own. Kafka keeps services decoupled.
Sync payment vs async order flow	Simpler state, but whole order waits on gateway response time.	Order created immediately. Payment result drives next step via Kafka.
Redis menu cache vs DB reads	Always fresh. No cache invalidation logic needed.	10x faster reads at scale. Invalidation needed on menu updates.
Strong vs eventual consistency	Payments and order status must be strongly consistent.	Notifications and tracking can lag a few seconds. That is fine.
GPS updates in DB vs Redis	Durable and queryable. 4,000 writes/sec will stress the main DB badly.	Fast, auto-expires stale data. Use a time-series store for history.
Assign partner immediately vs after restaurant accepts	Faster but partner may be dispatched before kitchen confirms.	Safer, no wasted trips. Most real food apps wait for acceptance.
Idempotency: Redis only vs Redis + DB unique constraint	Redis alone is faster and simpler but silently fails when the TTL expires.	Redis + DB constraint covers both cases. Small extra cost on writes, much stronger guarantee.

WATCH OUT

Common Fresher Mistakes to Avoid:

- Designing everything as one giant orders table with no state machine.
- Ignoring the payment failure and restaurant-rejection refund case.
- Forgetting idempotency keys on order creation and payment.
- Treating notifications as synchronous calls inside the order flow.
- Writing every GPS update directly into the orders table.
- Assuming the restaurant always accepts the order.
- Not explaining what happens when no delivery partner is available.

12

How to Answer This in 10 Minutes

HOW TO ANSWER THIS IN 10 MINUTES

1. Open with scope. Three users, three flows: customer orders, restaurant accepts, partner delivers. Say this first. 30 seconds.
2. Mention the state machine immediately. The order moves through `CREATED` → `PAID` → `PREPARING` → `DELIVERY_ASSIGNED` → `PICKED_UP` → `DELIVERED`. Only valid transitions are allowed. This one sentence separates a senior answer from a junior one. 30 seconds.
3. Draw the architecture. Client → API Gateway → Order Service → Kafka → downstream services. Show PostgreSQL for orders, Redis for cache and location, Kafka for async events. 2 minutes.
4. Walk through the full order flow. Placement, payment, restaurant response, delivery assignment, tracking. Use Kafka to connect each step asynchronously. 2 minutes.
5. Call out the GPS problem. 4,000 pings per second cannot go into the orders table. Redis for current location, time-series or Kafka log for history. This detail stands out. 1 minute.
6. Handle one failure in depth. Payment succeeds but order update fails. Idempotency key in Redis. Kafka event drives retry. More useful than listing five failures shallowly. 1.5 minutes.
7. Close with one tradeoff. Immediate assignment vs assignment after restaurant acceptance. Pick one, explain your reasoning. 1.5 minutes.

FINAL INTERVIEW CLOSING LINE

For a food delivery system, the core design idea is: Kafka handles async coordination between services so no single call blocks the order flow, Redis handles fast location reads and idempotency checks, and PostgreSQL owns the consistent state for orders and payments. The tradeoff is between strong consistency where money and order status are involved, and eventual consistency for everything else.

QUESTION 06 OF 15 · MODULE 14

E-Commerce Order System

Every fresher can write an endpoint that inserts a row into an orders table. That is not this question. The real question is what happens when two people buy the last item at the same time, when payment succeeds but your database write fails, and when a network retry quietly tries to create the same order twice.

01 PROBLEM STATEMENT

We are building the order flow behind a product like Amazon, Flipkart, or Myntra. Not the whole e-commerce platform, just the part that starts when a customer clicks "Place Order" and ends when that order is confirmed and handed off to fulfillment. Browsing and search are mentioned only where they touch this flow.

Four kinds of people touch this system. Customers placing orders. Sellers or admins managing product listings and pricing. The warehouse or fulfillment team that packs and ships once an order is confirmed. And a support team that needs to see what happened to a specific order when a customer calls in angry about it.

The core flow in one line: customer places an order, stock gets reserved, payment completes, the order is confirmed, fulfillment starts, the package ships. Every word in that sentence is a place where something can go wrong, and that is exactly what this question is testing.

INTERVIEW FOCUS

Most freshers describe this system as "a table called orders with a status column." That answer survives about thirty seconds before the interviewer asks what happens if two people order the last unit of a product at the same moment. An interviewer wants to see a state machine, an inventory story, and a payment story, not a CRUD table.

02 REQUIREMENTS

Split functional from non-functional before drawing anything. The non-functional list is where this question gets interesting, because order systems cannot lean on "eventual consistency everywhere" the way a feed or a chat app can.

Functional requirements

Requirement	What it means
Browse and add to cart	Customer can view products and build a cart before checkout.

Requirement	What it means
Place order	Convert a cart into an order with a fixed price and item list.
Reserve inventory	Stock is held for the order so it cannot be sold twice.
Apply coupon	Discount is validated and locked in at order time.
Make payment	Customer pays through a gateway, order reflects the outcome.
Track, cancel, refund	Customer can see order status, cancel before shipping, get refunded.
Notifications & history	Confirmation messages go out, past orders stay visible.

Non-functional requirements

Requirement	What it means
Strong consistency at checkout	Order, payment, and inventory must agree. No fuzzy state allowed here.
High availability during sales	Checkout cannot fall over the one day it matters most.
Idempotent order creation	A retried request must never create a second order.
Scalable reads	Product browsing traffic is far higher than order writes.
Async outside checkout	Shipping, notifications, invoices do not need to be instant.
Auditability	Every status change on an order should be traceable later.

Notice the split. Browsing is the part that behaves like every other read-heavy system you have studied, cache it, scale it horizontally, do not think too hard. Checkout is the opposite, it behaves like a banking problem. Saying this distinction out loud early is usually the moment an interviewer relaxes, because it tells them you are not going to cache your way into double-selling someone's last pair of shoes.

02.5 CAPACITY ESTIMATION

Keep the math rough. The point is to justify two design decisions: why browsing and checkout get treated differently, and why the database, not a cache, stays the source of truth for orders.

CAPACITY ESTIMATION WALKTHROUGH

Assume 10 million daily active users

Order volume

Assume 1 million orders/day across the platform

1,000,000 / 86,400 sec ~ 12 orders/sec average

Sale events run 5x to 10x that, ~60 to 120 orders/sec at peak

Browsing vs checkout

Product page views per DAU are easily 20 to 50x the order rate

That puts browsing reads at thousands of requests/sec, mostly cacheable

Checkout traffic is far lower volume but cannot tolerate a wrong answer

Storage growth

Each order touches 4 to 5 rows: order, order_items, payment, status events

1 million orders/day x ~5 rows ~ 5 million new rows/day across these tables

Why this matters

Low write volume, high correctness requirement, this is a transactional workload, not a big-data workload. A well-indexed relational database handles this comfortably, the hard part is correctness, not throughput

This is the number that should change how you think about this system. Twelve orders a second, even sixty at peak, is not a scale problem. Plenty of single PostgreSQL instances handle far more transactional writes than that. The actual difficulty is correctness under concurrency, not raw throughput, and that changes which tradeoffs are worth making.

03 APIs

Ten endpoints across browsing, cart, and order management. Notice which ones the client controls and which ones it does not.

Method & Path	Notes
GET /api/v1/products	Browse listing, cached heavily, paginated.
GET /api/v1/products/{id}	Single product detail, cached with short TTL.
POST /api/v1/cart/items	Add an item to the cart, does not touch inventory yet.
GET /api/v1/cart	Read the current cart with live prices for display.
POST /api/v1/orders	Places the order. Requires an idempotencyKey, see below.
POST /api/v1/orders/{id}/payment	Initiates payment for an existing order.
GET /api/v1/orders/{id}	Fetch one order with current status.

Method & Path	Notes
GET /api/v1/orders	Order history for the logged-in customer.
POST /api/v1/orders/{id}/cancel	Customer-initiated cancellation, only valid in certain states.
POST /api/v1/orders/{id}/refund	Triggers a refund, usually support or system-initiated.

INTERVIEW MOMENT

Three things worth saying without being asked. POST /orders must carry an idempotencyKey generated by the client, usually once per checkout attempt, so a retried request returns the existing order instead of creating a new one. The client never sends an order status in any request, status is a backend decision, never client input. And the payment gateway's webhook callback is a separate, signature-verified endpoint the backend controls, not something the browser calls directly.

04 DATABASE DESIGN

Relational, not NoSQL. Orders, payments, and inventory need transactions and constraints, this is the rare system design question where the safe interview answer really is PostgreSQL or MySQL without much debate.

Core tables

Table	Key columns	Notes
products	id, name, current_price, seller_id, status	Live catalog data, price here can change anytime.
inventory	product_id, available_quantity, reserved_quantity	reserved_quantity exists so available stock is never overstated.
carts / cart_items	cart_id, user_id, product_id, quantity	Pre-checkout state, no inventory impact yet.
coupons	code, discount_type, value, max_usage, status	Validated and locked in at order time, not re-checked later.

Order tables (the ones that matter most)

Table	Key columns	Notes
orders	id, user_id, status, total_amount, idempotency_key	idempotency_key has a unique constraint, this is your last line of defense.
order_items	order_id, product_id, quantity, unit_price_snapshot	Price is frozen here. The live products table can change tomorrow.

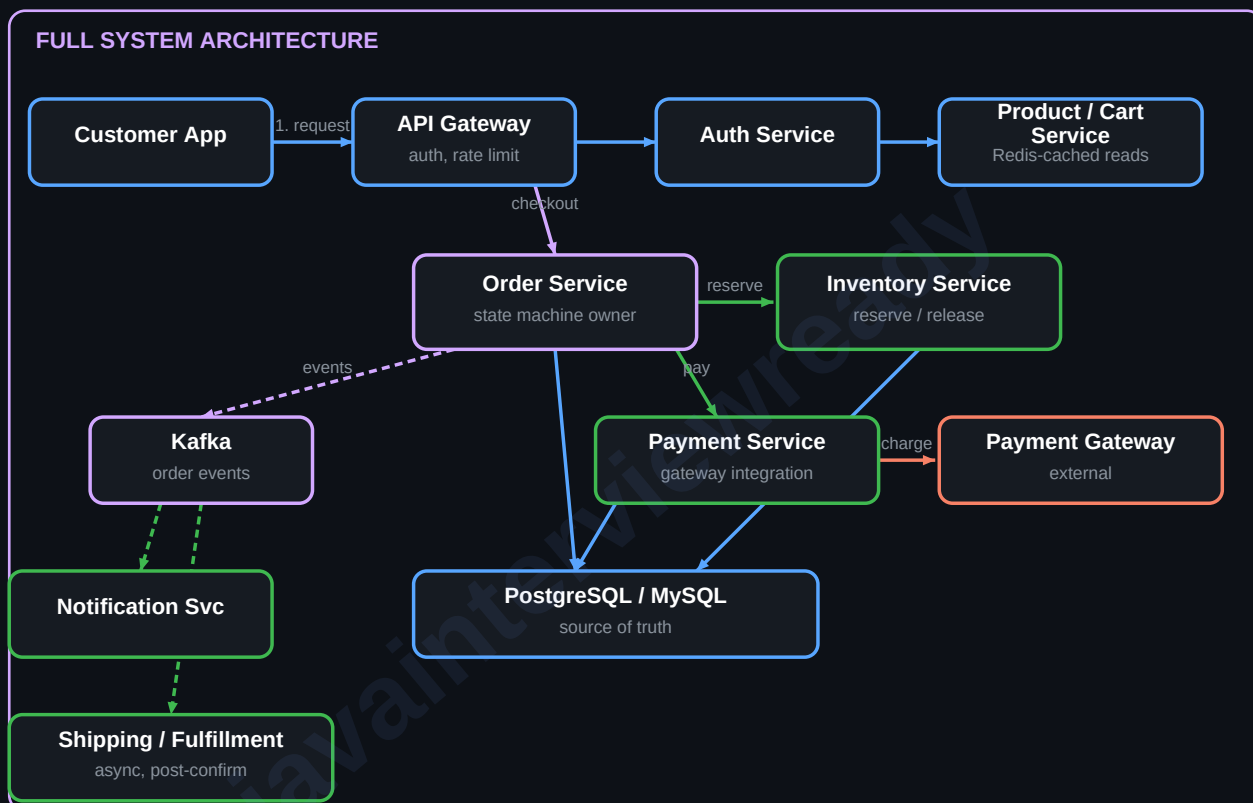
Table	Key columns	Notes
payments	id, order_id, payment_gateway_ref, status, amount	payment_gateway_ref ties your record back to the gateway's own ID.
inventory_reservations	order_id, product_id, quantity, status, expires_at	Holds stock temporarily. expires_at releases it if payment never completes.
order_status_events	order_id, from_status, to_status, actor, reason	Append-only audit log. Every transition leaves a row here.

unit_price_snapshot is not optional, it is the whole point of order_items existing as a separate table from products. A product's price can change an hour after someone orders it, but their order should always reflect the price they actually paid. Many beginners join orders straight to products for price and get this wrong the moment a seller updates a listing.

idempotency_key needs a real unique constraint in the database, not just an application level check. Two requests can race past an application check at almost the same millisecond, the database constraint is what actually guarantees only one order gets created. inventory updates need a transaction with either optimistic locking, a version column, or a row-level lock, plain read-then-write logic will oversell under load. And order_status_events exists because the first question support asks when something goes wrong is always "what happened to this order, and when."

05 HIGH-LEVEL DIAGRAM

Two services do the heavy lifting: Order Service and Inventory Service, talking inside a transaction boundary. Everything after confirmation, notifications, shipping, fans out async through Kafka and never blocks the customer's checkout response.



TIP

The straight arrows from API Gateway down to Order, Inventory, and Payment are the synchronous, consistent checkout path. The dashed arrows out of Kafka into Notification and Shipping are the async path. If you only remember one thing about this diagram for the interview, remember which arrows are dashed and why.

06 KEY FLOW DIAGRAM

Place Order to Confirmation, the flow every interviewer wants you to narrate end to end. Pay close attention to step 5, this is where two customers can collide over the same unit of stock.

- 1 Customer clicks Place Order, request carries an idempotencyKey

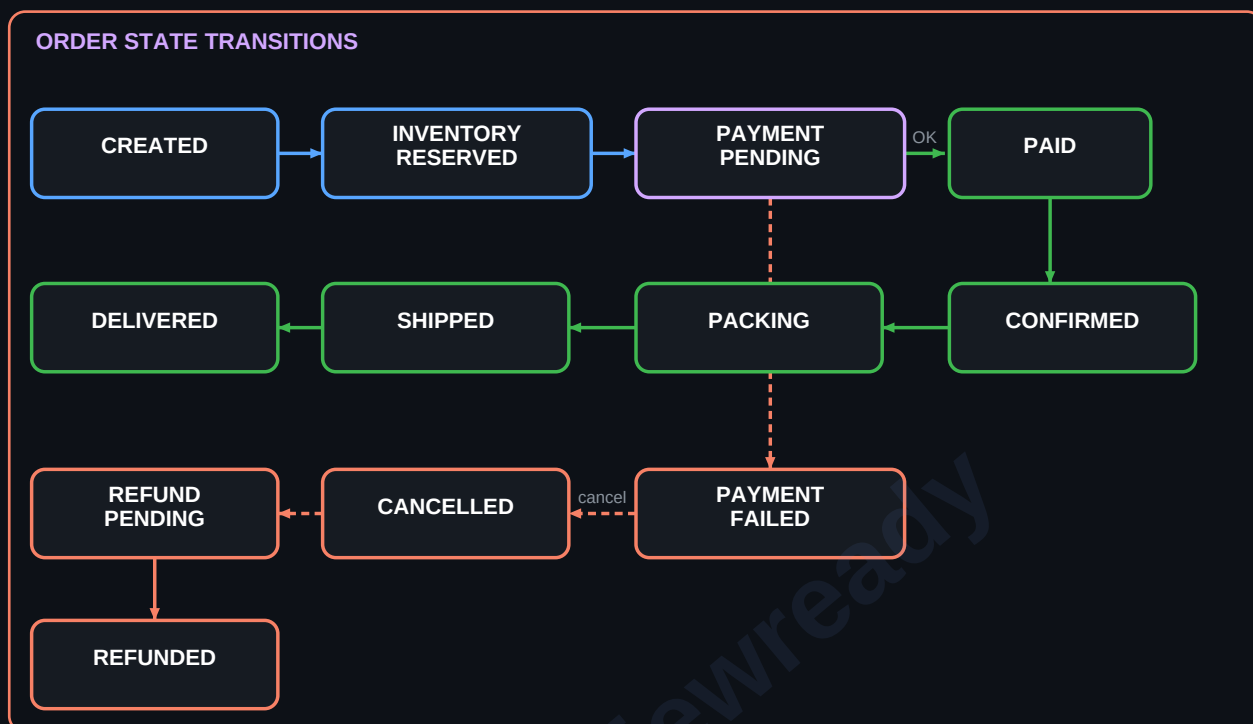
- 2 Order Service checks the key against existing orders before doing anything else
- 3 Order Service validates the cart and snapshots current prices and discounts
- 4 Order is created with status CREATED, then moves to PENDING_PAYMENT
- 5 Inventory Service reserves stock inside a transaction with row-level locking
- 6 Payment Service initiates payment against the gateway for the snapshotted amount
- 7 Payment gateway confirms success asynchronously through a webhook callback
- 8 Order status moves to PAID, then CONFIRMED once reservation is finalized
- 9 Outbox publisher sends OrderConfirmedEvent to Kafka after the confirmation transaction commits
- 10 Notification Service sends confirmation, Shipping Service starts packing

WATCH OUT

Step 5 is the one to slow down on. If two customers check out the last unit at nearly the same moment, whoever's transaction acquires the row lock first wins, the second one fails the reservation and their order moves to a failed or rejected state before payment is even attempted. This is exactly why inventory reservation happens before payment, not after, you do not want to charge someone's card for an item you cannot actually ship.

07 ORDER STATE MACHINE

An order is not a row with a status string customers can poke at. It is a state machine with a fixed set of legal transitions, and the backend is the only thing allowed to move an order from one state to another.



CANCELLED and REFUNDED are terminal, an order does not come back from them. Inventory reservations carry their own expiry, if payment does not complete in time the reservation lapses and stock returns to available automatically, the order moves to PAYMENT_FAILED or CANCELLED instead of sitting in limbo holding stock hostage forever.

INTERVIEW FOCUS

If asked "can a customer set their order to DELIVERED from the app," the answer is no, and explaining why matters. Every transition is a backend method call guarded by business rules, never a raw status field update from a client request. Letting clients set status directly is how orders end up DELIVERED before they have even shipped.

08 COMPONENTS EXPLANATION

Component	Why it exists
API Gateway	Auth and rate limiting in front of everything, especially important on checkout endpoints during a sale.
Product / Cart Service	Owns the read-heavy browsing path. Lives mostly off Redis and a search index, separate from order logic.
Order Service	Owns the state machine. The only service allowed to move an order between states.

Component	Why it exists
Inventory Service	Owns reserve and release of stock. Every reservation is a transaction, not a best-effort decrement.
Payment Service	Talks to the external gateway, handles the webhook callback, never stores card details itself.
Kafka	Decouples order confirmation from everything that happens after it. Notification and shipping do not block checkout.
Notification Service	Consumes order events, sends confirmation messages. Fully async, a delay here never affects checkout.
Shipping / Fulfillment Service	Consumes OrderConfirmedEvent, starts packing. Has no bearing on whether the order itself succeeded.
Redis	Product and cart caching for fast reads, plus an idempotency guard for fast-path duplicate detection.
Database	Source of truth for orders, payments, and inventory. Never bypassed for the checkout path.

Redis is a speed layer here, not a source of truth. It can tell you fast that a request with this idempotency key was seen recently, but the database unique constraint is what actually prevents a duplicate order if Redis is unavailable or the key expired. Treat Redis as an optimization on top of correctness, never as the thing providing correctness.

09 SCALING APPROACH

Scaling browsing

Product listing and detail pages are classic read-heavy traffic. Cache aggressively in Redis, put a CDN in front of images, use a search index like Elasticsearch for filtering and search. None of this needs strong consistency, a product page that is a few seconds stale is a non-event.

Scaling checkout

This is the opposite instinct. Do not cache inventory counts or order writes. Scale the Order Service and Inventory Service horizontally as stateless services, but every instance still goes through the same database transactions for the actual reservation and order write. During a flash sale, protect checkout with rate limiting per user and, if stock is extremely limited, a queue that admits a controlled number of checkouts at a time instead of letting every request race for the same row lock simultaneously.

Scaling everything after confirmation

Notifications, invoice generation, and shipping triggers all run through Kafka consumers that scale independently of checkout. If the shipping team's system is slow during a sale, that shows up as consumer lag, not as a failed checkout. Order history reads can use a read replica so reporting and support queries never compete with live checkout traffic on the primary database.

TRADEOFF

If asked how you would handle a flash sale with very limited stock, like 500 units of a phone for 50,000 interested buyers, the honest answer is that no amount of horizontal scaling fixes contention over the same 500 rows. The real fix is a queue or waiting room in front of checkout that admits requests at a controlled rate, so the database only ever sees a manageable number of concurrent reservation attempts.

10 FAILURE HANDLING

Distributed transactions across order, payment, and inventory are not realistic here. What actually keeps this system correct is idempotency, compensating actions, and a willingness to reconcile slightly after the fact instead of guaranteeing perfection in one atomic step.

Failure	What happens	How you handle it
Duplicate order from a retry	Client retries a timed-out request, same checkout attempt fires twice.	idempotencyKey checked first, backed by a DB unique constraint as the real guarantee.
Payment succeeds, order update fails	Gateway charged the customer but your write to mark PAID never committed.	Webhook event is retried by the gateway. A reconcile job cross-checks gateway status against local order state.
Inventory reserved, payment fails	Stock is held but the customer never actually paid.	Reservation expires automatically, or is explicitly released, stock returns to available.
Payment succeeds, reservation failed	Rare race: stock ran out after payment was already initiated.	Order is cancelled and the payment is refunded automatically, never silently kept.
Kafka consumer lag	Notification or shipping events sit unprocessed for a while.	Nothing is lost, the log retains events. Alert on lag, consumers catch up once healthy.
Payment gateway timeout	Your request to the gateway times out, unclear if the charge happened.	Order stays PAYMENT_PENDING. The webhook or a status poll confirms the real outcome later.
Shipping service down	Packing cannot start even though the order is confirmed.	Order safely sits at CONFIRMED. The shipping event is retried once the service recovers.

Failure	What happens	How you handle it
Coupon fails validation	Discount code is expired or already maxed out.	Rejected before payment is ever attempted, customer sees the corrected total immediately.
DB deadlock on inventory update	Two transactions lock rows in conflicting order.	Retried automatically with backoff. Keeping lock order consistent reduces how often this happens.

WATCH OUT

Notice the pattern across nearly every row: detect, do not prevent perfectly. You cannot wrap a database write, an external payment gateway call, and a stock update into one atomic operation across separate systems. What you can do is make every step idempotent, give every uncertain state an expiry or a reconciliation path, and make sure money is never silently kept for stock you cannot deliver.

11 JAVA + SPRING BOOT MAPPING

This is where the state machine and the transaction boundaries turn into code you could actually defend in review. Here is how the layers split.

Layer	Spring Boot mapping
Controller	OrderController exposes POST /orders, GET /orders/{id}, POST /orders/{id}/cancel, and the payment trigger endpoint.
Order Service	@Transactional on order creation and every valid state transition. This is the only place status changes happen.
Inventory Service	reserveInventory() and releaseInventory(), using @Version optimistic locking or SELECT FOR UPDATE depending on contention.
Payment Service	Calls the gateway through WebClient or OpenFeign, wrapped in Resilience4j CircuitBreaker and Retry.
Repositories	OrderRepository, OrderItemRepository, InventoryRepository, PaymentRepository, InventoryReservationRepository, all Spring Data JPA.
Redis	Idempotency guard for a fast existence check, plus product and cart caching. Never the source of truth.
Outbox	OutboxRepository saves OutboxEvent rows in the same transaction as the order. Guarantees the event is never lost or sent for a rolled-back order.

Layer	Spring Boot mapping
Kafka	A separate outbox publisher reads unpublished OutboxEvent rows and sends OrderCreatedEvent or OrderConfirmedEvent to Kafka after the database commit. @KafkaListener drives Notification and Shipping consumers.
Security	Spring Security with JWT. Customers can only fetch their own orders, support roles get a separate operational view.

ORDERSERVICE.JAVA: IDEMPOTENT ORDER CREATION WITH OUTBOX EVENT

CODE

```

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final InventoryService inventoryService;
    private final OutboxRepository outboxRepository;

    @Transactional
    public OrderResponseDto createOrder(CreateOrderRequest req, Long userId) {
        Optional<Order> existing = orderRepository.findByIdempotencyKey(
            req.getIdempotencyKey());
        if (existing.isPresent()) {
            return OrderResponseDto.from(existing.get());
        }

        Order order = new Order(
            userId,
            req.getIdempotencyKey(),
            OrderStatus.CREATED
        );

        order.applyPriceSnapshot(req.getCartItems());
        orderRepository.save(order);

        inventoryService.reserveInventory(order.getId(), order.getItems());

        order.setStatus(OrderStatus.PENDING_PAYMENT);
        orderRepository.save(order);

        outboxRepository.save(
            OutboxEvent.orderCreated(order.getId())
        );

        return OrderResponseDto.from(order);
    }
}

```

JAVA MAPPING

The idempotency check catches most retries early, but the real guarantee comes from the database unique constraint on `idempotency_key`. Inventory reservation and order state update happen inside the checkout transaction. Instead of sending Kafka directly inside the transaction, we save an `OutboxEvent` with the order. A separate outbox publisher sends `OrderCreatedEvent` to Kafka only after the database commit succeeds. This avoids the bug where Kafka receives an event for an order that was never committed.

12 INTERVIEW TRADE-OFFS

Pick three to five depending on time left. These are the decisions interviewers actually want you to reason through, not just name.

Decision	Option A	Option B
Reserve stock before or after payment	After payment: simpler, but you might charge someone for stock you cannot deliver.	Before payment: never overcharge for unavailable stock, but reservation can expire and waste effort.
Sync checkout vs event-driven	Fully synchronous: simple to reason about, but checkout latency includes every downstream step.	Sync for order/payment/inventory, async for the rest: lower checkout latency, more moving parts.
Monolith order service vs microservices	Monolith: simpler transactions, one deploy, harder to scale teams independently.	Microservices: independent scaling and ownership, distributed transaction complexity.
Consistency: checkout vs post-confirmation	Strong consistency everywhere: safest, but shipping and notifications do not need it and pay the cost anyway.	Strong at checkout, eventual after: matches reality, requires discipline about where the line is.
Redis cache vs DB for product reads	Cache aggressively: fast browsing, occasional stale price shown until checkout re-validates.	Always hit DB: always correct, does not scale to browsing traffic without heavy hardware.

Common mistakes worth naming: checking inventory without locking it, trusting the client for price or discount amounts, updating order status directly from a client request, treating payment and order update as one atomic step instead of two reconciled ones, and skipping the idempotency key because "retries probably will not happen." At my level, I would rather call out one of these clearly than pretend this system has no sharp edges.

13 HOW TO ANSWER THIS IN 10 MINUTES

HOW TO ANSWER THIS IN INTERVIEW IN 10 MINUTES

1. State the scope: product, cart, order, payment, inventory, shipping, and that checkout is the hard part, not browsing.
2. Mention the state machine early. Status is backend-owned, never client input.
3. Explain inventory reservation and why it happens before payment, with locking.
4. Explain idempotency: the key, the DB unique constraint, why retries are guaranteed to happen.
5. Draw the architecture: Order and Inventory Service in the sync path, Kafka fanning out to Notification and Shipping after confirmation.
6. Explain the Kafka async flow and why shipping/notification never block checkout.
7. Go deep on one failure, payment succeeds but the order update fails, rather than listing all of them shallowly.
8. Close with the consistency tradeoff: strong inside checkout, eventual outside it.

14 FINAL INTERVIEW CLOSING LINE

FINAL INTERVIEW CLOSING LINE

For an e-commerce order system, the core design idea is that checkout needs strong consistency for order, payment, and inventory, while everything after confirmation such as notification, invoice, and shipping can move asynchronously through Kafka. Redis helps with fast reads and idempotency, but the database remains the source of truth for orders and inventory.

QUESTION 07 OF 15 · MODULE 14

Payment System

Every e-commerce app, food delivery app, subscription product, and booking platform eventually needs this. We are not building Visa, UPI, or Razorpay. We are building the backend service inside our own application that talks to one of those gateways and keeps our own records correct no matter what the network does in between.

01 PROBLEM STATEMENT

Calling a payment gateway's API is maybe twenty lines of code. That is not the interview question. The real question is what your system does in the gap between "we asked the gateway to charge the user" and "we are completely sure what happened." That gap is where every hard problem in this design lives.

Order Service, Booking Service, and Subscription Service all call this system the same way. The customer app never talks to it directly, it goes through whichever service owns the thing being paid for. The gateway calls back into it through a webhook, and admin or support staff query it when a customer says "I paid but my order still shows unpaid."

The hard parts are specific: not charging someone twice when their app retries a slow request, handling a gateway timeout where you genuinely do not know if the charge went through, handling a webhook that arrives twice or arrives before your own write finishes, and never trusting the client's word that a payment succeeded. Every one of those is a real production incident waiting to happen if you design this casually.

INTERVIEW FOCUS

A common mistake is treating this like any other CRUD service: client calls API, API writes a row, done. Money is different. The interviewer wants to see you slow down on idempotency, auditability, and who is allowed to say a payment succeeded, before you draw a single box.

02 REQUIREMENTS

Functional	Non-functional
Create a payment attempt for an order or booking	Strong consistency for payment state, this is money
Initiate the payment with an external gateway	Every write must be idempotent, retries cannot double charge
Receive and verify the gateway's webhook callback	High reliability, this service cannot silently lose an update
Look up current payment status	Full auditability, every status change must be traceable
Support refund requests	Webhook signature must be verified, never trust an unsigned call
Support reconciliation for uncertain payments	Low latency on payment initiation, high tolerance for confirmation delay
Publish payment events for other services	Eventual consistency is fine for notifications and downstream updates

Read that non-functional column again. Strong consistency for the money itself, eventual consistency for everything downstream of it. Mixing those up in either direction is where most of the failure scenarios in this question come from.

02.5 CAPACITY ESTIMATION

Worth doing briefly, mostly to make one point: this traffic is small compared to browsing traffic, and that is exactly why correctness gets more attention than throughput in this question.

CAPACITY ESTIMATION WALKTHROUGH

Assume 5 million daily active users, 1 million orders/day

70% pay online ~ 700,000 payment attempts/day

700,000 / 86,400 sec ~ 8 payments/sec average

Sale day peak (10x) ~ 80 payments/sec

Webhook traffic roughly matches payment volume, plus retries from the gateway if our endpoint is slow to respond

Storage

payments + attempts + events rows are small, a few KB per payment lifecycle

700,000/day x ~2 KB ~ 1.4 GB/day, trivial compared to chat or notification data

Eighty payments a second at peak is nothing compared to the redirect traffic from earlier questions. A single well-tuned database handles this volume without strain. The actual engineering effort goes into never getting a single one of these payments wrong, not into scaling raw requests per second.

03 APIs

Six endpoints. Notice which one is missing: there is no endpoint where the client tells us a payment succeeded. The client can only ask, never declare.

Method & Path	Payload	Notes
POST /api/v1/payments	orderId, amount, currency, idempotencyKey	Creates a payment attempt. Same idempotencyKey on retry returns the same payment, never a new one.
GET /api/v1/payments/{paymentId}	none	Current status, safe to poll from the calling service.
POST /api/v1/payments/{paymentId}/confirm	internal only	Internal use only, for trusted reconciliation flows. Never called from the client app.
POST /api/v1/payments/webhook	gateway payload + signature header	The main external callback path that can move a payment to SUCCESS or FAILED. Signature must be verified first. Reconciliation can also update status after checking the gateway directly.
POST /api/v1/payments/{paymentId}/refund	amount, reason	Starts a refund workflow, does not refund synchronously.
GET /api/v1/payments/{paymentId}/attempts	none	Support and admin view, every gateway call we made for this payment.

WATCH OUT

Say this out loud in the interview even if not asked: the frontend is never trusted to report payment success. The backend trusts only a verified webhook signature, or its own reconciliation call to the gateway's status API. A client that calls confirm directly should not be able to move money.

04 DATABASE DESIGN

Five tables. Each one exists because squashing it into another table would erase history you will eventually need, usually during a support escalation at the worst possible time.

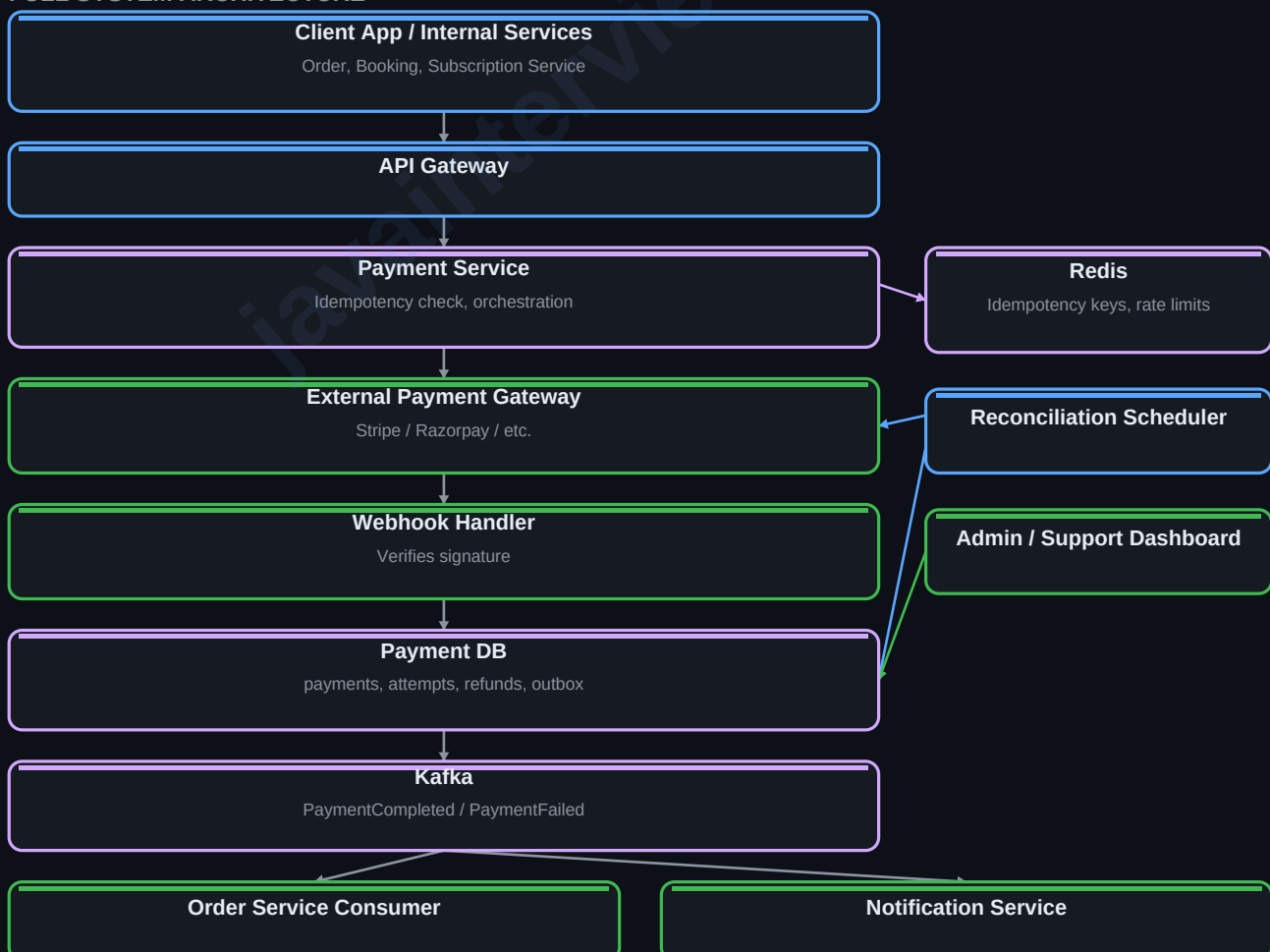
Table	Key fields	Answers
payments	payment_id, order_id, user_id, amount, currency, status, idempotency_key (unique), gateway_name, gateway_payment_id, created_at, updated_at	What is the current truth about this payment, right now.
payment_attempts	attempt_id, payment_id, gateway_request_id, gateway_response_code, status, error_message, created_at	What happened on every single call we made to the gateway for this payment.

Table	Key fields	Answers
refunds	refund_id, payment_id, amount, status, gateway_refund_id, reason, created_at	Refunds get their own table because a refund has its own lifecycle, separate from the original charge.
payment_events	event_id, payment_id, event_type, old_status, new_status, source, created_at	The audit trail. Every status transition, and what triggered it, forever.
outbox_events	event_id, payment_id, event_type, payload, published, created_at	Events waiting to be published to Kafka, written in the same transaction as the status change that created them.

Many beginners try to make `payments.status` do double duty as the audit trail by just overwriting it. That tells you the current state but throws away the story of how it got there. `payment_events` keeps that story, separately and permanently, and it is usually the first table a support engineer opens when a customer disputes a charge.

05 HIGH-LEVEL DIAGRAM

FULL SYSTEM ARCHITECTURE



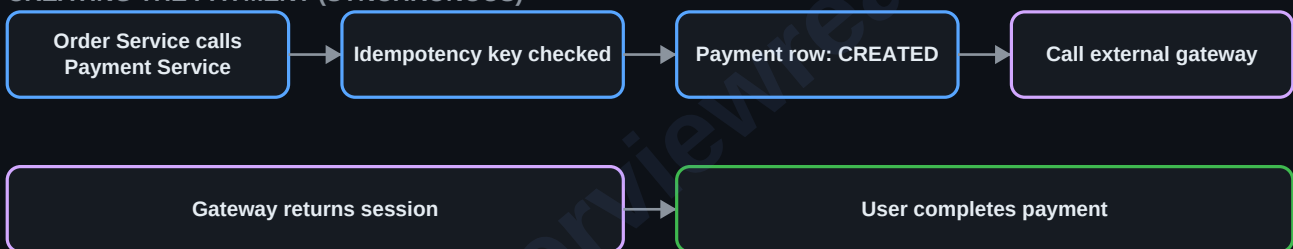
There are two sources of truth in this diagram, not one. Payment Service is the source of truth for everything inside our application. The external gateway is the source of truth for whether money actually moved. A reconciliation job, covered in scaling, exists specifically to keep those two in sync when a webhook gets lost or arrives late.

Payment Service also writes the initial **CREATED** row to Payment DB before it ever calls the gateway, this diagram only draws the webhook's update arrow into the DB to keep the page readable. Both the creation step and the confirmation step write to the same source of truth.

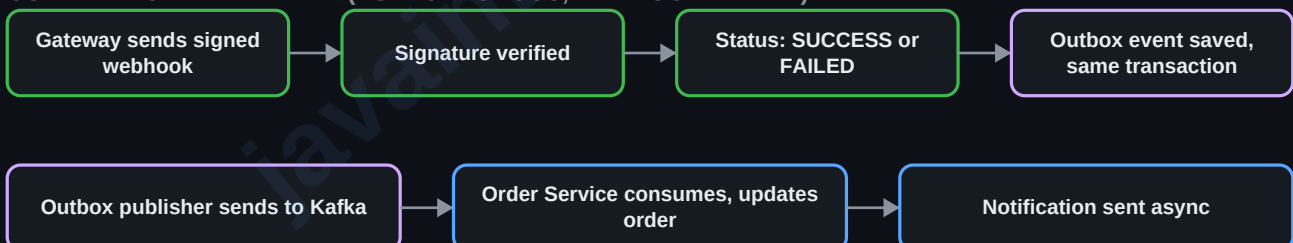
06 KEY FLOW DIAGRAM

Two halves. The first half is synchronous and fast. The second half is async and is where the actual money confirmation happens.

CREATING THE PAYMENT (SYNCHRONOUS)



CONFIRMING THE PAYMENT (ASYNCHRONOUS, WEBHOOK DRIVEN)

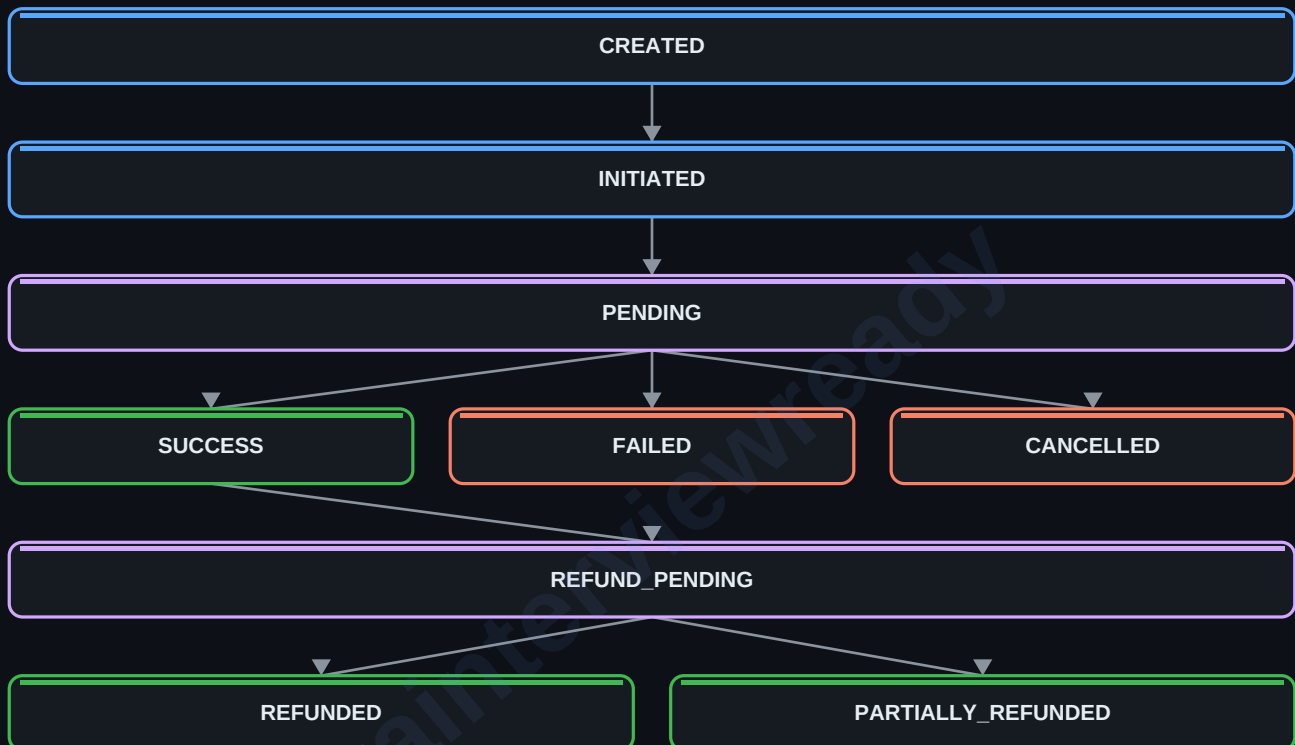


Notice where the transaction boundary actually sits. The status update and the outbox event commit together, in one transaction. Sending to Kafka happens afterward, by a separate publisher. If that publish step is the part that fails, the payment status is already correctly **SUCCESS**, it is only the downstream notification that is delayed.

07 PAYMENT STATE MACHINE

Draw this even if not asked. It answers half the failure handling questions before the interviewer gets to ask them.

PAYMENT STATUS LIFECYCLE



SUCCESS and **FAILED** are final for that payment attempt, nothing transitions out of **FAILED**. **REFUND_PENDING** and its children belong to a separate refund lifecycle that only starts after a payment has already reached **SUCCESS**. The client can request a state change, but it can never directly set **SUCCESS**, only the verified webhook or the reconciliation job holds that authority.

INTERVIEW MOMENT

A good follow-up answer here: "what stops a compromised or buggy client from calling an endpoint that sets status to **SUCCESS**?" The honest answer is that no such endpoint exists. The confirm endpoint is internal only, and the only two writers of **SUCCESS** in the whole system are the webhook handler and the reconciliation job, both of which talk directly to the gateway, never to the client.

08 COMPONENTS EXPLANATION

Component	Responsibility
Payment Service	Owns the payment lifecycle inside our app. Checks idempotency, creates the CREATED row, calls the gateway, and is the only thing other internal services ever talk to directly.
Webhook Handler	Verifies the gateway's signature first, before reading a single field of the payload. Moves status to SUCCESS or FAILED and writes the outbox event in one transaction.
External Payment Gateway	The actual source of truth for whether money moved. We never assume, we only react to what it tells us through a verified channel.
Outbox Publisher	A small background process that reads unpublished rows from outbox_events and sends them to Kafka, marking them published only after a successful send.
Reconciliation Scheduler	Periodically asks the gateway directly about any payment stuck in PENDING or INITIATED too long, for cases where the webhook never arrived.
Admin/Support Dashboard	Read-only view over payments, attempts, and events, for investigating the "I paid but it still shows unpaid" ticket.

TIP

Redis only does two jobs here: a fast idempotency check before hitting the database, and short rate limit buckets so one buggy retry loop cannot flood the gateway. It is never the source of truth for payment status, payments.status in the database is. If Redis goes down tonight, the database unique constraint on idempotency_key still protects you, just a little slower.

09 SCALING APPROACH

Scaling this system is less about request volume, which is small, and more about protecting correctness while still running multiple instances of everything.

Lever	What it does	When to reach for it
Stateless Payment Service	Run many instances behind the load balancer, none of them hold session state	Default setup, this is what makes horizontal scaling possible at all
Idempotency key + DB unique constraint	Guarantees one logical payment attempt creates exactly one row, even with concurrent retries	Always, this is not optional for a payment system
Circuit breaker + timeout on gateway calls	Stops a slow gateway from tying up every Payment Service thread	Any external call you do not control the latency of

Lever	What it does	When to reach for it
Reconciliation scheduler	Catches payments stuck in PENDING when a webhook never arrives, by asking the gateway directly	Required, webhooks get lost in production more often than people expect
Read replicas for admin queries	Support dashboard and reporting read from a replica instead of the primary	Once admin/support query volume starts competing with live traffic

If I were explaining this in an interview, I would spend most of my scaling time on the reconciliation scheduler. It is the least glamorous component in this whole design and the one that actually keeps the numbers honest when the network does something weird at 2am.

10 FAILURE HANDLING

Failure	How you handle it
Client retries the same payment request	idempotencyKey plus a DB unique constraint means the retry returns the existing payment instead of creating a second one.
Gateway call times out	Treat it as unknown, not as failed. Mark the attempt as PENDING and let reconciliation resolve it, do not assume failure and let the user retry into a double charge.
Gateway says success but our DB update fails	The transaction rolls back, status stays unresolved, and reconciliation catches it on the next pass since the gateway already has the truth.
Webhook arrives twice	The handler is idempotent on gateway_payment_id, the second call sees the status already updated and exits without re-processing.
Webhook signature is invalid	Reject immediately, log it, never touch the payment row. An unverified webhook is treated as an attack, not a bug.
Service crashes after DB commit, before Kafka publish	No event is lost, the outbox row is still there unpublished. The publisher picks it up on the next run.
Refund request fails partway	Refund stays in its own REFUND_PENDING state, retried independently of the original payment's status.
Redis is down	Idempotency check falls back to the database unique constraint, slower but still correct.
Kafka lag or a slow database	Payment status itself is unaffected since it already committed, only downstream order updates and notifications are delayed.

WATCH OUT

The mistakes that come up again and again: trusting a frontend success callback, creating a brand new payment row on every client retry instead of checking the idempotency key, publishing to Kafka directly inside the same transaction as the status update, and updating the order to paid before the payment is actually confirmed. Any one of these is the kind of detail that separates a fresher answer from a senior one.

11 JAVA + SPRING BOOT MAPPING

Layer	Spring Boot mapping
Controller	PaymentController for creation, status, and refund endpoints. WebhookController is separate, with its own security rules.
Service	PaymentService first checks the DB by idempotencyKey. If payment already exists, it returns the existing response and does not call the gateway again. Redis is only a fast duplicate marker. New payments are saved first, then the gateway session is created outside the DB transaction.
Repository	PaymentRepository, PaymentAttemptRepository, RefundRepository, PaymentEventRepository, and OutboxRepository, one Spring Data JPA repository per table.
Gateway client	A thin WebClient or OpenFeign client, wrapped in Resilience4j @CircuitBreaker, @Retry, and @TimeLimiter.
Idempotency	RedisTemplate for the fast setIfAbsent check, backed by a unique constraint on idempotency_key in the JPA entity.
Webhook security	WebhookSignatureVerifier checks the HMAC signature before the controller method body runs anything else.
Outbox publisher	A separate component, not the service itself, reads unpublished outbox rows and calls KafkaTemplate. The service method never calls KafkaTemplate directly.
Reconciliation	@Scheduled ReconciliationJob, queries payments stuck in PENDING past a threshold and asks the gateway directly.
Security	Spring Security with JWT for internal service to service calls, separate from the webhook's signature based verification.

PAYMENTSERVICE.JAVA: IDEMPOTENT PAYMENT CREATION WITHOUT DUPLICATE GATEWAY CALL

```
public PaymentResponse createPayment(PaymentRequest request) {
    Optional<Payment> existing =
        paymentRepository.findByIdempotencyKey(request.getIdempotencyKey());

    if (existing.isPresent()) {
        return PaymentResponse.from(existing.get());
    }

    Payment payment = createPaymentRecord(request);

    GatewaySession session = gatewayClient.createSession(payment);

    return markPaymentInitiated(payment.getId(), session);
}

@Transactional
public Payment createPaymentRecord(PaymentRequest request) {
    redisTemplate.opsForValue()
        .setIfAbsent("pay:" + request.getIdempotencyKey(), "1", Duration.ofHours(24));

    Payment payment = Payment.createFrom(request); // status CREATED
    return paymentRepository.save(payment); // unique idempotency_key protects concurrency
}

@Transactional
public PaymentResponse markPaymentInitiated(Long paymentId, GatewaySession session) {
    Payment payment = paymentRepository.findById(paymentId).orElseThrow();

    payment.markInitiated(session.getGatewayPaymentId());
    paymentRepository.save(payment);
    attemptRepository.save(PaymentAttempt.from(payment, session));

    return PaymentResponse.from(payment);
}
```

The first database check makes duplicate retries safe because an existing payment returns immediately and does not call the gateway again. Redis is only a fast duplicate marker, not the final guarantee. The database unique constraint on `idempotency_key` is the final protection during concurrent requests. The gateway call stays outside the database transaction so a slow gateway does not hold DB locks.

WEBHOOKSERVICE.JAVA: VERIFIED STATUS UPDATE WITH CORRECT OUTBOX EVENT

```
@Transactional
public void handleWebhook(WebhookPayload payload, String signature) {
    signatureVerifier.verifyOrThrow(payload, signature);

    Payment payment = paymentRepository
        .findByGatewayPaymentId(payload.getGatewayPaymentId())
        .orElseThrow();

    if (payment.getStatus() == payload.getStatus()) {
        return; // duplicate webhook, already applied, nothing to do
    }

    if (payment.isFinalStatus() && payment.getStatus() != payload.getStatus()) {
        eventRepository.save(PaymentEvent.conflict(payment, payload));
        return; // conflicting final status, route to manual review
    }

    payment.applyGatewayResult(payload.getStatus()); // SUCCESS or FAILED
    paymentRepository.save(payment);
    eventRepository.save(PaymentEvent.statusChanged(payment));

    if (payment.getStatus() == PaymentStatus.SUCCESS) {
        outboxRepository.save(OutboxEvent.paymentCompleted(payment.getId()));
    } else if (payment.getStatus() == PaymentStatus.FAILED) {
        outboxRepository.save(OutboxEvent.paymentFailed(payment.getId()));
    }
}
```

12 INTERVIEW TRADEOFFS

Decision	Option A	Option B
Confirmation model	Sync gateway call: simple, but you are guessing at success the moment the call returns.	Async webhook confirmation: slightly more moving parts, but the gateway tells you the truth instead of you assuming it.
Trust boundary	Trusting a frontend success callback: fast to build, trivially exploitable.	Verified webhook only: the only safe option for anything involving money.
Event publishing	Direct Kafka publish inside the transaction: simple, but the event can leak before the DB commit or vanish if the commit rolls back.	Transactional outbox: one extra table and a publisher process, but the event and the DB state can never disagree.
Idempotency protection	Redis only: fast, but a Redis outage at the wrong moment lets a duplicate through.	Redis plus a DB unique constraint: Redis for speed, the constraint as the guarantee that never goes down quietly.
Refund handling	Immediate synchronous refund: simpler code, but you are now blocking on a second external call inside a request.	Refund workflow with its own state: matches how gateways actually process refunds, which is rarely instant anyway.

13. HOW TO ANSWER THIS IN 10 MINUTES

1. State the scope: this is the backend payment service inside our app, not the gateway itself, called by Order, Booking, and Subscription services. 30 seconds.
2. Name the real hard parts up front: duplicate charges, gateway timeouts, late webhooks, and never trusting the client for success. This is what signals seniority immediately. 1 minute.
3. Draw the diagram: Client, Gateway, Payment Service with Redis beside it, the external gateway, Webhook Handler, Payment DB, Kafka, downstream consumers. 2 minutes.
4. Walk through both halves of the key flow: synchronous creation, then async webhook confirmation through the outbox. 2 minutes.
5. Draw the state machine and say out loud that only the webhook or reconciliation can set SUCCESS, never the client. 1.5 minutes.
6. Cover one failure scenario in real depth, gateway timeout treated as unknown rather than failed is the strongest choice, rather than listing all nine shallowly. 1.5 minutes.
7. Close with the outbox versus direct publish tradeoff, it is the single most senior detail in this entire question. 1 minute.

14. FINAL INTERVIEW CLOSING LINE

For payment systems, the main design idea is simple: Payment Service is the source of truth inside our app, the external gateway is the source of truth for the money itself, and a verified webhook plus a reconciliation job is what keeps those two stories from ever disagreeing. The tradeoff is between confirming fast and confirming correctly, and a senior answer always picks correctly.

QUESTION 08 OF 15 · MODULE 14

Ride Booking App

Anyone can write code that inserts a ride row and sets a driver_id column. That is not what this question is testing. The real difficulty is finding a free driver in seconds, never handing that same driver to two riders at once, and keeping a phone that pings its GPS every few seconds from drowning your database.

01 PROBLEM STATEMENT

We are designing the backend behind a ride booking app like Uber, Ola, Rapido, or Lyft. Not the whole company, just the flow from a rider requesting a ride to driver assignment, the trip running, and payment closing it out.

Four parties touch this system: the rider booking a trip, the driver receiving requests and updating location, a support team that needs to inspect a ride when something goes wrong, and two external dependencies you do not control, a payment gateway and a maps or ETA provider.

The part that actually makes this hard is never the ride row. It is finding nearby available drivers fast, making sure two riders never get assigned the same driver, handling a driver who rejects or never responds, and keeping a flood of GPS pings from ever touching your main transactional database.

INTERVIEW FOCUS

Most freshers jump straight into "rides table with a driver_id column" and stop there. That design quietly assumes a driver only ever gets offered one ride at a time, which is exactly the assumption that breaks the moment two ride requests query the same nearby driver within the same second. Bring up matching and locking yourself, before the interviewer has to drag it out of you.

02 REQUIREMENTS

Split functional from non-functional before drawing anything. The non-functional column is where this question actually gets decided, because driver matching and GPS tracking have completely different consistency needs.

Functional requirements

Requirement	What it means
Request a ride	Rider gives pickup and drop, system returns a fare and ETA estimate.
Find nearby drivers	System locates available drivers close to the pickup point.

Requirement	What it means
Assign a driver	One ride request reaches one driver at a time, with accept or reject.
Track in real time	Rider sees the assigned driver's live location as they approach.
Move through valid states	Ride status changes only through backend rules, not client input.
Cancel and pay	Either side can cancel under rules, payment runs after completion.

Non-functional requirements

Requirement	What it means
Low latency matching	Finding a driver should feel near-instant, not a multi-second wait.
High availability at peak	Evening rush and rain surges cannot take the system down.
Scalable GPS ingestion	Thousands of drivers pinging every few seconds, constantly.
Strong consistency	Ride assignment and payment must never be fuzzy or double-booked.
Eventual consistency elsewhere	Live tracking and notifications can lag by a second or two.
Idempotent and auditable	No duplicate rides on retry, every state change is traceable.

Notice the split in the non-functional list. Ride assignment and payment behave like a banking problem, you cannot afford to be wrong. GPS location is the opposite, it changes every few seconds and being a second stale never hurts anyone. Many beginners design this whole system around one consistency model and either slow down matching for no reason or accidentally make ride assignment loose enough to double-book a driver.

02.5 CAPACITY ESTIMATION

Keep this rough. The number that actually matters here is not ride volume, it is GPS ping volume, and seeing that gap is the whole point of this section.

CAPACITY ESTIMATION WALKTHROUGH

Assume 5 million daily active users

Ride volume: 500,000 rides/day ~ 6 requests/sec average, 30 to 50/sec at peak

Driver location: 200,000 drivers online, pinging every 3 to 5 sec ~ 50,000 writes/sec

Storage growth: 500,000 rides/day x ~4 rows ~ 2M new rows/day

Why this matters: 6 requests/sec is nothing for PostgreSQL, 50,000 writes/sec would crush it. That gap is why driver location lives in Redis, not the rides DB

This is the single insight that should shape your whole answer. Ride requests are rare and need to be correct. GPS pings are constant and only need to be recent. If you design one storage layer for both, you either slow down rides protecting GPS writes, or you let GPS writes overwhelm the table that holds your actual business data.

03 APIs

Nine endpoints. Notice that ride status never appears as something the client can set directly, it is always read or driven through a specific action endpoint.

Method & Path	Notes
POST /api/v1/rides/estimate	Returns fare estimate and ETA before the rider commits.
POST /api/v1/rides	Creates the ride request. Requires an idempotencyKey.
GET /api/v1/rides/{rideId}	Fetch ride details and current status.
POST /api/v1/rides/{rideId}/driver-response	Driver accepts or rejects the assignment.
PATCH /api/v1/rides/{rideId}/status	Backend-only state transition, never called by client UIs directly.
POST /api/v1/drivers/location	Driver app pushes its latest GPS fix.
POST /api/v1/rides/{rideId}/cancel	Rider or driver cancels, subject to state rules.
POST /api/v1/rides/{rideId}/complete	Marks the trip done and triggers payment.
GET /api/v1/rides/{rideId}/track	Rider polls or subscribes to the driver's live location.

INTERVIEW MOMENT

Say this without being asked: the client cannot freely set ride status. PATCH /rides/{rideId}/status exists, but it is called by backend services enforcing transition rules, never directly wired to a button in the rider or driver app. A driver's app calling an endpoint that says "mark this COMPLETED" before the ride ever started is exactly the kind of bug this design has to make structurally impossible.

04 DATABASE DESIGN

Relational for everything that needs to be correct and auditable. Redis for the one thing that changes every few seconds and never needs to be perfect.

Core tables

Table	Key columns	Notes
riders / drivers	id, name, phone, status	Standard profile data, rarely the interesting part of this question.
vehicles	id, driver_id, plate, type	One driver can have history across vehicles over time.
pricing_quotes	quote_id, fare_estimate, expires_at	Quote is locked briefly so the fare cannot drift before the rider confirms.

Ride tables (the ones that matter most)

Table	Key columns	Notes
rides	ride_id, rider_id, driver_id, status, fare_estimate, final_fare, idempotency_key	idempotency_key has a unique constraint, your real duplicate guard.
driver_assignments	assignment_id, ride_id, driver_id, status, requested_at, expires_at	One row per offer made to a driver. expires_at drives the timeout and retry.
ride_events	event_id, ride_id, from_status, to_status, actor, reason	Append-only audit log, the first thing support opens on a complaint.
payment_records	payment_id, ride_id, amount, status, gateway_ref	gateway_ref ties this row back to the payment provider's own record.
outbox_events	event_id, aggregate_id, type, payload, published	Holds RideRequestedEvent, DriverAssignedEvent, RideCompletedEvent until Kafka has them.

driver_assignments deserves its own table instead of a driver_id column on rides, because an offer can be rejected or time out and move to the next candidate. A single driver_id field would lose the history of who was offered the ride first and why they did not take it, exactly what support needs when a rider complains

their driver took five minutes to show up as "assigned."

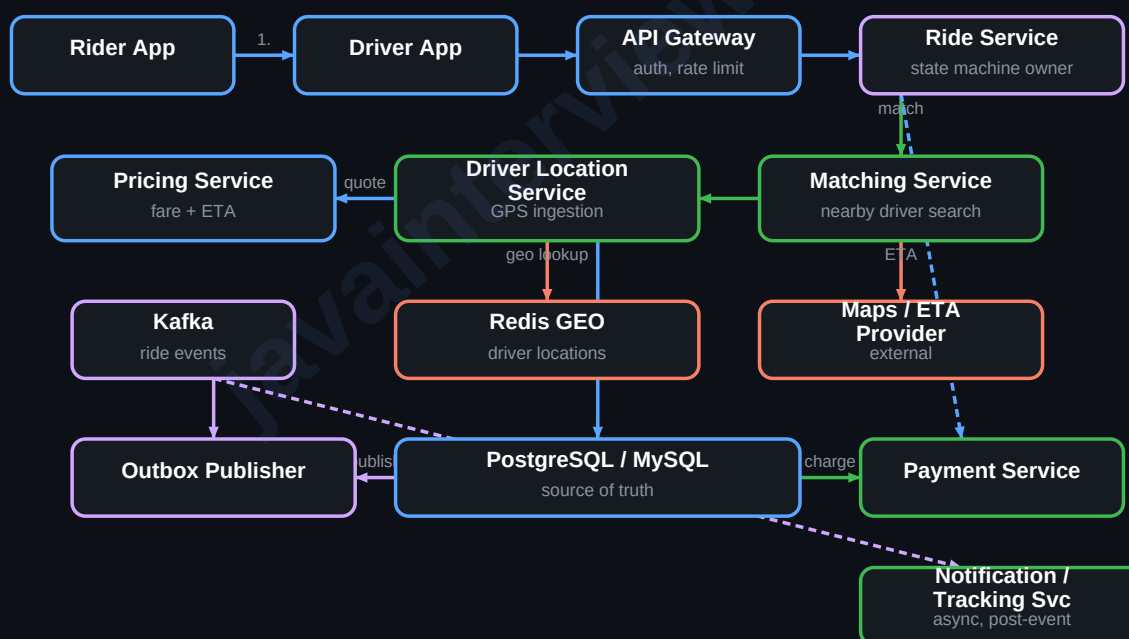
TIP

Driver GPS location does not get its own table here, and that is deliberate. Redis GEO holds the latest position per driver with a short TTL, Kafka can carry a location stream for analytics if you need history later. Writing every 3 to 5 second ping into PostgreSQL is the single most common mistake freshers make on this question, and it is exactly the 50,000 writes/sec problem from the capacity section.

05 HIGH-LEVEL DIAGRAM

Ride Service owns the state machine and talks to Matching, Pricing, and Payment inside the consistent path. Driver location lives entirely in Redis. Notification and tracking consume events off Kafka, well outside the request that the rider is waiting on.

FULL SYSTEM ARCHITECTURE



TIP

Notice Redis GEO sits between Matching Service and Driver Location Service, not behind the database. The dashed lines into Kafka are the async path, ride events flow out after commit and notification/tracking consume them independently. If you only remember one thing from this diagram, remember that GPS never reaches PostgreSQL.

06 KEY FLOW DIAGRAM

Request to assignment to completion, the flow interviewers want narrated end to end. Step 7 is where this question either shows you understand matching or shows you do not.

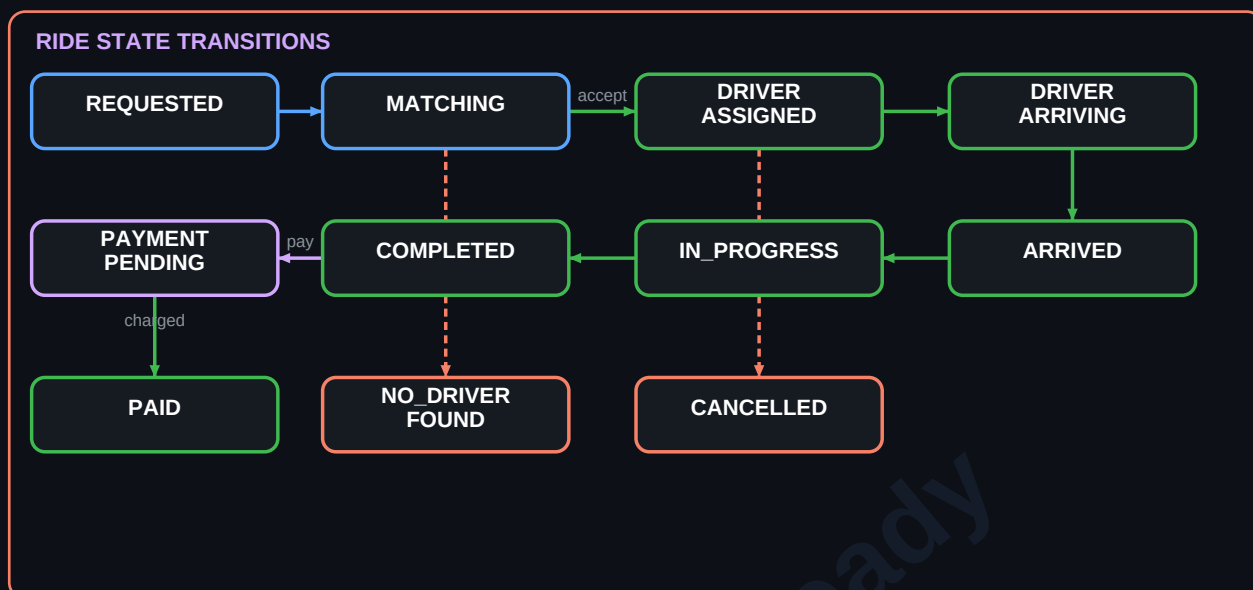
- 1 Rider requests a ride with pickup, drop, and an idempotencyKey
- 2 Ride Service checks the key against existing rides before doing anything else
- 3 Pricing Service calculates a fare estimate and ETA using the maps provider
- 4 Ride record is created with status REQUESTED inside one transaction
- 5 An OutboxEvent is saved in that same transaction, not published yet
- 6 Outbox publisher sends RideRequestedEvent to Kafka only after commit
- 7 Matching Service finds nearby available drivers from Redis GEO
- 8 Matching Service offers the ride to the best candidate with a short timeout
- 9 Driver accepts before the timeout, ride status moves to DRIVER_ASSIGNED
- 10 Driver location streams into Redis, rider tracks it live
- 11 Trip starts and completes, Payment Service charges the rider
- 12 Outbox publisher sends RideCompletedEvent to Kafka only after the ride completion transaction commits

WATCH OUT

Step 7 and 8 together are the heart of this design. If two ride requests both query Redis GEO and both see the same nearby driver as available, both could try to offer that driver the ride. The fix is a short-lived lock on the driver ID at offer time, plus a final database check before driver_assignments commits. Redis gives you speed, the database transaction gives you the actual guarantee.

07 RIDE STATE MACHINE

A ride is a state machine with a backend-owned set of legal transitions, the same idea you would apply to an order or a payment, just with a driver in the loop.



PAID, **CANCELLED**, and **NO_DRIVER_FOUND** are terminal, nothing moves out of them. A driver cannot mark a ride **COMPLETED** while it still sits in **DRIVER_ASSIGNED**, and a rider cannot directly set **DRIVER_ASSIGNED** on their own ride. Every arrow in this diagram is a backend method call guarded by a rule, never a raw status field a client can overwrite.

INTERVIEW FOCUS

A realistic follow-up here: "what happens if the driver never responds?" The ride stays in **MATCHING**, the assignment row expires, and Matching Service retries with the next nearby driver. Only after candidates run out does the ride move to **NO_DRIVER_FOUND**. Saying this out loud shows you thought about the unhappy path, not just the diagram.

08 COMPONENTS EXPLANATION

Component	Why it exists
Ride Service	Owns the state machine. Every status change is a method call here, never a client write.
Matching Service	Finds nearby drivers from Redis GEO and runs the offer-and-timeout loop.
Driver Location Service	Absorbs the constant stream of GPS pings, writes only to Redis, never to Postgres.
Pricing Service	Calculates fare and ETA using the maps provider, locks the quote briefly.
Payment Service	Talks to the external gateway after trip completion, never stores card data.
Redis GEO	Latest driver location and availability, built for fast radius queries.

Component	Why it exists
Kafka	Carries ride, assignment, and payment events to consumers that do not need to block checkout.
Outbox Publisher	Reads unpublished outbox rows and sends them to Kafka only after the DB commit.
Maps / ETA Provider	External dependency for distance and time estimates, wrapped behind a circuit breaker.
PostgreSQL / MySQL	Source of truth for rides, assignments, and payments. Never bypassed for these.

Redis earns five separate jobs here: latest driver location, driver availability, the nearby-driver radius lookup, a short-lived lock during the offer step, and a fast idempotency check before the database is touched. None of those jobs make Redis the source of truth. If Redis restarts and loses its data, a ride already marked PAID in Postgres is still PAID. That distinction is worth saying explicitly if asked.

09 SCALING APPROACH

Scaling matching and GPS

Ride Service and Matching Service are stateless, scale them horizontally without a second thought. The real scaling problem is GPS volume, and Redis GEO is built exactly for this, fast writes and fast radius queries. Put a TTL on every driver location key, a driver whose app crashed five minutes ago should fall out of nearby search instead of looking falsely available forever.

Scaling everything after assignment

Kafka decouples notification, tracking fan-out, and analytics from the ride request path, none of them block a rider waiting for a driver. For live tracking, prefer WebSocket or server-sent events over polling, polling at scale means thousands of clients hammering an endpoint every few seconds for data that barely changed. At very large scale, partition ride data by city, most queries are naturally scoped to one city anyway, and support dashboards should hit a read replica, not the primary handling live traffic.

TRADEOFF

If asked how you would handle surge demand during heavy rain, when ride requests spike but available drivers do not, the honest answer is that no amount of horizontal scaling fixes a driver shortage. What actually helps is surge pricing to balance demand, and Matching Service falling back to a wider search radius before declaring NO_DRIVER_FOUND.

10 FAILURE HANDLING

Nothing here gets solved by one perfect atomic operation across services. What actually keeps this system correct is idempotency, short expiries on uncertain state, and reconciliation jobs that clean up after the fact.

Failure	How you handle it
Duplicate ride request on retry	idempotencyKey checked first, backed by a DB unique constraint as the real guarantee.
No drivers available	Widen the search radius once, then move to NO_DRIVER_FOUND rather than waiting forever.
Driver accepts, response is lost	Driver app retries the same idempotent accept call, server returns the same result.
Same driver offered to two riders	Short Redis lock at offer time plus a DB check before assignment commits.
Driver rejects or times out	Assignment expires, Matching Service retries with the next nearby candidate.
Driver location updates stop	TTL expires the stale key, driver drops out of nearby search, alert on the gap.
Redis is down	Matching degrades or pauses briefly, circuit breaker avoids hammering a dead dependency.
Maps provider is slow	Timeout and fallback estimate, never block ride creation on a slow external call.
Payment succeeds, ride update fails	Webhook retried by the gateway, a reconcile job cross-checks gateway state against local rows.
Ride Service crashes after DB commit, before Kafka publish	Outbox row is already saved, the publisher picks it up once the service recovers. Nothing is lost.
Rider cancels after assignment	Driver is released back to available, a cancellation fee may apply per business rules.

WATCH OUT

Notice the row about the Ride Service crashing between commit and publish. That scenario is exactly why the outbox pattern exists in this design. The OutboxEvent row is already sitting in the same transaction as the ride, durable in Postgres the moment that transaction commits. The publisher does not need to be running at that exact instant, it just needs to eventually come back and read what is already saved.

11 JAVA + SPRING BOOT MAPPING

This is where the state machine and the matching logic turn into code you could open a pull request with. Here is how the layers split.

Layer	Spring Boot mapping
Controllers	RideController and DriverLocationController expose the endpoints from section 03.
Ride Service	@Transactional on ride creation and every valid status transition.
Matching Service	Queries Redis GEO for nearby drivers, runs the offer-and-timeout loop.
Pricing / Tracking	PricingService calls the maps provider, TrackingService reads from Redis for live location.
PaymentClient	WebClient or OpenFeign call to the gateway, wrapped in Resilience4j CircuitBreaker and Retry.
Repositories	RideRepository, DriverAssignmentRepository, RideEventRepository, OutboxRepository, all Spring Data JPA.
Redis	RedisTemplate or Redisson for GEO commands and short-lived assignment locks.
Kafka	KafkaTemplate is used only inside the outbox publisher. @KafkaListener drives matching, notification, and payment consumers.
Scheduled jobs	@Scheduled cleanup for expired driver_assignments rows that never got a response.
Security	Spring Security with JWT, separate token scopes for rider and driver apps.

RIDESERVICE.JAVA: IDEMPOTENT RIDE REQUEST WITH OUTBOX EVENT**CODE**

```
@Service
public class RideService {

    private final RideRepository rideRepository;
    private final RideEventRepository rideEventRepository;
    private final OutboxRepository outboxRepository;
    private final RedisTemplate<String, String> redisTemplate;

    @Transactional
    public RideResponseDto createRide(CreateRideRequest req, Long riderId) {
        String idemKey = req.getIdempotencyKey();

        Optional<Ride> existing = rideRepository.findByIdempotencyKey(idemKey);
        if (existing.isPresent()) {
            return RideResponseDto.from(existing.get());
        }

        redisTemplate.opsForValue()
            .setIfAbsent("idem:ride:" + idemKey, "1", Duration.ofMinutes(10));

        Ride ride = new Ride(riderId, idemKey, RideStatus.REQUESTED);
        ride.applyPickupAndDrop(req.getPickup(), req.getDrop());
        rideRepository.save(ride);

        rideEventRepository.save(RideEvent.created(ride.getId()));
        outboxRepository.save(OutboxEvent.rideRequested(ride.getId()));

        return RideResponseDto.from(ride);
    }
}
```

JAVA MAPPING

The database check by idempotencyKey runs first and is what the method actually trusts. Redis setIfAbsent right after is only a fast duplicate guard for the cheap, common case, it does not gate the response. The database unique constraint on idempotency_key is the final guarantee underneath both checks. The Ride row, the RideEvent row, and the OutboxEvent row all save inside the same transaction. Kafka is never called directly inside this method, a separate outbox publisher sends RideRequestedEvent only after that transaction commits.

12 INTERVIEW TRADE-OFFS

Pick three to five depending on time left. These are the decisions that actually separate a thoughtful answer from a memorized one.

Decision	Option A	Option B
Driver selection	Nearest only: simple, fast, sometimes worse match.	Scoring (distance, rating, acceptance): better outcomes, more compute.
Geo lookup	Redis GEO: built for this, one more moving part.	DB geo query: one less system, slower at scale.
GPS storage	Every ping in Postgres: simple, collapses under load.	Redis plus Kafka: scales easily, more infra to run.
Event publishing	Direct Kafka call: simpler, can publish for an uncommitted ride.	Transactional outbox: event matches what was saved, extra part.
Live tracking	Polling: simple, expensive at scale, stale data.	WebSocket/SSE: efficient, more connection state.

Common mistakes worth naming: storing every GPS ping in the rides table, letting two riders get assigned the same driver, trusting the client to set ride status, skipping the idempotency key, assuming a driver always accepts, and calling `kafkaTemplate.send` directly inside a `@Transactional` method. At my level, I would rather call out one of these clearly than pretend this system has no sharp edges.

13 HOW TO ANSWER THIS IN 10 MINUTES

HOW TO ANSWER THIS IN INTERVIEW IN 10 MINUTES

1. State the scope: rider, driver, matching, tracking, payment, matching plus GPS volume is the hard part.
2. Mention the state machine early. Status is backend-owned, never client input.
3. Explain why driver location lives in Redis GEO, not the main database.
4. Explain idempotency: the key, the DB unique constraint, why retries happen.
5. Draw the architecture: Ride and Matching Service sync, Kafka fanning out after assignment.
6. Explain the offer-and-timeout loop and the lock against double-assigning a driver.
7. Go deep on one failure rather than listing all of them shallowly.
8. Close with the consistency tradeoff: strong for assignment and payment, eventual elsewhere.

14 FINAL INTERVIEW CLOSING LINE

FINAL INTERVIEW CLOSING LINE

For a ride booking system, the core idea is that ride assignment and payment need strong consistency, while driver location and notifications stay eventually consistent, living almost entirely in Redis and Kafka. The database remains the source of truth for rides, but never has to see a single GPS ping to do its job.

javainterviewready

Instagram Feed

Designing a feed sounds simple until you think about a celebrity with 20 million followers posting once. Do you write that post into 20 million feed rows? Do you recompute the feed on every read? Neither is fully right. The interesting design decision here is fan-out strategy, and that is exactly what the interviewer is waiting for you to discuss.

01

Problem Statement

We are designing the backend feed system for an Instagram-like app. We are not designing all of Instagram. The scope is: a user creates a post with media, followers see those posts in their home feed, users can open a profile feed, images and videos are served through object storage and a CDN.

Core users: content creators who post, followers who browse, an admin/moderation team, and the media storage layer that handles uploads.

The hard parts are not storing one post. They are: building a fast home feed without heavy joins on every request, deciding how to distribute posts to followers, handling celebrity accounts efficiently, and serving images at scale through CDN instead of through Spring Boot controllers.

INTERVIEW FOCUS

Many freshers start by saying "I will join the follows table with the posts table for every feed request." That works for 100 users. At 10 million users it is a disaster. The whole design is about avoiding that join on the read path.

02

Requirements

Functional Requirements

Requirement	What it means
Create post	User uploads media, then creates post with caption and media references
Home feed	Follower sees recent posts from accounts they follow
Profile feed	All posts by a specific user, sorted by time
Follow / Unfollow	User follows or unfollows another user
Like and comment	User can interact with posts (not the main focus here)
Pagination	Feed loads next page using cursor, not page number
Media delivery	Images and videos served from CDN, not from backend API
Moderation	Admin can hide or soft-delete a post from feeds

Non-Functional Requirements

Requirement	What it means
Low latency feed reads	Home feed must load fast, under 200ms target
High availability	Feed reads must work even during partial failures
Eventual consistency	New post can appear in followers feeds within a few seconds
High read-to-write ratio	Feed reads far outnumber post creations: sign for reads
Scalable media delivery	CDN absorbs image/video bandwidth, not our servers
No heavy joins on feed	Feed store should allow direct row lookup by user_id

02.5

Capacity Estimation

Keep this directional. The point is to identify what kind of system we are building, not to get exact numbers.

Metric	Estimate	Impact on Design
Daily active users	10 million	Base reference for all estimates
Posts created per day	2 million (1M creators x 2)	Write volume is relatively low
Feed reads per day	100 million (10x per user)	Read volume dominates design decisions
Posts per feed page	20 posts per request	Cursor-based pagination needed
feed_items growth	Avg 500 followers per creator	2M posts x 500 = 1B rows/day at scale
Media storage	CDN + S3	Backend never streams media directly

TIP

The key insight: this is a read-heavy system. Post creation is relatively rare. Feed reads happen constantly. The entire design (a feed store, Redis caching, fanout workers, CDN) exists to make those reads fast. Say this out loud in the interview before you start drawing.

03

APIs

Method	Path	Request	Response
POST	/api/v1/posts	mediaIds[], caption, userId	201: postId, status, createdAt
GET	/api/v1/feed/home?cursor=	JWT (identifies user)	200: posts[], nextCursor
GET	/api/v1/users/{userId}/posts?cursor=	userId (path param)	200: posts[], nextCursor
POST	/api/v1/users/{userId}/follow	followerId (from JWT)	200: followed
DELETE	/api/v1/users/{userId}/follow	followerId (from JWT)	200: unfollowed
POST	/api/v1/posts/{postId}/likes	userId (from JWT)	200: likeCount
POST	/api/v1/posts/{postId}/comments	text, userId	201: commentId
DELETE	/api/v1/posts/{postId}	userId/admin (auth)	200: soft deleted

INTERVIEW FOCUS

Notice that POST /posts does not accept the raw image. Media is uploaded separately through the Media Service first, then the post is created with the resulting mediaIds. This separates media upload latency from post creation. A failed upload does not create a broken post row.

04

Database Design

Table	Key Fields
users	user_id, username, display_name, bio, avatar_url, follower_count, created_at
follows	follower_id, followee_id, created_at: Index on (follower_id), (followee_id)
posts	post_id, author_id, caption, status (ACTIVE/HIDDEN/DELETED), created_at, updated_at
post_media	media_id, post_id, media_url, thumbnail_url, media_type, width, height, order_index
feed_items	user_id, post_id, author_id, created_at, score: Partitioned by user_id
likes	user_id, post_id, created_at: Index(user_id, post_id)
comments	comment_id, post_id, author_id, text, created_at
outbox_events	id, event_type, payload, status, created_at: Indexed for reliable Kafka publish

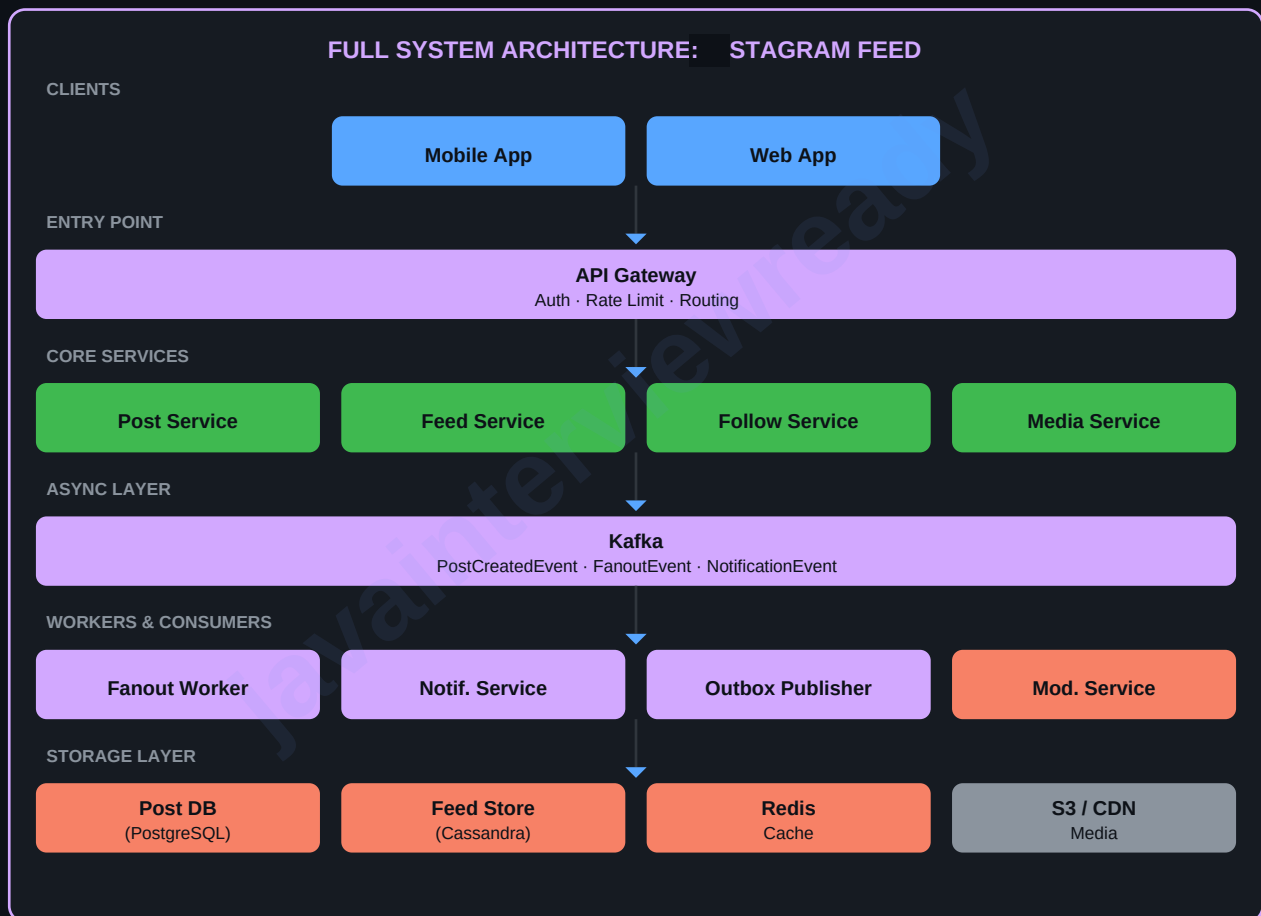
Database Choices

Store	Used For	Why
PostgreSQL / MySQL	users, follows, posts, likes, comments	Relational data with clear foreign keys. Works well at beginner/interview scale. Read replicas handle follow/post reads.
Cassandra / DynamoDB	feed_items	Append-only, partitioned by user_id. Huge write volume from fanout. This is the hot table: Wide-column fits perfectly.
Redis	Hot home feed cache, profile feed cache, like counts	First-page cache for active users. Fast reads, TTL handles staleness. Invalidate on post delete or hide.
S3 / Object Storage	Images, videos, thumbnails	Media never goes into the database. S3 stores it, CDN serves it.

Store	Used For	Why
Kafka + Outbox	PostCreatedEvent, fanout events	Post row and OutboxEvent saved in same transaction. Publisher sends to Kafka after commit. Fanout Worker reads and distributes.

05

High-Level Architecture Diagram



06

Key Flow: Post Creation to Feed Delivery

The most interview-relevant flow starts the moment a user creates a post and ends when a follower sees it in their feed.

POST CREATION TO FEED DELIVERY

1. User uploads media → Media Service stores in S3 → returns mediaId and CDN URL
2. User calls POST /api/v1/posts with caption + mediaIds
3. Post Service saves Post row (status=ACTIVE) + OutboxEvent in same @Transactional block
4. Outbox Publisher detects committed OutboxEvent, sends PostCreatedEvent to Kafka
5. Fanout Worker receives PostCreatedEvent, reads author's follower list
6. For normal users: Fanout Worker inserts one feed_item row per follower (fan-out on write)
7. For celebrity authors: skip full fanout: mark for pull-based read (fan-out on read)
8. Feed Service receives GET /feed/home: checks Redis cache first
9. Cache hit: return cached feed page. Cache miss: query feed_items from Cassandra
10. Feed response includes CDN URLs for media: client fetches images directly from CDN

Note: The outbox publisher only sends PostCreatedEvent after the database transaction commits. If the transaction rolls back, no Kafka event is sent.

07

Feed Generation Strategy

This is the section that separates a good interview answer from a great one. There are three approaches, and knowing when to use each one is what interviewers listen for.

FEED GENERATION STRATEGIES: COMPARISON

Fan-out on Write

- Push post into follower feeds on write
- Fast read: just look up by user_id
- Works well for normal-sized accounts
- Expensive for users with many followers
- Celebrity post = millions of writes

Fan-out on Read

- Store post once, build feed on request
- No fanout cost at write time
- Slow read: must query many users
- Good for celebrity accounts only
- Bad default for most users

Hybrid (Best Answer)

- Fan-out on write for normal users
- Fan-out on read for celebrities
- Cache final feed page in Redis
- Defines "celebrity" by follower count
- Most real systems use this approach

FEED STRATEGY

In an interview, say this clearly: "For normal users I fan out on write. For accounts above a threshold (by 500K followers) skip the fanout and merge celebrity posts at read time. The final feed is cached in Redis." That is the complete answer. You do not need to mention ML ranking here.

On ranking: start with reverse chronological. Mention that engagement score, relationship strength, and freshness can be added later. Do not over-engineer ranking in an interview unless specifically asked.

08

Components Explanation

Component	Responsibility
API Gateway	Auth via JWT, rate limiting, routing. Ensures only authenticated users access feed and create posts.
Post Service	Validates media ownership, creates Post + PostMedia rows, saves OutboxEvent in same transaction. Does not publish to Kafka directly.
Feed Service	Serves home feed and profile feed. Checks Redis cache first, falls back to Feed Store (Cassandra) on miss, merges celebrity posts on read.

Component	Responsibility
Follow Service	Manages follow/unfollow relationships. Updates follows table. Triggers feed rebuild hint on unfollow.
Media Service	Handles media upload to S3 and returns CDN-accessible URLs. Post Service consumes these URLs, never stores raw bytes.
Fanout Worker	@KafkaListener on PostCreatedEvent. Reads follower list from Follow DB. Inserts feed_item rows into Cassandra for normal users. Skips fanout for celebrities.
Redis Cache	Caches first page of home feed per user. Invalidated on post delete, hide, or unfollow. Also caches like counts.
Feed Store	Cassandra or DynamoDB table partitioned by user_id. Stores feed_items with post_id, author_id, score, created_at. Optimised for fast range reads.
Post DB	PostgreSQL. Stores posts, users, follows, likes, comments. Read replicas for follows/post lookups.
Outbox Publisher	@Scheduled job or CDC. Reads committed PENDING OutboxEvent rows, publishes them to Kafka, and marks them as SENT. Decouples Kafka publish from the DB transaction.
CDN	Serves all images and videos. Clients fetch media directly from CDN URLs returned in feed response. Backend never streams media.
Moderation Svc	Subscribes to moderation events or runs periodic checks. Soft-deletes posts, invalidates Redis cache, hides post from feed reads.
Notif. Service	Listens to PostCreatedEvent and like/comment events. Sends push notifications to followers. Async, does not block feed flow.

09

Scaling Approach

Concern	Approach
CDN for all media	Backend never streams images or videos. CDN absorbs all media bandwidth. Spring Boot controllers should never return image bytes.
Redis first-page cache	Cache the first 20 posts of home feed per active user. TTL around 5 minutes. On post delete or hide, invalidate the cache.
Cursor-based pagination	Use created_at or score as cursor instead of LIMIT/OFFSET. OFFSET scans skipped rows every time, usable at scale.

Concern	Approach
Fanout workers scale out	Fanout Worker is stateless. Scale horizontally. Partition Kafka by authorId so all events for one author go to same partition.
feed_items by user_id	Cassandra partitioning on user_id means all feed items for a user are co-located. Range query by created_at is cheap.
Profile feed from DB	Profile feed is just posts by one author. Index on author_id + created_at in Post DB. No fanout needed.
Celebrity threshold	Define a follower count threshold (e.g. 500K) above which fan-out on write is skipped. Merge at read time for these accounts.
Read replicas for Post DB	Follow lookups and post metadata reads hit replicas, not the primary. Reduces primary write-path load.
Precompute active users	For users who open the app frequently, precompute and warm Redis cache periodically via a background job.

10

Failure Handling

Failure Scenario	How to Handle
Post saved, Kafka event not sent	Transactional Outbox Pattern. OutboxEvent committed with post. Publisher retries until sent.
Fanout Worker crashes midway	Kafka consumer offset not committed. Worker restarts and resumes from last offset.
Duplicate PostCreatedEvent	Fanout uses unique key (user_id + post_id) in feed_items. Duplicate insert is idempotent.
Kafka consumer lag	Feed updates delayed but not lost. Alert on lag threshold. Scale consumer group.
Redis cache down	Feed Service falls back to Feed Store directly. Slower but correct.
Celebrity post overloads fanout	Celebrity threshold check in Fanout Worker. Skip fanout, serve via pull at read time.
Failure Scenario (continued)	How to Handle
User unfollows after feed_item exists	Feed items already in Cassandra remain. Filter at read time using current follow state, or clean up async.

Failure Scenario (continued)	How to Handle
Post deleted or hidden	Soft delete: set status=DELETED. Invalidate Redis cache. Feed Service filters DELETED posts during reads.
CDN unavailable	Client receives feed with media URLs. CDN outage means images fail to load, but feed structure still works.
Fanout Worker DB slow	Fanout inserts batch into Cassandra. Use batch writes and backoff. Monitor fanout lag separately.
Feed Store slow	Redis absorbs most traffic. Only cache misses hit Cassandra. Circuit breaker on Cassandra client.

11

Java + Spring Boot Mapping

Component	Spring Boot Mapping
PostController	POST /posts, DELETE /posts/{postId}. Validates request, calls PostService.
FeedController	GET /feed/home, GET /users/{userId}/posts. Reads from FeedService.
FollowController	POST/DELETE /users/{userId}/follow. Delegates to FollowService.
PostService	@Transactional. Saves Post + PostMedia + OutboxEvent in one transaction. Does NOT call KafkaTemplate directly.
FeedService	Checks RedisTemplate for cached feed. On miss, queries FeedItemRepository. Merges celebrity posts on read.
FollowService	Saves to FollowRepository. On unfollow, publishes UnfollowEvent to trigger async feed cleanup.
FanoutWorker	@KafkaListener on "post-created" topic. Reads follower list, inserts feed_items into Cassandra. Skips fanout for celebrity accounts.
PostRepository	Spring Data JPA. findByAuthorIdOrderByCreatedAtDesc() for profile feed.
FollowRepository	findAllByFolloweeId(authorId): ed by FanoutWorker to get follower list.
FeedItemRepository	Spring Data Cassandra. findByUserIdAndCreatedAtBefore(userId, cursor) for paginated home feed.
OutboxRepository	findByStatus(PENDING): lled by OutboxPublisher scheduler.

Component	Spring Boot Mapping
RedisTemplate	opsForValue() caches feed page JSON per userId. opsForValue() also caches like counts.
KafkaTemplate	Used only by OutboxPublisher (not PostService). Publishes PostCreatedEvent after DB commit.
@KafkaListener	FanoutWorker subscribes to "post-created". groupId="fanout-worker".
@Scheduled	OutboxPublisher polls OutboxEvent table every few seconds for PENDING events.
Spring Security/JWT	All write endpoints (post, follow, like) require auth. Feed reads require auth for personalisation.
@Transactional	PostService.createPost() wraps Post + PostMedia + OutboxEvent saves atomically.

Code Snippet: PostService.java: create Post with Outbox Event

```
@Service
public class PostService {

    @Autowired private PostRepository postRepo;
    @Autowired private PostMediaRepository mediaRepo;
    @Autowired private OutboxRepository outboxRepo;

    @Transactional
    public PostResponse createPost(CreatePostRequest req, Long authorId) {

        // Validate media ownership: media must belong to this user
        mediaService.validateOwnership(req.getMediaIds(), authorId);

        Post post = Post.builder()
            .authorId(authorId)
            .caption(req.getCaption())
            .status(PostStatus.ACTIVE)
            .build();
        postRepo.save(post);

        // Save media records linked to this post
        List mediaList = req.getMediaIds().stream()
            .map(id -> PostMedia.of(post.getId(), id))
            .toList();
        mediaRepo.saveAll(mediaList);

        // Save outbox event in same transaction: kafka publish happens after commit
        outboxRepo.save(
            OutboxEvent.postCreated(post.getId(), authorId)
        );

        return PostResponse.from(post);
    }
}
```

JAVA MAPPING

The Post, PostMedia, and OutboxEvent are all saved in one @Transactional method. If anything fails, the whole transaction rolls back and no Kafka event is sent. The OutboxPublisher is a separate @Scheduled job that reads PENDING outbox events and calls KafkaTemplate.send() only after they are committed to the database.

In real code, if the DB unique constraint on outbox prevents a duplicate, the publisher safely skips it and moves on.

Fanout Worker (simplified)

```
@KafkaListener(topics = "post-created", groupId = "fanout-worker")
public void onPostCreated(PostCreatedEvent event) {

    Author author = authorService.findById(event.getAuthorId());

    if (author.getFollowerCount() > CELEBRITY_THRESHOLD) {
        // Skip fan-out: feed built at read time for celebrity posts
        return;
    }

    // Normal user: fan out to all followers
    List followerIds = followRepo.findFollowerIds(event.getAuthorId());

    List items = followerIds.stream()
        .map(fid -> FeedItem.of(fid, event.getPostId(), event.getAuthorId()))
        .toList();

    feedItemRepo.saveAll(items); // Batch insert into Cassandra
}
```

12

Interview Tradeoffs

Decision	Option A	Option B
Fan-out on write vs read	Write-time fanout: fast reads, expensive for popular accounts.	Read-time fanout: cheap writes, slow reads. Hybrid is the real answer.
Redis cache vs Feed Store	Redis: ultra fast, but bounded memory. Only cache first page for active users.	Cassandra/DynamoDB: durable, scales well, but slower than Redis.
Reverse chronological vs ranking	Chronological: simple, predictable, easy to implement.	Ranked by engagement: better UX, needs ML/scoring pipeline. Start chronological.
S3 + CDN vs storing media in DB	Storing in DB: terrible, binary data in relational tables kills performance.	S3 + CDN: correct approach. Media stays out of the database entirely.
Cursor vs offset pagination	Offset: simple but scans all skipped rows on every request. Unusable at scale.	Cursor: efficient range query on created_at or score. Always use this for feeds.
Direct Kafka publish vs Outbox Pattern	Direct kafka.send() inside @Transactional: event can be sent even if DB rolls back.	Outbox: Post and OutboxEvent committed atomically. Kafka publish only after commit.

WATCH OUT

Common Fresher Mistakes:

- Building home feed by joining follows + posts on every request.
- Storing images directly in the database.
- Serving media through Spring Boot controllers instead of CDN.
- Fanning out to celebrity followers without any threshold check.
- Using LIMIT/OFFSET pagination on large feed tables.
- Not invalidating Redis cache when a post is deleted or hidden.
- Calling `kafka.send()` directly inside the `@Transactional` post creation method.
- Assuming feed must be perfectly real-time: eventual consistency is fine here.
- Ignoring duplicate fanout events: fanout must be idempotent.

javainterviewready

13

How to Answer This in 10 Minutes

HOW TO ANSWER THIS IN 10 MINUTES

1. Open with scope. Say clearly: "I am designing the feed backend, not all of Instagram. The scope is post creation, home feed, profile feed, and media delivery." 30 seconds.
2. State the key insight immediately. This is a read-heavy system. Feed reads are 50x the write volume. The whole design exists to serve reads fast. Say this before drawing anything. 30 seconds.
3. Draw the architecture. Client → API Gateway → Post/Feed/Follow Services → Kafka → Fanout Worker → Feed Store / Redis. Show S3 + CDN for media. 2 minutes.
4. Explain the fan-out strategy: This is the core of the question. "Fan-out on write for normal users, fan-out on read for celebrities, first page cached in Redis." One sentence per approach. 2 minutes.
5. Walk through the post creation flow. Post saved + OutboxEvent in same transaction. Publisher sends to Kafka after commit. Fanout Worker distributes to followers. Feed Service reads from cache or Feed Store. Media from CDN. 2 minutes.
6. Cover one failure. Post saved but Kafka event not sent: Transactional Outbox fixes it. This is more useful than listing five failures shallowly. 1 minute.
7. Close with one tradeoff. Fan-out on write vs read, and why cursor pagination is mandatory at scale. 1 minute.

FINAL INTERVIEW CLOSING LINE

For an Instagram-style feed, the core design decisions are: use the Transactional Outbox pattern so post events reach Kafka reliably, fan-out on write for normal users and fan-out on read for celebrities, cache the first feed page in Redis, store feed_items in Cassandra partitioned by user_id, and always serve media from CDN. The tradeoff is write-time cost versus read-time latency, and the hybrid strategy is what real systems use.

YouTube Video Upload

This question trips up many freshers because they picture it as "save a file, done." The real design challenge starts before the upload even lands. How does a 4GB video get to your system without going through Spring Boot? What happens if the upload fails at 80%? Who decides when the video is ready to play, and what does the user see while transcoding runs? Those are the questions an interviewer actually wants to hear you answer.

01

Problem Statement

We are designing the backend for video upload on a platform like YouTube. The scope: a creator uploads a raw video, the backend stores metadata, workers transcode the video into multiple resolutions, and a viewer later streams the processed video through a CDN. We are not designing recommendations, comments, subscriptions, or the home feed.

Core users: content creators who upload, viewers who watch, and background workers that process video asynchronously.

The hard parts: a large video file should never pass through Spring Boot API servers. Upload can fail midway and must be resumable. Transcoding is slow and must be fully async. A viewer should see upload and processing status rather than an infinite spinner. The same raw file may produce four or five output resolutions, and the video should not be marked READY until at least the required outputs exist.

INTERVIEW FOCUS

The single most common mistake in this question: the candidate designs the video upload to go through their Spring Boot API server. At 200 MB average file size with thousands of concurrent uploads, that will kill your API servers instantly. Say this out loud early: the client uploads directly to object storage.

02

Requirements

Functional Requirements

Requirement	What it means
Request upload URL	Backend validates metadata and returns a presigned URL or multipart session
Direct upload to storage	Client uploads raw video bytes directly to object storage, <code>not</code> through API
Confirm upload complete	Client calls backend to confirm upload finished; backend verifies object exists
Async processing	Workers transcode video and generate thumbnails without blocking upload response
Video status polling	Creator checks status: UPLOADING, PROCESSING, READY, FAILED
Playback	Viewer gets CDN URL or streaming manifest for a READY video
Delete video	Creator soft-deletes video; physical cleanup runs async

Non-Functional Requirements

Requirement	What it means
No backend bandwidth	Large file bytes never touch API servers: <code>object storage</code> handles it
Resumable uploads	If upload fails midway, client can resume from the last byte
Async transcoding	Processing can take minutes: <code>must not block</code> the upload request
Eventual consistency	Video readiness can propagate with a few seconds lag
Strong consistency	Video ownership, metadata, and status must always be accurate
Retry support	Failed processing jobs must retry with backoff, not silently drop
High availability	Video metadata reads must work even during partial failures

02.5

Capacity Estimation

Metric	Estimate	Implication
Daily active users	5 million	Mostly viewers, small fraction are creators

Metric	Estimate	Implication
Video uploads per day	100K	Write volume is low compared to views
Avg raw video size	200 MB	100K x 200 MB = 20 TB raw storage/day
Processed versions	2x to 4x raw size	Multiple resolutions add 40-80 TB/day total
Video views per day	500 million+	View traffic dwarfs upload traffic: 30X critical
Peak upload concurrency	~5,000 at once	Presigned URLs offload bandwidth to S3/GCS

TIP

The main insight here: upload traffic is heavy in bytes but low in request count. View traffic is massive in both. Neither should touch your Spring Boot servers directly. Object storage absorbs uploads. CDN absorbs streams. Your backend coordinates both but handles almost no bytes.

03

APIs

Method	Path	Request	Response
POST	/api/v1/videos/upload-request	title, description, fileSize, contentType	201: videoId, sessionId, presignedUrl or multipartUploadId
POST	/api/v1/videos/upload-complete	videoId, sessionId, eTag (for multipart)	200: video status updated to UPLOAD_COMPLETE
GET	/api/v1/videos/{videoId}	JWT (auth)	200: video metadata, status, thumbnailUrl
GET	/api/v1/videos/{videoId}/status	JWT (auth)	200: status enum: 3LOADING, PROCESSING, READY, FAILED
GET	/api/v1/videos/{videoId}/playback	JWT or public	200: CDN URL or HLS manifest URL for READY videos only
DELETE	/api/v1/videos/{videoId}	JWT (owner or admin)	200: soft-deleted, cleanup scheduled

INTERVIEW FOCUS

Two things to say unprompted on the API design. First: upload-request returns a presigned URL: the video bytes go directly from the client to S3 or GCS without touching your API servers. Second: the playback endpoint returns a CDN URL or HLS manifest, not the video file itself. Your backend never streams video bytes.

04

Database Design

Table	Key Fields
videos	video_id, owner_id, title, description, status (enum), visibility, duration_seconds, thumbnail_url, created_at, updated_at
video_files	file_id, video_id, resolution, format, storage_key, size_bytes, bitrate, status, created_at
upload_sessions	session_id, video_id, owner_id, raw_storage_key, expected_size, content_type, status, expires_at
processing_jobs	job_id, video_id, job_type (TRANSCODE/THUMBNAIL), status, retry_count, error_message, created_at, updated_at
video_events	event_id, video_id, old_status, new_status, reason, created_at
outbox_events	id, event_type, payload, status (PENDING/SENT), created_at

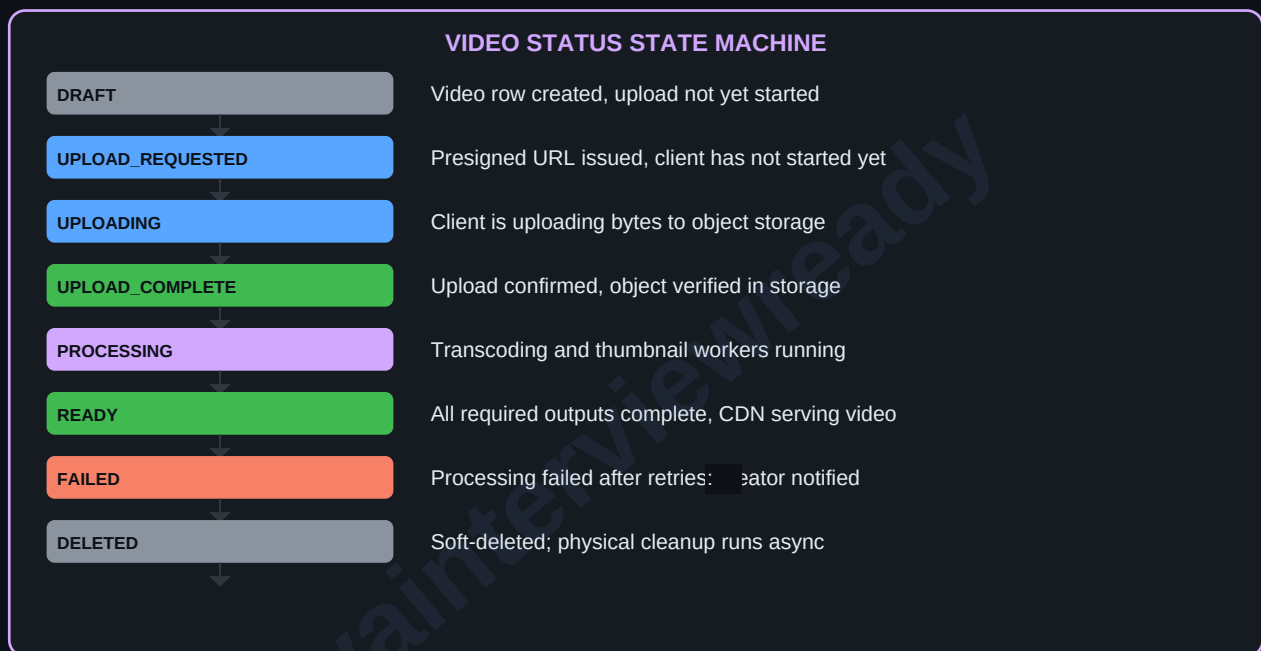
Storage Choices

Store	Used For	Why
PostgreSQL / MySQL	videos, video_files, upload_sessions, processing_jobs, video_events	Relational. Strong consistency for ownership and status. Supports transactions across tables.
Object Storage (S3/GCS)	Raw video files, transcoded versions, thumbnails	Video bytes never go in the DB. Storage keys in video_files reference objects.
Redis	Upload progress cache, processing status fast-read, rate limits	Optional. Avoid a DB query on every status poll. TTL keeps it fresh.
Kafka + Outbox	VideoUploadedEvent, TranscodeRequestedEvent, VideoReadyEvent	Video status update and OutboxEvent committed together. Publisher sends to Kafka after commit.

Store	Used For	Why
CDN	Processed video files, HLS segments, thumbnails	Viewers never hit your backend for video bytes. CDN serves everything from the storage edge.

04.5

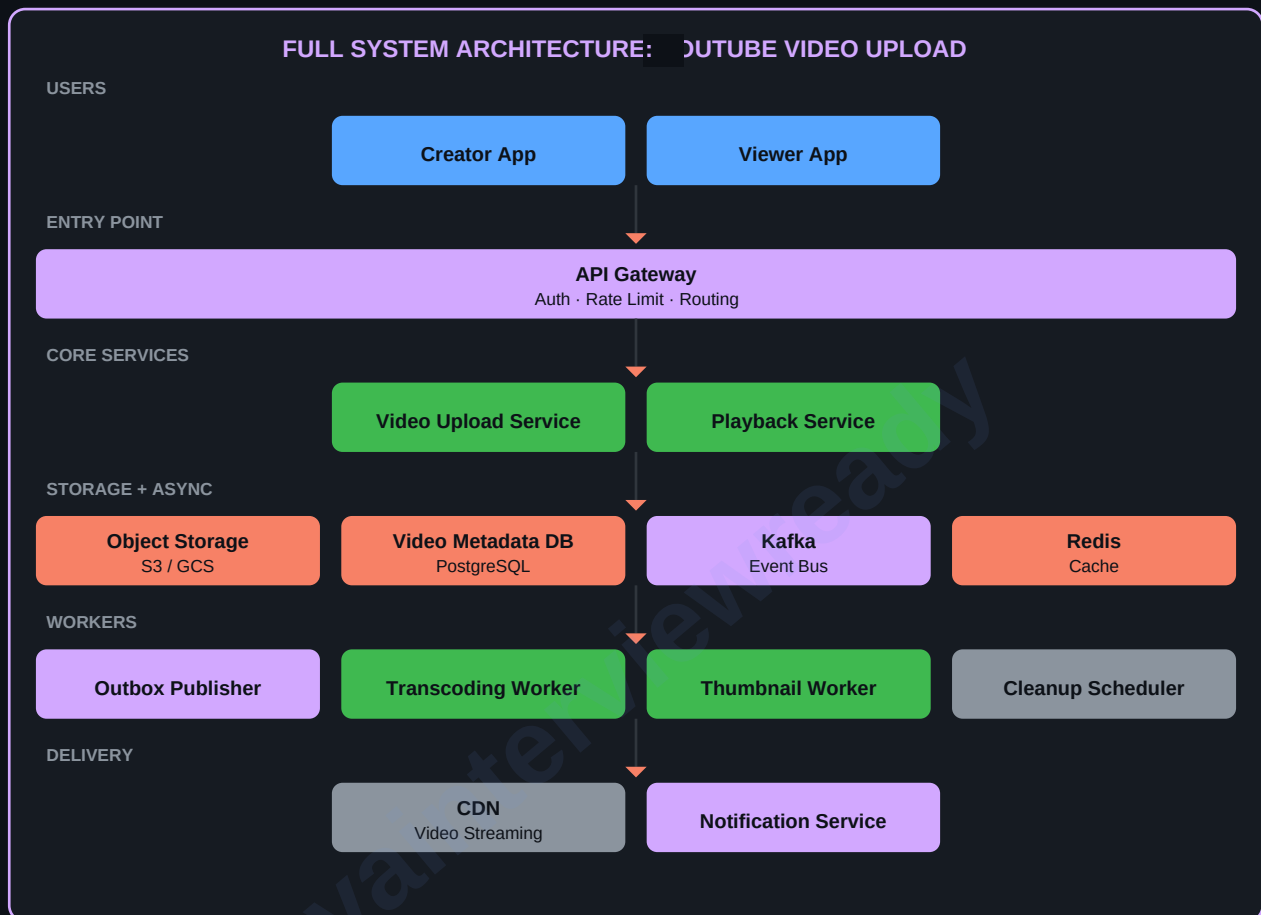
Video Status State Machine

**WARNING**

The client cannot directly set **READY**. Only the processing workers can do that, and only after all required output resolutions are saved to object storage. **FAILED** and **DELETED** are terminal: both require explicit action to recover from. Mention these rules unprompted in an interview.

05

High-Level Architecture Diagram



06

Key Flow: Upload Request to Video Ready

The most interview-relevant flow starts at the upload request and ends when the video is available to viewers.



Note: The VideoUploadedEvent is published by the Outbox Publisher only after the database transaction commits. If the transaction rolls back, no Kafka event is sent.

07

Video Processing Pipeline

Processing is where the real engineering lives. Once the `VideoUploadedEvent` lands in Kafka, a chain of workers kicks off. The key idea for beginners: upload should be fast, processing can be slow, and the user should see status instead of a frozen screen.

Step	What Happens
1. Raw upload validation	Check file format, codec, and expected size against the upload session. Reject corrupt or mismatched uploads early.
2. Malware / content scan	Optional virus scan or content policy check before processing starts. Can run in parallel.
3. Metadata extraction	Extract duration, codec, resolution, and frame rate from the raw file. Update videos table.
4. Transcoding	Create 240p, 360p, 720p, 1080p (or 4K) versions. Each resolution runs as a separate job. Write each to object storage.
5. Thumbnail generation	Extract a frame from the middle of the video or use creator-provided thumbnail. Save to storage.
6. Save video_files rows	Insert a row per completed output into video_files. Include storage_key, resolution, bitrate, size.
7. Mark READY	Video status moves to READY only after minimum required outputs (e.g. 360p + 720p) are saved. Not before.
8. Failure handling	Failed job: increment retry_count, retry with backoff. After max retries, move to dead letter topic and mark FAILED.

TIP

A common mistake is to mark the video READY as soon as one output is ready. In practice, you define a minimum required set: y 360p and 720p must both exist. Once those are done, mark READY. Continue generating 1080p and 4K in the background. Viewers get a watchable video sooner without waiting for every resolution.

08

Components Explanation

Component	Responsibility
API Gateway	Auth, rate limiting, routing. Rejects unauthenticated upload requests.

Component	Responsibility
Video Upload Service	Validates creator, creates Video + UploadSession rows, generates presigned URL. Handles upload-complete confirmation.
Playback Service	Returns CDN URL or HLS manifest for READY videos. Never streams bytes itself.
Object Storage	S3, GCS, or similar. Stores raw video and all processed outputs. Handles massive file bandwidth that API servers never should.
Video Metadata DB	PostgreSQL. Stores videos, video_files, upload_sessions, processing_jobs, and video_events. Transactional for status changes.
Kafka	Event backbone. Carries VideoUploadedEvent, TranscodeRequestedEvent, ThumbnailGeneratedEvent, VideoReadyEvent between services.
Outbox Publisher	@Scheduled job. Reads committed PENDING OutboxEvent rows, publishes to Kafka, marks as SENT. Kafka is never called inside @Transactional.
Transcoding Worker	@KafkaListener on VideoUploadedEvent. Creates processing_jobs, runs FFmpeg or cloud transcoder, saves output to object storage.
Thumbnail Worker	Extracts a frame or uses creator thumbnail. Saves to storage, updates thumbnail_url on video row.
Redis	Optional. Caches video metadata and processing status to reduce DB reads during status polling.
CDN	Serves all processed video files and thumbnails. Viewers stream from CDN edge nodes, not from backend servers.
Notification Service	Fires push or email notification when video moves to READY or FAILED.
Cleanup Scheduler	@Scheduled. Finds expired upload sessions, orphan objects in storage, and stuck PROCESSING jobs. Retries or marks FAILED.

09

Scaling Approach

Concern	Approach
Direct upload to object storage	Eliminates API server bandwidth. S3 multipart handles gigabyte files. Presigned URLs expire after a set time.
Multipart upload for large files	Upload session splits large files into parts. If any part fails, only that part retries. Critical for 1GB+ files.

Concern	Approach
Horizontal transcoding workers	Workers are stateless. Scale consumer group horizontally. Kafka lag monitoring triggers auto-scaling.
CDN for video delivery	Processed video is cached at CDN edge. Viewers worldwide get low latency without touching your servers.
Redis for status polling	Creator polls for status every few seconds. Redis cache avoids hitting PostgreSQL on every poll.
Read replicas for metadata	Video metadata reads (title, description, thumbnail) can hit read replicas. Writes stay on primary.
Separate storage prefixes	raw/ prefix for uploads, processed/ prefix for outputs. Lifecycle policy can auto-delete raw files after N days.
Lifecycle policies	S3 lifecycle rules automatically abort incomplete multipart uploads older than N days. Removes orphan parts.
Partition processing jobs	Partition by video_id for ordering. Each video's jobs stay on the same Kafka partition.

10

Failure Handling

Failure Scenario	How to Handle
Upload fails midway	Upload session tracks status. Client retries using multipart session ID. Only failed parts re-upload.
Upload-complete but object missing	Service checks object exists in storage before changing status. Returns 400 if missing. Client re-uploads.
Duplicate upload-complete call	If video is already UPLOAD_COMPLETE, PROCESSING, READY, or FAILED, return existing response. Idempotent by design.
Object storage temporarily down	Upload fails gracefully. Session stays UPLOADING. Client retries when storage recovers.
DB commit succeeds, Kafka publish fails	OutboxEvent is committed with video status. Publisher retries on next poll cycle. No event is lost.
Transcoding worker crashes	Kafka consumer offset not committed. Worker restarts and reprocesses the event. Use idempotent job creation.

Failure Scenario	How to Handle
Thumbnail generation fails	Mark thumbnail job FAILED. Use a default thumbnail. Retry separately. Video can still become READY.
Video stuck in PROCESSING	@Scheduled job scans videos in PROCESSING longer than N minutes. Retries or marks FAILED. Notifies creator.
Failure Scenario (continued)	How to Handle
User deletes video during processing	Soft delete first. Mark video DELETED. Workers detect DELETED status and stop further processing.
Raw upload exists but session expired	@Scheduled cleanup job finds orphan raw files and deletes them. Lifecycle policy is the safety net.
Processing creates 720p but fails at 1080p	Video still moves to READY if minimum outputs exist. 1080p job retried separately in background.
CDN serves stale or missing video	CDN cache key tied to storage path version. Use cache invalidation on video delete or re-process.

11

Java + Spring Boot Mapping

Component	Spring Boot Mapping
VideoController	GET /videos/{videoid}, GET /videos/{videoid}/status, DELETE /videos/{videoid}.
UploadController	POST /videos/upload-request, POST /videos/upload-complete.
PlaybackController	GET /videos/{videoid}/playback: Returns CDN URL or HLS manifest.
VideoUploadService	@Transactional. Validates ownership, verifies object in S3, updates Video status, saves VideoEvent + OutboxEvent.
VideoProcessingService	Receives VideoUploadedEvent via @KafkaListener. Creates ProcessingJob rows. Coordinates worker execution.
PlaybackService	Reads video_files for READY video, constructs CDN URL or HLS manifest URL.
VideoRepository	Spring Data JPA. findByIdAndOwnerId for ownership checks.
VideoFileRepository	findByIdAndStatus for checking completed transcoding outputs.
UploadSessionRepository	findBySessionIdAndOwnerId. findExpiredSessions for cleanup job.

Component	Spring Boot Mapping
ProcessingJobRepository	findByVideoIdAndStatus. findStuckJobs for scheduler.
OutboxRepository	findByStatus(PENDING): lled every few seconds by OutboxPublisher.
S3Service	Uses AWS SDK S3Client and S3Presigner to generate presigned URLs and verify object existence.
RedisTemplate	opsForValue() caches video status per videoId. TTL of 30 seconds for status polling.
KafkaTemplate	Used ONLY by OutboxPublisher: nds VideoUploadedEvent and VideoReadyEvent after DB commit.
@KafkaListener	TranscodingWorker and ThumbnailWorker: parate consumer groups.
@Scheduled	OutboxPublisher (polls every 5s), CleanupScheduler (scans expired sessions + stuck jobs).
@Transactional	VideoUploadService.completeUpload(): aps Video status update + VideoEvent + OutboxEvent save.
Spring Security/JWT	Creator endpoints require auth. Playback can be public or token-gated.

JAVA MAPPING

The Video status update, UploadSession status update, VideoEvent row, and OutboxEvent row are saved in the same transaction. The VideoEvent gives us an audit trail. The OutboxEvent tells the publisher to send VideoUploadedEvent to Kafka only after the database commit succeeds. VideoUploadService never calls KafkaTemplate directly.

Transcoding Worker (simplified)

```
@KafkaListener(topics = "video-uploaded", groupId = "transcoding-worker")
public void onVideoUploaded(VideoUploadedEvent event) {

    Video video = videoRepo.findById(event.getVideoId()).orElseThrow();
    if (video.getStatus() == VideoStatus.DELETED) return; // skip deleted

    // Transcoding runs OUTSIDE the DB transaction: t is slow, no need to hold a lock
    List targets = List.of(R240P, R360P, R720P, R1080P);
    for (Resolution res : targets) {
        String outputKey = transcode(event.getRawStorageKey(), res);
        videoFileRepo.save(VideoFile.of(video.getId(), res, outputKey));
    }

    if (videoFileRepo.countByVideoIdAndStatus(video.getId(), DONE) >= MIN_OUTPUTS) {
        markVideoReady(video.getId()); // status + VideoEvent + OutboxEvent, one transaction
    }
}

@Transactional
public void markVideoReady(Long videoId) {
    Video video = videoRepo.findById(videoId).orElseThrow();
    VideoStatus oldStatus = video.getStatus();

    video.setStatus(VideoStatus.READY);
    videoRepo.save(video);

    eventRepo.save(VideoEvent.statusChanged(videoId, oldStatus, READY));
    outboxRepo.save(OutboxEvent.videoReady(videoId));
}
```

JAVA MAPPING

Transcoding itself runs outside any database transaction since it can take minutes and should never hold a DB lock open. The final step is different: the READY status update, the VideoEvent audit row, and OutboxEvent.videoReady() are all saved together inside one short @Transactional method. KafkaTemplate is never called from the worker directly. The OutboxPublisher sends VideoReadyEvent to Kafka after that transaction commits.

12

Interview Tradeoffs

Decision	Option A	Option B
Upload through backend vs direct to storage	Backend upload: simple to implement, kills API servers at scale with large files.	Direct to object storage via presigned URL: correct approach. Backend only coordinates.
Single upload vs multipart	Single upload: simple for small files, fails completely on network interruption.	Multipart / resumable: required for large files. Failed parts retry independently.
Sync vs async processing	Sync: user waits until transcoding finishes. At 10 minutes per video this is unusable.	Async: upload request returns fast. User polls status. Standard approach.
All resolutions immediately vs progressive	All at once: higher latency to READY, more processing load for unpopular videos.	Progressive: mark READY after 360p + 720p. Continue 1080p in background. Better UX.
Backend streaming vs CDN	Streaming through Spring Boot: bandwidth kills your API tier. Never do this.	CDN: serves processed video at edge. No backend bytes. Scales to millions of views.
Direct Kafka publish vs Outbox Pattern	Direct <code>kafka.send()</code> inside <code>@Transactional</code> : event fired even if DB rolls back.	Outbox: <code>VideoStatus</code> and <code>OutboxEvent</code> committed atomically. Publisher fires after commit.

WATCH OUT

Common Fresher Mistakes:

- Uploading large video bytes through Spring Boot API servers.
- Storing video files as BLOBs in the database.
- Making the user wait while transcoding runs inside the upload request.
- Not checking that the object actually exists in storage before processing.
- Marking the video READY before all required outputs are transcoded.
- Calling `kafka.send()` directly inside the `@Transactional` method.
- Serving video streams through Spring Boot instead of CDN.
- Not handling duplicate upload-complete calls (must be idempotent).
- Ignoring orphan raw files when upload sessions expire.

13

How to Answer This in 10 Minutes

HOW TO ANSWER THIS IN 10 MINUTES

1. Open with scope. "We are building video upload and processing, not recommendations or comments. Focus: creator uploads, backend coordinates, workers process, CDN serves." 30 seconds.
2. State the key constraint immediately. "Video bytes should never go through Spring Boot API servers. The client uploads directly to object storage using a presigned URL." This one sentence shows you understand the core design. 30 seconds.
3. Walk through the upload flow. upload-request returns presigned URL. Client uploads to S3. Client calls upload-complete. Backend verifies object exists. Status moves to UPLOAD_COMPLETE. OutboxEvent saved in same transaction. Publisher sends to Kafka after commit. 2 minutes.
4. Explain the state machine. DRAFT → UPLOAD_REQUESTED → UPLOADING → UPLOAD_COMPLETE → PROCESSING → READY. Only workers can set READY. FAILED and DELETED are terminal. 1 minute.
5. Explain the processing pipeline. Workers receive VideoUploadedEvent from Kafka. Transcode to multiple resolutions. Mark READY after minimum outputs exist. Keep processing the rest in background. 2 minutes.
6. Cover one failure. Transcoding worker crashes: Kafka offset not committed, worker restarts and reprocesses. Scheduled job finds stuck PROCESSING videos. 1.5 minutes.
7. Close with one tradeoff. Sync vs async processing. Direct upload vs presigned URL. Outbox vs direct Kafka publish. Pick one and explain it clearly. 1.5 minutes.

FINAL INTERVIEW CLOSING LINE

For a YouTube-style video upload system, the core decisions are: use presigned URLs so video bytes go directly to object storage and never touch your API servers, use async transcoding workers so upload is fast and processing happens in the background, use the Transactional Outbox Pattern so Kafka events are published only after the DB commits, and use CDN so viewers stream from the edge, not from your backend. The tradeoff is complexity of coordination versus bandwidth and scalability at each step.

QUESTION 11 OF 15 · MODULE 14

BookMyShow Ticket Booking

Listing movies and showtimes is the easy fifteen percent of this question. The real test is what happens when six thousand people open the same Friday night show at the same second and all try to grab the same forty good seats before anyone else does.

01 PROBLEM STATEMENT

We are designing the booking flow behind a product like BookMyShow. Not the whole platform, just the part from searching a movie or event to picking a showtime, holding seats, paying, and getting a confirmed ticket.

Five parties touch this system: the customer booking tickets, theatre or event partners who own the show data, a support team that inspects bookings when something goes wrong, the payment gateway, and the notification layer that emails the ticket.

The hard part is never listing movies. It is what happens when a popular show goes live and thousands of people try to book the same limited set of seats at once. You have to hold seats temporarily while someone pays, release that hold if they abandon it, and make absolutely sure one seat never gets confirmed for two different people.

INTERVIEW FOCUS

Most freshers describe this as "a bookings table with a seat number column." That survives until the interviewer asks what happens when two people click the same seat within the same second. Bring up seat holds, locking, and expiry yourself, before the interviewer has to pull it out of you.

02 REQUIREMENTS

Split functional from non-functional before drawing anything. The non-functional column is where this question gets decided, because browsing and seat booking need completely different consistency guarantees.

Functional requirements

Requirement	What it means
Search and browse	Find movies or events by city, view theatres and showtimes.
View seat layout	See which seats are available, held, or booked for a show.
Hold seats	Selected seats are reserved temporarily while paying.

Requirement	What it means
Pay and confirm	Payment success turns a hold into a confirmed booking.
Generate ticket	A confirmed booking produces a ticket the customer can show.
Release expired holds	Seats nobody paid for in time go back to available.

Non-functional requirements

Requirement	What it means
Low latency browsing	Movie and showtime pages should feel instant.
High availability at release	A blockbuster's opening weekend cannot take the system down.
Strong consistency for seats	One seat can never be confirmed for two people.
Idempotent booking and payment	A retried request must never double-charge or double-book.
Eventual consistency elsewhere	Ticket emails and notifications can lag by a few seconds.
Auditable	Every booking state change is traceable later.

Notice the split. Browsing behaves like any other read-heavy catalog, cache it without worry. Seat booking behaves like a banking problem, the moment two requests touch the same seat you need a real guarantee, not a best effort. Many beginners apply one consistency model everywhere and either slow down browsing for no reason or leave seat booking loose enough to double-sell during a big release.

02.5 CAPACITY ESTIMATION

Keep this rough. The number that matters is not total volume, it is how many people can collide on the same small set of seats at once.

CAPACITY ESTIMATION WALKTHROUGH

Assume 5 million daily active users

Booking volume: 500,000 bookings/day ~ 6 bookings/sec average

Peak concentration: one popular show with ~200 seats can see thousands of users hit it within the same minute on release day

Read vs write: seat availability reads far outnumber confirmed bookings, most viewers never finish a purchase

Why this matters: total volume is small for PostgreSQL, the real danger is many requests converging on the same 200 rows at once, a concurrency problem, not throughput

This is the insight that should shape your whole answer. Five million daily users sounds big, but spread across thousands of shows it is not the bottleneck. The bottleneck is one popular show with 200 seats getting hammered by far more interest than supply, at the exact same moment. That is a locking and correctness problem, not a scaling problem.

CAPACITY TAKEAWAY

Total booking volume is not the real problem here, six bookings a second is nothing for PostgreSQL. The real problem is many users fighting for the same small set of show_seats rows at the same time. That is exactly why DB locking or constraints matter more than raw horizontal scaling, adding more application servers does not make 200 contested rows any less contested.

03 APIs

Eight endpoints. Notice there is no endpoint that lets the frontend mark a seat BOOKED directly, that decision only ever comes out of the payment workflow.

Method & Path	Notes
GET /api/v1/movies?city=	List movies or events showing in a city.
GET /api/v1/movies/{movieId}/shows	Theatres and showtimes for a movie, filtered by city and date.
GET /api/v1/shows/{showId}/seats	Seat layout with AVAILABLE, HELD, or BOOKED status per seat.
POST /api/v1/bookings/hold	Temporarily holds selected seats. Requires an idempotencyKey.
POST /api/v1/bookings/{bookingId}/payment	Starts payment for the held seats.
POST /api/v1/payments/webhook	Payment gateway confirms success or failure, backend-only.
GET /api/v1/bookings/{bookingId}	Check current booking status.
POST /api/v1/bookings/{bookingId}/cancel	Cancels a booking if cancellation rules allow it.

INTERVIEW MOMENT

Say this without being asked: the frontend cannot directly mark a seat as BOOKED. The client calls hold, then payment, and the backend decides the seat's real status based on what the payment gateway actually confirms. A booking flow that lets the client set seat status is the most common way this design goes wrong in an interview.

04 DATABASE DESIGN

Relational, no real debate here. Shows, seats, bookings, and payments need transactions and constraints, and the access pattern is straightforward joins, not document lookups.

Catalog tables

Table	Key columns	Notes
movies / theatres / screens	id, name, city, capacity	Slow-changing catalog data, cached heavily for reads.
shows	show_id, movie_id, screen_id, start_time, status	One row per showtime on one screen.

Table	Key columns	Notes
seats	seat_id, screen_id, row_name, seat_number, seat_type	Physical seat layout, fixed per screen.
show_seats	show_seat_id, show_id, seat_id, status, price	One row per seat per show. This is the row everyone fights over.

Booking tables (the ones that matter most)

Table	Key columns	Notes
bookings	booking_id, user_id, show_id, status, idempotency_key	idempotency_key has a unique constraint, your real duplicate guard.
booking_seats	booking_id, show_seat_id, price_snapshot, status	Price is frozen at hold time, never re-read from show_seats later.
seat_holds	hold_id, show_id, user_id, status, expires_at	Tracks who is holding what and when that hold expires.
booking_events	event_id, booking_id, old_status, new_status, reason	Append-only audit log, the first thing support opens.
outbox_events	event_id, aggregate_id, type, payload, published	Holds SeatsHeldEvent and BookingConfirmedEvent until Kafka has them.

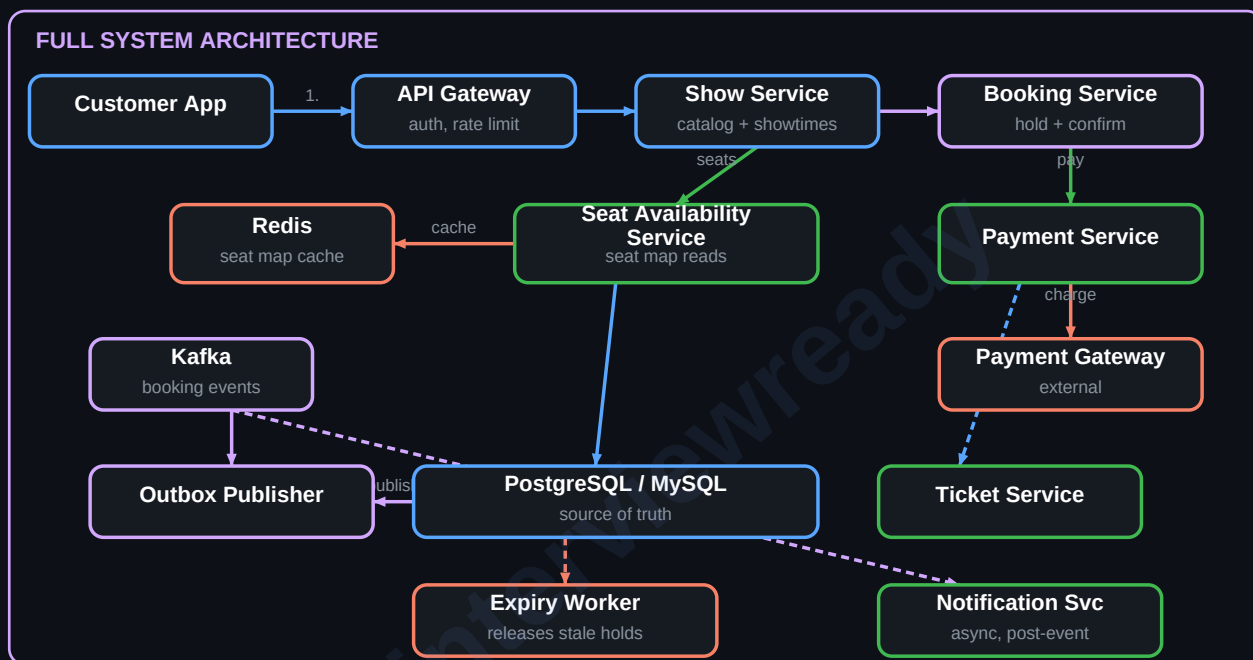
show_seats is the single most important table in this whole design. It is one row per seat per show, and every hold or booking attempt for that seat has to pass through a transaction touching that exact row. price_snapshot on booking_seats matters too, a theatre can change pricing between when you opened the seat map and when you pay, and your booking should reflect the price you actually agreed to, not whatever it is now.

TIP

Redis is genuinely useful here for two things: caching the seat map for fast reads, and a quick lookup for hold status without hitting Postgres every time. It is not useful as the only guarantee against double booking. The database transaction on show_seats is what actually decides the truth, Redis just makes reading that truth fast.

05 HIGH-LEVEL DIAGRAM

Booking Service owns the hold-and-confirm flow inside one transaction boundary. Redis sits beside it as a cache, never the source of truth. Ticket generation and notifications consume events off Kafka, well after the booking already committed.



TIP

The dashed lines into Kafka are the async path, ticket generation and notification fire after commit and never block the booking response. The Expiry Worker runs independently, scanning for holds that ran past their expiry and never converted to a payment.

06 KEY FLOW DIAGRAM

Seat selection to confirmed ticket, the flow interviewers want narrated end to end. Steps 4 and 5 are where this question is actually won or lost.

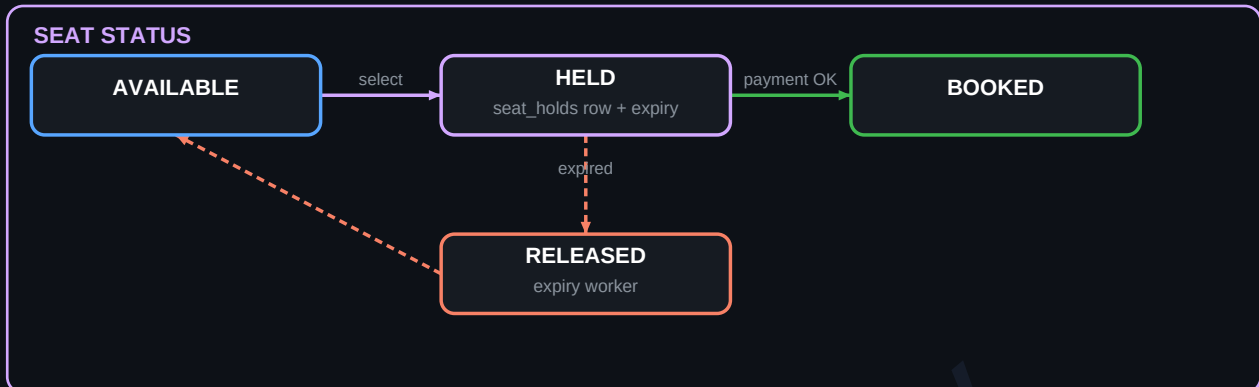
- 1 User opens the seat layout for a show, reads come from Redis or DB
- 2 User selects seats and calls hold with an idempotencyKey
- 3 Booking Service checks the key against existing bookings first
- 4 Booking Service loads the selected show_seats with a row lock
- 5 All seats are verified AVAILABLE before anything is created
- 6 Booking is created with status SEATS_HELD, seat_holds rows get an expiry
- 7 show_seats move to HELD inside that same transaction
- 8 User starts payment, payment gateway processes the charge
- 9 Payment gateway sends a verified webhook back to the backend
- 10 Booking Service confirms payment, HELD seats move to BOOKED
- 11 Booking moves to CONFIRMED, an OutboxEvent is saved in that transaction
- 12 Outbox publisher sends BookingConfirmedEvent to Kafka only after commit
- 13 Ticket Service generates the ticket, Notification Service sends confirmation

WATCH OUT

Step 4 is the whole interview signal in one line. Loading show_seats with a row lock, or relying on a unique constraint on an active hold for that show_seat_id, is what actually prevents two people from both holding the same seat. Checking availability with a plain SELECT and then writing afterward is a race condition waiting to double-book.

Seat status moves through a small state machine, and booking status moves through a wider one alongside it. Both are backend-owned, never client input.

07 SEAT LOCKING / BOOKING STATE MACHINE



Booking status is a wider chain: INITIATED to SEATS_HELD once seats are locked, then PAYMENT_PENDING while the gateway processes the charge, then CONFIRMED on success or PAYMENT_FAILED on failure. A held booking nobody paid for in time moves to EXPIRED. CONFIRMED and CANCELLED never accept a direct client write, only backend transitions.

WATCH OUT

Frontend seat color is a UI hint, not a guarantee, two browsers can both render a seat as green right before both users click it. Redis-only locking is risky as the only guarantee too, a lock living purely in memory can be lost or expire at the wrong moment with nothing in the database to fall back on. A held seat should expire in 5 to 10 minutes, long enough to pay, short enough that abandoned carts do not starve buyers.

The recommended approach: cache the seat map in Redis for fast reads, but move a seat from AVAILABLE to HELD only inside a database transaction with a row lock or a unique constraint on an active hold for that show_seat_id. Update Redis only after that transaction commits. idempotencyKey stops a retried hold from creating a second booking, and price_snapshot stops a mid-flow price change from altering what the customer agreed to pay.

08 COMPONENTS EXPLANATION

Component	Why it exists
Show / Seat Availability	Owns catalog reads and serves the seat map fast, backed by Redis with DB as fallback.
Booking Service	Owns the hold-and-confirm transaction. The only place seat status actually changes.
Payment Service	Talks to the external gateway, handles the webhook callback.
Ticket Service	Generates the ticket only after a booking reaches CONFIRMED.
Expiry Worker	Scheduled job that releases holds nobody paid for in time.
Redis	Seat map cache and hold status lookup. Never the final word on a seat.
Kafka & Outbox Publisher	Kafka carries booking and ticket events. The publisher reads unpublished outbox rows and sends them only after the DB commit.
PostgreSQL / MySQL	Source of truth for shows, seats, bookings, and payments.

If Redis goes down completely, browsing gets slower because every read falls back to Postgres, but no seat gets double-booked, because the database transaction was always the real guarantee underneath the cache. That is the test of whether you have actually designed this correctly versus just drawn a Redis box because the question mentioned scale.

09 SCALING APPROACH

Movie and showtime browsing is read-heavy, cache it in Redis and use read replicas for catalog queries so they never compete with live booking traffic on the primary. Seat map reads can be cached too, but invalidate the cache the moment a hold or booking commits, a stale seat map showing an already-booked seat as available creates a support ticket.

Booking Service is stateless and scales horizontally, but that scaling never removes the need for row-level locking on `show_seats`, more instances just means more concurrent attempts to grab the same lock. At very large scale, partition bookings by `show_id`, a blockbuster's release traffic should never slow down bookings for an unrelated small show running the same evening.

TRADEOFF

If asked how you would handle the exact moment tickets open for a huge release, the honest answer is that horizontal scaling does not fix contention over the same 200 seats. What helps is a virtual waiting room that admits a controlled number of users into the seat selection page at a time, so the database only ever sees a manageable number of concurrent hold attempts instead of every interested user hitting it at once.

10 FAILURE HANDLING

Nothing here is solved by one perfect atomic step across services. What keeps this system correct is locking at the database, idempotency, hold expiry, and reconciliation jobs that clean up after the fact.

Failure	How you handle it
Two users select the same seat at once	Row lock or unique constraint on the active hold for that show_seat_id, only one wins.
Payment succeeds, booking update fails	Webhook retried by the gateway, a reconcile job cross-checks gateway state against local rows.
Payment fails after seats are held	Booking moves to PAYMENT_FAILED, seats release back to AVAILABLE.
User closes the app after holding seats	Hold expiry handles it automatically, no special case needed.
Hold expires while payment is in progress	Reject the late confirmation and instruct the user to retry, do not silently confirm.
Duplicate hold request	idempotencyKey checked first, backed by a DB unique constraint as the real guarantee.
Duplicate payment webhook	Webhook handler is idempotent, a webhook for an already-CONFIRMED booking is a no-op.
Redis shows stale availability	DB transaction is still correct, cache is invalidated right after commit to catch up.
Booking Service crashes after commit, before Kafka publish	OutboxEvent row is already saved, the publisher picks it up once it recovers. Nothing is lost.
Ticket Service fails after confirmation	Booking stays CONFIRMED regardless, ticket generation retries independently off the event.
DB deadlock during seat hold	Retried automatically with backoff, consistent lock ordering reduces how often this happens.

WATCH OUT

The crash-between-commit-and-publish row is exactly why the outbox pattern exists in this design. The OutboxEvent is durable in Postgres the instant the booking transaction commits. The publisher does not need to be running at that exact moment, it only needs to eventually come back and read what is already saved.

11 JAVA + SPRING BOOT MAPPING

This is where the locking strategy and the state machine turn into code you could open a pull request with.

Layer	Spring Boot mapping
Controllers	MovieController, ShowController, SeatController, BookingController. PaymentWebhookController verifies the payment webhook and triggers backend confirmation, the frontend never marks seats BOOKED.
Booking Service	@Transactional on seat hold and booking confirmation, the only place status changes.
SeatHoldService / PaymentService	Loads show_seats with a lock and marks them HELD, then verifies the webhook and moves HELD seats to BOOKED.
TicketService / NotificationService	@KafkaListener consumers triggered off BookingConfirmedEvent.
BookingSeatRepository	Saves the selected seats for a booking with price_snapshot frozen at hold time.
BookingEventRepository	Stores the audit trail for SEATS_HELD, PAYMENT_PENDING, CONFIRMED, EXPIRED, and CANCELLED.
Other repositories	ShowSeatRepository, BookingRepository, SeatHoldRepository, OutboxRepository, all Spring Data JPA.
Locking	@Lock(PESSIMISTIC_WRITE) or a SELECT FOR UPDATE style query on show_seats during hold.
Redis & Kafka	RedisTemplate for seat map cache and hold-status lookup. KafkaTemplate used only inside the outbox publisher.
Scheduled jobs & Security	@Scheduled expiry worker releases stale seat_holds. Spring Security with JWT guards booking and payment endpoints.

BOOKINGSERVICE.JAVA: HOLD SEATS WITH DB LOCK + PRICE SNAPSHOT + OUTBOX EVENT**CODE**

```
@Service
public class BookingService {

    private final BookingRepository bookingRepository;
    private final BookingSeatRepository bookingSeatRepository;
    private final ShowSeatRepository showSeatRepository;
    private final SeatHoldRepository seatHoldRepository;
    private final BookingEventRepository bookingEventRepository;
    private final OutboxRepository outboxRepository;

    @Transactional
    public BookingResponseDto holdSeats(HoldSeatsRequest req, Long userId) {
        Optional<Booking> existing =
            bookingRepository.findByIdempotencyKey(req.getIdempotencyKey());
        if (existing.isPresent()) {
            return BookingResponseDto.from(existing.get());
        }

        List<ShowSeat> seats = showSeatRepository
            .findByIdsForUpdate(req.getShowSeatIds());
        if (seats.size() != req.getShowSeatIds().size()) {
            throw new SeatNotFoundException(req.getShowSeatIds());
        }
        for (ShowSeat seat : seats) {
            if (seat.getStatus() != SeatStatus.AVAILABLE) {
                throw new SeatNotAvailableException(seat.getId());
            }
        }

        Booking booking = new Booking(userId, req.getShowId(),
            req.getIdempotencyKey(), BookingStatus.SEATS_HELD);
        bookingRepository.save(booking);

        Instant expiresAt = Instant.now().plus(Duration.ofMinutes(8));
        for (ShowSeat seat : seats) {
            bookingSeatRepository.save(new BookingSeat(
                booking.getId(), seat.getId(), seat.getPrice()));
            seatHoldRepository.save(new SeatHold(booking.getId(), seat.getId(), expiresAt));
            seat.setStatus(SeatStatus.HELD);
        }

        bookingEventRepository.save(BookingEvent.seatsHeld(booking.getId()));
        outboxRepository.save(OutboxEvent.seatsHeld(booking.getId()));
        return BookingResponseDto.from(booking);
    }
}
```

JAVA MAPPING

The row lock on show_seats is the real protection against two users holding the same seat. The booking_seats rows store price_snapshot so the customer pays the price they agreed to during hold, even if the show price changes later. The Booking row, BookingSeat rows, SeatHold rows, show_seats status updates, BookingEvent, and OutboxEvent are saved in the same transaction. Kafka is not called directly. The outbox publisher sends SeatsHeldEvent only after commit.

BOOKINGSERVICE.JAVA: CONFIRM BOOKING AFTER VERIFIED PAYMENT**CODE**

```
@Transactional
public void confirmBooking(String bookingId, PaymentVerification payment) {
    if (!payment.isVerified()) {
        throw new PaymentNotVerifiedException(bookingId);
    }

    Booking booking = bookingRepository.findById(bookingId).orElseThrow();
    if (booking.getStatus() == BookingStatus.CONFIRMED) {
        return;
    }
    if (booking.getStatus() != BookingStatus.SEATS_HELD
        && booking.getStatus() != BookingStatus.PAYMENT_PENDING) {
        throw new InvalidBookingStateException(bookingId);
    }

    SeatHold hold = seatHoldRepository.findActiveByBookingId(bookingId)
        .orElseThrow(() -> new HoldNotFoundException(bookingId));
    if (hold.getExpiresAt().isBefore(Instant.now())) {
        // Late payment confirmation is rejected safely. The booking is
        // marked EXPIRED and the caller can show a retry message.
        booking.setStatus(BookingStatus.EXPIRED);
        bookingEventRepository.save(BookingEvent.expired(bookingId));
        outboxRepository.save(OutboxEvent.bookingExpired(bookingId));
        return; // do not throw after saving EXPIRED state
    }

    List<ShowSeat> seats = showSeatRepository.findByBookingIdForUpdate(bookingId);
    for (ShowSeat seat : seats) {
        if (seat.getStatus() != SeatStatus.HELD) {
            throw new InvalidSeatStateException(seat.getId());
        }
        seat.setStatus(SeatStatus.BOOKED);
    }
    bookingSeatRepository.markBooked(bookingId);

    booking.setStatus(BookingStatus.CONFIRMED);
    bookingEventRepository.save(BookingEvent.confirmed(bookingId));
    outboxRepository.save(OutboxEvent.bookingConfirmed(bookingId));
}
```

JAVA MAPPING

The hold expiry check prevents a late payment webhook from confirming seats after the user's hold window has already expired. We do not throw after marking the booking EXPIRED, because that could roll back the transaction. Instead, the expired state and BookingEvent are committed, and the caller can show a retry message.

12 INTERVIEW TRADE-OFFS

Pick three to five depending on time left. These are the decisions that actually separate a thoughtful answer from a memorized one.

Decision	Option A	Option B
Seat locking	Redis-only lock: fast, risky as the sole guarantee.	DB lock/constraint: the real guarantee, a bit slower per hold.
Locking style	Pessimistic (row lock): simple to reason about, holds the lock until commit.	Optimistic (version check): no lock held, must retry on conflict.
Seat map reads	Cache in Redis: fast, needs invalidation after every hold/booking.	Always read DB: always correct, does not scale to browsing traffic.
Ticket generation	Sync in the request: simple, slows down the booking response.	Async via Kafka: fast response, ticket appears a moment later.
Hold expiry duration	Short (5 min): frees seats fast, can rush slower payers.	Long (15+ min): friendlier, holds up inventory longer during a rush.

Common mistakes worth naming: trusting the frontend's seat color, using only a Redis lock with no database guarantee, never expiring held seats, letting the client mark a seat BOOKED directly, ignoring duplicate payment webhooks, generating a ticket before a booking is actually CONFIRMED, skipping price_snapshot, and calling kafkaTemplate.send directly inside a @Transactional method. At my level, I would rather name one of these clearly than pretend this system has no sharp edges.

13 HOW TO ANSWER THIS IN 10 MINUTES

HOW TO ANSWER THIS IN INTERVIEW IN 10 MINUTES

1. State the scope: browse, seat selection, hold, payment, confirmation, ticket, and that seat correctness is the hard part, not the catalog.
2. Mention the seat and booking state machines early. Status is backend-owned.
3. Explain why seat holds exist and why expiry is necessary.
4. Explain the locking strategy: DB transaction and row lock on show_seats as the real guarantee, Redis only as a cache.
5. Explain idempotency: the key, the DB unique constraint, why retries happen.
6. Draw the architecture: Booking Service in the sync path, Kafka fanning out to ticket and notification after confirmation.
7. Go deep on one failure rather than listing all of them shallowly.
8. Close with the consistency tradeoff: strong for seats and payment, eventual elsewhere.

14 FINAL INTERVIEW CLOSING LINE

FINAL INTERVIEW CLOSING LINE

For a ticket booking system, the core idea is that seat selection and payment need strong consistency enforced by the database, while ticket generation and notifications can stay eventually consistent and live in Kafka. Redis makes browsing and seat map reads fast, but the database transaction is what decides whether a seat is really yours.

Rate Limiter

We are not building a full API Gateway product here. We are building the rate limiting layer that sits inside one, the piece that decides in under a millisecond whether a request gets to continue or gets a 429 and stops right there. Every backend system you will ever ship eventually needs this, usually after the first time someone's retry loop takes a service down.

javainterviewready

TOPIC 01

Problem Statement

Returning HTTP 429 is one line of code. That is not what this question is testing. The real difficulty shows up the moment you have more than one gateway instance running, which in production is always.

If each gateway instance counts requests in its own memory, a client can get three times the limit you intended just by getting load balanced across three instances. So the count has to live somewhere shared, the check on that shared count has to be atomic under concurrent traffic, and the whole thing has to run fast enough that it never becomes the slowest part of the request path it is supposed to be protecting.

On top of that, you need different rules for different callers. A free tier user, a premium tier user, an internal microservice, and an admin tool should not share one limit. A login endpoint should be far stricter than a product listing endpoint. And you need an answer ready for the question every interviewer eventually asks: what happens when Redis itself is down?

INTERVIEW FOCUS

A common mistake is jumping straight to 'I'll use a counter in Redis' without addressing identity, atomicity, or the Redis-down case. Say all three out loud early. It signals you understand this is a distributed systems problem wearing a simple disguise.

TOPIC 02

Requirements

FUNCTIONAL

Limit by user, IP, API key, or tenant
Support endpoint-specific rules
Allow short bursts without allowing abuse
Return 429 with useful headers when exceeded
Admins can configure rules without redeploying
Support different plans: free, premium, internal, admin
Temporarily block clearly abusive clients

NON-FUNCTIONAL

Very low latency added per request, this sits on the hot path
Correctness across many gateway instances, not just one
Counter or token updates must be atomic under concurrency
Scales to high request volume without becoming the bottleneck
Graceful, predictable behaviour when Redis is unavailable
Configurable per endpoint and per plan without a code change
Rejected requests stop at the gateway, never reach backend services

Notice the shape of the non-functional column. Every line is really about one thing: this component cannot become the reason requests get slow, even though its entire job is to stand in the way of every single request.

TOPIC 02.5

Capacity Estimation

Worth doing here because it justifies the single biggest design decision in this question: why this check has to live at the gateway and has to be fast.

```
Assume 10 million daily active users, 100 million API requests/day
100,000,000 / 86,400 sec ~ 1,200 requests/sec average
Traffic spike (10x) ~ 12,000 requests/sec at peak

Every single request needs one rate limit check, no exceptions
Redis operations need to stay sub-millisecond to a few ms at this volume

Rejected requests must stop at the gateway
A 429 should never reach a backend service or its database
```

At 12,000 checks a second, even a few extra milliseconds per check adds up to real latency across the whole platform. That single number is why this lives at the API Gateway, in front of every backend service, and why the algorithm choice matters more here than in almost any other question in this book.

TOPIC 03

APIs / Rate Limit Response Contract

This is not a public API in the usual sense, it is a gateway-level contract. The interesting part is what headers come back, not what endpoints exist.

ALLOWED

```
GET /api/v1/orders
Authorization: Bearer <token>

Response headers
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 73
X-RateLimit-Reset: 1712345678
```

DENIED

```
HTTP 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0

{ "error": "RATE_LIMIT_EXCEEDED",
  "message": "Too many requests. Please retry later.",
  "retryAfterSeconds": 30 }
```

On the admin side, there is one real endpoint: **POST /api/v1/rate-limit-rules**, taking scope, endpointPattern, limit, windowSeconds, burstCapacity, and plan. This is how support raises a limit for one customer without anyone redeploying the gateway.

INTERVIEW MOMENT

Always mention Retry-After explicitly. It tells a well-behaved client exactly when to try again instead of guessing, and a missing Retry-After header is one of the most common things freshers forget to include in their answer.

TOPIC 04

Data Model / Redis Key Design

Redis earns its place here for two reasons: it is fast, and it gives you atomic operations that a relational database cannot match at this volume.

```
Fixed window / sliding window counter
rate:{scope}:{id}:{endpoint}:{window}
rate:user:123:/api/v1/orders:2026-07-01T10:30
rate:ip:49.204.x.x:/api/v1/login:2026-07-01T10:30

Token bucket
bucket:{scope}:{id}:{endpoint}
bucket:apikey:abc123:/api/v1/search
value: tokens_remaining, last_refill_timestamp

Sliding window counter (two keys, weighted together)
rate:{id}:{endpoint}:{currentWindow}
rate:{id}:{endpoint}:{previousWindow}
```

Configuration lives somewhere slower and more durable. **rate_limit_rules** holds rule_id, scope, endpoint_pattern, plan, limit, window_seconds, burst_capacity, enabled, created_at, and updated_at. **rate_limit_events** optionally logs event_id, identifier, endpoint, decision, and reason, for abuse investigation later.

TIP

Redis stores fast-changing counters and tokens. The database stores configuration rules, which change rarely. Mixing these up, putting per-request counters in a relational table, is a fast way to turn your rate limiter into the slowest part of the request. Always set a TTL on the Redis keys too, or they grow forever and never get cleaned up.

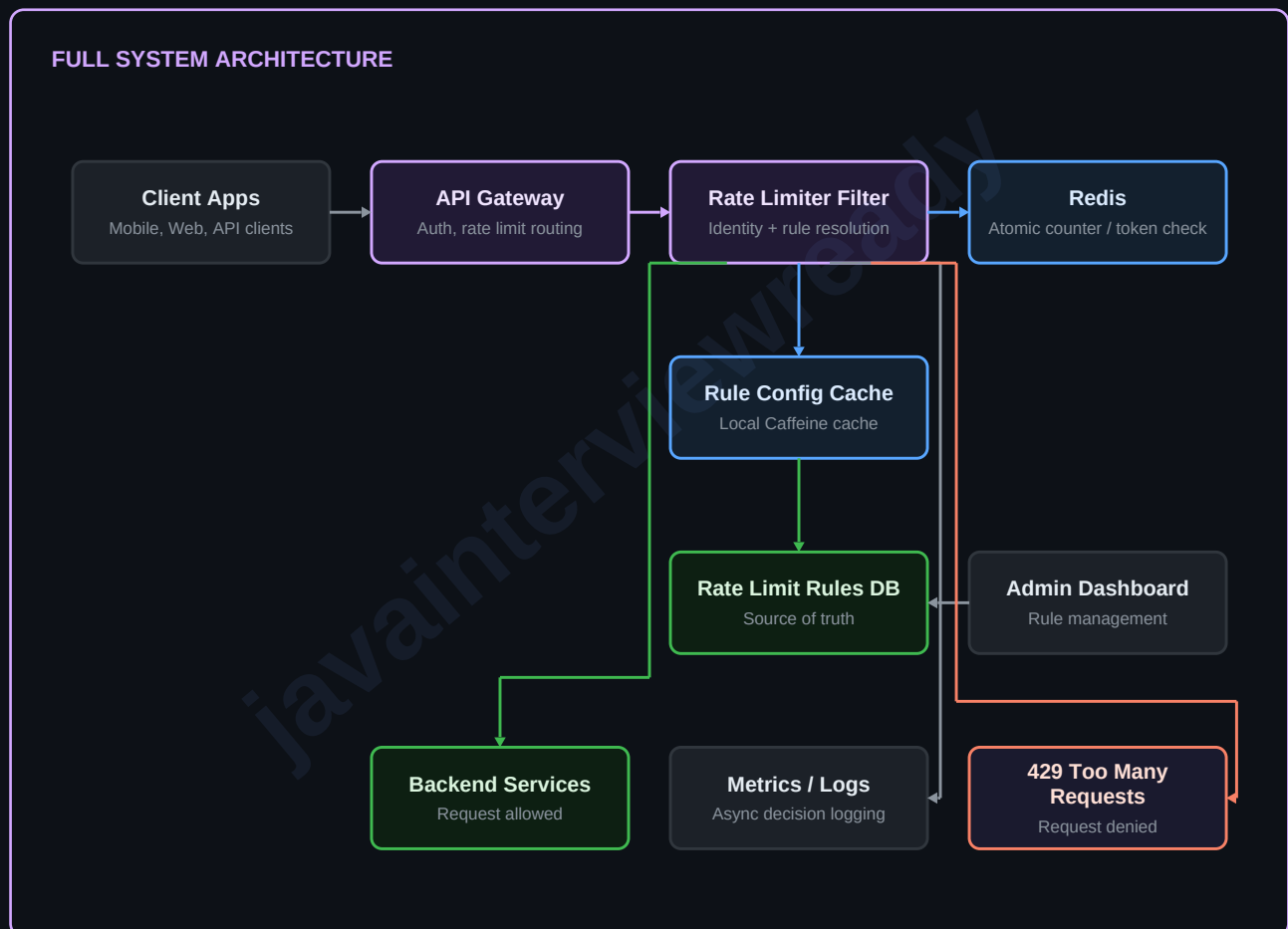
KEY DESIGN TAKEAWAY

Redis stores fast-changing counters or token bucket state. The database stores slow-changing configuration rules. The gateway keeps rules in a local cache so every request does not hit the database. Redis keys must use TTL so old window counters disappear automatically and stale keys never accumulate.

TOPIC 05

High-Level Diagram

Two flows live in this one diagram. The hot path runs on every single request: Client, Gateway, Filter, Redis, then a fork into either Backend Services or a 429. The cold path runs rarely: Admin Dashboard writes a rule to the database, and that rule eventually refreshes into the local cache the filter reads from, without ever touching the hot path directly.



TOPIC 06

Key Flow Diagram

Walk through this exact sequence in an interview. It is short, but every step answers a question the interviewer was about to ask anyway.



The backend service is only ever called on the allowed branch. On the denied branch, the gateway responds immediately, the request never gets anywhere near a database or a downstream call. That single property is the entire point of putting this check at the gateway instead of inside each service.

TOPIC 07

Rate Limiting Algorithms

Five algorithms come up in interviews. You do not need to memorize implementations, you need to know the tradeoff each one makes.

Algorithm	How it works	Tradeoff
Fixed window counter	Count requests in a fixed time slot, reset at the boundary.	Simple, but a client can send double the limit by timing requests around the window edge.
Sliding window log	Store the timestamp of every request, count how many fall in the last window.	Very accurate, but memory cost grows with traffic, not practical at high volume.
Sliding window counter	Blend the current and previous window counts using a weighted average.	Smooths out the boundary problem at a fraction of the memory cost of a full log.
Token bucket	Tokens refill at a steady rate, a request costs one token, bucket has a max capacity.	Allows short bursts up to the bucket size while still enforcing an average rate. The strongest default interview answer.
Leaky bucket	Requests queue and drain at a constant rate, like water through a fixed hole.	Produces smooth, steady output, useful when the system behind it cannot handle bursty traffic at all.

INTERVIEW FOCUS

For most APIs, token bucket backed by Redis is the strongest default answer. It lets a user who has been idle send a quick burst of requests, which matches how real clients behave, while still capping the average rate over time. Reach for sliding window counter when the interviewer specifically wants smoother enforcement with less burst tolerance.

TOPIC 08

Components Explanation

Component	Responsibility
Rate Limiter Filter	Runs on every request at the gateway. Resolves identity, looks up the matching rule, builds the Redis key, and makes the allow or deny decision.
Identity Resolver	Pulls userId from the JWT, falls back to API key or IP depending on what the caller actually presents.
Rule Config Cache	A local in-memory cache of active rules, read on every request so the hot path never queries the database directly.
Redis	Holds the fast-changing counters or tokens. The only component that needs to support atomic, concurrent updates at this volume.
Rate Limit Rules DB	Durable storage for rules. Changes rarely, read only to refresh the local cache, never on the hot path.
Admin Dashboard	Where support and platform teams adjust limits per customer or per endpoint without a deployment.
Metrics / Logs	Records allow and deny decisions asynchronously, used to spot abuse patterns and tune limits over time.

TOPIC 09

Scaling Approach

Scaling this system is about keeping the hot path thin while still enforcing limits correctly across however many gateway instances you are running.

Lever	What it does	When to reach for it
Stateless gateway instances	Scale gateways horizontally with no shared state between them	Default setup, this is what makes the architecture scale at all
Redis as shared counter store	Every gateway instance checks the same counters, so limits stay correct regardless of which instance handled the request	Always, this is the whole reason Redis is in this design
Lua script or atomic command	Performs check-and-update as one operation, removing the race condition between reading and writing the count	Always, never split a counter check into separate GET and INCR calls
Redis Cluster	Shards counters across multiple Redis nodes for very high request volume	Once a single Redis instance becomes the bottleneck
Locally cached rules	Avoids a database lookup on every single request	Always, refresh on an interval or on an admin-triggered push instead
Stricter limits on sensitive endpoints	Login, OTP, search, and payment endpoints get tighter limits than a simple GET	From day one, these are the endpoints attackers actually target

TOPIC 10

Failure Handling

Failure	How you handle it
Redis is down	Decide per endpoint. Fail open for a product listing page, fail closed for a payment or login endpoint, with a small local emergency limit as a fallback either way.
Redis is slow	Set an aggressive timeout on the Redis call itself, do not let a slow rate limit check become the slowest part of the request.
Gateway cannot resolve identity	Fall back to IP-based limiting rather than letting the request through unlimited.
Rule config DB is down	The local cache keeps serving the last known rules, this is exactly why rules are cached locally instead of queried per request.
Local rule cache is stale	Acceptable for a short window, refresh on a schedule, this is far better than a per-request database call.
Hot key for one abusive identifier	Apply temporary blocking, a local deny cache, WAF or bot protection, or stricter limits for that identifier so Redis is not repeatedly hit for clearly abusive traffic.
Attacker rotates IPs	Layer identity checks, combine IP with API key or account identity so rotating IPs alone does not reset the limit.
Legitimate client hits limit during a retry storm	Retry-After plus client side exponential backoff guidance prevents the retries themselves from making things worse.

WARNING

The mistakes that come up again and again: rate limiting inside every individual service instead of at the gateway, using an in-memory counter when there are multiple gateway instances, doing a separate GET then INCR in Redis without atomicity, forgetting a TTL on the keys, and applying the exact same limit to a login endpoint as to a harmless GET.

TOPIC 11

Java + Spring Boot Mapping

Layer	Spring Boot mapping
Gateway filter	A Spring Cloud Gateway GlobalFilter or GatewayFilter, running before the request reaches any backend route.
Rate limiter logic	RateLimiterFilter coordinates identity resolution, rule lookup, and the Redis check.
Identity	IdentityResolver reads userId from the JWT via Spring Security, falls back to an API key header or the client IP.
Rule lookup	RateLimitRuleService backed by RateLimitRuleRepository, with rules held in a local Caffeine cache, refreshed on an interval.
Redis access	RedisTemplate or ReactiveRedisTemplate, with the actual check-and-update done through a Lua script for atomicity.
Token bucket library	Bucket4j is a common choice if you want a ready-made token bucket instead of hand-rolled Lua.
Resilience	Resilience4j TimeLimiter wraps the Redis call so a slow Redis cannot stall the whole gateway.
Metrics	Micrometer counters for allowed and rejected requests, exported asynchronously, never written to a database on the hot path.
Admin API	AdminController exposes rule create and update endpoints, secured separately from the public-facing gateway routes.

JAVA MAPPING

In Spring Cloud Gateway, this filter must run before any backend route is called. Once the request is denied, the filter chain must stop immediately by returning the response directly. The backend service, its database, and any expensive downstream calls should never see rejected traffic. That is the whole point of checking at the gateway.

TOPIC 11.1

RateLimiterFilter.java

Gateway rate limit check with Redis and fallback.

```
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    String identity = identityResolver.resolve(exchange);
    String endpoint = exchange.getRequest().getPath().value();
    RateLimitRule rule = ruleCache.match(endpoint, identity);

    RateLimitResult result;
    try {
        String key = "rate:" + rule.getScope() + ":" + identity + ":" + endpoint;
        result = redisRateLimiter.allowRequest(key, rule);
    } catch (Exception e) {
        result = fallbackPolicy.decide(rule, endpoint); // fail-open or fail-closed
    }

    HttpHeaders headers = exchange.getResponse().getHeaders();
    headers.add("X-RateLimit-Limit", String.valueOf(rule.getLimit()));
    headers.add("X-RateLimit-Remaining", String.valueOf(result.getRemaining()));
    headers.add("X-RateLimit-Reset", String.valueOf(result.getResetTimestamp()));

    if (!result.isAllowed()) {
        headers.add("Retry-After", String.valueOf(result.getRetryAfterSeconds()));
        exchange.getResponse().setStatusCode(HttpStatus.TOO_MANY_REQUESTS);
        exchange.getResponse().getHeaders().setContentType(MediaType.APPLICATION_JSON);

        String body = "{\"error\":\"RATE_LIMIT_EXCEEDED\", \""
            + "\"message\":\"Too many requests. Please retry later.\", \""
            + "\"retryAfterSeconds\":\"" + result.getRetryAfterSeconds() + "\"}";

        DataBuffer buf = exchange.getResponse().bufferFactory()
            .wrap(body.getBytes(StandardCharsets.UTF_8));
        return exchange.getResponse().writeWith(Mono.just(buf)); // chain stops here
    }

    return chain.filter(exchange);
}
```

The Redis check must be atomic. Do not do GET and INCR as two separate calls, because two concurrent requests can both read the same count before either updates it. Use a Lua script, a Redis atomic command, or a library like Bucket4j. Also keep the Redis timeout very small. If Redis is unavailable, the gateway should follow endpoint-specific fallback rules instead of blindly failing every request the same way.

TOPIC 12

Interview Tradeoffs

Decision	Option A	Option B
Algorithm	Fixed window: simple to reason about, but allows a burst right at the window boundary.	Token bucket: smooths bursts properly, slightly more state to track per key.
Where state lives	Local in-memory counter: zero network cost, but wrong the moment you run more than one gateway instance.	Centralized in Redis: correct across all instances, adds one fast network hop per request.
Redis down behaviour	Fail open: availability stays high, but limits are unenforced during the outage.	Fail closed: limits hold, but a Redis outage can turn into a full outage for protected endpoints.
Limit scope	Per-IP: works without authentication, but shared networks and IP rotation make it easy to dodge.	Per-user or per-API-key: harder to dodge, but only works once the caller is identified.
Where the check happens	Gateway-level: one place to reason about, protects every backend service uniformly.	Service-level: more flexible per service, but duplicated logic and easy to miss on a new service.

TRADEOFF

If asked which single tradeoff matters most, pick centralized-versus-local state. Everything else in this question, the algorithm, the fail-open decision, the limit scope, is downstream of accepting that state has to live somewhere shared.

TOPIC 13

How to Answer This in 10 Minutes

1. State the scope: the rate limiting layer at the gateway, not a full gateway product, protecting backend services from abuse and spikes. (30 seconds)
2. Name the real hard part immediately: staying correct and atomic across many gateway instances, not just returning 429. This is what signals seniority. (1 minute)
3. Draw the diagram: Client, Gateway, Rate Limiter Filter, Redis, the allowed/denied split, with Rule Config Cache and the Rules DB beside it. (2 minutes)
4. Walk through the key flow in order, ending on the point that a denied request never reaches a backend service. (1.5 minutes)
5. Cover the algorithms briefly, then commit to token bucket as your default answer and explain why. (1.5 minutes)
6. Go deep on one failure scenario, Redis being down and the fail-open versus fail-closed decision, rather than listing all of them shallowly. (2 minutes)
7. Close with the centralized-versus-local-state tradeoff, it is the single idea this entire question is built around. (1 minute)

TOPIC 14

Final Interview Closing Line

CLOSING LINE

For rate limiters, the main design idea is simple: Redis gives every gateway instance a shared, atomic view of how many requests an identity has made, the gateway is where the check belongs so denied requests never reach a backend service, and the algorithm choice is really a tradeoff between strict fairness and tolerating short bursts. A senior answer picks token bucket by default and explains exactly why.

Logging System

Printing logs is the easy part. Every fresher can do `System.out.println`. The actual question is what happens when 100 microservices, each running 20 instances, all start writing logs at once during an error storm. Who collects them, who searches them six months later, and why should none of this ever slow down the request that triggered the log in the first place?

01

Problem Statement

We are designing a centralized logging system for backend microservices. The scope is application logs, error logs, request logs, and audit logs: collecting them, searching them, and retaining them sensibly. We are not building full observability with metrics, tracing, or APM dashboards, though we will mention where those connect to logs.

Core users: backend developers debugging an incident, the DevOps or SRE team watching for error spikes, the support team investigating a user complaint, the security team reviewing audit trails, and automated alerting that reacts to patterns.

The hard parts are not obvious until you have lived through an incident. Collecting logs from hundreds of instances without slowing down user requests. Searching by `requestId`, `traceId`, or `userId` across a massive volume. Surviving an error storm that multiplies log volume by 10x in seconds. Keeping recent logs fast to search while archiving old logs cheaply. And never, ever logging a password or an OTP by accident.

INTERVIEW FOCUS

A common fresher answer here is "I will write logs to a database." That breaks almost immediately. Logs are high-volume, append-only, and rarely need relational joins. The moment you say database for logs, the interviewer knows you have not operated a logging pipeline under real traffic.

02

Requirements

Functional Requirements

Requirement	What it means
Collect logs from all services	Every microservice instance ships logs to a central pipeline
Structured JSON logs	Logs are structured fields, not free-form text strings
Correlation IDs	Every log carries requestId / traceId so a request can be traced end to end
Search by multiple fields	Search by service, level, timestamp, traceId, userId, or keyword
Store ERROR and WARN logs	These are the logs developers actually need during debugging
Separate audit/security logs	Compliance-sensitive logs are isolated from regular app logs
Retention policies	Different log types keep different hot/cold retention windows
Alerting on error spikes	Automated detection when error rate crosses a threshold

Non-Functional Requirements

Requirement	What it means
Non-blocking logging	Writing a log must never slow down or block the user request
High write throughput	System absorbs thousands of log lines per second, including spikes
Near real-time search	Recent logs (last 7-15 days) searchable within seconds of being written
Cheap long-term archival	Old logs move to low-cost storage rather than staying in the search engine
PII and secret protection	Sensitive fields are masked before logs are stored anywhere
Graceful degradation	Under extreme load, the system drops low-priority logs rather than crashing

02.5

Capacity Estimation

Metric	Estimate	Implication
Microservices	100 services	Each ships logs independently to the pipeline

Metric	Estimate	Implication
Instances per service	20	2,000 total instances writing logs concurrently
Logs per instance/sec	50 (normal traffic)	2,000 x 50 = 100,000 logs/sec baseline
Error storm multiplier	5x to 10x	Peak can spike to 500K-1M logs/sec briefly
Average log size	1 KB	100K logs/sec x 1KB = ~100 MB/sec baseline
Hot retention	7-15 days	Search engine sized for ~2 weeks of logs
Cold retention	30-90+ days	Object storage holds the long tail cheaply

TIP

The key insight: logging is write-heavy, and most search activity happens on recent logs. Nobody searches six-month-old DEBUG logs interactively. So the design splits into a fast searchable hot layer for recent logs and a cheap cold archive for everything older. This single decision shapes almost every component that follows.

03

Log Format / API Contract

Before any pipeline design, agree on the log format. Every service should emit structured JSON, not random printed strings. This is the contract the entire system is built around.

STRUCTURED LOG EXAMPLE: JSON

```
{
  "timestamp": "2026-07-01T10:15:30Z",
  "level": "ERROR",
  "service": "order-service",
  "environment": "prod",
  "traceId": "abc-123",
  "requestId": "req-789",
  "userId": "u_456",
  "endpoint": "/api/v1/orders",
  "method": "POST",
  "statusCode": 500,
  "message": "Order creation failed",
  "errorType": "PaymentTimeoutException",
  "stackTrace": "...",
  "durationMs": 842
}
```


WARNING

timestamp, level, service, traceId, and message are mandatory on every log line. stackTrace is captured for ERROR logs specifically. Passwords, tokens, OTPs, and card numbers must never appear in a log, full stop. A log should carry enough context to debug without ever exposing a secret.

04

Data Model / Storage Design

Storage here is two layers, not one. Many freshers try to force everything into a single database, and that is exactly where this design falls apart at scale.

Layer	Stores	Why
Hot Store (OpenSearch / Elasticsearch / Loki)	Recent logs, indexed by timestamp, service, level, traceId, requestId	Used for dashboards and live debugging. Fast search, expensive to keep forever.
Cold Archive (S3 / GCS)	Compressed old logs beyond the hot retention window	Cheap long-term retention. Used for compliance, audits, and rare investigations.
Kafka	Buffer between log producers and storage systems	Absorbs spikes. Decouples app instances from the speed of the search engine.

Optional Relational Tables (config and audit, not log storage)

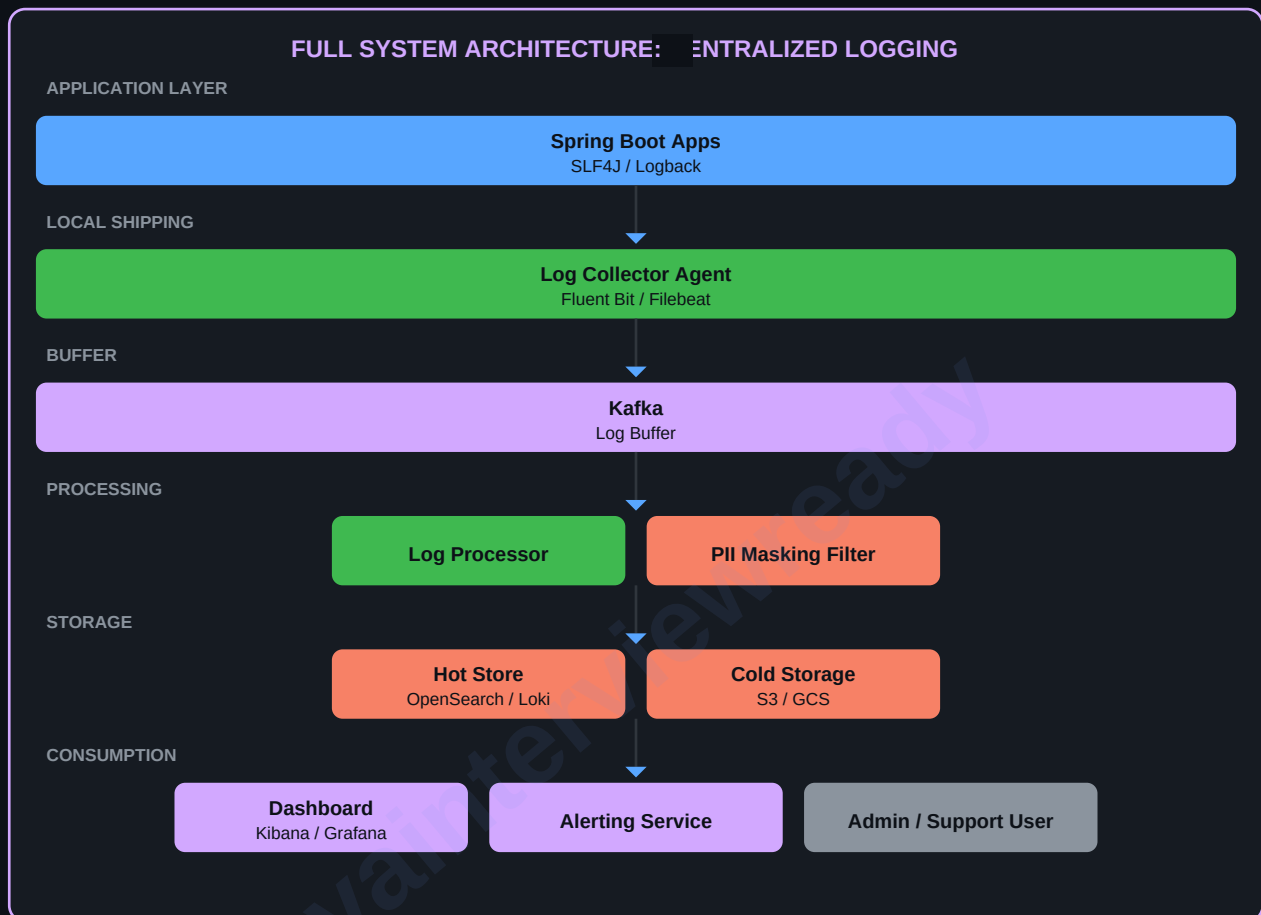
Table	Key Fields
log_retention_policies	policy_id, log_type, environment, hot_retention_days, cold_retention_days, enabled
log_alert_rules	rule_id, service, level, threshold, time_window, action
audit_log_access	access_id, user_id, query, reason, created_at

INTERVIEW FOCUS

Notice what is NOT in a relational database: the logs themselves. A relational DB is wrong for huge append-only volume with no real join requirements. The small tables above are just configuration and audit metadata, a few hundred rows, not billions.

05

High-Level Architecture Diagram



06

Key Flow: Request to Searchable Log

This flow shows a single log line traveling from inside a request handler to becoming searchable in the dashboard.



Note: the application never calls the search engine or Kafka synchronously inside the request path. Everything past stdout happens asynchronously.

07

Log Processing Pipeline

For beginners, the main idea is simple: logging should be fast and local, shipping should be async, and processing should happen far away from the request path.

Step	What Happens
1. Log generation	Application writes structured JSON via SLF4J/Logback. traceId/requestId injected from MDC.
2. Async appender / stdout	Logback AsyncAppender or container stdout. Never a synchronous network call from the app.
3. Collector reads logs	Fluent Bit or Filebeat tails stdout/files per pod or instance. Runs as a sidecar or daemonset.
4. Kafka buffers	Collector ships to Kafka. Buffer absorbs spikes so the processor is never overwhelmed directly.
5. Schema validation	Log Processor parses each message, drops or quarantines malformed entries.
6. PII masking	Sensitive fields stripped or redacted before anything touches storage.
7. Enrichment	Add service name, environment, host, pod, and region metadata for richer search.
8. Indexing	Recent logs indexed into the hot store with time-based indices.
9. Archival	Older logs compressed and moved to cold object storage on a retention job.
10. Alerting	Pattern detection on the processed stream triggers alerts for error spikes.

WARNING

Say this clearly in an interview: applications should never synchronously call the logging database or search engine inside the request flow. Logs are emitted locally and shipped asynchronously. If the search engine is slow or down, the user request should never know about it.

08

Components Explanation

Component	Responsibility
Application Services	Write structured JSON logs via SLF4J/Logback. Logging is local and non-blocking by design.
SLF4J / Logback	Standard Java logging facade and implementation. Configured with a JSON encoder, not plain text.

Component	Responsibility
MDC	Mapped Diagnostic Context. Thread-local storage that carries traceId/requestId/userId into every log line automatically.
Log Collector Agent	Fluent Bit or Filebeat. Tails stdout/files, ships to Kafka. Runs alongside each instance or pod, scales with the app.
Kafka	Buffer between producers and storage. Absorbs traffic spikes, protects downstream systems from being overwhelmed.
Log Processor	Kafka consumer group. Validates schema, enriches metadata, routes to hot or cold storage.
PII Masking Filter	Strips or redacts sensitive fields (passwords, tokens, card numbers) before persistence. A deny-list of field names.
Hot Log Store	OpenSearch, Elasticsearch, or Loki. Indexed by time, service, level, traceId for fast search on recent logs.
Cold Object Storage	S3 or GCS. Compressed archives for compliance and rare long-term investigation. Far cheaper per GB than the hot store.
Dashboard	Kibana or Grafana. Developers and support staff search and visualize logs from here.
Alerting Service	Watches the processed stream for error rate spikes or known failure patterns. Notifies on-call.
Retention Manager	Scheduled job that enforces hot/cold retention windows per log_retention_policies.

09

Scaling Approach

Concern	Approach
Local stdout, not direct writes	Apps write to stdout/local file, never directly to the search engine. Decouples app speed from storage speed.
Collector scales with instances	One collector per pod/instance. Scales naturally as the fleet grows, no central bottleneck.
Kafka absorbs spikes	Error storms hit Kafka first. Downstream processor and search engine never see the raw spike directly.
Partition by service/environment	Kafka topics partitioned so one noisy service does not starve others in the consumer group.
Horizontal log processors	Processor is a Kafka consumer group. Scale instances based on consumer lag.

Concern	Approach
Time-based indices	Hot store indexes logs by day or hour. Old indices drop off cleanly per retention policy.
Sample noisy DEBUG logs	In production, sample or disable DEBUG level. Keep ERROR and WARN at full fidelity.
Separate audit pipeline	Security/audit logs get their own topic and retention, isolated from general app noise.
Backpressure handling	If the pipeline falls behind, drop low-priority logs before the logging system can hurt the main app.

10

Failure Handling

Failure Scenario	How to Handle
Kafka is down	Collector buffers locally with a size limit. Drops oldest/lowest-priority logs if buffer fills. App is unaffected.
Log collector is down	Logs accumulate on local disk briefly. Alert on disk usage. Collector restarts and resumes shipping.
Hot store (OpenSearch/Loki) slow	Kafka holds the backlog. Indexing lags but no logs are lost. Alert on consumer lag.
Log processor crashes	Kafka consumer offset not committed. Processor restarts and reprocesses from last committed offset.
Sudden volume spike	Backpressure kicks in. Sample or drop DEBUG/INFO first. ERROR and audit logs stay protected.
Disk fills (collector cannot ship)	Local buffer has a hard size cap. Oldest non-critical logs are evicted first. Alert fires on disk threshold.
Duplicate logs	Mostly harmless for logs (unlike orders/payments). Dedup at query time using requestId if it ever matters.
Logs arrive out of order	Search engine sorts by timestamp at query time. Order of arrival does not need to match order of creation.
Failure Scenario (continued)	How to Handle
PII/token accidentally logged	Masking filter is the safety net, but the real fix is code review and lint rules at the source.

Failure Scenario (continued)	How to Handle
Dashboard query is slow	Limit query time range by default. Index lifecycle keeps hot indices small enough to stay fast.
Alerting service is down	Logs and indexing continue unaffected. Alerting is a consumer of the pipeline, not a dependency of it.
One service floods the pipeline	Per-service rate limiting at the collector or Kafka partition level protects the shared pipeline.

11

Java + Spring Boot Mapping

Component	Spring Boot Mapping
SLF4J Logger	Standard logging facade used in every Spring Boot service. Never <code>System.out.println</code> .
Logback	Default Spring Boot logging implementation. Configured with <code>logstash-logback-encoder</code> for JSON output.
MDC	<code>org.slf4j.MDC</code> . Carries <code>traceId</code> , <code>requestId</code> , <code>userId</code> across the request thread automatically into each log line.
OncePerRequestFilter	Adds <code>requestId</code> / <code>traceId</code> to MDC at the start of each request, clears it in a finally block at the end.
Sleuth / Micrometer Tracing	Optional. Generates and propagates <code>traceId</code> / <code>spanId</code> across service calls if distributed tracing is in scope.
@ControllerAdvice	Global exception handler. Logs unhandled exceptions with full context before returning an error response.
AsyncAppender	Logback appender that writes logs on a separate thread, so logging never blocks the request thread.
Spring Boot Actuator	<code>/actuator/loggers</code> endpoint allows changing log levels at runtime without a redeploy.
Fluent Bit / Filebeat	Collector agent, not a Spring component. Runs alongside the app in the container or pod.
Kafka Consumer (LogProcessor)	Separate service. Consumes raw logs, validates, masks PII, enriches, and writes to the hot store.
OpenSearch / Elasticsearch Client	Used only inside the LogProcessor service, never inside individual microservices.

Component	Spring Boot Mapping
Micrometer	Pipeline health metrics: collector lag, Kafka consumer lag, indexing throughput.

Code Snippet: RequestLoggingFilter.java: Id Traceld to MDC and Log Request Summary

```

@Component
public class RequestLoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
        FilterChain chain) throws IOException, ServletException {

        String requestId = Optional.ofNullable(req.getHeader("X-Request-Id"))
            .orElse(UUID.randomUUID().toString());
        String traceId = MDC.get("traceId") != null ? MDC.get("traceId") : requestId;

        MDC.put("requestId", requestId);
        MDC.put("traceId", traceId);
        if (req.getUserPrincipal() != null) {
            MDC.put("userId", req.getUserPrincipal().getName());
        }

        long start = System.currentTimeMillis();
        try {
            chain.doFilter(req, res);
        } finally {
            long duration = System.currentTimeMillis() - start;
            log.info("{} {} -> {} ({} ms)",
                req.getMethod(), req.getRequestURI(), res.getStatus(), duration);
            MDC.clear(); // always clear, even on exception
        }
    }
}

```

JAVA MAPPING

MDC.clear() in the finally block is the part freshers forget. Threads are reused in a thread pool. If you do not clear MDC, the next request handled by that thread inherits the previous request's traceId, and your logs become unreliable for tracing.

Logback JSON Encoder Concept

LOGBACK CONFIG SNIPPET

```
<appender name="JSON_CONSOLE"
  class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="net.logstash.logback.encoder.LogstashEncoder">
    <includeMdcKeyName>traceId</includeMdcKeyName>
    <includeMdcKeyName>requestId</includeMdcKeyName>
    <includeMdcKeyName>userId</includeMdcKeyName>
  </encoder>
</appender>

<appender name="ASYNC_JSON"
  class="ch.qos.logback.classic.AsyncAppender">
  <queueSize>8192</queueSize>
  <discardingThreshold>0</discardingThreshold>
  <appender-ref ref="JSON_CONSOLE"/>
</appender>

<root level="INFO">
  <appender-ref ref="ASYNC_JSON"/>
</root>
```

JSON_CONSOLE writes structured logs to stdout. ASYNC_JSON wraps it so request threads are never blocked by log writing. The collector reads stdout and ships logs asynchronously. In production, avoid logging secrets and avoid enabling DEBUG everywhere.

12

Interview Tradeoffs

Decision	Option A	Option B
Plain text vs structured JSON logs	Plain text: easy to read by eye, terrible to search or parse programmatically.	JSON: machine-parseable, indexable by field. Slightly more verbose. Always the right call at scale.
App writes directly vs collector-based shipping	Direct writes: simpler at first, but couples app latency to the search engine's availability.	Collector-based: app stays decoupled. Industry standard for containerized services.
OpenSearch/Elasticsearch vs Loki	OpenSearch: full-text indexing of every field, more storage and compute cost.	Loki: indexes only labels, logs stay compressed. Cheaper, less flexible full-text search.
Hot storage vs cold archive	All logs hot: fast search always, but storage cost grows unbounded over time.	Hot + cold split: recent logs fast, old logs cheap. The standard real-world approach.
Sync vs async logging	Sync: log write blocks the request thread. Unacceptable under load.	Async appender: write happens off the request thread. Slight risk of losing the last few logs on a hard crash.
Log everything vs sampling noisy logs	Log everything: complete picture, but DEBUG/INFO volume can drown out ERROR signal and cost.	Sample noisy levels: keep ERROR/WARN at full fidelity, sample DEBUG. Standard production practice.

WATCH OUT

Common Fresher Mistakes:

- Using `System.out.println` everywhere instead of a real logging framework.
- Not adding `traceId/requestId`, making cross-service debugging impossible.
- Writing logs synchronously to a database inside the request path.
- Storing all logs in a relational database instead of a search engine + archive.
- Logging passwords, tokens, OTPs, or card numbers.
- Leaving DEBUG logs enabled everywhere in production.
- Not setting a retention policy, letting storage costs grow unbounded.
- Not separating audit/security logs from regular application noise.
- Making logs searchable only by raw message text instead of structured fields.

13

How to Answer This in 10 Minutes

HOW TO ANSWER THIS IN 10 MINUTES

1. Open with scope. "We are designing centralized logging for microservices: collection, search, and retention. Not full observability with metrics and tracing." 30 seconds.
2. State the key constraint immediately. "Logging must never block the request path. Apps write locally, shipping happens async." This sentence shows real production experience. 30 seconds.
3. Draw the architecture. App → stdout → Collector → Kafka → Processor → Hot Store + Cold Archive → Dashboard/Alerting. 2 minutes.
4. Explain the hot/cold split. Recent logs in a search engine for fast debugging. Old logs compressed in object storage for cheap retention. One sentence each. 1 minute.
5. Walk through the flow. Request comes in, MDC gets traceId, structured JSON log written to stdout, collector ships to Kafka, processor masks PII and indexes. 2 minutes.
6. Cover one failure. Kafka goes down: Collector buffers locally with a size cap, drops lowest priority first. This shows you think about backpressure. 1.5 minutes.
7. Close with one tradeoff. Structured JSON vs plain text, or hot/cold storage split. Pick one and explain it with confidence. 1.5 minutes.

FINAL INTERVIEW CLOSING LINE

For a centralized logging system, the core decisions are: applications log locally and asynchronously so logging never blocks a request, Kafka buffers logs between producers and storage so spikes never overwhelm the pipeline, recent logs live in a fast search engine while old logs archive to cheap object storage, and sensitive data is masked before anything is persisted. The tradeoff is search flexibility and cost versus retention length and write throughput.

Online Code Judge

Most candidates hear "design a coding platform" and jump straight to the database schema. That is the wrong starting point. The hardest part of this question is a single sentence: you need to execute arbitrary, untrusted code written by strangers, at scale, without letting any of it harm your infrastructure. Everything else in the design follows from solving that problem first.

01

Problem Statement

We are designing the backend for a competitive coding platform like LeetCode, HackerRank, or CodeChef. The scope is: a user submits code for a problem, the system compiles and executes it against hidden test cases, and returns a verdict. We are not designing the learning portal, editorial system, or video content side. The focus is submission, judging, and result.

Core users: students and candidates submitting code, problem setters uploading test cases, judge workers executing submissions, and support staff reviewing system errors.

The hard parts: running untrusted code safely, enforcing time and memory limits, supporting multiple languages, preventing hidden test cases from leaking, scaling during contest spikes without making the main API suffer, and making sure a worker crash does not silently drop a submission.

INTERVIEW FOCUS

A very common mistake in this interview: the candidate tries to run user code synchronously inside the POST /submissions handler. That design fails at the first contest with 10,000 submissions at once. Say early that judging is async and workers are separate.

02

Requirements

Functional Requirements

Requirement	What it means
View problem	User reads problem statement, examples, constraints, and time/memory limits
Submit code	User submits source code for a specific problem and language
Async judging	Backend stores submission with QUEUED status; judging happens separately
Test case execution	Workers run both sample and hidden test cases with enforced limits
Verdict	System returns ACCEPTED, WRONG_ANSWER, TLE, MLE, RUNTIME_ERROR, or COMPILATION_ERROR
Submission history	User can view all past submissions for a problem
Multi-language support	At least Java, Python, C++; configuration-driven, not hardcoded
Admin panel	Admin creates problems and uploads hidden test cases securely
Contest mode (optional)	Submissions linked to a contest, leaderboard updated asynchronously

Non-Functional Requirements

Requirement	What it means
Safe code execution	Untrusted user code runs inside an isolated sandbox with no network access
Time and memory limits	Enforced per language and problem; violations return TLE or MLE, not crashes
Async judging	POST /submissions never waits for test execution to complete
High availability at peaks	Contest traffic (50x-100x normal) handled by queue and auto-scaled workers
Hidden test case privacy	Test inputs/outputs never exposed through any user-facing API
Durability	Submission records persist even if a worker crashes mid-execution
Fair execution	One bad submission cannot starve other users sharing the same worker pool

02.5

Capacity Estimation

Metric	Estimate	Implication
Daily active users	1 million	Baseline for API and DB sizing
Submissions per day	200K	Roughly 2-3 submissions/sec on average
Contest peak multiplier	50x to 100x	Queue absorbs the spike; workers auto-scale
Test cases per submission	20 to 100	Judge time dominated by test case count
Average judge time	2-5 seconds per submission	CPU/memory bound, not network bound
Hidden test case files	KB to MB each	Stored in object storage, not in the DB
Source code size	1-10 KB typically	Can store in DB or object storage

TIP

The key insight here: the bottleneck is CPU and memory on judge workers, not database rows or API throughput. You cannot scale this just by adding more API pods. The queue and worker pool need to scale separately from the API layer.

03

APIs

Method	Path	Request	Response
GET	/api/v1/problems/{id}	JWT (optional)	200: problem statement, examples, constraints, time/memory limits
POST	/api/v1/submissions	problemId, language, sourceCode, contestId (optional)	201: submissionId, status=QUEUED (no verdict yet)
GET	/api/v1/submissions/{id}	JWT (owner or admin)	200: submission status, verdict, execution stats, per-test results
GET	/api/v1/problems/{id}/submissions	JWT (current user)	200: paginated list of user submissions for this problem

Method	Path	Request	Response
POST	/api/v1/admin/problems	title, statement, constraints, limits	201: problemId
POST	/api/v1/admin/problems/{id}/testcases	type (SAMPLE/HIDDEN), inputFile, outputFile, scoreWeight	201: testCaseId, stored securely in object storage
GET	/api/v1/contests/{id}/leaderboard	contestId (public or JWT)	200: ranked list, updated async

WARNING

POST /submissions returns immediately with QUEUED status and a submissionId. The client then polls GET /submissions/{id} or listens on a WebSocket. If a candidate in an interview makes POST wait for the full verdict, that is a red flag.

04

Database Design

Table	Key Fields
problems	problem_id, title, difficulty, statement, time_limit_ms, memory_limit_kb, status, created_at
test_cases	test_case_id, problem_id, type (SAMPLE/HIDDEN), input_storage_key, output_storage_key, score_weight
submissions	submission_id, user_id, problem_id, language, source_code_storage_key, source_hash, idempotency_key, status, verdict, execution_time_ms, memory_used_kb, created_at, updated_at
submission_results	result_id, submission_id, test_case_id, status, actual_output_key, execution_time_ms, memory_used_kb, error_message
judge_runs	run_id, submission_id, worker_id, status, started_at, finished_at, retry_count
language_configs	language, compiler_command, run_command, time_multiplier, memory_limit_mb, enabled
contests	contest_id, title, start_time, end_time, problem_ids[]
outbox_events	id, event_type, payload, status (PENDING/SENT), created_at

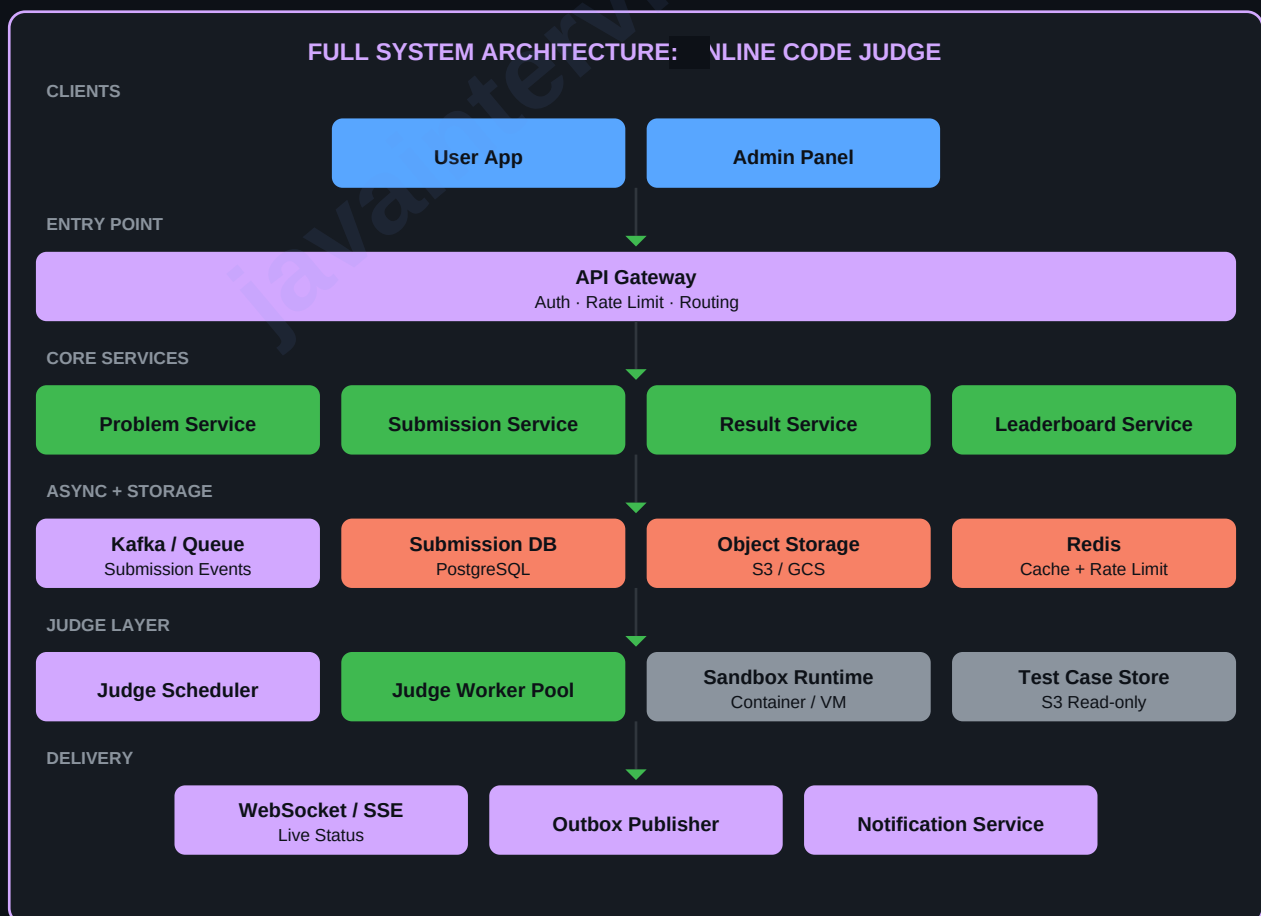
source_hash and idempotency_key help detect duplicate retries. A DB unique constraint can be added on idempotency_key when provided, or on a short-window combination like user_id + problem_id + language + source_hash.

Storage Choices

Store	Used For	Why
PostgreSQL / MySQL	Problems, submissions, results, judge_runs, language_configs	Strong consistency needed for submission status and result. Foreign keys help.
Object Storage (S3/GCS)	Source code files, hidden test case inputs/outputs	Keeps secrets off the public API path. Test cases never served directly to users.
Redis	Contest rate limiting, live leaderboard cache, submission count cache	Fast counters and TTL-based limits without hitting the DB on every request.
Kafka / Queue	SubmissionQueuedEvent for judging, LeaderboardUpdateEvent	Buffer between API and workers. Workers process at their own pace.

05

High-Level Architecture Diagram



06

Key Flow: Submission to Verdict

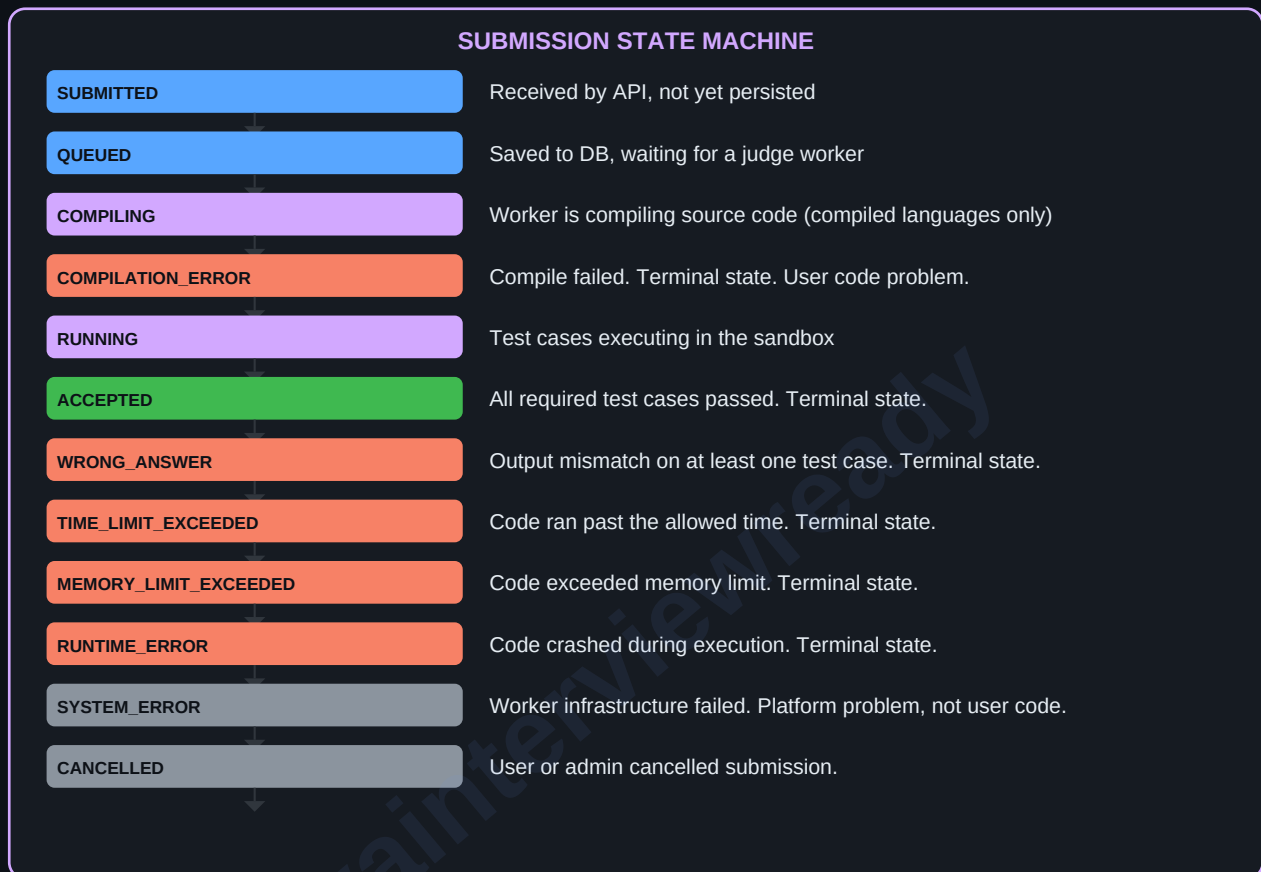
This is the flow that interviewers care about most. Everything from the API call to the final verdict.



Note: the Submission Service never executes code. It creates the record and queues the job. The Judge Worker is the only component that touches code execution.

07

Submission State Machine



WARNING

SYSTEM_ERROR is important to get right. If a worker crashes or times out on infrastructure, that is not the user's fault. The submission should be retried automatically, and if it keeps failing, marked SYSTEM_ERROR. WRONG_ANSWER, TLE, and MLE all mean the code ran but produced the wrong output or violated limits. These are user code problems.

08

Sandbox and Security

SANDBOX AND SECURITY

This is the most important section in this entire design.

User code is untrusted. All of it.

- Never run user code inside the main Spring Boot API server.
- Run code inside an isolated container or VM with these constraints enforced:
CPU time limit, wall clock limit, memory limit, process count limit,
no outbound network access, read-only base filesystem, no write to shared paths.
- Destroy the sandbox completely after execution completes or fails.
- Hidden test case files should be read via stdin injection, not passed as file paths.
- Judge workers must run on machines or nodes completely separate from the API tier.
- Log sandbox creation failures separately for investigation.
- Do not trust any output the sandbox process writes outside of stdout/stderr.
- Consider Docker with seccomp profiles, gVisor, Firecracker, or similar hardening.

09

Components Explanation

Component	Responsibility
API Gateway	Auth, rate limiting per user per minute, routing. Rejects unauthenticated submission attempts.
Problem Service	Serves problem statements, examples, constraints. Cached in Redis since problems rarely change.
Submission Service	Saves submission row with QUEUED status, stores code in Object Storage, saves OutboxEvent. Never executes code.
Judge Scheduler	Consumes SubmissionQueuedEvent from Kafka. Assigns submissions to available Judge Workers. Tracks which worker is handling which submission.
Judge Worker	The execution engine. Pulls source code and test cases, creates sandbox, compiles, runs, collects results, writes verdict. Stateless and horizontally scalable.

Component	Responsibility
Sandbox Runtime	Isolated container or VM environment per submission. Enforces CPU, memory, time, and network constraints. Destroyed after execution.
Object Storage	Stores source code files and hidden test case inputs/outputs. Workers read test cases at run time. Never exposed through user-facing APIs.
Submission DB	PostgreSQL. Stores submission records, per-test-case results, judge_runs, and language_configs. Transactional for status updates.
Result Service	Reads submission_results for a submission and aggregates the final verdict display for the user.
WebSocket / SSE	Pushes live status updates to the user's browser as the submission moves through states. Eliminates constant polling.
Leaderboard Service	Updates contest rankings asynchronously when a submission is judged. Serves from Redis cache for speed.
Outbox Publisher	@Scheduled job. Reads committed PENDING OutboxEvents, publishes to Kafka, marks as SENT. Kafka never called inside @Transactional.

10

Scaling Approach

Concern	Approach
API services stay stateless	Problem Service and Submission Service scale horizontally. No shared state.
Queue absorbs contest spikes	Kafka or SQS buffers submissions during peak. Workers process at their own pace.
Worker auto-scaling	Scale Judge Workers based on Kafka consumer lag or queue depth. More lag, more workers.
Separate queues per language	Java jobs and Python jobs can have separate queues so one slow language does not block others.
Pre-warm sandbox images	Popular language containers (Java, Python, C++) kept warm on worker nodes to reduce startup latency.
Cache problem metadata	Problem statements rarely change. Cache in Redis with a long TTL. Invalidate only on admin update.
Object storage for test cases	Workers read test case files from S3 at run time. No need to move them over the network in advance.

Concern	Approach
Rate limit per user	Enforce submission limits per user per minute using Redis counters. Prevents one user flooding the queue.
Contest queue priority	Active contest submissions can be routed to a dedicated worker pool so regular submissions do not compete.
Partition by contestId	Kafka topics partitioned so submissions for the same contest stay ordered per user.

11

Failure Handling

Failure Scenario	How to Handle
Judge Worker crashes mid-run	Kafka offset not committed. Scheduler detects no heartbeat. Submission returns to queue. Retry with different worker.
Compilation takes too long	Wall clock timeout at the compiler step. Mark <code>COMPILATION_ERROR</code> or <code>SYSTEM_ERROR</code> depending on cause.
User code infinite loop	Sandbox enforces wall clock limit. Process killed. Submission marked <code>TIME_LIMIT_EXCEEDED</code> .
User code uses too much memory	Sandbox cgroup memory limit. Process killed. Submission marked <code>MEMORY_LIMIT_EXCEEDED</code> .
Sandbox creation fails	Mark <code>SYSTEM_ERROR</code> . Retry automatically. Alert if consistently failing on a worker node.
Object storage unavailable	Worker cannot pull source code or test cases. Retry with backoff. Dead letter queue after max retries.
Hidden test case file missing	Worker logs the missing file ID, marks <code>SYSTEM_ERROR</code> , alerts admin. Submission held pending fix.
Queue backlog during contest	Auto-scale workers. Prioritize contest queue. Alert on lag above threshold.
Duplicate submission request	Idempotency key or DB unique constraint on (user_id, problem_id, source_hash) within a short window.
Failure Scenario (continued)	How to Handle
Result update fails after judging	Worker retries the result save. judge_runs table tracks which worker last updated. Admin can rejudge.

Failure Scenario (continued)	How to Handle
Leaderboard update fails	Leaderboard is eventually consistent. Retry asynchronously. Contest result not affected.
User floods submissions	Redis rate limiter per user enforces max submissions per minute. Returns 429 Too Many Requests.
Language runtime image broken	Disable that language in language_configs. All submissions using it return SYSTEM_ERROR until fixed.

12

Java + Spring Boot Mapping

Component	Spring Boot Mapping
ProblemController	GET /problems/{id}. Returns problem metadata. Reads from Redis cache or DB.
SubmissionController	POST /submissions, GET /submissions/{id}, GET /problems/{id}/submissions.
AdminProblemController	POST /admin/problems, POST /admin/problems/{id}/testcases. Admin-only endpoints.
SubmissionService	Validates problem and language, checks duplicate retries, uploads source code to Object Storage, then calls a short @Transactional method to save Submission + OutboxEvent.
JudgeWorkerService	@KafkaListener on submission queue. Creates sandbox, compiles, runs test cases, saves results, updates status.
ResultService	Aggregates submission_results into final verdict display. Called by SubmissionController.
LeaderboardService	Updates contest ranking async after verdict. Reads from Redis cache. Writes async via Kafka event.
SubmissionRepository	Spring Data JPA. findByIdAndProblemId for history. findByStatusAndCreatedAtBefore for stuck job cleanup.
SubmissionResultRepository	saveAll() for batch insert of per-test-case results after judge run.
OutboxRepository	findByStatus(PENDING). Polled by OutboxPublisher scheduler every few seconds.
ObjectStorageService	Wraps AWS SDK S3Client. putObject() for source code upload. getObject() for test case download in workers.
RedisTemplate	opsForValue() for rate limiting counters (incr + expire). opsForZSet() for contest leaderboard cache.

Component	Spring Boot Mapping
KafkaTemplate	Used ONLY by OutboxPublisher. Sends SubmissionQueuedEvent and LeaderboardUpdateEvent after DB commit.
@Scheduled	OutboxPublisher (polls every 5s). StuckSubmissionJob (finds QUEUED/RUNNING submissions older than N minutes).
Spring Security/JWT	All write endpoints require auth. Admin endpoints require ROLE_ADMIN.
WebSocket / SSE	Spring WebSocket or SseEmitter pushes status updates to client when judge result arrives.
@Transactional	createQueuedSubmission() wraps only the Submission save + OutboxEvent save atomically. The Object Storage upload happens before this transaction opens.

Code Snippet: SubmissionService.java -- Save Submission and Queue Judge Event

```

@Service
public class SubmissionService {

    @Autowired private ProblemRepository problemRepo;
    @Autowired private SubmissionRepository submissionRepo;
    @Autowired private OutboxRepository outboxRepo;
    @Autowired private ObjectStorageService storage;
    @Autowired private LanguageConfigRepository langRepo;

    // No @Transactional here - storage upload happens before DB transaction opens
    public SubmissionResponse submit(SubmitRequest req, Long userId) {

        problemRepo.findById(req.getProblemId()).filter(Problem::isActive)
            .orElseThrow(() -> new NotFoundException("Problem not found"));
        langRepo.findByLanguageAndEnabled(req.getLanguage(), true)
            .orElseThrow(() -> new UnsupportedLanguageException());

        // Duplicate check - avoid judging the same retry twice
        Optional existing = submissionRepo.findRecentDuplicate(
            userId, req.getProblemId(), req.getLanguage(), hash(req.getSourceCode()));
        if (existing.isPresent()) {
            return SubmissionResponse.queuedOrCompleted(existing.get());
        }

        // Upload BEFORE opening the DB transaction - no lock held during storage call
        String sourceKey = storage.upload(
            req.getSourceCode(), userId, req.getProblemId());

        return createQueuedSubmission(req, userId, sourceKey);
    }
}

```

```
@Transactional
public SubmissionResponse createQueuedSubmission(
    SubmitRequest req, Long userId, String sourceKey) {
    Submission sub = Submission.builder()
        .userId(userId).problemId(req.getProblemId())
        .language(req.getLanguage())
        .sourceCodeStorageKey(sourceKey)
        .sourceHash(hash(req.getSourceCode()))
        .status(SubmissionStatus.QUEUED)
        .build();
    submissionRepo.save(sub);
    outboxRepo.save(OutboxEvent.submissionQueued(sub.getId()));
    return SubmissionResponse.queued(sub.getId());
}
```

JAVA MAPPING

The source code upload happens before the DB transaction opens, so a slow Object Storage call never holds DB locks open. The `@Transactional` method only saves the `Submission` row and `OutboxEvent` together. If the DB transaction fails after upload, an orphan-cleanup job or object storage lifecycle rule removes unused source files later. Kafka is still not called from `SubmissionService`.

For duplicate retries: use an `idempotencyKey` when the client provides one. Otherwise use a short-window source hash check (`user_id + problem_id + language + source_hash`). A DB unique constraint is still the final protection against concurrent duplicate retries.

Judge Worker (simplified)

```
@KafkaListener(topics = "submission-queued", groupId = "judge-worker")
public void onSubmissionQueued(SubmissionQueuedEvent event) {

    Submission sub = submissionRepo.findById(event.getSubmissionId()).orElseThrow();
    sub.setStatus(COMPILING); submissionRepo.save(sub);

    // Pull code and test cases from Object Storage
    String code = storage.download(sub.getSourceCodeStorageKey());
    List tests = testCaseRepo.findByProblemIdAndType(sub.getProblemId(), HIDDEN);

    // Compile if needed (outside any DB transaction)
    CompileResult cr = sandbox.compile(code, sub.getLanguage());
    if (!cr.isSuccess()) { markVerdict(sub, COMPILATION_ERROR); return; }

    sub.setStatus(RUNNING); submissionRepo.save(sub);

    List results = new ArrayList<>();
    for (TestCase tc : tests) {
        TestResult r = sandbox.run(cr, tc, timeLimitMs, memoryLimitKb);
        results.add(r);
    }

    markFinalVerdict(sub, results); // @Transactional: status + per-test rows + OutboxEvent
}
```

JAVA MAPPING

Compile and run steps happen outside any long DB transaction since they can take seconds. The `markFinalVerdict()` call is a short `@Transactional` method that saves verdict, per-test-case results, and `OutboxEvent` together. `SYSTEM_ERROR` is used only for platform or sandbox infrastructure failures. User code problems become `WA`, `TLE`, `MLE`, `RUNTIME_ERROR`, or `COMPILATION_ERROR`.

13

Interview Tradeoffs

Decision	Option A	Option B
Sync vs async judging	Sync: <code>POST /submissions</code> waits for all test cases. Simple but unusable under any real load.	Async with queue: API returns immediately. Workers judge independently. Standard approach.
API server runs code vs isolated workers	API server executes code: one bad submission can crash your entire API. Never do this.	Isolated workers: sandboxed, separate from the API tier. The only safe option.

Decision	Option A	Option B
One queue vs separate queues per language	Single queue: simple. One slow language (e.g. Java startup) backs up Python too.	Separate queues: language runtimes isolated. Workers can specialise per language.
Source code in DB vs Object Storage	DB: simple for small scale. Large code files inflate DB tables and backups.	Object Storage: scales well. DB only stores the storage key. Preferred at scale.
Polling vs WebSocket/SSE	Polling: simple to implement, wastes requests when judging takes multiple seconds.	WebSocket/SSE: push update when verdict arrives. Better UX, slightly more complex.
Stop on first failure vs run all tests	Stop early: faster feedback, less compute used per wrong submission.	Run all tests: full partial scoring data. Needed for IOI-style problems.

WATCH OUT

Common Fresher Mistakes:

- Running user code inside the Spring Boot API server.
- Making POST /submissions wait for all test cases to finish.
- Not using sandbox isolation at all.
- Not enforcing time and memory limits, letting code run forever.
- Exposing hidden test case inputs through any API endpoint.
- Storing only the final verdict with no per-test-case breakdown.
- Not handling worker crashes; submissions stuck in RUNNING forever.
- Mixing SYSTEM_ERROR with user code errors (TLE, WA, RUNTIME_ERROR).
- Updating the contest leaderboard synchronously inside the submission flow.
- Not rate limiting submissions, allowing one user to flood the queue.

14

How to Answer This in 10 Minutes

HOW TO ANSWER THIS IN 10 MINUTES

1. Open with the hard constraint. "The entire design is shaped by one requirement: we must execute untrusted user code safely, at scale." Say this first. 30 seconds.
2. State the key design decisions immediately. "POST /submissions is async. Judging happens in isolated workers. The queue absorbs contest spikes." Three sentences, 30 seconds.
3. Draw the architecture. Client → API Gateway → Submission Service → Queue → Judge Workers → Sandbox → DB. Show Object Storage for code and test cases. 2 minutes.
4. Walk through the submission flow. Submit → QUEUED → Worker picks it up → Compile → Run test cases → Verdict → User notified. Mention the state machine naturally. 2 minutes.
5. Explain the sandbox. No network, CPU and memory limits, destroyed after execution. Workers are on separate machines from the API. Test cases never exposed through APIs. This is what separates a serious answer from a shallow one. 1.5 minutes.
6. Cover one failure. Worker crashes mid-run. Kafka offset not committed. Submission goes back to the queue. Retry with a different worker. 1 minute.
7. Close with one tradeoff. Single queue vs separate queues per language, or stop-on-first-failure vs run-all-tests. Pick one and explain it clearly. 1.5 minutes.

FINAL INTERVIEW CLOSING LINE

For an Online Code Judge, the design centers on one non-negotiable constraint: user code is untrusted and must never run inside the main API server. Submissions are queued and processed asynchronously by isolated workers. Sandboxes enforce CPU, memory, and network limits. The queue protects the system from contest traffic spikes. The tradeoff is between isolation strength and execution speed, and between operational simplicity and language fairness across separate queues.

QUESTION 15 OF 15 · MODULE 14

WhatsApp Last Seen / Online Status

Designing real-time presence for millions of unreliable mobile connections

Most candidates hear this question and relax a little. It sounds small. One boolean, one timestamp, how hard can it be. That relaxed feeling is exactly what gets people into trouble in the interview.

The actual difficulty here has nothing to do with storing a flag. It is about detecting state changes on networks you do not control, on devices that do not always tell you when they are leaving, at a scale where writing to a database on every heartbeat would fall over in minutes.

In real projects, this usually becomes a problem the first time someone tests it on a train or in a lift. The phone loses signal, the socket dies silently, and if your only signal for "user left" is a clean disconnect event, that user stays online forever in your system. That single gap is where this entire design lives.

1 . PROBLEM STATEMENT

We are building the backend that powers WhatsApp style presence: showing whether a contact is online right now, and if not, showing when they were last seen. Millions of phones open the app at roughly the same time every morning, hold a live connection while the app is active, and drop that connection in all kinds of ugly ways.

We are scoping this narrowly. We are not designing chat delivery, media sharing, group messaging, calls, or encryption here. Just presence: online/offline state, last seen, heartbeat handling, multi device support, and privacy controls around who can see this information.

INTERVIEW MOMENT

The hard parts interviewers are actually probing for: detecting offline reliably when the disconnect event never arrives, avoiding a database write on every heartbeat at hundreds of millions of users, handling multi-device logins correctly, and enforcing privacy before any status reaches anyone, not just when it is read directly.

Core actors in this system: the user whose presence is tracked, the contact viewing that presence, the WebSocket gateway handling live connections, the Presence Service that owns the state logic, the Privacy Service enforcing visibility rules, and a support engineer who occasionally needs to debug why someone's status looks stuck.

2 . REQUIREMENTS

FUNCTIONAL	NON-FUNCTIONAL
User goes online when the app connects and sends a heartbeat	Low latency for online/offline transitions
User goes offline on clean disconnect or heartbeat timeout	Highly available, survives gateway crashes
Last seen timestamp updates only on the offline transition	Scales to millions of concurrent WebSocket connections
Multiple devices per user, one active device keeps user online	Redis backed state lookups, sub-millisecond reads
Privacy modes: everyone, contacts only, nobody	No database write on every heartbeat
Blocked users never see presence or last seen	Eventual consistency acceptable for last seen, by a few seconds
Active chat screens get near real-time presence pushes	Privacy enforced strictly, no leaks through side channels
	Stale sessions expire automatically without manual cleanup

Many freshers skip straight to "I'll store isOnline as a boolean column" here. That answer tells the interviewer you have not thought about scale or network reality. Say it out loud in the interview: current state belongs somewhere fast and self-expiring, durable history belongs in the database. That one sentence usually earns you a nod.

3 . CAPACITY ESTIMATION

Keep this light. The interviewer is not checking your arithmetic, they want to see that scale assumptions shape your design choices.

ASSUMPTION	ROUGH NUMBER	WHAT IT TELLS US
Daily active users	50 million	Presence has to work at real scale, not toy scale
Concurrent online users	5 million	Redis must hold 5M live keys comfortably
Heartbeat interval	Every 25-30 sec	~170K-200K heartbeat writes/sec into Redis
Heartbeat DB writes	Zero	DB would collapse under this write rate, so it never sees a heartbeat
Last seen DB writes	Only on offline transition	Orders of magnitude fewer writes than heartbeats
Presence reads	Spikes during active chats	Read path needs cache, not DB, for hot lookups

THE CORE INSIGHT

Presence is a high frequency, short lived state. Redis fits naturally because keys expire on their own. The database only needs to know the durable facts: when a user last went offline, and what their privacy setting is. Never let heartbeats touch the database.

4 . APIs / WEBSOCKET EVENTS

Presence is mostly event driven, not request driven. REST covers on-demand lookups and settings. WebSocket events carry the live state.

REST endpoints

METHOD + PATH	PURPOSE
GET /api/v1/users/{userId}/presence	Return online status and last seen, only if the viewer is allowed to see it
GET /api/v1/users/me/privacy/presence	Fetch the caller's own presence privacy setting
PATCH /api/v1/users/me/privacy/presence	Update visibility: EVERYONE, CONTACTS_ONLY, or NOBODY

WebSocket events, client to server

EVENT	WHEN IT FIRES
presence.connect	App starts or the socket connects for a device
presence.heartbeat	Every 20-30 seconds while the app is active
presence.disconnect	App closes cleanly, when the OS actually allows it

WebSocket events, server to client

EVENT	WHEN IT FIRES
presence.online	Pushed to allowed contacts when the user's first device connects
presence.offline	Pushed when the user's last active device disappears
presence.last_seen_updated	Pushed to active chat viewers, if privacy allows it

WARNING

Mobile disconnects are not clean most of the time. Airplane mode, a dead battery, a killed background process, none of these fire presence.disconnect. If your design depends on that event always arriving, it will fail the first time someone tests it on a real phone. The heartbeat TTL is not optional, it is the actual detection mechanism.

5 . DATABASE DESIGN

Split clearly: durable facts live in a relational database, live state lives in Redis. Do not blur this line, it is the whole point of the design.

Relational tables (PostgreSQL / MySQL)

TABLE	KEY FIELDS
users	user_id, phone_number, name, created_at
contacts	owner_id, contact_user_id, added_at
blocked_users	blocker_id, blocked_id, blocked_at
presence_privacy_settings	user_id, visibility, show_online, show_last_seen, updated_at
user_last_seen	user_id, last_seen_at, updated_at
device_sessions (optional)	session_id, user_id, device_id, connected_at, disconnected_at, last_heartbeat_at, status

visibility on presence_privacy_settings takes one of three values: EVERYONE, CONTACTS_ONLY, or NOBODY. This is a small table, but every read path and every fanout has to check it before doing anything.

Redis key design

KEY	VALUE	TTL
presence:user:{userId}	ONLINE	60 sec
presence:user:{userId}:devices	set of active device/connection ids	per device
presence:device:{deviceId}	userId + connectionId	60 sec
presence:last_seen:{userId}	cached last seen (optional read-through)	short

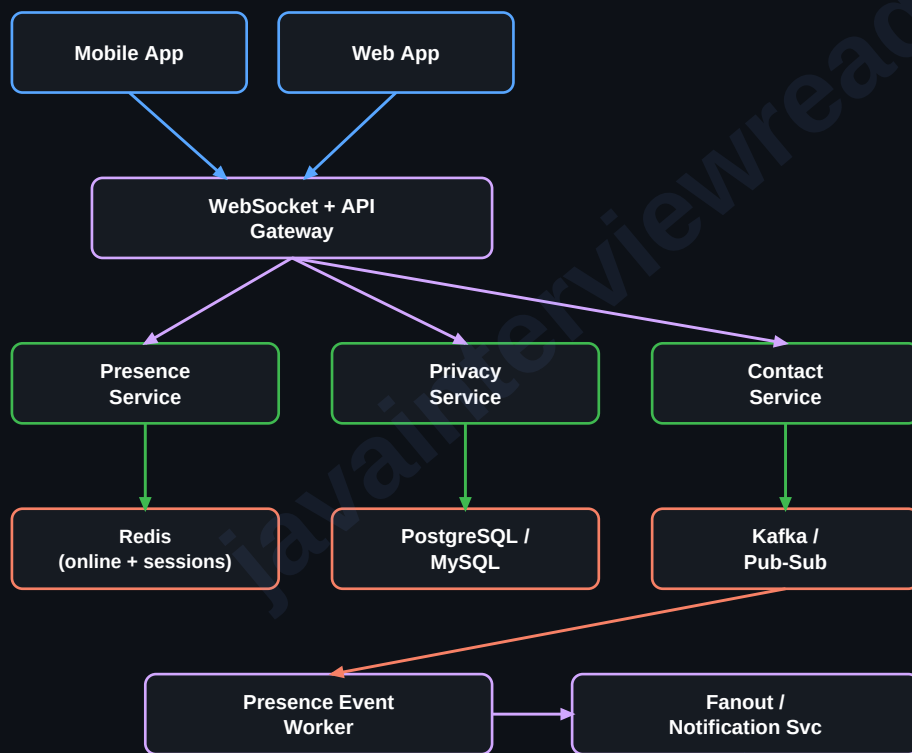
Every heartbeat just refreshes the TTL on these keys. If the app stops sending heartbeats for any reason, Redis quietly expires the key on its own, nobody has to run a job to notice the user vanished. That is the entire trick behind why this scales. For multi device users, the user stays marked online as long as one device key is alive, and last_seen is written only once, at the moment the very last device's key disappears.

THE DEVICES SET IS NOT THE SOURCE OF TRUTH

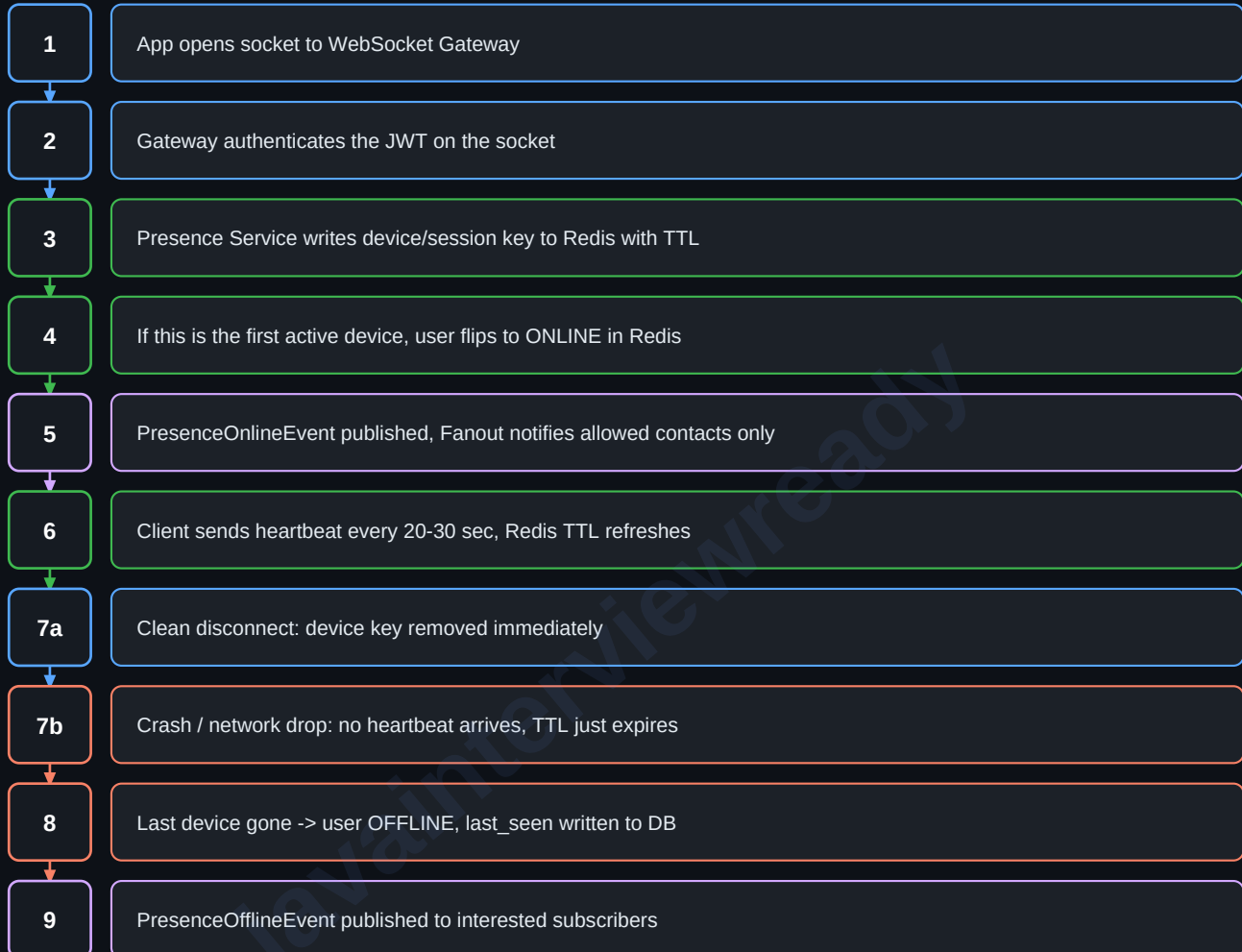
Redis Set members do not expire individually. If `presence:device:{deviceId}` expires on its own, that device id can still sit inside `presence:user:{userId}:devices` unless something cleans it up.

The devices set is only an index, not the final truth. Individual device keys with TTL are the real truth. When checking if a user is still online, remove expired device ids from the set and count only the ones whose `presence:device:{deviceId}` key still exists.

Another option: use a Redis sorted set with the last heartbeat timestamp as the score, and clean stale devices by score range instead of trusting plain set membership.

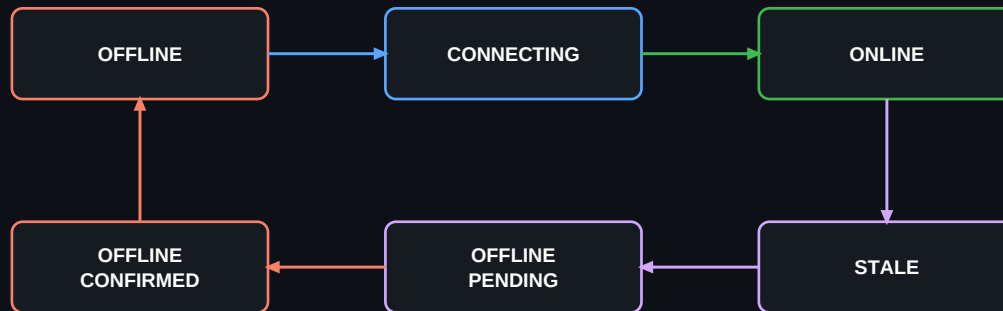
6 . HIGH-LEVEL ARCHITECTURE**Full system architecture**

Notice there is no arrow from Redis straight to the phone. Presence changes flow through the worker and fanout layer so privacy checks and subscriber filtering happen in one place, not scattered across every service that touches Redis.

7 . KEY FLOW: CONNECT, HEARTBEAT, DISCONNECT**End to end presence lifecycle**

8 . PRESENCE STATE MACHINE

How a user's state actually moves



heartbeat missing -> STALE -> grace period -> OFFLINE_PENDING -> other devices checked -> OFFLINE_CONFIRMED

CONNECTING means the socket passed authentication but we have not seen a stable heartbeat yet. ONLINE means a live heartbeat exists in Redis right now. STALE means the heartbeat is missing but the TTL grace window has not fully closed, this absorbs short network blips without flipping someone's status on every train tunnel. OFFLINE_PENDING fires when one device disappears but other devices still need to be checked before deciding the user is really gone. OFFLINE_CONFIRMED is the only point where last_seen actually gets written.

PRIVACY RULES

SETTING	BEHAVIOUR
EVERYONE	Online status and last seen visible to any non-blocked viewer
CONTACTS_ONLY	Visible only if the viewer exists in the user's contact list
NOBODY	Last seen hidden, online status hidden or shown as generic, per product rule
Blocked users	Always hidden, regardless of the visibility setting above

WARNING

Privacy has to be checked at read time and at fanout time, not just at the API layer. If your fanout service pushes presence.online to every contact without filtering, you have just leaked presence through a side channel even though your GET endpoint correctly hides it. This is a very common gap in fresher designs.

9 . COMPONENTS EXPLAINED

COMPONENT	WHY IT EXISTS
WebSocket Gateway	Holds the live socket per device, authenticates it, forwards presence events, stays stateless beyond the active connection
Presence Service	Owns the online/offline decision logic, writes to Redis, decides when to write last_seen
Privacy + Contact Service	Single source of truth for who is allowed to see whose presence, and who is in whose contact list
Redis	Fast, self-expiring store for current online state and active device sessions
PostgreSQL / MySQL	Durable store for last_seen, privacy settings, contacts, blocks
Kafka / Pub-Sub	Decouples state changes from the fanout of who needs to be told
Fanout Service	Applies privacy filtering, then pushes updates only to allowed, currently interested subscribers
Presence Event Worker / TTL Watcher	Consumes and orders state change events, triggers last_seen writes, backstops stale sessions Redis missed

10 . SCALING APPROACH

- WebSocket gateways scale horizontally behind a load balancer, each instance stateless beyond its active sockets.
- Redis holds the shared source of truth for presence, so any gateway instance can answer "is this user online" without owning that user's socket. Move to Redis Cluster once one node cannot hold the key volume.
- Heartbeat interval is a real tuning knob: too short multiplies write load for no benefit, too long makes detection feel sluggish. 20-30 seconds is a reasonable starting point.
- Never let heartbeats touch the database. This single rule is what keeps the write path cheap at 5 million concurrent users.
- Fanout only to contacts who are currently relevant (active chat viewers), not the owner's entire contact list every time. Shard presence events by userId to preserve ordering.
- Cache privacy settings for read-heavy paths, but invalidate the cache the moment a user changes their setting.
- last_seen writes can be batched or async, they do not need to block the offline transition. Multi-region setups can treat presence as eventually consistent across regions.

11 . FAILURE HANDLING

WHAT GOES WRONG	HOW IT IS HANDLED
App closes cleanly	presence.disconnect removes the device key immediately
App crashes / network drops	No clean event, but the heartbeat stops, TTL expires it

WebSocket Gateway crashes	Its sockets die, their Redis keys just expire naturally, no special recovery code needed
Redis is down	Serve last known / cached presence, or return "unknown", never guess ONLINE
Heartbeat delayed (bad network)	Grace period (STALE state) absorbs short delays before flipping offline
User on multiple devices	Stays ONLINE while at least one device key is alive, offline only once every device key is gone
last_seen DB write fails	Retry with backoff, the Redis offline state is already correct so users see it either way
Fanout service lagging / blocked user checks	Consumers catch up from Kafka offsets; blocked users always get hidden/unknown, no exception thrown
Events arrive out of order	Each event carries a timestamp/version, an older OFFLINE event cannot override a newer ONLINE one

12 . JAVA + SPRING BOOT MAPPING

CONCERN	SPRING BOOT PIECE
REST endpoints	PresenceController
Live socket handling	PresenceWebSocketHandler (Spring WebSocket / STOMP)
Core presence logic	PresenceService
Visibility rules	PrivacyService, PresencePrivacyRepository
Contact lookups	ContactService, backed by ContactRepository
Blocking checks	BlockedUserRepository
Fanout to subscribers	PresenceFanoutService
Event consumption	PresenceEventWorker with @KafkaListener
Durable last seen	UserLastSeenRepository, write wrapped in @Transactional
Redis access	RedisTemplate for TTL keys, device sets, online flags
Socket auth	Spring Security + JWT validated at handshake time
Event publishing	KafkaTemplate on the offline/online transition
Stale session backstop	@Scheduled cleanup job, or Redis keyspace notifications if enabled

PresenceService.java - heartbeat updates Redis, not the DB

```
public void handleHeartbeat(String userId, String deviceId) {
    String deviceKey = "presence:device:" + deviceId;
    String userKey = "presence:user:" + userId;
    String devicesKey = "presence:user:" + userId + ":devices";
    Duration ttl = Duration.ofSeconds(60);

    // 1. refresh this device's own key, this is the real truth
    redisTemplate.opsForValue().set(deviceKey, deviceId, ttl);

    // 2. devices set is only an index, keep it updated too
    redisTemplate.opsForSet().add(devicesKey, deviceId);
    redisTemplate.expire(devicesKey, ttl); // cleanup safety, not the source of truth

    // 3. atomic first-online transition, no read-then-write race
    Boolean firstOnline = redisTemplate.opsForValue()
        .setIfAbsent(userKey, "ONLINE", ttl);
    if (Boolean.TRUE.equals(firstOnline)) {
        kafkaTemplate.send("presence-events", new PresenceOnlineEvent(userId));
    } else {
        redisTemplate.expire(userKey, ttl); // already online, just refresh TTL
    }

    // note: no database write happens in this method, on purpose
}
```

setIfAbsent is what removes the duplicate-event bug. If two devices for the same user connect within the same second, only one of them can win the setIfAbsent race and only that one publishes PresenceOnlineEvent. The other device just refreshes the existing key's TTL, no second event, no double notification to contacts.

On disconnect or TTL cleanup, the service removes the device id from the devices set. Before marking the user offline, it checks only live device keys. If no live device keys remain, it deletes the user online key, writes last_seen in the database, and publishes PresenceOfflineEvent. This avoids marking a user offline while another device is still active.

13 . INTERVIEW TRADEOFFS**Redis online state vs a database online flag**

Redis gives self-expiring keys, cheap reads, and natural handling of unclean disconnects. A DB flag needs an external process to notice staleness and never expires on its own. Redis wins for anything at this scale, the DB flag only looks simpler on a whiteboard.

Short heartbeat interval vs long interval

Shorter intervals mean faster, more accurate offline detection but higher write and network load. Longer intervals cut load but make status feel laggy and delay offline detection. 20-30 seconds balances the two for a chat-style product.

Clean disconnect events vs TTL based expiry

Clean events are faster when they arrive, but you cannot depend on them on mobile networks. TTL expiry is slower by design (bounded by the TTL window) but it is the only mechanism that is actually reliable. Use both, trust only the TTL for correctness.

Fanout to everyone vs strong consistency, the two things freshers over-engineer

Broadcasting presence to every contact wastes bandwidth for people not even looking at that chat, so filter to active viewers instead. Similarly, presence does not need to be perfectly consistent across every server at every millisecond, a few seconds of lag is invisible to users. Chasing strong consistency here adds cost for a guarantee nobody needs.

14 . HOW TO ANSWER THIS IN INTERVIEW IN 10 MINUTES**10-MINUTE ANSWER STRUCTURE**

Minute 1-2: Scope it down out loud. Say you are covering online/offline and last seen only, not chat delivery or media. State the scale assumption (millions of concurrent users, heartbeat every 25-30 seconds).

Minute 2-4: Split state into two stores. Redis for current online state with TTL, database for durable last_seen and privacy settings. Say clearly that heartbeats never touch the DB.

Minute 4-6: Walk the flow. Connect, authenticate, write Redis keys, publish an online event only on the first device, fanout filtered by privacy. Then disconnect or TTL expiry, last device gone triggers the DB write and offline event.

Minute 6-8: Call out multi-device handling and privacy enforcement at fanout time, not just at the API layer. This is where most answers lose points, so naming it earns them back.

Minute 8-10: Give two or three tradeoffs unprompted, heartbeat interval, TTL vs clean disconnect, fanout scope. Close by saying you would start simple (single Redis instance, basic TTL) and scale to Redis Cluster and sharded events as load grows.

This question rewards candidates who talk about failure before being asked about it. Bring up the unclean disconnect problem yourself in the first two minutes. It signals you have actually thought about how mobile apps behave in the real world, not just how the happy path looks in a diagram.

15 . FINAL INTERVIEW CLOSING LINE**SAY THIS IF YOU ONLY REMEMBER ONE THING**

For WhatsApp-style presence, the main design idea is that current online state should live in Redis with TTL, while durable last_seen and privacy settings stay in the database. Heartbeats refresh Redis only, not the database. A user stays online while at least one device session is alive. Clean disconnects are useful, but heartbeat expiry is the real protection against mobile network drops. Privacy must be checked before returning or pushing presence updates.

Presence looks small on paper. It is one of the better questions to show that you understand the gap between what a network promises and what it actually delivers, and that is exactly the instinct a senior engineer needs day to day.

MODULE 15

Java + Spring Boot System Design Mapping

How every system design component maps to actual Java code, including Controller, Service, Repository, Redis, Kafka, database, external APIs, security, resilience, file storage, scheduled jobs, and observability.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers & 1-3 Years of Experience

Module 15 Overview

01 Spring Boot Layer Map	02 API Gateway
03 Load Balancing + Discovery	04 Redis Caching
05 Kafka Async Processing	06 JPA, Hibernate + MongoDB
07 WebClient + OpenFeign	08 Spring Security + JWT
09 Resilience4j Patterns	10 S3 Presigned URLs
11 Scheduled Jobs	12 Actuator + Metrics
13 Component-to-Code Reference	

WHAT YOU WILL LEARN

- Where Controller, Service, Repository and DTOs fit in the architecture
- How Redis, Kafka, DB and external APIs map to Spring Boot code
- How API Gateway, Eureka and LoadBalancer work together
- How Spring Security, JWT and OAuth2 connect to system design
- How Resilience4j, Actuator, Micrometer and tracing fit into real systems

Here is the thing most freshers miss in interviews. They explain the system beautifully at the architecture level. API Gateway, Load Balancer, Cache, Database, Kafka. All correct. Then the interviewer asks: "So in your Spring Boot service, where exactly does the caching happen?" And they freeze.

System design and actual Java code are not two separate things. Every component you draw on the whiteboard has a real home in your Spring Boot project. This module maps them precisely so you never have to guess.

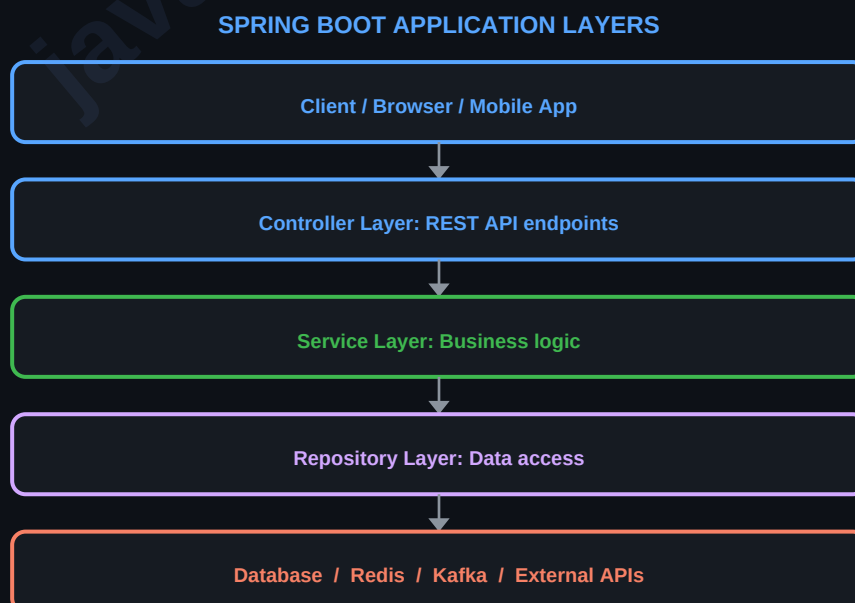
■ INTERVIEW FOCUS

Interviewers at mid-level companies almost always ask "how would you implement this in Spring Boot?" right after the architecture discussion. Knowing the mapping sets you apart.

TOPIC 01

The Spring Boot Layer Map

Every Spring Boot service follows the same layered pattern. This is not just convention. Each layer has a clear responsibility, and system design components map cleanly into these layers.



Controller: Receives HTTP requests, validates input, calls the service, and returns DTOs. Nothing else.

Service: Owns all business logic. Talks to the repository, cache, Kafka, and external APIs.

Repository: Only talks to the database. A JPA interface or a custom query class.

Entity: Your database row mapped as a Java class using JPA annotations.

DTO: The shape of data going to or coming from the client. Never expose your entity directly.

■ WARNING

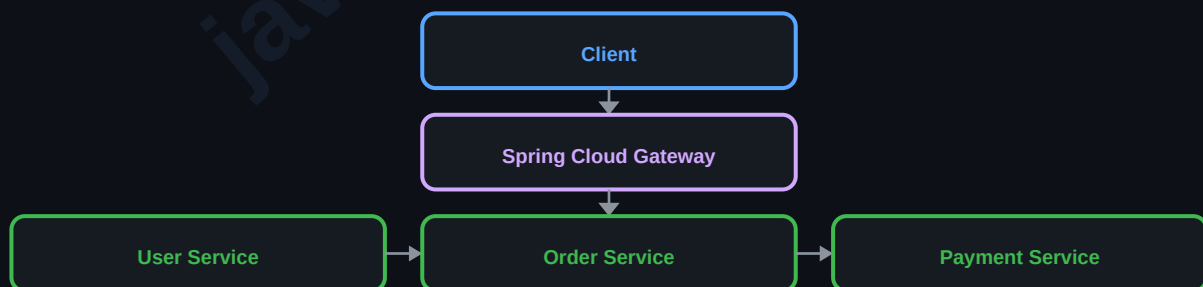
Freshers often write business logic directly in the Controller, or they return the JPA entity straight to the API response. Both are red flags in any code review or interview walkthrough.

TOPIC 02

API Gateway in Spring Boot

In system design, the API Gateway is the single entry point for all client requests. In the Spring ecosystem, this maps to Spring Cloud Gateway.

Request routing flow: Client → Gateway → Microservices



Spring Cloud Gateway lets you route requests, apply rate limiting, handle auth checks, add circuit breakers, and log requests, all before the call even reaches your microservice.

Responsibility	Spring Cloud Gateway Component
Request routing	Route config in application.yml

Rate limiting	Bucket4j + RequestRateLimiterGatewayFilterFactory
Auth token check	Spring Security filter in the Gateway
Circuit breaking	Resilience4j GatewayFilter
Request/response logs	Custom GlobalFilter implementation
Path rewriting	RewritePath filter in route config

■ JAVA MAPPING

"The API Gateway in our design maps to Spring Cloud Gateway. Each microservice registers with Eureka. The gateway uses service discovery to route by logical service name and applies Resilience4j circuit breaking at the gateway level before traffic reaches any service."

TOPIC 03

Load Balancing + Service Discovery

Two things work together here. Service discovery finds where instances are running. Load balancing decides which instance gets the next request.

EUREKA: Service Discovery

- Each service registers on startup
- Gateway queries Eureka for live IPs
- @EnableEurekaClient on each service
- Deregisters cleanly on shutdown
- Heartbeat keeps registry current

SPRING CLOUD LOADBALANCER

- Replaces Netflix Ribbon (deprecated)
- Round-robin by default
- Pluggable custom strategy
- @LoadBalanced on RestTemplate
- Works with WebClient too

```
// Eureka Server:
@SpringBootApplication
@EnableEurekaServer
class EurekaServerApp { }

// Each microservice in application.yml:
spring.application.name=order-service
eureka.client.serviceUrl.defaultZone=http://eureka:8761/eureka

// Load-balanced caller:
@Bean @LoadBalanced
RestTemplate restTemplate() { return new RestTemplate(); }

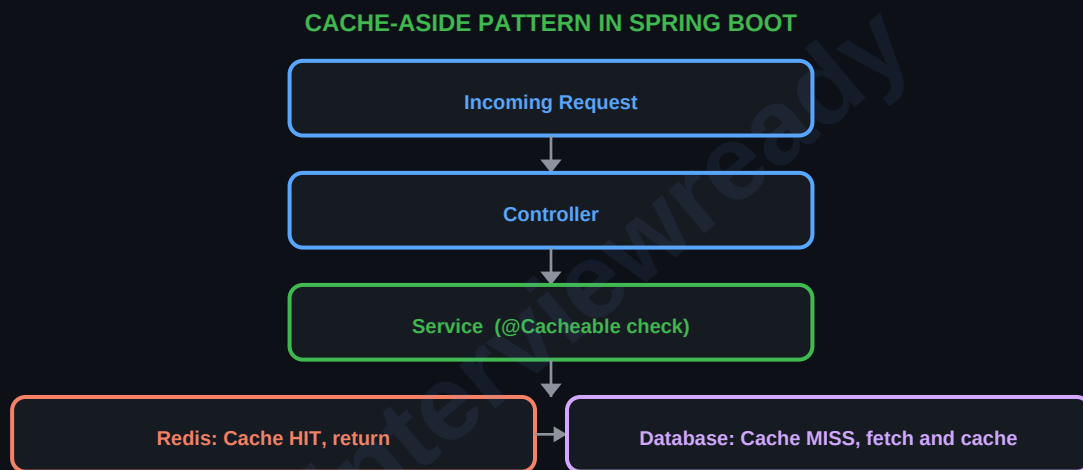
// Call using logical service name: o hardcoded IP:
restTemplate.getForObject("http://order-service/api/orders/{id}", Order.class, id);
```

Spring Cloud LoadBalancer intercepts the call, looks up "order-service" in Eureka, picks an instance using round-robin, and makes the actual HTTP call. You just use the name.

TOPIC 04

Caching with Redis in Spring Boot

Redis in system design covers a lot of ground. Cache, session store, rate limiter counter, distributed lock, leaderboard. In Spring Boot you interact with it through Spring Data Redis or the Spring Cache abstraction.



```
// 1. Enable caching in main class:
@EnableCaching
@SpringBootApplication
class App { }

// 2. Cache on the service method:
@Cacheable(value = "users", key = "#userId")
public UserDTO getUser(Long userId) {
    return userRepository.findById(userId)
        .map(userMapper::toDTO)
        .orElseThrow(...);
}

// 3. Evict on update:
@CacheEvict(value = "users", key = "#userId")
public void updateUser(Long userId, UpdatedDTO dto) { ... }

// 4. Fine-grained Redis control:
@Autowired RedisTemplate<String, Object> redisTemplate;
redisTemplate.opsForValue().set("session:" + token, user, 30, TimeUnit.MINUTES);
```

The `@Cacheable` annotation is what Spring Cache gives you. Under the hood it stores the result in Redis (or any configured cache). First call hits the DB. Every call after that hits Redis until the TTL expires or you evict it.

Use Case	Spring Redis Implementation
Method-level caching	<code>@Cacheable</code> + Spring Cache abstraction
TTL key storage	<code>RedisTemplate.opsForValue().set(key, val, ttl, unit)</code>
Distributed sessions	Spring Session + Redis (spring-session-data-redis)
Rate limit counter	<code>RedisTemplate.opsForValue().increment(key)</code>
Pub/Sub events	<code>RedisMessageListenerContainer</code> + <code>MessageListener</code>
Distributed lock	Redisson (tryLock) or Redis SETNX pattern
Sorted leaderboard	<code>RedisTemplate.opsForZSet().add()</code> / <code>range()</code>
Bloom filter	Redisson <code>RBloomFilter</code> for duplicate detection

TOPIC 05

Async Processing with Kafka in Spring Boot

Kafka shows up in almost every system design that needs async or event-driven processing. Order placed, send notification. File uploaded, kick off processing. Payment done, update inventory. In Spring Boot, this is Spring Kafka.

Event flow: Producer → Kafka Topic → Multiple Consumer Groups




```
// Producer: publish event after saving order:
@Autowired KafkaTemplate<String, OrderEvent> kafkaTemplate;

public OrderResponse placeOrder(OrderRequest req) {
    Order order = orderRepository.save(buildOrder(req));
    kafkaTemplate.send("order.placed",
                      order.getId().toString(),
                      new OrderEvent(order.getId(), order.getUserId()));
    return toResponse(order); // returns immediately, no waiting
}

// Consumer: Notification Service:
@KafkaListener(topics = "order.placed", groupId = "notification-group")
public void onOrderPlaced(OrderEvent event) {
    notificationService.sendEmail(event.getUserId(), "Order confirmed!");
}

// Consumer: Inventory Service (same topic, different group):
@KafkaListener(topics = "order.placed", groupId = "inventory-group")
public void reserveStock(OrderEvent event) {
    inventoryService.reserve(event.getItems());
}

# application.yml
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.auto-offset-reset=earliest
```

◆ TIP

Use a different consumer group ID per service. "notification-group" and "inventory-group" both consume the same topic independently. Each tracks its own offset. One slow consumer never blocks the other.

The key benefit: the Order Service does not wait for the email to go out. It saves the order, fires the event, and returns immediately. Notification and inventory pick it up async.

■ WARNING

Do not use `@Async` alone for critical business events. If the server restarts between the `@Async` task being scheduled and actually running, the event is silently lost. Kafka gives you durability and replay. Use it for anything that must not be dropped.

TOPIC 06

Database Layer: JPA, Hibernate, Queries

In system design you choose SQL or NoSQL. In Spring Boot, SQL databases go through JPA and Hibernate. NoSQL like MongoDB goes through Spring Data MongoDB. The repository pattern looks the same either way, which is intentional.

```
// Entity: maps to a DB table:
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    private LocalDateTime createdAt;
}

// Repository: Spring Data generates queries automatically:
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByUserId(Long userId);

    @Query("SELECT o FROM Order o WHERE o.status = :s AND o.createdAt > :since")
    List<Order> findRecentByStatus(@Param("s") OrderStatus s,
                                   @Param("since") LocalDateTime since);
}
```

SQL: JPA + PostgreSQL

- JpaRepository<Entity, ID>
- @Entity, @Table, @Column
- @OneToMany, @ManyToOne joins
- Hibernate maps entities to tables.
- Use Flyway or Liquibase in production.
- ACID transactions built-in
- @Transactional on service methods

NoSQL: Spring Data MongoDB

- MongoRepository<Document, ID>
- @Document, @Field annotations
- Embedded docs, no joins needed
- Schemaless: evolves freely
- Eventual consistency by default
- Aggregation pipeline for analytics
- Good for flexible or nested data

■ TRADEOFF

Hibernate maps your entities and can generate the schema in development. For production migrations, use Flyway or Liquibase. They give you versioned, repeatable, and reviewable schema changes. Never rely on auto-ddl in production.

◆ TIP

Always put @Transactional on the service method, not on repository methods. If your service calls two repositories in one business flow, they both join the same transaction. If you annotate only the repo, each call is its own transaction and you lose atomicity.

TOPIC 07**External API Calls: WebClient and OpenFeign**

In microservices, services call each other all the time. Order Service needs to call Payment Service. You have three common options in Spring Boot. Each has a different use case.

Option	When to Use	Notes
RestTemplate	Legacy projects, simple sync calls	Deprecated in newer Spring. Avoid for new code.
WebClient	Modern reactive or blocking HTTP	Current standard. Works sync or async.
OpenFeign	Service-to-service in microservices	Cleanest option. Declarative interface.

```
// OpenFeign: leanest for microservice calls:
@FeignClient(name = "payment-service", fallback = PaymentFallback.class)
public interface PaymentClient {
    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest req);
}

// Use it like a local service bean:
@Autowired PaymentClient paymentClient;
PaymentResponse result = paymentClient.processPayment(request);

// Fallback when payment service is down:
@Component
public class PaymentFallback implements PaymentClient {
    public PaymentResponse processPayment(PaymentRequest req) {
        return PaymentResponse.pending("queued for retry");
    }
}

// WebClient: or reactive or mixed flows:
webClient.post().uri("/api/payments")
    .bodyValue(request)
    .retrieve()
    .bodyToMono(PaymentResponse.class)
    .block(); // blocking call in non-reactive context
```

■ INTERVIEW FOCUS

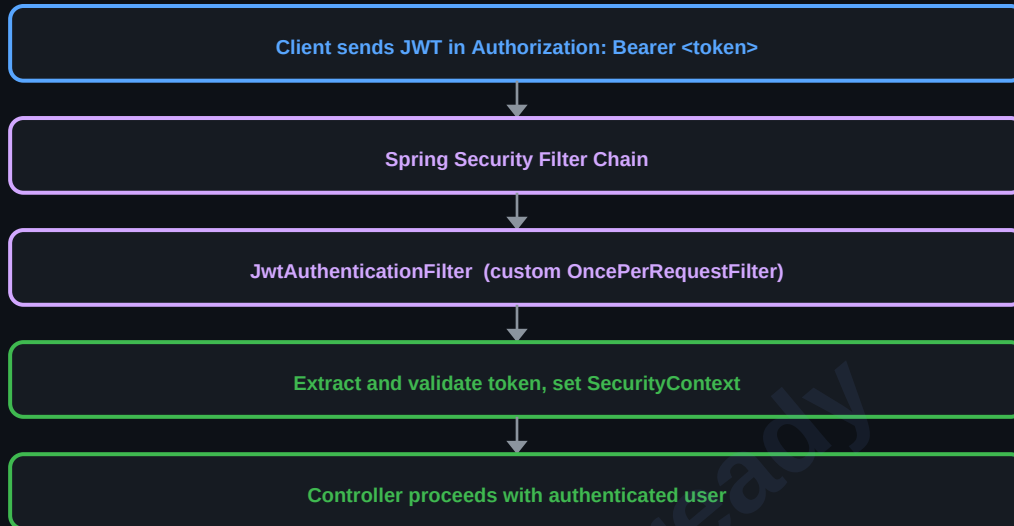
"For inter-service calls we use OpenFeign, which is declarative and integrates cleanly with Eureka and Resilience4j. For flows where we do not need a synchronous response, like sending a notification after an order, we publish to Kafka instead of making a direct HTTP call."

TOPIC 08

Authentication: Spring Security, JWT and OAuth2

In system design you say "JWT for stateless auth" or "OAuth2 for third-party login". Here is exactly where that lives in the Spring Boot code.

JWT AUTH FLOW IN SPRING SECURITY



```

@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res,
                                    FilterChain chain) throws ... {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtUtil.validate(token)) {
                String username = jwtUtil.extractUsername(token);
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        username, null, jwtUtil.extractRoles(token));
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}

// SecurityConfig:
http.authorizeHttpRequests()
    .requestMatchers("/api/auth/**").permitAll()
    .anyRequest().authenticated();
http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
  
```

💡 TIP

In many microservice systems, the API Gateway validates JWTs before routing traffic. Internal services may trust the gateway inside a private network, but high-security systems can also validate tokens again at the service level. Store JWT secrets in environment variables or Spring Cloud Config. Never hardcode them in yml.

TOPIC 09

Resilience: Circuit Breaker, Retry and Rate Limiting

Distributed systems fail. Services go slow. External APIs time out. Resilience4j is the standard library in Spring Boot for handling this gracefully.

CIRCUIT BREAKER

- `@CircuitBreaker(name="paymentSvc")`
- CLOSED: normal traffic flows
- OPEN: fails fast on threshold
- HALF_OPEN: tests recovery
- Prevents cascade failures
- `fallbackMethod` handles degradation

RETRY + RATE LIMITER

- `@Retry(name="svc", fallback=...)`
- Retries on transient failures
- Exponential backoff config
- `@RateLimiter` limits calls/second
- Protects downstream services
- `Bucket4j` for API-level limiting

```
// Combine circuit breaker + retry on same method:
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService")
public PaymentResponse processPayment(PaymentRequest req) {
    return paymentClient.process(req); // OpenFeign call
}

public PaymentResponse paymentFallback(PaymentRequest req, Exception ex) {
    kafkaTemplate.send("payment.retry.queue", req);
    return PaymentResponse.pending("queued for retry");
}

# application.yml:
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
  retry:
    instances:
      paymentService:
        maxAttempts: 3
        waitDuration: 500ms
```

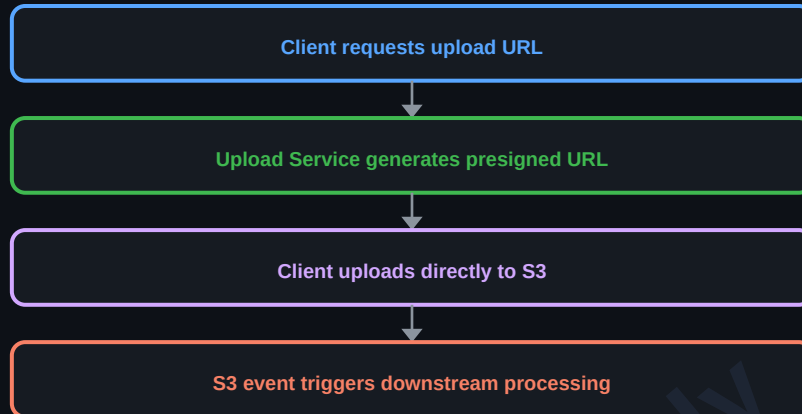
■ INTERVIEW MOMENT

"If the payment service is down, the circuit breaker opens after hitting the failure threshold. New calls hit the fallback immediately instead of waiting for timeouts. We queue them in Kafka for retry. Once the service recovers and the circuit goes HALF_OPEN, it allows test traffic through before fully closing again."

TOPIC 10

File Storage: S3 and Presigned URLs in Spring Boot

In design questions like "design a file upload system" or "design YouTube upload", you say "store files in S3". Here is how that actually maps to Java code.

PRESIGNED URL UPLOAD FLOW

```
@Service
public class S3Service {
    private final S3Client s3Client;
    private final S3Presigner presigner;

    // Generate presigned URL: Client uploads directly to S3:
    public String generateUploadUrl(String key) {
        PutObjectPresignRequest req = PutObjectPresignRequest.builder()
            .signatureDuration(Duration.ofMinutes(15))
            .putObjectRequest(r -> r
                .bucket("my-bucket")
                .key(key)
                .build())
            .build();
        return presigner.presignPutObject(req).url().toString();
    }

    // Generate presigned download URL:
    public String generateDownloadUrl(String key) {
        GetObjectPresignRequest req = GetObjectPresignRequest.builder()
            .signatureDuration(Duration.ofMinutes(60))
            .getObjectRequest(r -> r.bucket("my-bucket").key(key))
            .build();
        return presigner.presignGetObject(req).url().toString();
    }
}
```


Presigned URL pattern: client asks your service for an upload URL. Your service generates a time-limited URL from S3. Client uploads directly to S3. Your server never touches the file bytes. Way less server load, no file size limits on your API layer.

TOPIC 11

Scheduled Jobs: Spring Scheduler and Quartz

Any system that needs daily reports, cleanup jobs, or retry queues uses scheduled tasks. Spring Boot makes this easy with `@Scheduled`. The catch is multi-instance deployments.

```
@EnableScheduling
@SpringBootApplication
class App { }

@Component
public class OrderCleanupJob {

    // Cron: every day at 2 AM
    @Scheduled(cron = "0 0 2 * * ?")
    @SchedulerLock(name = "cancelAbandonedOrders", lockAtMostFor = "PT10M")
    public void cancelAbandonedOrders() {
        orderService.cancelOrdersOlderThan(Duration.ofHours(24));
    }

    // Every 30 seconds
    @Scheduled(fixedDelay = 30_000)
    @SchedulerLock(name = "retryFailedNotifications", lockAtMostFor = "PT1M")
    public void retryFailedNotifications() {
        notificationService.retryPending();
    }
}

// ShedLock config: prevents duplicate runs across pods:
@Bean
public LockProvider lockProvider(DataSource dataSource) {
    return new JdbcTemplateLockProvider(dataSource);
}
```

■ WARNING

@Scheduled runs on every pod in a multi-instance deployment. Without a distributed lock, your 3-pod setup will run the same job 3 times simultaneously. Use ShedLock (backed by DB or Redis) or Quartz with a clustered JobStore to ensure the job runs exactly once across all instances.

TOPIC 12

Observability: Actuator, Metrics and Tracing

This comes up less often in fresher interviews but you should know the basics. In system design you say "monitor the system". In Spring Boot, this is how it works.

Need	Spring Boot Tool
Health check endpoints	Spring Boot Actuator (/actuator/health)
Metrics collection	Micrometer (auto-configured with Actuator)
Metrics storage + dashboards	Prometheus + Grafana
Distributed tracing	Micrometer Tracing + Zipkin or Jaeger
Log correlation	MDC (Mapped Diagnostic Context) + trace IDs
Error tracking	Sentry or custom exception handler + alerting

```
# application.yml: xpose actuator and metrics:
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, prometheus
  tracing:
    sampling:
      probability: 1.0 # 100% in dev, lower in prod

# Custom counter metric in service:
@Autowired MeterRegistry meterRegistry;

public void placeOrder(OrderRequest req) {
    meterRegistry.counter("orders.placed",
        "status", "success").increment();
    // ... rest of logic
}
```

TOPIC 13

Complete Component-to-Code Reference

Every system design component you draw on the whiteboard, mapped to its Spring Boot home. Use this as your quick-reference before interviews.

System Design Component	Spring Boot / Java Implementation
API Gateway	Spring Cloud Gateway
Load Balancer	Spring Cloud LoadBalancer + Eureka
Service Registry	Eureka Server (@EnableEurekaServer)
REST API	@RestController + @RequestMapping
Input Validation	@Valid + javax/jakarta ConstraintValidator
Business Logic	@Service layer
Database Access	JpaRepository / MongoRepository
SQL Database	Spring Data JPA + Hibernate + HikariCP pool

NoSQL Database	Spring Data MongoDB / Cassandra
In-memory Cache	@Cacheable + Spring Cache + Redis
Session Store	Spring Session + Redis
Async Event Queue	KafkaTemplate.send() + @KafkaListener
Circuit Breaker	Resilience4j @CircuitBreaker
Retry Logic	Resilience4j @Retry with backoff
API Rate Limiting	Bucket4j + Spring Cloud Gateway filter
Authentication	Spring Security + JWT filter + OAuth2
Authorization	@PreAuthorize, @Secured, method-level security
Service-to-service calls	OpenFeign (@FeignClient) / WebClient
File / Media Storage	AWS SDK S3Client + S3Presigner
Scheduled Tasks	@Scheduled + ShedLock for cluster safety
Distributed Lock	Redisson tryLock() or ShedLock + Redis/DB
Config Management	Spring Cloud Config Server + Config Client
Health Checks	Spring Boot Actuator /actuator/health
Metrics	Micrometer + Prometheus + Grafana dashboard
Distributed Tracing	Micrometer Tracing + Zipkin / Jaeger
Full-text Search	Elasticsearch + Spring Data Elasticsearch

Module 15 Quick Revision Cheat Sheet

System Design Component	Spring Boot Mapping
API Gateway	Spring Cloud Gateway: routing, auth, rate limit
Load Balancer	Spring Cloud LoadBalancer: round-robin by default
Service Registry	Eureka Server: services register and discover each other
REST API	@RestController + @GetMapping / @PostMapping
Controller Layer	Handles HTTP in/out. No business logic here.
Service Layer	@Service: owns all business logic and coordination

Repository Layer	JpaRepository or MongoRepository: DB access only
SQL Database	Spring Data JPA + Hibernate + PostgreSQL/MySQL
NoSQL Database	Spring Data MongoDB / Cassandra / DynamoDB
Redis Cache	@Cacheable + RedisTemplate: cache-aside pattern
Kafka	KafkaTemplate (produce) + @KafkaListener (consume)
External API Calls	OpenFeign for service-to-service, WebClient for reactive
JWT Authentication	Spring Security + JwtAuthFilter + SecurityContext
Circuit Breaker	Resilience4j @CircuitBreaker with fallbackMethod
File Storage	AWS S3Client + S3Presigner for presigned URLs
Scheduled Jobs	@Scheduled + ShedLock for safe cluster execution
Observability	Actuator + Micrometer + Prometheus + Zipkin tracing

■ INTERVIEW MOMENT

When an interviewer asks "how would you implement X in Spring Boot?" you now have an answer for every component. The mapping is not just for interviews. Every project you build will use these same connections. The faster this becomes muscle memory, the better your system design conversations will flow.

Next: Module 16 gives the final system design cheat sheet, interview tips, quick component reference, red flags, and 30-minute pre-interview revision checklist.

MODULE 16

System Design Cheat Sheet + Interview Tips

Final 30-minute revision module. One-page visual summaries, the 8-step answer framework, what to say in interviews, what to avoid, phrase cards you can use directly, and a complete component reference sheet.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers and 1-3 Years of Experience

You have about 30 minutes before the interview. This module is for that exact moment. No long explanations. No new concepts. Just the things you need to remember, in the order you need them.

Go through each topic quickly. If something feels shaky, spend two extra minutes on it. Then close the PDF, take a breath, and trust the work you have already done.

TOPIC 01

The 8-Step Answer Framework

Every system design answer benefits from a clear structure. Not a rigid script, but a thinking sequence. This is what interviewers at good companies want to see.

01**CLARIFY**

Ask what we are building, who uses it, and what scale we expect.

02**REQUIREMENTS**

List functional requirements first. Then non-functional ones.

03**ESTIMATE SCALE**

Daily active users, requests per second, storage. Keep it rough and fast.

04**API DESIGN**

Key endpoints, HTTP methods, request and response shapes.

05**DATABASE CHOICE**

SQL or NoSQL. Sketch the schema. Justify your choice.

06**HIGH-LEVEL DESIGN**

Draw the main components and show the primary request flow.

07**SCALE IT UP**

Add caching, async queues, sharding, replication where the design needs it.

08**TRADEOFFS**

Explain what you chose and what you gave up. This is what seniors do.

INTERVIEW FOCUS

Start every answer with: "Before I start drawing, let me clarify a few things." That one sentence signals process thinking, and interviewers notice it immediately.

TOPIC 02

Interview Phrases That Actually Work

These are not scripts. They are ideas expressed the way experienced engineers talk. Read them once, internalize the pattern, and say them in your own words.

STARTING OUT

OPENING	Before I start designing, let me clarify a few requirements.
READ/WRITE	Is this system more read-heavy or write-heavy?
USERS	What is the expected number of daily active users?
LATENCY	Do we need real-time responses or can some operations be delayed?

EXPLAINING DESIGN

READ HEAVY	Since this is read-heavy, I would add a cache in front of the database.
ASYNC	Instead of a direct API call here, I would publish to Kafka so the main flow does not block.
GATEWAY	The API Gateway centralizes auth and routing before traffic reaches any service.
LB	Multiple service instances sit behind a load balancer to distribute traffic.

DATABASE

SQL PICK	For structured relational data with transactions, I would go with PostgreSQL.
----------	---

NOSQL PICK	Since we need flexible schema and high write throughput, Cassandra fits better.
CACHE	We can cache the top 1000 most-read items in Redis to cut database reads by 80 percent.
REPLICAS	With a 10-to-1 read-write ratio, read replicas make sense here.

SCALE

ESTIMATE	At 1 million DAU with 10 requests per user per day, we get roughly 120 RPS average.
HORIZONTAL	We scale the API layer horizontally. The load balancer distributes across instances.
SHARD	Database sharding by user ID gives us linear write scalability as we grow.
CDN	A CDN handles static assets and reduces latency for users in different regions.

TRADEOFFS

CAP	The tradeoff here is between consistency and availability. For this use case I choose availability.
KAFKA COST	Kafka adds complexity, but it decouples services and gives us durability and replay.
CACHE RISK	Caching improves read speed but we need a clear invalidation strategy.
MONOLITH	I would start with a monolith and split into microservices as the team or traffic demands it.

TOPIC 03

Fresher vs Senior: Same Question, Different Answer

Most of the time the knowledge gap is not that big. The communication gap is. Here is what that looks like.

X I will use MySQL.	V The read-write ratio is 10 to 1, so I would use PostgreSQL with read replicas and Redis for hot data.
X I will add a cache.	V I would apply cache-aside with a 5-minute TTL for user profiles. Cache key: user:{id}.
X The system can scale.	V The API layer scales horizontally. DB reads go to replicas. Heavy writes batch through Kafka.
X Kafka is for messaging.	V Kafka gives us durability, replay, and fan-out. Multiple consumer groups process the same event independently.
X Use microservices.	V For this scale I would start with a monolith and extract services where traffic or team size demands it.
X I will use NoSQL.	V The data is document-shaped and schema evolves quickly, so MongoDB is the right fit here.

TOPIC 04

Red Flags That Kill Interviews

These are patterns interviewers see constantly. Every one of them is fixable with a small habit change.

- X** **Jumping straight to drawing**
You skip requirements. The interviewer sees no process. Always clarify first.
- X** **"It depends" and then nothing**
"It depends on the use case" must be followed by stating which case you are assuming.
- X** **Over-engineering from the start**
Kafka, Redis, Kubernetes for 1000 users is a red flag. Know when simple is correct.

- X Ignoring failure scenarios**
Any senior will ask "what happens when X fails?" Have an answer for DB, Redis, and Kafka.
- X Buzzwords without depth**
"We use microservices" without knowing when, why, or what the tradeoffs are.
- X No scale reasoning**
"We add a load balancer" without explaining when it becomes necessary or what it solves.
- X Skipping tradeoffs**
Every design decision has a cost. Not mentioning it looks like shallow thinking.
- X Not clarifying ambiguity**
Designing for 1M users when they meant 10K wastes everyone's time.

TOPIC 05

Scaling Decision Map

When traffic or data hits a wall, the fix usually fits one of these patterns. Know them by instinct so you can reach for the right one in the interview.

PROBLEM VS SOLUTION (1 OF 2)

Slow reads	→ Add Redis cache with cache-aside pattern
High write load	→ Kafka queue + async consumer workers
DB becoming bottleneck	→ Read replicas + HikariCP connection pooling
Single DB too large	→ Shard by user_id or region
Service under traffic	→ Horizontal scale + load balancer
Slow full-text search	→ Elasticsearch as secondary index

PROBLEM VS SOLUTION (2 OF 2)

Global users, high latency	→ CDN for static, edge caching
Spiky traffic bursts	→ Auto-scaling + Kafka as traffic buffer
Cascade failures	→ Resilience4j circuit breaker
Duplicate requests	→ Idempotency key in DB or Redis
Large file uploads	→ Presigned S3 URLs, bypass server
Notification at scale	→ Kafka fan-out to per-channel consumers

TOPIC 06

CAP Theorem: One-Minute Version

You will hear "CAP theorem" in almost every distributed systems conversation. Network partitions happen in distributed systems eventually. So the real choice in practice is between CP and AP.

Choice	What You Get	Real Examples
CP	No stale reads. May be unavailable during a partition.	HBase, ZooKeeper, Redis Cluster
AP	Always responds. Data might be slightly stale.	Cassandra, DynamoDB, CouchDB
CA	Only works without network partitions. Not realistic.	Single-node PostgreSQL only

INTERVIEW MOMENT

"In a distributed system, partitions always happen eventually. So the real choice is CP vs AP. For our payment service we choose CP because stale data means wrong balances. For our activity feed we choose AP because slightly stale content is acceptable."

TOPIC 07

When to Use What: Component Quick Reference

You should be able to make these decisions quickly in an interview. If the "when to skip" column surprises you, spend a minute on that row.

Component	Use When	Skip When
Redis Cache	Read-heavy, hot data, latency-sensitive	Data changes constantly, strong consistency needed
Kafka	Async processing, fan-out, event durability	Simple one-to-one sync call is enough
PostgreSQL	Relational data, ACID transactions	Unstructured data, massive write scale
MongoDB	Flexible schema, document data, rapid iteration	Complex joins or strong consistency required
Cassandra	Massive write scale, time-series, distributed	Complex queries, low-latency reads only
Elasticsearch	Full-text search, log aggregation	Primary data store. It is a secondary index.
CDN	Static assets, global users, high bandwidth	Dynamic personalised content, private data
Load Balancer	Multiple service instances, traffic distribution	Single-instance small apps
API Gateway	Multiple services, centralised auth, routing	Single service, internal-only APIs
Microservices	Large teams, independent deployments, per-service scale	Small teams, early stage, simple apps
Kafka vs @Async	Critical events that must not be lost	@Async fine for non-critical background tasks

TOPIC 08

Numbers Worth Knowing

You do not need to memorise every figure. But rough numbers let you do scale estimation confidently and make you sound like someone who has actually thought about this in production.

What	Rough Number	Why It Matters
1M DAU x 10 req/user/day	~120 RPS average	Baseline for "medium scale" systems
Peak traffic multiplier	2x to 5x of average	Design for peak, not average
Redis read latency	Under 1ms	Cache is fast. DB is not.
PostgreSQL write limit (single)	~5,000 TPS	Beyond this: sharding or replicas

Kafka throughput	Millions of msg/sec	Async fan-out at real scale
Object storage cost	Low per GB, varies by region	Good for files and media. Check current cloud pricing.
CDN cache hit target	80 to 95 percent	Lower means CDN costs without benefit
JWT access token expiry	15 minutes typical	Balance security vs user friction
JWT refresh token expiry	7 days typical	Long enough to avoid constant re-login
99.9 percent uptime	~8.7 hours downtime/yr	Three nines SLA means this gap
99.99 percent uptime	~52 minutes downtime/yr	Four nines needs real HA design

TOPIC 09

SQL vs NoSQL: The Decision in Under 30 Seconds

The question is not which is better. The question is which fits the access pattern. If you cannot explain the access pattern, you cannot justify the choice.

Signal in the Requirements	Pick SQL	Pick NoSQL
Data has relationships	Yes	No
Need ACID transactions	Yes	Depends on DB
Schema is stable and known	Yes	No
Complex joins needed	Yes	No
Schema changes often	No	Yes
Massive horizontal write scale	No	Yes
Flexible or nested document data	No	Yes
Financial or compliance data	Yes	Rarely

TIP

Never just say "I will use MongoDB" or "I will use MySQL". Always follow it with: because the data is [structured/document-shaped], we need [transactions/schema flexibility], and the access pattern is [joins/key lookup].

TOPIC 10

Sync vs Async: When to Use Which

Most freshers default to direct API calls for everything. The real skill is recognising when a synchronous response is not actually needed.

Use Synchronous (REST / gRPC)	Use Asynchronous (Kafka / MQ)
Client needs the result immediately	Fire-and-forget is acceptable
Payment confirmation response	Sending notification after order
User login and token return	Updating inventory after purchase
Real-time order status check	Generating and emailing a report
Simple service-to-service query	Audit logging and event tracking
OpenFeign or WebClient call	KafkaTemplate.send()

TOPIC 11

Failure Scenarios: Quick Answers

Interviewers always ask "what happens when X fails?" Have a confident answer for each of these before you walk in.

What Fails	Impact	How to Handle
Redis goes down	All reads hit DB	Fall through to DB. Redis restarts fast. Alert.
Kafka consumer lag	Async processing delayed	Scale consumers, check offsets, alert on lag.
DB primary fails	Write outage	Failover to replica. RDS Multi-AZ auto-fails over.
Payment API fails	Orders stuck in PENDING	Circuit breaker, Kafka retry queue, idempotency key.

Service instance crash	Some requests fail	Load balancer routes to healthy instances.
Duplicate request	Duplicate order or payment	Check idempotency key in DB or Redis before processing.
File upload fails midway	Partial file in S3	S3 multipart upload, lifecycle cleanup rule.
Gateway overload	Gateway becomes a bottleneck	Rate limit at gateway, horizontal scale of gateway.

TOPIC 12

Spring Boot Mapping: Quick Reference

Every component you draw on the whiteboard lives somewhere in your Spring Boot project. Know this table cold before the interview.

System Design Component	Spring Boot Implementation
API Gateway	Spring Cloud Gateway
Load Balancer	Spring Cloud LoadBalancer + Eureka
Service Registry	Eureka Server (@EnableEurekaServer)
REST API Endpoint	@RestController + @GetMapping / @PostMapping
Business Logic	@Service layer, no logic in Controller
Database Access	JpaRepository or MongoRepository
SQL Database	Spring Data JPA + Hibernate + HikariCP
Redis Cache	@Cacheable + Spring Cache + RedisTemplate
Session Store	Spring Session + Redis
Kafka Producer	KafkaTemplate.send(topic, key, event)
Kafka Consumer	@KafkaListener(topics, groupId)
Circuit Breaker	Resilience4j @CircuitBreaker + fallbackMethod
Retry Logic	Resilience4j @Retry with exponential backoff
Rate Limiting	Bucket4j + Spring Cloud Gateway filter

JWT Authentication	Spring Security + custom JwtAuthFilter
Service-to-Service Call	OpenFeign @FeignClient or WebClient
File Storage	AWS SDK S3Client + S3Presigner
Scheduled Job	@Scheduled + ShedLock for cluster safety
Observability	Actuator + Micrometer + Prometheus + Zipkin

TOPIC 13

30-Minute Pre-Interview Revision Checklist

Go through this list. Each item should take under 2 minutes. If any item feels weak, spend the time on it now.

ARCHITECTURE BASICS

- ✓ Client, API Gateway, Load Balancer, Service, Cache, DB flow
- ✓ Controller, Service, Repository layers in Spring Boot
- ✓ What an API Gateway does differently from a Load Balancer

DATABASES

- ✓ SQL vs NoSQL: when to use which, and why
- ✓ Read replicas for read-heavy, sharding for write scale
- ✓ Redis: cache, session, rate limit counter, distributed lock

ASYNC AND EVENTS

- ✓ Kafka: producer publishes, consumer group subscribes per service
- ✓ Different consumer group IDs let multiple services consume one topic
- ✓ Use Kafka when you do not need an immediate response or need fan-out

RESILIENCE

- ✓ Circuit breaker states: CLOSED, OPEN, HALF_OPEN

- ✓ Retry with exponential backoff for transient failures
- ✓ Fallback method returns a safe default or queues for later

SECURITY

- ✓ JWT: short-lived access token plus long-lived refresh token
- ✓ Validate JWT at the API Gateway. Internal services may trust the gateway inside a private network, but high-security systems can validate again at service level.
- ✓ Spring Security filter chain runs before Controller

SCALE SIGNALS

- ✓ Read-heavy: Redis cache
- ✓ Write-heavy: Kafka async queue
- ✓ Global users: CDN
- ✓ DB bottleneck: read replicas or sharding
- ✓ Traffic spikes: auto-scale plus queue as buffer

INTERVIEW PROCESS

- ✓ Clarify requirements before drawing anything
- ✓ Estimate scale before picking components
- ✓ Start simple, then layer on scale components
- ✓ Mention tradeoffs for every major decision
- ✓ Have failure answers ready: Redis down, Kafka lag, DB failover

INTERVIEW MOMENT

You have done the work. Trust it. In the interview: think out loud, ask questions before you draw, and explain your reasoning as you go. The interviewer wants to see how you think, not just what you know. Good luck.

This is the final module of The System Design Interview Playbook. Everything you need to answer system design questions with confidence is in this guide.

BONUS SECTION

System Design Interview Answer Templates

Ready-to-use language for starting your answer, drawing your diagram, explaining your database choice, and defending your tradeoffs out loud.

Part of The System Design Interview Playbook

For Java Backend Developers, Freshers & 1-3 Years of Experience

javainterviewready

BONUS SECTION OVERVIEW

What This Section Covers

Knowing system design concepts and being able to talk through them out loud under interview pressure are two different skills. This section gives you the actual sentences experienced engineers use to start an answer, draw a diagram out loud, justify a database choice, defend a scaling plan, and handle a follow-up question without freezing.

Topic Roadmap

01	How to Start Your Answer
02	How to Draw the Diagram Out Loud
03	How to Explain Your Database Choice
04	How to Explain Your Scaling Approach
05	How to Discuss Tradeoffs
06	Handling Follow-Up Questions
07	Common Mistakes Freshers Make

WHAT YOU WILL LEARN

- Exact phrases to open a system design answer with confidence
- How to narrate a diagram while you draw it
- Ready language for justifying SQL vs NoSQL out loud
- How to explain scaling decisions without rambling
- A simple structure for discussing any tradeoff
- How to answer follow-up questions without panicking
- The mistakes that quietly cost freshers the interview

TOPIC 1

How to Start Your Answer

Most interviews are lost in the first ninety seconds. The interviewer says "design a URL shortener" and the candidate immediately grabs a marker and starts drawing boxes. No questions asked, no scope clarified, no scale discussed. Five minutes in, the interviewer has to interrupt and redirect the whole conversation, and that costs you more than the slow start ever would have.

What interviewers are actually checking for in the opening minute is not your system design knowledge. It is whether you treat an ambiguous problem the way a real engineer would: by asking questions before committing to an architecture. A senior engineer who jumps straight to drawing without clarifying scope looks just as inexperienced as a fresher doing the same thing.

Silence after the question is fine. Drawing before you understand the problem is not. Interviewers read a pause as thinking. They read a rushed diagram as a candidate who has memorized one answer and is pattern-matching it onto whatever gets asked.

Opening Lines That Actually Work

You do not need a clever opening. You need a calm, structured one. Here is language that works almost every time, regardless of which system gets asked.

```
"Before I start designing, I want to clarify a few things about  
scope and scale, so I do not design for the wrong problem."
```

```
"Can I assume this is for a single region to start with, and we  
can talk about multi-region later if it comes up?"
```

```
"Roughly how many users are we expecting, and is this more  
read-heavy or write-heavy?"
```

Notice none of these lines mention a single technology. That is deliberate. At this stage you are not designing anything yet, you are scoping the problem. Many freshers skip this and try to sound smart early by naming Redis or Kafka in the first thirty seconds. It usually backfires, because the interviewer then asks why, and you do not have a scale-based reason yet.

Once scope is roughly set, move into requirements. This is where you separate functional requirements (what the system does) from non-functional requirements (how well it needs to do it). Most candidates either skip non-functional requirements entirely, or list them without connecting them to any design decision later.

Read-heavy system	→	Mention caching early, design around it
Write-heavy system	→	Mention queues and async processing early
Strict consistency needed	→	Flag this before picking a database
High availability needed	→	Mention redundancy before drawing

The reason this matters: every answer you give later in the interview should trace back to something you said in this first minute. If you said the system is read-heavy, and then twenty minutes later you have not mentioned caching once, the interviewer notices the gap. Setting these flags early gives your whole answer a thread that ties together.

A Simple Opening Framework



At my level, I would not spend more than two or three minutes here. This is not the part of the interview where depth wins you points. It is the part where a missing step costs you points. Get through it cleanly, then move into the actual design.

TOPIC 2

How to Draw the Diagram Out Loud

Here is a scenario that plays out constantly in mock interviews. A candidate draws a perfectly correct architecture diagram in total silence, then turns around and says "so that's the design." The interviewer has been watching boxes and arrows appear for four minutes with no narration, and now has no idea why any of those choices were made.

A diagram on its own does not prove you understand the system. Narrating it while you draw is what proves that. The interviewer is not grading your diagram, they are grading your thinking, and the only way they can see your thinking is if you say it out loud as your hand moves.

A useful habit: draw one component, say what it does and why it is there, then draw the next one. Do not draw the whole diagram and explain it afterward. By the time you finish drawing, the interviewer has already formed half their opinion of your answer based on silence.

TIP

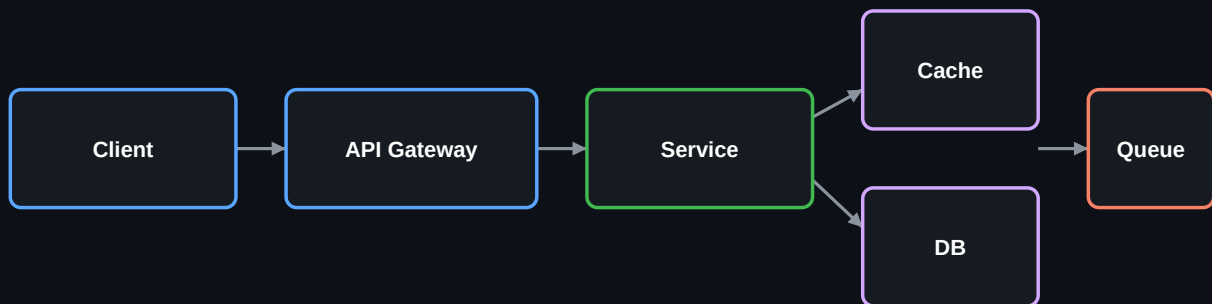
Draw left to right, following the direction a request actually flows: client, then gateway, then service, then database. This mirrors how the interviewer is mentally tracing the request, so your diagram reads naturally instead of looking like a random collection of boxes.

Narration Language for Each Component

COMPONENT	WHAT TO SAY WHILE DRAWING IT
Client	"This is the client, could be a mobile app or web app."
API Gateway	"All requests come through a gateway, this handles routing and auth."
Load Balancer	"This distributes traffic across multiple instances so no single one is overloaded."
Service	"This is the core service that handles the actual business logic."
Cache	"Since this is read-heavy, I am adding a cache here to reduce database load."
Database	"I am picking [SQL/NoSQL] here because..."
Queue	"This decouples the slow part of the flow so the main request does not wait on it."

Notice the cache and database rows both end with "because." That word is doing a lot of work. Anytime you add a component, finish the sentence with a reason tied back to something from your requirements gathering. A diagram full of boxes with no reasons attached looks memorized. A diagram where every box has a one-sentence justification looks reasoned.

DRAW IN THIS ORDER, NARRATING EACH STEP



This left-to-right order is not the only correct way to draw a system, but it gives you a default you can fall back on when the pressure of the room makes it hard to think about layout. Following the request path keeps your narration in sync with your drawing.

TOPIC 3

How to Explain Your Database Choice

"I would use MongoDB because it is flexible." This is probably the single most common answer interviewers hear when they ask why a candidate picked a particular database, and it is also the answer that gets the least credit. Flexible is not a reason, it is a word that sounds like a reason.

A database choice that actually earns points connects back to something specific about the system: the shape of the data, how it gets queried, whether relationships matter, and what consistency guarantee the system actually needs. None of those things are about which database is newer or more popular.

WATCH OUT

If your reason for picking a database does not change when the use case changes, it is not a real reason. "It is scalable" applies to almost every modern database. Push yourself to name something specific to this system's data and access pattern.

Language for SQL

```
"I am going with a relational database here because orders,
users, and payments have clear relationships, and I need
transactions to guarantee an order and a payment are
created together or not at all."
```

```
"The schema is well defined and unlikely to change often,
so I do not need the flexibility a document store gives me."
```

Language for NoSQL

```
"This data is read by ID almost every time, with no joins,
so a key-value or document store gives me faster reads
without the overhead of relational lookups."
```

```
"Write volume is very high and the schema differs slightly
between records, so I would rather not force a rigid table
structure on this."
```

Many beginners pick a database first and then try to justify it. That order is backwards, and interviewers can usually tell. Start from the data and the access pattern, and let the database choice fall out of that naturally. If you genuinely are not sure, saying so honestly works better than guessing confidently.

```
"I could see this working with either a relational database
with proper indexing, or a document store if the access
pattern stays simple. Given the relationships between
entities here, I would lean SQL, but I am open to the
tradeoff if write volume becomes the bigger concern."
```

That last line matters. It shows you can hold two options in your head at once and pick based on a real signal, rather than defending a choice you made too early. Interviewers like seeing that more than they like seeing a confident wrong answer.

PICK SQL WHEN

- Relationships between entities matter
- You need transactions across multiple writes
- Schema is stable and well understood
- Strong consistency is a hard requirement

PICK NoSQL WHEN

- Access pattern is simple, mostly by key
- Write volume is very high
- Schema varies between records
- Horizontal scaling matters more than joins

TOPIC 4

How to Explain Your Scaling Approach

A common interviewer move: you finish your initial design, and they say "now this needs to handle ten times the traffic, what changes?" This question is not really about scaling. It is testing whether you designed something rigid or something that can evolve.

The trap most freshers fall into here is naming every scaling technique they know at once: add a cache, add a queue, shard the database, add a CDN, all in one breath. That sounds like a list you memorized, not a plan. A stronger answer picks the one or two bottlenecks that actually exist in the design you just drew, and fixes those first.

"At my level, I would first explain the simple design and then improve it step by step." This single sentence, said early when you present your first diagram, buys you room to start simple without the interviewer assuming you do not know how to scale it.

A Structure for Answering Scaling Questions



That last step, stating the new tradeoff, is the part almost everyone skips. Every scaling fix introduces a new problem. Caching introduces staleness. Sharding introduces cross-shard query complexity. Saying this out loud is what separates someone who has actually operated a scaled system from someone reciting a checklist.

"The database read load is the first thing that breaks at this scale, since every request hits it directly. I would add a cache in front of it for the hot read paths. The tradeoff is that cached data can be slightly stale, which is acceptable here since this is not financial data."

If the interviewer pushes further and asks what happens at a hundred times the original traffic, do not panic and start guessing. Walk the same structure again: what breaks next, why, what fixes it, what tradeoff comes with the fix. The structure is reusable at any scale, which is exactly why it is worth memorizing instead of memorizing answers for specific numbers.

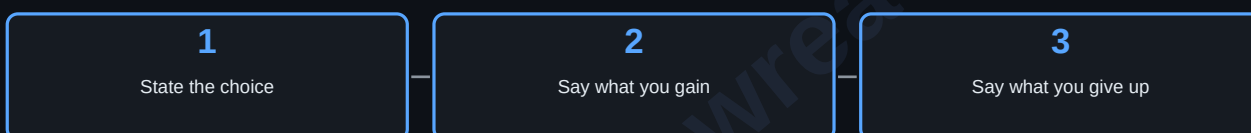
TOPIC 5

How to Discuss Tradeoffs

If there is one skill that separates a strong system design answer from a weak one, it is this: weak answers present a choice as obviously correct. Strong answers present a choice as the better option given specific constraints, while acknowledging what you give up to get it.

Every real decision in system design is a tradeoff. There is no universally correct answer between SQL and NoSQL, sync and async, or strong and eventual consistency. The interviewer already knows this. What they are testing is whether you know it too, or whether you think one option is just better in every situation.

A Three-Part Structure for Any Tradeoff



This structure works for literally any tradeoff in system design. Here it is applied to a few of the most commonly asked ones.

```
"The tradeoff here is between consistency and availability.  
If I want every read to return the latest write, I need  
strong consistency, but that means the system has to wait  
or reject requests during a network partition. If I am okay  
with reads occasionally being slightly stale, eventual  
consistency keeps the system available even during issues."
```

Notice the language never says one side is wrong. It explains the mechanics of what each choice costs, then connects it back to what this specific system actually needs. That last connection is what makes it sound like reasoning rather than a memorized definition.

SYNCHRONOUS CALL

- Simple to reason about, immediate response
- Caller waits for the full operation to finish
- A slow downstream service slows everything
- Good for: payment confirmation, auth checks

ASYNC VIA QUEUE

- Caller does not wait, feels faster to the user
- Harder to reason about ordering and failures
- Needs retry and dead-letter handling
- Good for: notifications, video processing

In real projects, this usually becomes a problem when a team picks async everywhere because it feels more scalable, then spends weeks debugging why events arrive out of order or get processed twice. Sync

is not the lesser option. It is the correct option for anything the user is actively waiting on and needs an immediate, certain answer to.

TRADEOFF

If the interviewer asks "why not just use Kafka for everything," a strong answer is: "Kafka adds value when something can happen later without the user noticing. For anything the user is waiting on right now, like a payment result, adding a queue just adds latency and complexity without a real benefit."

TOPIC 6

Handling Follow-Up Questions

You finish your design. It looks solid. Then the interviewer asks something you did not prepare for: "what happens if two requests try to update the same row at the same time?" Your mind goes blank for a second. This moment, more than anything else in the interview, is what actually gets evaluated.

Follow-up questions are not designed to catch you out. They are designed to see how you think when you do not already have a memorized answer. Interviewers expect some hesitation here. What they do not want to see is you inventing a confident-sounding answer that does not actually address the question.

It is completely fine to say "I have not thought about that specific case, give me a second." A short pause followed by a reasoned answer beats an instant answer that falls apart under a second follow-up.

A Way to Buy Yourself Thinking Time

Repeating the question back, slightly rephrased, gives you a few seconds to think without going silent. It also confirms you understood what they are actually asking, which matters more than people expect.

```
"So you are asking what happens if there is a race condition  
on the same row, let me think through that."
```

```
"Just to confirm, you mean if the payment service crashes  
after charging the card but before marking the order as paid?"
```

Once you have bought yourself a moment, work through the scenario out loud step by step instead of jumping to a solution. Interviewers can tell the difference between someone narrating their reasoning and someone guessing and hoping it lands.

FOLLOW-UP TYPE	WHAT IT IS REALLY TESTING
What if this component fails?	Do you design for failure, not just happy path
What if traffic spikes 10x?	Do you understand your own bottlenecks
What if two requests race?	Do you understand concurrency at all
Why not use [other technology]?	Can you defend a choice without dismissing alternatives
How would you monitor this?	Do you think about production, not just design

If you genuinely do not know the answer to a follow-up, naming the tradeoff you would need to investigate is a respectable fallback. "I would need to check whether the database gives me row-level locking by default, but if not, I would add an optimistic lock using a version column" shows real engineering instinct even without a complete answer.

TOPIC 7

Common Mistakes Freshers Make

Most of what costs freshers points in system design interviews has nothing to do with not knowing enough technologies. It is a small set of habits that repeat across almost every weak interview. Here are the ones that show up most often, and what to do instead.

MISTAKE	WHY IT HURTS	DO THIS INSTEAD
Jumping straight to drawing	Skips requirement gathering entirely	Spend 2-3 minutes on scope and scale first
Naming every technology at once	Sounds memorized, not reasoned	Introduce each component only when it solves a stated problem
Silent diagramming	Interviewer cannot see your thinking	Narrate every box and arrow as you draw it
Picking a database with no reason	Looks like a guess	Tie the choice to data shape and access pattern
Treating tradeoffs as one-sided	Misses the point of the question	Always state what you gain and what you give up
Freezing on follow-ups	Looks like memorized knowledge only	Repeat the question back, then reason out loud
Ignoring failure scenarios	Design looks naive and untested	Mention what breaks and how you would detect it

Of these, the one that costs the most points without the candidate ever realizing it is treating tradeoffs as one-sided. It is such a small habit to fix, just one extra sentence after every decision, and it changes how the entire answer is perceived from start to finish.

WATCH OUT

Watch for filler phrases that sound technical but say nothing: "it is scalable," "it is industry standard," "everyone uses this." None of these explain a decision. If you catch yourself saying one, stop and replace it with an actual reason tied to this specific system.

Another pattern worth naming: over-engineering a simple system because you are nervous and want to show range. If someone asks you to design a URL shortener for a small internal team and you immediately propose a multi-region, Kafka-backed, sharded architecture, that is not impressive, it is a signal that you do not calibrate design to actual requirements.

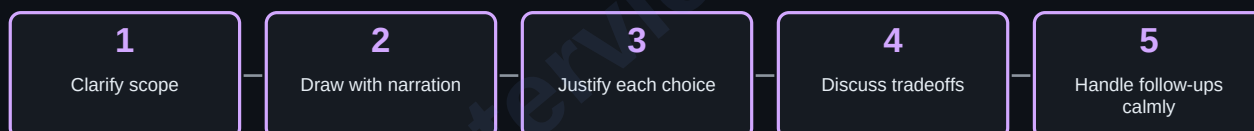
Many beginners design this directly because they think more components equals more credit. It is the opposite. A simple design that matches the stated scale, with a clear path to scale it further if asked, reads as more senior than an overbuilt design from the first sentence.

BONUS SECTION SUMMARY

Putting the Language Into Practice

None of the phrases in this section are meant to be memorized word for word and recited. They are meant to be practiced until the underlying structure becomes automatic: clarify before designing, narrate while drawing, justify every choice with a reason, name the tradeoff, and stay calm on follow-ups.

The Full Answer Structure, End to End



INTERVIEW FOCUS

If you remember nothing else from this section, remember this: every sentence in a strong system design answer either states a requirement, justifies a decision, or names a tradeoff. If a sentence does none of those three things, it is probably filler, and filler is what makes a correct answer sound unconvincing.

The goal of this section was never to give you a script. It was to give you a way of thinking out loud that holds up regardless of which system gets asked, because the structure stays the same even when the technology details change.